MANUAL TÉCNICO DE FORMACIÓN PROFESIONAL

Administración de Servidores Linux

Web, bases de datos y correo electrónico desde cero

Ubuntu Server 24.04 LTS · Apache 2.4 · PHP-FPM 8.3 · MariaDB 10.11

Postfix · Dovecot 2.3 · OpenDKIM · SPF · DMARC · Let's Encrypt · UFW · Fail2ban · vsftpd

Proyecto guía: ejemplo1.com y ejemplo2.com alojados en un único servidor

Edición: noviembre de 2026

Índice

Introducción 7
A quién va dirigido este manual 7
Cómo está organizado 7
Convenciones tipográficas 8
Plataforma de referencia y versiones 8
Antes de empezar: lo que necesitas saber de tu entorno 9
Los datos del proyecto guía 10
Módulo 1. Fundamentos de infraestructura 12
Objetivos del módulo 12
1.1 Qué es un servidor Linux 12
1.2 Servidor local y servidor público 12
1.3 Direcciones IP públicas y privadas 13
1.4 Qué es un dominio 13
1.5 Qué es el DNS 14
1.6 Tipos de registros DNS 14
1.7 Qué es un puerto 15
1.8 TCP y UDP 15
1.9 Cómo se comunican los servicios 16
1.10 Qué es SSH 16
1.11 Preparar el sistema 18
1.12 Usuarios administrativos 20
1.13 Permisos y privilegios en Linux 23
1.14 Ejercicios prácticos del Módulo 1 25
Módulo 2. Instalación y configuración de Apache 27
Objetivos del módulo 27
2.1 Qué es Apache y cómo funciona 27
2.2 Gestión del servicio con systemctl 29
2.3 Estructura de directorios de Apache en Ubuntu 30
2.4 El directorio raíz de las webs, propietarios y permisos 32
2.5 Comprobación de errores de configuración: apachectl 33
2.6 Activación y desactivación de módulos, sitios y configuraciones 34
2.7 Logs de acceso y de errores 35
2.8 Ejercicios prácticos del Módulo 2 37
Módulo 3. Configuración de Virtual Hosting 38
Objetivos del módulo 38

3.1 Qué es un VirtualHost 38
3.2 Diseño de carpetas y usuarios 38
3.3 Los archivos de configuración de los VirtualHosts 43
3.4 Probar los dominios 45
3.5 Solucionar conflictos entre VirtualHosts 47
3.6 HTTPS para cada dominio 48
3.7 Ejercicios prácticos del Módulo 3 54
Módulo 4. Instalación y configuración de MariaDB 55
Objetivos del módulo 55
4.1 Qué es MariaDB 55
4.2 La consola de MariaDB 58
4.3 Bases de datos y usuarios independientes para cada VirtualHost 59
4.4 Copias de seguridad y restauración 65
4.5 Referencia rápida de SQL 67
4.6 Ejercicios prácticos del Módulo 4 67
Módulo 5. Integración de MariaDB con Apache y PHP 68
Objetivos del módulo 68
5.1 Cómo funciona una página dinámica 68
5.2 Instalación de PHP y PHP-FPM 69
5.3 Estructura de configuración de PHP-FPM 70
5.4 Conexión de Apache con PHP-FPM 74
5.5 Conexión de PHP con MariaDB 77
5.6 Aplicación completa: agenda de contactos (CRUD) 82
5.7 Protección de credenciales: resumen 88
5.8 Ejercicios prácticos del Módulo 5 88
Módulo 6. Instalación y configuración de Postfix 90
Objetivos del módulo 90
6.1 Qué es SMTP y cómo viaja un correo 90
6.2 Los puertos SMTP: recepción, envío y envío autenticado 91
6.3 Instalación de Postfix 93
6.4 Estructura y arquitectura de Postfix 94
6.5 Los parámetros fundamentales 95
6.6 Configuración de Postfix 96
6.7 Pruebas, logs y cola de correo 99
6.8 Si tu proveedor bloquea el puerto 25: relayhost 103
6.9 Ejercicios prácticos del Módulo 6 103
Módulo 7. Instalación y configuración de Dovecot 104

Objetivos del módulo 104
7.1 Qué son IMAP y POP3 104
7.2 Puertos de IMAP y POP3 105
7.3 Maildir: dónde y cómo se guardan los mensajes 105
7.4 Estructura de configuración de Dovecot 106
7.5 Integración de Dovecot con Postfix 116
7.6 Pruebas de funcionamiento 120
7.7 Ejercicios prácticos del Módulo 7 123
Módulo 8. Creación de cuentas de correo 125
Objetivos del módulo 125
8.1 Cuentas del sistema frente a cuentas virtuales 125
8.2 Qué hace falta para que una dirección exista 126
8.3 Proteger la identidad del remitente 126
8.4 Scripts de administración de cuentas 128
8.5 Crear las cuentas del proyecto 131
8.6 Gestión de buzones y permisos 134
8.7 Configurar un cliente de correo 135
8.8 Ejercicios prácticos del Módulo 8 136
Módulo 9. Configuración DNS del servidor de correo 137
Objetivos del módulo 137
9.1 Por qué el DNS es crítico para el correo 137
9.2 Registros A y MX 138
9.3 El DNS inverso (PTR) 139
9.4 SPF (Sender Policy Framework) 139
9.5 DKIM y OpenDKIM 141
9.6 DMARC 146
9.7 Comprobar la propagación de los registros 147
9.8 Verificación final de la autenticación 148
9.9 Reputación de la IP y entregabilidad 149
9.10 Ejercicios prácticos del Módulo 9 149
Módulo 10. Certificados SSL/TLS 150
Objetivos del módulo 150
10.1 Cómo funciona TLS 150
10.2 Let's Encrypt y ACME 151
10.3 Los certificados de nuestra infraestructura 151
10.4 Instalación de Certbot: apt o snap 152
10.5 Renovación automática 153

10.6 Verificar los certificados de todos los servicios 155
10.7 Protección de las conexiones de autenticación 157
10.8 Operaciones habituales con certificados 157
10.9 Ejercicios prácticos del Módulo 10 158
Módulo 11. Integración completa de los servicios 159
Objetivos del módulo 159
11.1 Inventario de la infraestructura 159
11.2 El sistema web: INTERNET → DNS → APACHE → PHP → MARIADB 160
11.3 El sistema de correo: INTERNET → DNS MX → POSTFIX → DOVECOT → BUZÓN → CLIENTE 160
11.4 Cómo se relacionan ambos sistemas: la web que envía correos 161
11.5 Dependencias entre servicios 166
11.6 Ejercicios prácticos del Módulo 11 166
Módulo 12. Seguridad y mantenimiento 168
Objetivos del módulo 168
12.1 El modelo de seguridad por capas 168
12.2 Configuración definitiva de UFW 168
12.3 Protección de SSH 170
12.4 Protección de MariaDB 170
12.5 Seguridad de Apache 171
12.6 Seguridad de Postfix y Dovecot 172
12.7 Protección contra fuerza bruta con Fail2ban 173
12.8 Actualizaciones automáticas de seguridad 176
12.9 Copias de seguridad automáticas 177
12.10 Monitorización de servicios 180
12.11 Revisión de logs 182
12.12 Recuperación ante fallos 183
12.13 Ejercicios prácticos del Módulo 12 183
Módulo 13. Pruebas y diagnóstico 184
Objetivos del módulo 184
13.1 Método de diagnóstico 184
13.2 Procedimiento de verificación completo 184
13.3 Hoja de verificación final 188
13.4 Catálogo de errores reales y soluciones 188
13.5 Caja de herramientas de diagnóstico 191
13.6 Ejercicios prácticos del Módulo 13 191
Módulo 14. Proyecto final 192
Objetivos del proyecto 192

14.1 Requisitos de partida 192
14.2 Plan de construcción 192
14.3 Pruebas de aceptación 195
14.4 Entregables del proyecto 196
14.5 Retos de ampliación 196
14.6 Archivo main.cf completo de referencia 197
14.7 Conclusión 198
Módulo 15. Servidor FTP con vsftpd (complementario) 200
Objetivos del módulo 200
15.1 Qué es FTP y para qué lo queremos 200
15.2 Preparar el DNS y el certificado 201
15.3 Instalación de vsftpd 202
15.4 Usuarios FTP y jaula chroot 203
15.5 Configuración de vsftpd 206
15.6 Pruebas de funcionamiento 209
15.7 Integración con la infraestructura 212
15.8 Gestión del día a día 214
15.9 Diagrama de la integración 215
15.10 Alternativa: SFTP con OpenSSH 215
15.11 Ejercicios prácticos del Módulo 15 216
Glosario técnico 217
Anexos 220
Anexo A. Tabla de puertos 220
Anexo B. Mapa de archivos de la infraestructura 220
Anexo C. Comandos esenciales por servicio 221
Anexo D. Lista de tareas para añadir un nuevo dominio 222

Introducción

A quién va dirigido este manual

Este manual está escrito para una persona que nunca ha administrado un servidor y quiere aprender a hacerlo de verdad. No basta con instalar paquetes: el objetivo es entender qué se hace, por qué se hace, qué pasa por dentro y cómo comprobar que funciona.

Al terminar, serás capaz de montar, desde un servidor vacío, una infraestructura completa con:

- Dos sitios web independientes (`ejemplo1.com` y `ejemplo2.com`) servidos por Apache mediante Virtual Hosting.
- Una aplicación PHP en cada sitio, ejecutada con PHP-FPM y un usuario del sistema propio para cada web.
- Una base de datos MariaDB independiente para cada aplicación (`web1_db` y `web2_db`), cada una con su propio usuario y permisos limitados.
- Un servidor de correo completo: Postfix (SMTP) para enviar y recibir, y Dovecot (IMAP/POP3) para consultar los buzones.
- Cuentas de correo virtuales como `info@ejemplo1.com`, `administracion@ejemplo1.com` y `soporte@ejemplo2.com`.
- Firma DKIM, políticas SPF y DMARC, y DNS inverso para que tus correos no acaben en la carpeta de spam.
- Certificados SSL/TLS gratuitos de Let's Encrypt para la web y el correo, con renovación automática.
- Cortafuegos UFW, protección contra fuerza bruta con Fail2ban, copias de seguridad automáticas y procedimientos de diagnóstico.
- Como complemento, un servidor FTP cifrado (FTPS) con vsftpd para que el responsable de cada web suba sus archivos, encerrado en su propia carpeta.

Cómo está organizado

El manual tiene 14 módulos que se apoyan unos en otros, más un módulo complementario (el 15) dedicado al servidor FTP. Cada módulo empieza con sus objetivos y termina con ejercicios prácticos. Las operaciones están divididas en **PASOS** que siempre siguen la misma estructura:

Apartado	Qué contiene
Objetivo	Qué vamos a conseguir con este paso.
Explicación	Qué estamos haciendo, por qué y qué ocurre internamente.
Comando	El comando completo, listo para copiar y pegar.
Análisis del comando	Qué significa cada parte y cada parámetro.
Resultado esperado	Lo que debería aparecer en el terminal.
Comprobación	Cómo verificar que el paso ha funcionado antes de seguir.
Errores habituales	Qué puede fallar y cómo solucionarlo.
Siguiente paso	Qué haremos a continuación.

> **ATENCIÓN: Regla de oro del manual:** no pases nunca al paso siguiente sin haber hecho la comprobación del paso actual. Casi todos los problemas difíciles de un servidor son en realidad tres problemas fáciles que se han acumulado sin que nadie los detectara.

Convenciones tipográficas

Los bloques de código llevan una etiqueta que indica qué son:

TERMINAL
```
# TERMINAL: comandos que se escriben en la consola del servidor (o del equipo que se indique).
# Las líneas que empiezan por # son comentarios: no hacen nada, solo explican.
sudo systemctl status apache2
```

ARCHIVO DE CONFIGURACIÓN
```
# ARCHIVO DE CONFIGURACIÓN: contenido de un archivo. Encima se indica siempre su ruta exacta.
ServerName www.ejemplo1.com
```

SALIDA ESPERADA
```
SALIDA ESPERADA: lo que el terminal debería mostrarte. Puede variar ligeramente
(fechas, números de proceso, versiones menores), pero la estructura será la misma.
```

Además:

- El indicador `sysadmin@mail:~$` significa «terminal del servidor, como usuario normal `sysadmin`». Si ves `root@mail:~#` estás como superusuario. Nunca copies el indicador, solo lo que va detrás.
- `sudo` delante de un comando significa «ejecútalo con privilegios de administrador». Te pedirá tu contraseña de usuario (no la de root).
- Las palabras entre `<` y `>` (por ejemplo `<CONTRASEÑA_SEGURA>`) son marcadores: sustitúyelas por tu valor real, **sin** los signos `<` y `>`.

> **NOTA:** Este manual no contiene contraseñas reales. Todos los valores como `Cambia.Esta.Clave.01` son ejemplos y **debes** sustituirlos por contraseñas largas y únicas generadas por ti (más adelante verás cómo generarlas con `openssl rand`).

Plataforma de referencia y versiones

Para que todos los comandos, rutas y archivos sean exactos, el manual usa una única plataforma de referencia:

Componente	Versión de referencia (Ubuntu Server 24.04 LTS)
Sistema operativo	Ubuntu Server 24.04 LTS «Noble Numbat», 64 bits (amd64 o arm64)
Servidor web	Apache HTTP Server 2.4.58 o superior (paquete `apache2`)
PHP	PHP 8.3 con PHP-FPM (paquetes `php8.3-*`)
Base de datos	MariaDB 10.11 LTS (paquete `mariadb-server`)
SMTP	Postfix 3.8

Componente	Versión de referencia (Ubuntu Server 24.04 LTS)
IMAP / POP3	Dovecot 2.3.21
Firma DKIM	OpenDKIM 2.11
Certificados	Certbot 2.x (paquete `certbot` + `python3-certbot-apache`)
Cortafuegos	UFW 0.36
Anti fuerza bruta	Fail2ban 1.0
FTP (módulo complementario)	vsftpd 3.0.5

Ubuntu 24.04 LTS tiene soporte estándar hasta abril de 2029, utiliza Dovecot 2.3 (cuya sintaxis de configuración está muy documentada) y es la plataforma que la mayoría de proveedores de servidores ofrecen como imagen preinstalada.

> **ATENCIÓN: Si usas otra versión del sistema, lee esto.** Debian 12 «bookworm» es casi idéntica a Ubuntu 24.04 en todo lo que hace este manual; la diferencia principal es que trae PHP 8.2, así que donde ponga `8.3` deberás escribir `8.2` (por ejemplo `/etc/php/8.2/fpm/`). En cambio, **Debian 13 «trixie» y Ubuntu 26.04 incluyen Dovecot 2.4**, cuya sintaxis de configuración cambió de forma importante (por ejemplo `mail_location` se divide en `mail_driver` y `mail_path`, y el bloque `passdb` usa otro formato). Los módulos 7 y 8 están escritos para Dovecot 2.3. Comprueba siempre tu versión con `dovecot --version` antes de copiar su configuración.

Antes de empezar: lo que necesitas saber de tu entorno

Un administrador nunca empieza a instalar sin conocer el terreno. Rellena esta ficha antes de continuar; la usarás durante todo el manual.

Dato	Cómo averiguarlo	Valor en el manual	Tu valor
Sistema operativo y versión	`lsb_release -a` o `cat /etc/os-release`	Ubuntu 24.04 LTS	
Arquitectura	`uname -m` (x86_64 = amd64; aarch64 = arm64)	x86_64	
IP pública IPv4	Panel del proveedor o `curl -4 ifconfig.me`	203.0.113.10	
IP pública IPv6 (si tienes)	Panel del proveedor o `ip -6 addr`	2001:db8:10::10	
Dominio 1	El que hayas registrado	ejemplo1.com	
Dominio 2	El que hayas registrado	ejemplo2.com	
Acceso	SSH con usuario y contraseña, o consola web	SSH	
Proveedor DNS	Donde gestionas los registros (registrador, Cloudflare...)	—	
¿Puedes configurar el DNS inverso (PTR)?	Panel del proveedor del servidor (no del dominio)	Sí	
¿El puerto 25 saliente está abierto?	Documentación del proveedor; prueba del Módulo 6	Sí	
RAM y disco	`free -h` y `df -h`	2 GB / 40 GB	

> **NOTA:** Las direcciones `203.0.113.10` y `2001:db8:10::10` pertenecen a rangos reservados para documentación (RFC 5737 y RFC 3849). No existen en Internet: sustitúyelas siempre por las tuyas. Lo mismo ocurre con `ejemplo1.com` y `ejemplo2.com`.

Restricciones de proveedores que debes conocer

- **Puerto 25 bloqueado.** Muchas nubes (AWS, Google Cloud, Azure, y la mayoría de conexiones domésticas) bloquean por defecto el tráfico saliente al puerto 25 para frenar el spam. Sin él tu servidor podrá **recibir** correo, pero no **entregarlo** a Gmail, Outlook, etc. Algunos proveedores lo desbloquean bajo petición; otros nunca. Si es tu caso, tendrás que usar un *relay* (servidor intermediario) de envío. Lo verás en el Módulo 6.
- **DNS inverso (PTR).** Lo configura el dueño de la IP (tu proveedor del servidor), no tu registrador de dominios. Sin un PTR correcto, gran parte de tus correos será rechazada.
- **IP doméstica o dinámica.** Puedes seguir todo el manual en una máquina virtual de tu casa para aprender, pero un servidor de correo real necesita una IP pública fija con buena reputación. Las IP domésticas suelen estar en listas negras (por ejemplo la PBL de Spamhaus).
- **RAM.** 1 GB es el mínimo absoluto; 2 GB es lo recomendable para Apache + PHP + MariaDB + correo.

> **CONSEJO:** **Si estás aprendiendo en un laboratorio local** (VirtualBox, VMware, Proxmox, Hyper-V) sin IP pública ni dominios reales, puedes seguir todos los módulos. Las diferencias son tres: (1) en lugar de configurar el DNS público, añadirás los nombres al archivo `hosts` de tu equipo; (2) Let's Encrypt no podrá emitir certificados, así que usarás el certificado autofirmado que Ubuntu genera automáticamente (`/etc/ssl/certs/ssl-cert-snakeoil.pem`); (3) no podrás intercambiar correo con Internet, pero sí entre tus propias cuentas. Cada módulo indica cómo adaptarlo.

Los datos del proyecto guía

Durante todo el manual usaremos estos valores. Cuando adaptes el manual a tu caso, mantén la coherencia: si cambias un nombre, cámbialo en todas partes.

Elemento	ejemplo1.com	ejemplo2.com
Nombre web principal	www.ejemplo1.com	www.ejemplo2.com
Alias	ejemplo1.com	ejemplo2.com
Usuario Linux del sitio	web1	web2
Carpeta del sitio	/var/www/ejemplo1.com	/var/www/ejemplo2.com
Carpeta pública (DocumentRoot)	/var/www/ejemplo1.com/public_html	/var/www/ejemplo2.com/public_html
Archivo VirtualHost	/etc/apache2/sites-available/ejemplo1.com.conf	/etc/apache2/sites-available/ejemplo2.com.conf
Pool PHP-FPM	/etc/php/8.3/fpm/pool.d/web1.conf	/etc/php/8.3/fpm/pool.d/web2.conf
Base de datos	web1_db	web2_db
Usuario de base de datos	web1_user	web2_user

Elemento	ejemplo1.com	ejemplo2.com
Cuentas de correo	info@, administracion@, postmaster@ (alias)	soporte@, postmaster@ (alias)
Servidor de correo (MX)	mail.ejemplo1.com	mail.ejemplo2.com (misma IP)
Acceso FTP (Módulo 15)	ftp.ejemplo1.com, usuario web1	ftp.ejemplo2.com, usuario web2

El servidor se llamará `mail.ejemplo1.com`. Es su nombre «oficial» (el que aparecerá en el DNS inverso y en los saludos SMTP). El usuario administrador del sistema será `sysadmin`.

Módulo 1. Fundamentos de infraestructura

Objetivos del módulo

- Entender qué es un servidor Linux y qué diferencia un servidor local de uno público.
- Comprender las direcciones IP, los dominios, el DNS y sus tipos de registro.
- Saber qué son los puertos, TCP y UDP, y cómo se comunican los servicios.
- Conectarte a tu servidor por SSH de forma segura con claves.
- Dejar el sistema actualizado, con nombre, zona horaria y un usuario administrador propio.
- Dominar lo básico de usuarios, grupos y permisos en Linux.

1.1 Qué es un servidor Linux

Un **servidor** es un ordenador cuya función es ofrecer servicios a otros ordenadores, llamados **clientes**. Cuando abres `https://www.ejemplo1.com` en tu navegador, tu navegador es el cliente y el ordenador que te entrega la página es el servidor.

Un servidor Linux es simplemente un ordenador con el sistema operativo Linux, normalmente **sin entorno gráfico**: se administra escribiendo comandos en un terminal. No tener escritorio no es una limitación, es una ventaja: consume menos memoria, tiene menos programas que actualizar y menos puntos débiles para un atacante.

Dentro del servidor, cada servicio es un programa que se queda funcionando en segundo plano (un **demonio** o *daemon*) esperando peticiones:

Servicio	Programa (demonio)	Qué hace
Web	`apache2`	Entrega páginas web por HTTP/HTTPS.
Intérprete PHP	`php8.3-fpm`	Ejecuta el código PHP que Apache le pasa.
Base de datos	`mariadbd`	Guarda y entrega datos estructurados.
Correo SMTP	`master` (Postfix)	Recibe correo de Internet y envía el de tus usuarios.
Buzones	`dovecot`	Permite a los usuarios leer su correo (IMAP/POP3).
Firma DKIM	`opendkim`	Firma criptográficamente los correos salientes.
Acceso remoto	`sshd`	Te permite administrar el servidor a distancia.

En Ubuntu, todos estos demonios los arranca, detiene y vigila **systemd**, el gestor de servicios del sistema. La herramienta para hablar con systemd es `systemctl`, que usarás constantemente.

1.2 Servidor local y servidor público

Característica	Servidor local (laboratorio)	Servidor público (producción)
Dónde está	Tu casa, tu empresa, una máquina virtual.	Un centro de datos (VPS, servidor dedicado, nube).
Dirección IP	Privada (192.168.x.x, 10.x.x.x).	Pública, accesible desde todo Internet.
Quién puede acceder	Solo los equipos de tu red.	Cualquiera en el mundo (también atacantes).

Característica	Servidor local (laboratorio)	Servidor público (producción)
Dominios	Simulados con el archivo `hosts`.	Reales, resueltos por DNS público.
Certificados	Autofirmados.	Let's Encrypt u otra autoridad.
Correo	Solo entre cuentas internas.	Intercambio con todo Internet.

> **ATENCIÓN:** Un servidor público empieza a recibir intentos de intrusión **minutos** después de encenderse. Bots automáticos prueban contraseñas en SSH las 24 horas del día. Por eso en este módulo aseguraremos el acceso antes de instalar ningún servicio.

1.3 Direcciones IP públicas y privadas

Una **dirección IP** identifica a un equipo en una red, como el número de un portal en una calle. Hay dos versiones:

- **IPv4**: cuatro números de 0 a 255 separados por puntos, por ejemplo `203.0.113.10`. Hay unos 4.300 millones y están prácticamente agotadas.
- **IPv6**: ocho grupos hexadecimales separados por dos puntos, por ejemplo `2001:db8:10::10` (`::` abrevia grupos de ceros). Hay muchísimas más.

Las IP **privadas** solo valen dentro de una red local y no se pueden alcanzar directamente desde Internet. Los rangos reservados son:

Rango privado IPv4	Uso típico
10.0.0.0 – 10.255.255.255 (10.0.0.0/8)	Redes de empresa, nubes internas.
172.16.0.0 – 172.31.255.255 (172.16.0.0/12)	Redes de empresa, Docker.
192.168.0.0 – 192.168.255.255 (192.168.0.0/16)	Redes domésticas.
127.0.0.0/8 (sobre todo 127.0.0.1)	*Loopback*: el propio equipo hablándose a sí mismo.

Una IP **pública** es única en todo Internet. Tu servidor necesita una IP pública para que el mundo pueda llegar a tu web y a tu correo.

La notación `/24`, `/8`, etc. (llamada CIDR) indica cuántos bits de la dirección son fijos. `192.168.1.0/24` significa «todas las direcciones de 192.168.1.0 a 192.168.1.255».

> **CONSEJO:** La dirección `127.0.0.1` (y su equivalente IPv6 `::1`) es muy importante en este manual: los servicios que solo deben ser usados desde dentro del servidor (MariaDB, OpenDKIM) escucharán **solo** en esa dirección, de modo que sean invisibles desde Internet.

1.4 Qué es un dominio

Recordar `203.0.113.10` es incómodo; recordar `ejemplo1.com` es fácil. Un **dominio** es un nombre legible que se traduce a una dirección IP.

Los dominios se leen de derecha a izquierda, de lo general a lo particular:

DIAGRAMA

```
          www . ejemplo1 . com .
           │       │        │   └─ raíz (normalmente invisible)
           │       │        └──── dominio de nivel superior (TLD)
           │       └───────────── dominio que tú registras
           └───────────────────── subdominio (lo creas tú, gratis)
```

Tú **alquilas** `ejemplo1.com` a un **registrador** (por un año o varios). A partir de ahí puedes crear todos los subdominios que quieras: `www.ejemplo1.com`, `mail.ejemplo1.com`, `tienda.ejemplo1.com`...

1.5 Qué es el DNS

El **DNS** (*Domain Name System*) es la «guía telefónica» de Internet: un sistema distribuido de servidores que responde a preguntas como «¿qué IP tiene www.ejemplo1.com?» o «¿qué servidor recibe el correo de ejemplo1.com?».

Cuando tu navegador visita `www.ejemplo1.com` ocurre esto:

DIAGRAMA

```
Navegador ──1──► Resolvedor DNS (de tu operador, 1.1.1.1, 8.8.8.8...)
                        │
                        ├──2──► Servidores raíz:      "¿.com?"        → pregunta a los de .com
                        ├──3──► Servidores de .com:   "¿ejemplo1.com?" → pregunta a ns1.tuproveedor
                        └──4──► Servidor autoritativo de ejemplo1.com:
                                  "www.ejemplo1.com tiene la IP 203.0.113.10"
Navegador ◄──5── 203.0.113.10   (la respuesta se guarda en caché durante el TTL)
Navegador ──6──► Conecta con 203.0.113.10 puerto 443 y pide la página
```

El **servidor autoritativo** es el que tiene la última palabra sobre tu dominio. Normalmente es el de tu registrador o el de un servicio DNS como Cloudflare. Ahí es donde **tú** crearás los registros. En este manual no montaremos un servidor DNS propio (sería un riesgo y una complicación innecesaria): usaremos el panel de tu proveedor DNS.

El **TTL** (*Time To Live*) de cada registro indica cuántos segundos pueden guardar la respuesta en caché los resolvedores. Un TTL de 3600 significa que un cambio puede tardar hasta una hora en verse en todo el mundo. Antes de hacer cambios importantes, conviene bajar el TTL a 300 (5 minutos).

1.6 Tipos de registros DNS

Registro	Para qué sirve	Ejemplo
A	Asocia un nombre a una IPv4.	`www.ejemplo1.com. A 203.0.113.10`
AAAA	Asocia un nombre a una IPv6.	`www.ejemplo1.com. AAAA 2001:db8:10::10`
CNAME	Hace que un nombre sea alias de otro nombre.	`blog.ejemplo1.com. CNAME www.ejemplo1.com.`
MX	Indica qué servidor recibe el correo del dominio, con una prioridad.	`ejemplo1.com. MX 10 mail.ejemplo1.com.`
TXT	Texto libre; se usa para SPF, DKIM, DMARC y verificaciones.	`ejemplo1.com. TXT "v=spf1 mx -all"`
PTR	DNS inverso: asocia una IP a un nombre. Lo gestiona el dueño de la IP.	`10.113.0.203.in-addr.arpa. PTR mail.ejemplo1.com.`

Registro	Para qué sirve	Ejemplo
NS	Indica qué servidores son autoritativos para el dominio.	`ejemplo1.com. NS ns1.proveedor.net.`
CAA	Indica qué autoridades pueden emitir certificados para el dominio.	`ejemplo1.com. CAA 0 issue "letsencrypt.org"`

Detalles importantes:

- El **@** en los paneles DNS significa «el propio dominio» (`ejemplo1.com` sin nada delante).
- Un **CNAME** no puede coexistir con otros registros del mismo nombre, por eso nunca se pone CNAME en `@`.
- Un **MX** debe apuntar a un **nombre** que tenga registro A/AAAA, **nunca** a una IP ni a un CNAME.
- En el **MX**, el número es la prioridad: cuanto más bajo, más preferido. Con un solo servidor, el valor da igual (usaremos 10).
- El **PTR** lo verás en detalle en el Módulo 9. Es crucial para el correo.

1.7 Qué es un puerto

La IP lleva el paquete hasta el servidor; el **puerto** indica a qué programa dentro del servidor va dirigido. Si la IP es el portal, el puerto es el número de piso. Hay 65.535 puertos; los inferiores a 1024 están reservados para servicios conocidos y solo root puede abrirlos.

Puerto	Protocolo	Servicio	¿Abierto a Internet en este manual?
21	TCP	FTP con TLS (FTPS explícito, Módulo 15)	Solo si instalas el FTP
22	TCP	SSH (administración remota)	Sí (protegido con claves y Fail2ban)
25	TCP	SMTP entre servidores (recepción de correo)	Sí
53	UDP/TCP	DNS	No (no montamos DNS propio)
80	TCP	HTTP (web sin cifrar; redirige a 443)	Sí
110	TCP	POP3 sin cifrar	No (deshabilitado)
143	TCP	IMAP sin cifrar / STARTTLS	No (deshabilitado)
443	TCP	HTTPS (web cifrada)	Sí
465	TCP	SMTPS: envío autenticado con TLS implícito	Sí
587	TCP	Submission: envío autenticado con STARTTLS	Sí
993	TCP	IMAPS: IMAP cifrado	Sí
995	TCP	POP3S: POP3 cifrado	Sí
3306	TCP	MariaDB	**No, nunca** (solo 127.0.0.1)
8891	TCP	OpenDKIM (milter)	**No** (solo 127.0.0.1)
40000–40100	TCP	FTP pasivo: conexiones de datos (Módulo 15)	Solo si instalas el FTP

1.8 TCP y UDP

Ambos son protocolos de transporte: la forma en que viajan los datos entre dos puertos.

- **TCP** (*Transmission Control Protocol*) es orientado a conexión. Antes de enviar datos, cliente y servidor se saludan (el «saludo en tres pasos»: SYN, SYN-ACK, ACK). TCP garantiza que los datos llegan completos y en orden, y reenvía lo que se pierda. Lo usan la web, el correo, SSH y MariaDB.
- **UDP** (*User Datagram Protocol*) envía paquetes sueltos sin conexión ni garantías. Es más rápido y ligero. Lo usan el DNS (consultas cortas), la voz sobre IP, los juegos y el streaming.

Todos los servicios de este manual usan TCP, salvo las consultas DNS que tu servidor hace hacia fuera.

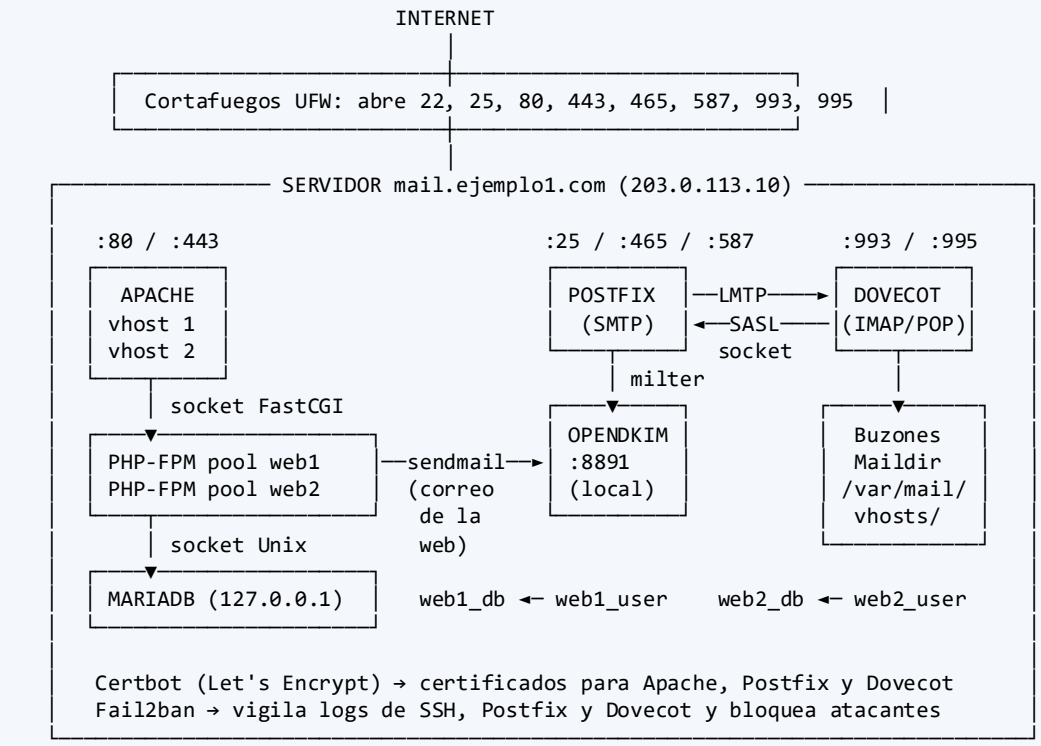
1.9 Cómo se comunican los servicios

Los servicios hablan entre sí de dos maneras:

1. **Por red (TCP/IP)**: un servicio escucha en `IP:puerto` y otro se conecta. Ejemplo: Postfix se conecta a OpenDKIM en `127.0.0.1:8891`.
2. **Por socket Unix**: un archivo especial del sistema por el que dos programas del **mismo** servidor se comunican sin pasar por la red. Es más rápido y se protege con permisos de archivo. Ejemplos: Apache habla con PHP-FPM por `/run/php/php8.3-fpm-web1.sock`; PHP habla con MariaDB por `/run/mysqld/mysqld.sock`; Postfix entrega el correo a Dovecot por `/var/spool/postfix/private/dovecot-lmtp`.

Esta es la infraestructura que vamos a construir:

DIAGRAMA



1.10 Qué es SSH

SSH (*Secure Shell*) es el protocolo que te permite abrir un terminal en el servidor desde tu ordenador, con todo el tráfico cifrado. En el servidor funciona el demonio `sshd` (puerto 22); en tu ordenador usas un cliente `ssh`.

SSH admite dos formas de autenticarte:

- **Contraseña**: cómoda, pero vulnerable a ataques de fuerza bruta.

- **Par de claves**: generas en tu ordenador una clave **privada** (que nunca sale de tu equipo) y una **pública** (que copias al servidor). El servidor comprueba matemáticamente que tienes la privada sin que esta viaje nunca por la red. Es muchísimo más segura.

Tu objetivo en este módulo: entrar por primera vez con contraseña, instalar tu clave pública y, después, desactivar el acceso por contraseña.

PASO 1.1 – Conectarte al servidor por primera vez

Objetivo: abrir un terminal remoto en el servidor.

Explicación: tu proveedor te habrá dado una IP, un usuario (a menudo `root` o `ubuntu`) y una contraseña o una clave. Windows 10/11, macOS y Linux incluyen el cliente `ssh` de serie. En Windows ábrelo desde «Terminal» o «PowerShell».

Comando: escríbelo en **tu ordenador** (no en el servidor):

TERMINAL

```
ssh root@203.0.113.10
```

Análisis del comando:

- `ssh`: el programa cliente de SSH.
- `root`: el usuario con el que entras. Si tu proveedor te dio `ubuntu`, escribe `ubuntu@203.0.113.10`.
- `@203.0.113.10`: la IP de tu servidor (sustitúyela por la tuya).

Resultado esperado: la primera vez verás un aviso de huella digital:

SALIDA ESPERADA

```
The authenticity of host '203.0.113.10 (203.0.113.10)' can't be established.
ED25519 key fingerprint is SHA256:k3Zr6c0f0yP1G0qkU8JxwO7V4p9k2aQ1Zb8s5m3Lq2E.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Escribe `yes` y pulsa Intro. Después introduce la contraseña (no verás nada mientras escribes: es normal). Llegarás a un indicador similar a `root@nombre-servidor:~#`.

Comprobación: ejecuta `whoami` (debe responder `root` o tu usuario) y `hostname` (el nombre actual del servidor).

Errores habituales:

- `Connection timed out`: la IP es incorrecta, el servidor está apagado o un cortafuegos del proveedor bloquea el puerto 22. Revisa el panel del proveedor.
- `Permission denied (publickey)`: el servidor no admite contraseñas, solo claves. Usa la clave que te dio el proveedor: `ssh -i ruta/a/clave root@IP`.
- `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`: la huella del servidor cambió (por ejemplo, reinstalaste el sistema). Si sabes por qué, borra la huella antigua con `ssh-keygen -R 203.0.113.10` y reconecta. Si no sabes por qué, **no continúes**: podría ser un ataque.

Siguiente paso: actualizar el sistema.

1.11 Preparar el sistema

PASO 1.2 – Actualizar el sistema

Objetivo: instalar todas las correcciones de seguridad publicadas desde que se creó la imagen del servidor.

Explicación: Ubuntu instala software con **APT** a partir de repositorios oficiales. `apt update` descarga la lista actualizada de paquetes disponibles (no instala nada); `apt full-upgrade` instala las versiones nuevas, incluyendo las que necesitan añadir o quitar dependencias.

Comando:

TERMINAL

```
sudo apt update
sudo apt full-upgrade -y
```

Análisis del comando:

- `sudo`: ejecuta como administrador (si ya eres root no hace falta, pero no molesta).
- `apt update`: refresca el índice de paquetes.
- `apt full-upgrade`: actualiza todo lo instalado.
- `-y`: responde «sí» automáticamente a la confirmación.

Resultado esperado: una lista de paquetes descargados e instalados que termina sin errores. Si aparece una pantalla morada preguntando qué hacer con un archivo de configuración modificado, elige la opción por defecto («keep the local version currently installed»).

Comprobación:

TERMINAL

```
apt list --upgradable
ls /var/run/reboot-required 2>/dev/null && echo "Hay que reiniciar"
```

La primera orden no debe listar nada. Si la segunda muestra «Hay que reiniciar», ejecuta `sudo reboot`, espera un minuto y vuelve a conectarte.

Errores habituales:

- `Could not get lock /var/lib/dpkg/lock-frontend`: otro proceso (normalmente las actualizaciones automáticas al arrancar) está usando APT. Espera unos minutos y repite.
- `Temporary failure resolving 'archive.ubuntu.com'`: el servidor no tiene DNS o red. Comprueba con `ping -c 3 1.1.1.1` y `resolvectl status`.

Siguiente paso: dar al servidor su nombre definitivo.

PASO 1.3 – Configurar el nombre del servidor (hostname)

Objetivo: que el servidor se llame `mail.ejemplo1.com` y sepa resolver su propio nombre.

Explicación: el nombre del servidor es fundamental para el correo: Postfix lo usa para presentarse ante otros servidores, y debe coincidir con el registro A y con el DNS inverso (PTR). Hay dos partes: el nombre corto (`mail`) y el nombre completo o **FQDN** (`mail.ejemplo1.com`).

Comando:

TERMINAL

```
sudo hostnamectl set-hostname mail.ejemplo1.com
```

Análisis del comando: `hostnamectl` es la herramienta de systemd para gestionar el nombre; `set-hostname` lo fija de forma permanente (lo guarda en `/etc/hostname`).

Ahora edita el archivo `/etc/hosts`, que el sistema consulta antes que el DNS:

TERMINAL

```
sudo nano /etc/hosts
```

`nano` es un editor de texto sencillo. Te mueves con las flechas, escribes normalmente, guardas con **Ctrl+O** seguido de Intro y sales con **Ctrl+X**.

Archivo: `/etc/hosts`. Deja su contenido así (conserva las líneas IPv6 que ya hubiera debajo):

ARCHIVO DE CONFIGURACIÓN

```
127.0.0.1       localhost
203.0.113.10    mail.ejemplo1.com mail

# Las siguientes líneas son las de IPv6 que trae el sistema; consérvalas
::1             localhost ip6-localhost ip6-loopback
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters
```

ATENCIÓN: Si tu proveedor usa **cloud-init**, puede sobrescribir `/etc/hosts` al reiniciar. Si al reiniciar tus cambios desaparecen, edita `/etc/cloud/cloud.cfg` y cambia `manage_etc_hosts: true` por `manage_etc_hosts: false` (o añade la línea `preserve_hostname: true`).

Comprobación:

TERMINAL

```
hostname
hostname -f
```

Resultado esperado:

SALIDA ESPERADA

```
mail.ejemplo1.com
mail.ejemplo1.com
```

Errores habituales: si `hostname -f` muestra solo `mail` o da error, revisa que la línea de `/etc/hosts` tenga primero el FQDN y luego el nombre corto.

Siguiente paso: configurar la hora.

PASO 1.4 – Configurar la zona horaria y la sincronización de hora

Objetivo: que los registros (logs) muestren tu hora local y que el reloj sea exacto.

Explicación: una hora incorrecta rompe los certificados TLS, confunde los logs y hace que otros servidores desconfíen de tus correos. Ubuntu sincroniza el reloj con `systemd-timesyncd`.

Comando:

TERMINAL

```
sudo timedatectl set-timezone Europe/Madrid
timedatectl
```

Análisis del comando: `set-timezone` fija la zona horaria (lista todas con `timedatectl list-timezones`; por ejemplo `America/Mexico_City` o `America/Argentina/Buenos_Aires`).

Resultado esperado:

SALIDA ESPERADA

```
               Local time: dom 2026-11-08 21:40:12 CET
           Universal time: dom 2026-11-08 20:40:12 UTC
                 Time zone: Europe/Madrid (CET, +0100)
System clock synchronized: yes
              NTP service: active
```

Comprobación: las líneas `System clock synchronized: yes` y `NTP service: active` deben aparecer así.

Errores habituales: si `NTP service` aparece `inactive`, actívalo con `sudo timedatectl set-ntp true`.

Siguiente paso: crear tu usuario administrador.

1.12 Usuarios administrativos

Trabajar siempre como `root` es peligroso: un error de escritura puede destruir el sistema y, si alguien roba la sesión, tiene control total. La práctica profesional es usar un usuario normal que pide privilegios **solo cuando los necesita**, mediante `sudo`. Así cada acción administrativa queda registrada en `/var/log/auth.log`.

PASO 1.5 – Crear el usuario administrador

Objetivo: crear el usuario `sysadmin` con permiso para usar `sudo`.

Comando (ejecútalo como root o con sudo):

TERMINAL

```
sudo adduser sysadmin
sudo usermod -aG sudo sysadmin
```

Análisis del comando:

- `adduser sysadmin`: crea el usuario, su grupo del mismo nombre y su carpeta `/home/sysadmin`. Te pedirá una contraseña (elige una larga) y datos opcionales (pulsa Intro para dejarlos vacíos).

- `usermod -aG sudo sysadmin`: añade (`-a`, *append*) el usuario al grupo (`-G`) `sudo`. En Ubuntu, los miembros de este grupo pueden usar `sudo`. Sin la `-a` el usuario sería **expulsado** de sus otros grupos: no la olvides.

Comprobación:

TERMINAL
```
groups sysadmin
```

SALIDA ESPERADA
```
sysadmin : sysadmin sudo users
```

(`users` puede aparecer o no; lo importante es `sudo`.)

Ahora, **sin cerrar la sesión actual**, abre otro terminal en tu ordenador y entra con el nuevo usuario:

TERMINAL
```
ssh sysadmin@203.0.113.10
sudo whoami
```

`sudo whoami` debe pedirte la contraseña de `sysadmin` y responder `root`.

Errores habituales: `sysadmin is not in the sudoers file` significa que el `usermod` no se aplicó; repítelo y vuelve a iniciar sesión (los grupos se cargan al iniciar sesión).

Siguiente paso: configurar el acceso por clave.

PASO 1.6 – Crear un par de claves SSH y copiar la pública al servidor

Objetivo: entrar al servidor sin contraseña, con una clave criptográfica.

Explicación: generarás las claves en **tu ordenador**. El algoritmo Ed25519 es moderno, rápido y seguro. La clave privada se puede proteger con una frase de paso: si alguien te roba el archivo, no podrá usarlo sin ella.

Comando (en **tu ordenador, Windows, macOS o Linux):**

TERMINAL
```
ssh-keygen -t ed25519 -C "sysadmin@portatil"
```

Análisis del comando:

- `-t ed25519`: tipo de clave.
- `-C "..."`: comentario para identificarla (por ejemplo, desde qué equipo se usa).

Acepta la ruta por defecto pulsando Intro y escribe una frase de paso. Se crean dos archivos en la carpeta `.ssh` de tu usuario: `id_ed25519` (privada, **no la compartas nunca**) y `id_ed25519.pub` (pública).

Para copiar la clave pública al servidor desde **macOS o Linux**:

TERMINAL
```
ssh-copy-id sysadmin@203.0.113.10
```

Desde **Windows (PowerShell)**, que no incluye `ssh-copy-id`:

POWERSHELL (WINDOWS)
```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh sysadmin@203.0.113.10 "mkdir -p ~/.ssh &&
chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Análisis del comando de Windows: `type` muestra el contenido de la clave pública; `|` lo envía como entrada al comando remoto, que crea la carpeta `~/.ssh` con permisos `700` (solo el dueño), añade la clave al final de `authorized_keys` y deja ese archivo con permisos `600`. SSH **rechaza** las claves si estos permisos son más abiertos.

Comprobación: cierra la sesión y vuelve a entrar con `ssh sysadmin@203.0.113.10`. Ahora te pedirá la **frase de paso de la clave** (o nada, si no pusiste) en lugar de la contraseña del servidor.

Errores habituales: si sigue pidiendo la contraseña del servidor, en el servidor revisa permisos con `ls -ld ~/.ssh && ls -l ~/.ssh/authorized_keys` (deben ser `drwx------` y `-rw-------`) y que el propietario sea `sysadmin`.

Siguiente paso: endurecer la configuración de SSH.

PASO 1.7 – Desactivar el acceso de root y por contraseña

Objetivo: que solo se pueda entrar con clave y nunca directamente como root.

Explicación: en Ubuntu 24.04, el archivo principal `/etc/ssh/sshd_config` incluye al principio todos los archivos de `/etc/ssh/sshd_config.d/*.conf` por orden alfabético, y **para cada opción gana el primer valor que se lee**. Algunos proveedores dejan un archivo `50-cloud-init.conf` con `PasswordAuthentication yes`. Por eso nuestro archivo debe empezar por `00-`, para leerse antes.

> **ATENCIÓN: Antes de este paso**, confirma que puedes entrar como `sysadmin` con tu clave y usar `sudo`. Mantén abierta una sesión mientras haces el cambio: si algo sale mal, podrás corregirlo desde ella.

Comando:

TERMINAL
```
sudo nano /etc/ssh/sshd_config.d/00-endurecimiento.conf
```

Archivo: `/etc/ssh/sshd_config.d/00-endurecimiento.conf` (archivo nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN
```
# Endurecimiento de SSH - se lee antes que el resto de archivos .conf
# Prohíbe entrar directamente como root
PermitRootLogin no
# Solo autenticación por clave pública
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
# Número máximo de intentos por conexión
MaxAuthTries 3
# Tiempo máximo para completar el inicio de sesión
LoginGraceTime 30
# Solo este usuario puede entrar por SSH (añade otros separados por espacios)
AllowUsers sysadmin
# Sin reenvío gráfico
X11Forwarding no
```

Comprueba la sintaxis y aplica:

TERMINAL

```
sudo sshd -t
sudo systemctl restart ssh
```

Análisis del comando: `sshd -t` (*test*) revisa la configuración sin aplicarla; si no muestra nada, está bien. `systemctl restart ssh` reinicia el servicio (en Ubuntu se llama `ssh`, no `sshd`).

Comprobación: desde **otro** terminal de tu ordenador:

TERMINAL

```
ssh root@203.0.113.10
ssh -o PubkeyAuthentication=no sysadmin@203.0.113.10
ssh sysadmin@203.0.113.10
```

Las dos primeras deben fallar con `Permission denied`; la tercera debe funcionar. También puedes consultar el valor efectivo de cada opción en el servidor:

TERMINAL

```
sudo sshd -T | grep -Ei 'permitrootlogin|passwordauthentication'
```

SALIDA ESPERADA

```
permitrootlogin no
passwordauthentication no
```

Errores habituales: si `sshd -t` muestra `Bad configuration option`, hay una errata en el archivo. Si te quedas fuera, usa la **consola web** del proveedor (todos la ofrecen) para entrar y borrar el archivo.

Siguiente paso: entender usuarios, grupos y permisos.

1.13 Permisos y privilegios en Linux

Cada archivo y carpeta tiene un **propietario** (un usuario), un **grupo** y tres juegos de permisos: para el propietario (*u*), para el grupo (*g*) y para el resto (*o*). Cada juego puede tener lectura (**r**), escritura (**w**) y ejecución (**x**).

TERMINAL

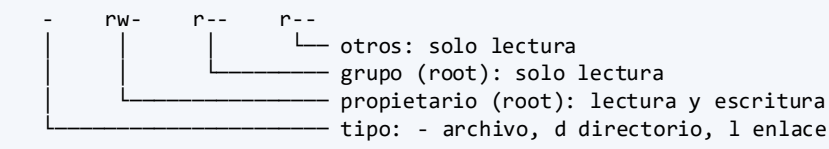
```
ls -l /etc/passwd
```

SALIDA ESPERADA

```
-rw-r--r-- 1 root root 1932 nov  8 21:45 /etc/passwd
```

Cómo se lee `-rw-r--r--`:

DIAGRAMA



En una **carpeta**, `r` permite listar su contenido, `w` crear o borrar archivos dentro y `x` entrar en ella (atravesarla). Sin `x` en una carpeta no puedes acceder a nada de lo que contiene, aunque tengas permiso sobre los archivos.

Los permisos se expresan también en **octal**: r = 4, w = 2, x = 1, y se suman por cada juego.

Octal	Simbólico	Uso típico en este manual
755	rwxr-xr-x	Carpetas públicas genéricas.
750	rwxr-x---	Carpetas web: dueño todo, grupo (www-data) entra y lee, otros nada.
700	rwx------	Carpetas privadas (~/.ssh, configuración con contraseñas).
644	rw-r--r--	Archivos de lectura pública.
640	rw-r-----	Archivos web: dueño escribe, grupo lee, otros nada.
600	rw-------	Secretos: claves privadas, archivos con contraseñas.

Comandos esenciales:

Comando	Qué hace	Ejemplo
`chmod`	Cambia permisos.	`chmod 640 index.php`
`chown`	Cambia propietario y grupo.	`sudo chown web1:www-data index.php`
`chown -R` / `chmod -R`	Lo mismo, recursivamente.	`sudo chown -R web1:www-data /var/www/ejemplo1.com`
`id`	Muestra UID, GID y grupos de un usuario.	`id web1`
`useradd` / `adduser`	Crea usuarios (bajo nivel / interactivo).	`sudo adduser sysadmin`
`usermod -aG`	Añade un usuario a un grupo.	`sudo usermod -aG www-data sysadmin`
`sudo -u`	Ejecuta un comando como otro usuario.	`sudo -u web1 ls /var/www/ejemplo1.com`

> **CONSEJO: Principio de mínimo privilegio:** cada programa y cada persona debe tener exactamente los permisos que necesita y ninguno más. Es la idea que guía todo el diseño de este manual: cada web tendrá su propio usuario Linux, su propio usuario de base de datos y su propio proceso PHP, de modo que si una web es atacada, el atacante no pueda tocar la otra.

PASO 1.8 – Instalar herramientas básicas de administración

Objetivo: disponer de las utilidades que usaremos en los diagnósticos.

Comando:

TERMINAL

```
sudo apt install -y curl wget dnsutils net-tools htop unzip ca-certificates rsyslog
```

Análisis del comando: `curl` y `wget` hacen peticiones web; `dnsutils` aporta `dig` para consultar el DNS; `net-tools` aporta comandos clásicos de red; `htop` muestra procesos y consumo; `unzip` descomprime; `ca-`

`certificates` contiene las autoridades de certificación de confianza; `rsyslog` escribe los logs clásicos en `/var/log` (`mail.log`, `auth.log`), que Fail2ban leerá más adelante (en Ubuntu 24.04 ya suele venir instalado).

Comprobación:

TERMINAL
```
dig +short ubuntu.com
ls /var/log/auth.log
```

Debe mostrar una o varias IP y existir el archivo `auth.log`.

Siguiente paso: activar un cortafuegos mínimo antes de exponer servicios.

PASO 1.9 – Activar el cortafuegos básico

Objetivo: bloquear todo el tráfico entrante salvo SSH. Iremos abriendo puertos a medida que instalemos servicios (el Módulo 12 lo trata en profundidad).

Comando:

TERMINAL
```
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status verbose
```

Análisis del comando: `ufw allow OpenSSH` permite el perfil de aplicación de SSH (puerto 22/tcp). `ufw enable` activa el cortafuegos (te avisará de que puede cortar conexiones SSH; responde `y`: ya has permitido SSH). `status verbose` muestra las reglas.

Resultado esperado:

SALIDA ESPERADA
```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
22/tcp (OpenSSH (v6))      ALLOW IN    Anywhere (v6)
```

> **ATENCIÓN:** **Nunca** ejecutes `ufw enable` sin haber permitido antes SSH: te quedarías fuera del servidor.

Siguiente paso: Módulo 2, instalación de Apache.

1.14 Ejercicios prácticos del Módulo 1

1. Consulta con `dig` los registros A, MX y TXT de un dominio conocido (`dig A google.com +short`, `dig MX google.com +short`, `dig TXT google.com +short`) y explica qué significa cada respuesta.
2. Averigua qué puertos están escuchando en tu servidor con `sudo ss -tlnp`. Identifica el proceso de cada uno.

3. Crea un usuario `prueba`, crea en su carpeta un archivo y ponle permisos `600`. Comprueba con `sudo -u sysadmin cat /home/prueba/archivo` que otro usuario no puede leerlo. Después elimina el usuario con `sudo deluser --remove-home prueba`.
4. Explica con tus palabras por qué un MX no puede apuntar a una IP y por qué el nombre del servidor debe coincidir con su PTR.
5. Calcula los permisos octales de `rwxr-x---` y de `rw-r-----`.

Módulo 2. Instalación y configuración de Apache

Objetivos del módulo

- Instalar Apache HTTP Server y dejarlo arrancando automáticamente.
- Conocer la estructura de directorios y los archivos de configuración de Apache en Ubuntu.
- Entender el directorio raíz de las webs, los propietarios y los permisos.
- Crear una página de prueba y verla desde el navegador.
- Activar y desactivar módulos, sitios y configuraciones con las herramientas `a2*`.
- Comprobar errores de configuración y leer los logs de acceso y de errores.

2.1 Qué es Apache y cómo funciona

Apache HTTP Server es un servidor web: un programa que escucha en los puertos 80 (HTTP) y 443 (HTTPS), recibe peticiones de los navegadores y responde con archivos (HTML, imágenes, CSS) o con el resultado de ejecutar un programa (por ejemplo, PHP).

Una petición HTTP tiene este aspecto (es texto plano):

```
TEXTO
GET /index.html HTTP/1.1
Host: www.ejemplo1.com
User-Agent: Mozilla/5.0 ...
```

Y la respuesta de Apache:

```
TEXTO
HTTP/1.1 200 OK
Server: Apache
Content-Type: text/html; charset=UTF-8

<html> ... contenido de la página ... </html>
```

La cabecera **Host** es la clave del Virtual Hosting (Módulo 3): gracias a ella, Apache sabe a qué web quiere ir el visitante aunque todas compartan la misma IP.

Apache atiende muchas peticiones a la vez gracias a un **MPM** (*Multi-Processing Module*). En Ubuntu 24.04 se usa por defecto **mpm_event**, que gestiona miles de conexiones con pocos procesos. Es el MPM recomendado cuando PHP se ejecuta aparte con PHP-FPM, como haremos en el Módulo 5.

Los códigos de respuesta más habituales:

Código	Significado	Causa típica
200	OK	Todo correcto.
301 / 302	Redirección permanente / temporal	Paso de HTTP a HTTPS, de `ejemplo1.com` a `www`.
403	Prohibido	Permisos de archivo o configuración `Require` que deniega.
404	No encontrado	El archivo no existe en el DocumentRoot.

Código	Significado	Causa típica
500	Error interno	Error en el código PHP o en un `.htaccess`.
502 / 503	Puerta de enlace / servicio no disponible	PHP-FPM detenido o socket incorrecto.

PASO 2.1 – Instalar Apache

Objetivo: instalar el servidor web.

Explicación: el paquete `apache2` instala el programa, los módulos básicos, las herramientas de administración y una configuración por defecto que ya sirve una página de bienvenida.

Comando:

TERMINAL

```
sudo apt update
sudo apt install -y apache2
```

Análisis del comando: `apt install` instala el paquete y todas sus dependencias (`apache2-bin`, `apache2-data`, `apache2-utils`...). `-y` acepta sin preguntar.

Resultado esperado: termina con líneas como `Setting up apache2 (2.4.58-1ubuntu8...)` y `Created symlink ... apache2.service`. Esa última indica que el servicio ya está registrado para arrancar con el sistema.

Comprobación:

TERMINAL

```
apache2 -v
```

SALIDA ESPERADA

```
Server version: Apache/2.4.58 (Ubuntu)
Server built:   2025-xx-xx...
```

Errores habituales: `Unable to locate package apache2` indica que no ejecutaste `apt update` o que no hay red.

Siguiente paso: abrir el cortafuegos y gestionar el servicio.

PASO 2.2 – Permitir el tráfico web en el cortafuegos

Objetivo: permitir conexiones a los puertos 80 y 443.

Comando:

TERMINAL

```
sudo ufw app list
sudo ufw allow "Apache Full"
sudo ufw status
```

Análisis del comando: Apache instala tres perfiles de UFW: `Apache` (solo 80), `Apache Secure` (solo 443) y `Apache Full` (80 y 443). Usamos el completo porque el puerto 80 es necesario para redirigir a HTTPS y para que Let's Encrypt valide el dominio.

Resultado esperado: en `ufw status` aparecerá la línea `Apache Full ALLOW Anywhere`, además de `OpenSSH`.

> **ATENCIÓN:** Si tu proveedor tiene además un **cortafuegos externo** (en AWS «Security Groups», en Oracle «Security Lists», en Hetzner «Firewalls»...), también debes abrir allí los puertos. UFW solo controla el cortafuegos del propio servidor.

Siguiente paso: aprender a gestionar el servicio.

2.2 Gestión del servicio con systemctl

`systemctl` es la herramienta con la que hablas con systemd. Funciona igual para todos los servicios del manual (`apache2`, `mariadb`, `postfix`, `dovecot`, `php8.3-fpm`, `opendkim`, `fail2ban`, `ssh`).

Comando	Qué hace
`sudo systemctl start apache2`	Arranca el servicio.
`sudo systemctl stop apache2`	Lo detiene.
`sudo systemctl restart apache2`	Lo detiene y lo arranca (corta las conexiones en curso).
`sudo systemctl reload apache2`	Relee la configuración sin cortar conexiones. **Úsalo tras cambiar configuración.**
`systemctl status apache2`	Muestra si está activo, desde cuándo y las últimas líneas del log.
`sudo systemctl enable apache2`	Hace que arranque automáticamente al iniciar el sistema.
`sudo systemctl disable apache2`	Quita el arranque automático.
`systemctl is-active apache2`	Responde solo `active` o `inactive` (útil en scripts).
`systemctl is-enabled apache2`	Responde `enabled` o `disabled`.
`journalctl -u apache2 -n 50`	Muestra las últimas 50 líneas del registro de systemd para ese servicio.

PASO 2.3 – Comprobar que Apache está activo y habilitado

Objetivo: verificar que Apache funciona y arrancará tras cada reinicio.

Comando:

TERMINAL

```
sudo systemctl enable --now apache2
systemctl status apache2 --no-pager
```

Análisis del comando: `enable --now` habilita el arranque automático y, a la vez, arranca el servicio si no lo estaba. `--no-pager` muestra la salida completa sin entrar en el visor paginado (si entras en él, sales con la tecla `q`).

Resultado esperado:

SALIDA ESPERADA

```
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-11-08 21:52:04 CET; 3min ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2741 (apache2)
      Tasks: 55 (limit: 2275)
     Memory: 5.3M
```

Las palabras clave son **enabled** (arranca solo) y **active (running)** (está funcionando).

Comprobación: abre en tu navegador `http://203.0.113.10` (tu IP). Debe aparecer la página **«Apache2 Default Page – It works!»**. Desde el propio servidor también puedes comprobarlo:

TERMINAL

```
curl -I http://localhost
```

SALIDA ESPERADA

```
HTTP/1.1 200 OK
Date: Sun, 08 Nov 2026 20:55:41 GMT
Server: Apache/2.4.58 (Ubuntu)
Content-Type: text/html
```

Errores habituales:

- El navegador no carga nada: revisa UFW (PASO 2.2) y el cortafuegos del proveedor.
- `Active: failed`: mira el motivo con `journalctl -u apache2 -n 30 --no-pager`. Suele ser que otro programa (por ejemplo nginx) ocupa el puerto 80; compruébalo con `sudo ss -tlnp | grep ':80'`.

Siguiente paso: conocer dónde vive la configuración.

2.3 Estructura de directorios de Apache en Ubuntu

Ubuntu (y Debian) organizan Apache de forma modular, distinta de otras distribuciones como Rocky Linux o Fedora (que usan `/etc/httpd/` y el servicio `httpd`). Todo lo que hagas en este manual corresponde a la organización de Debian/Ubuntu.

DIAGRAMA

```
/etc/apache2/
├── apache2.conf          ← archivo principal: incluye todos los demás
├── envvars               ← variables: usuario (www-data), rutas de logs
├── ports.conf            ← en qué puertos escucha (Listen 80, Listen 443)
├── conf-available/       ← fragmentos de configuración global disponibles
├── conf-enabled/         ← enlaces a los fragmentos activos  (a2enconf/a2disconf)
├── mods-available/       ← todos los módulos instalados (.load y .conf)
├── mods-enabled/         ← enlaces a los módulos activos      (a2enmod/a2dismod)
├── sites-available/      ← un archivo por cada sitio web (VirtualHost)
└── sites-enabled/        ← enlaces a los sitios activos       (a2ensite/a2dissite)

/var/www/html/            ← DocumentRoot del sitio por defecto
/var/log/apache2/         ← access.log, error.log, other_vhosts_access.log
/usr/lib/systemd/system/apache2.service  ← unidad de systemd
```

La idea clave del par **available / enabled**: en las carpetas `*-available` están todos los elementos que **podrías** usar; en `*-enabled` hay **enlaces simbólicos** a los que **estás** usando. Apache solo lee las carpetas `*-enabled`. Las herramientas `a2en*` crean esos enlaces y las `a2dis*` los borran. Así puedes activar o desactivar un sitio sin borrar su configuración.

Archivos principales

Archivo	Contenido	¿Lo modificaremos?
`/etc/apache2/apache2.conf`	Configuración global: tiempos de espera, usuario, permisos de directorios, inclusión del resto de archivos.	Casi nunca.
`/etc/apache2/ports.conf`	Directivas `Listen`.	No (ya escucha en 80 y en 443 cuando se activa SSL).
`/etc/apache2/envvars`	Define `APACHE_RUN_USER=www-data` y `APACHE_RUN_GROUP=www-data`.	No.
`/etc/apache2/conf-available/security.conf`	Cabeceras e información que Apache revela.	Sí (Módulo 12).
`/etc/apache2/sites-available/000-default.conf`	VirtualHost por defecto.	Lo usaremos como «sitio comodín».

Echa un vistazo a cómo `apache2.conf` incluye el resto:

TERMINAL

```
grep -E '^(Include|IncludeOptional)' /etc/apache2/apache2.conf
```

SALIDA ESPERADA

```
IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf
Include ports.conf
IncludeOptional conf-enabled/*.conf
IncludeOptional sites-enabled/*.conf
```

Y la sección que controla qué carpetas puede servir Apache:

TERMINAL

```
sed -n '/<Directory \/>/,/<\/Directory>/p;/<Directory \/var\/www\/>/,/<\/Directory>/p'
/etc/apache2/apache2.conf
```

SALIDA ESPERADA

```
<Directory />
        Options FollowSymLinks
        AllowOverride None
        Require all denied
</Directory>
<Directory /var/www/>
        Options Indexes FollowSymLinks
        AllowOverride None
        Require all granted
</Directory>
```

Esto significa: por defecto Apache **deniega** el acceso a todo el sistema de archivos (`/`) y solo **permite** servir lo que esté bajo `/var/www/`. Por eso todas nuestras webs vivirán en `/var/www/`.

> **CONSEJO:** La palabra `Indexes` permite que Apache muestre un listado de archivos si una carpeta no tiene `index.html`. Eso puede revelar archivos que no quieres publicar. En nuestros VirtualHosts lo desactivaremos con `Options -Indexes`.

2.4 El directorio raíz de las webs, propietarios y permisos

El **DocumentRoot** es la carpeta cuyo contenido se publica. Si el DocumentRoot es `/var/www/html`, la URL `http://servidor/fotos/a.jpg` corresponde al archivo `/var/www/html/fotos/a.jpg`.

Apache se ejecuta con el usuario **www-data**. Para servir un archivo estático, `www-data` necesita:

- Permiso `x` en **todas** las carpetas de la ruta (`/`, `/var`, `/var/www`, `/var/www/html`).
- Permiso `r` sobre el propio archivo.

Lo que **no** debe tener `www-data`: permiso de escritura sobre el código de la web. Si un atacante consiguiera ejecutar algo a través de Apache, no podría modificar tus páginas.

PASO 2.4 – Crear una página de prueba

Objetivo: publicar una página propia en el sitio por defecto y entender la relación entre archivo y URL.

Comando:

TERMINAL

```
ls -l /var/www/html/
echo '<h1>Servidor mail.ejemplo1.com funcionando</h1>' | sudo tee /var/www/html/prueba.html
sudo chmod 644 /var/www/html/prueba.html
ls -l /var/www/html/
```

Análisis del comando:

- `echo '...'` escribe el texto en la salida.
- `| sudo tee archivo` recibe ese texto y lo escribe en el archivo **con privilegios de root** (un simple `sudo echo ... > archivo` no funcionaría, porque la redirección `>` la hace tu shell sin privilegios).

- `chmod 644`: el propietario (root) lee y escribe; el resto, solo lee. `www-data` entra en «el resto», así que puede leerlo.

Resultado esperado:

SALIDA ESPERADA

```
-rw-r--r-- 1 root root 10671 nov  8 21:52 index.html
-rw-r--r-- 1 root root    49 nov  8 22:01 prueba.html
```

Comprobación: abre `http://203.0.113.10/prueba.html` o, desde el servidor:

TERMINAL

```
curl http://localhost/prueba.html
```

SALIDA ESPERADA

```
<h1>Servidor mail.ejemplo1.com funcionando</h1>
```

Ahora provoca un error 403 a propósito para entender los permisos:

TERMINAL

```
sudo chmod 600 /var/www/html/prueba.html
curl -I http://localhost/prueba.html
sudo chmod 644 /var/www/html/prueba.html
```

La respuesta intermedia será `HTTP/1.1 403 Forbidden`: `www-data` ya no podía leer el archivo. Tras restaurar `644`, vuelve a funcionar.

Errores habituales: un 404 significa que el archivo no está en la ruta que crees; comprueba mayúsculas y minúsculas (Linux las distingue: `Prueba.html` ≠ `prueba.html`).

Siguiente paso: aprender a comprobar la configuración.

2.5 Comprobación de errores de configuración: apachectl

Antes de recargar Apache tras un cambio, **comprueba siempre la sintaxis**. Si recargas con un error, Apache puede negarse a arrancar y dejar todas tus webs caídas.

Comando	Qué hace
`sudo apachectl configtest`	Comprueba la sintaxis. Debe responder `Syntax OK`.
`sudo apachectl -S`	Muestra los VirtualHosts cargados, sus puertos y archivos. Fundamental en el Módulo 3.
`sudo apachectl -M`	Lista los módulos cargados.
`sudo apachectl -t -D DUMP_INCLUDES`	Muestra qué archivos se incluyen y en qué orden.

PASO 2.5 – Comprobar la configuración y corregir el aviso de ServerName

Objetivo: obtener un `Syntax OK` limpio, sin avisos.

Comando:

TERMINAL
```
sudo apachectl configtest
```

Resultado esperado (en una instalación nueva):

SALIDA ESPERADA
```
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using
127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK
```

Explicación: no es un error, sino un aviso: Apache no tiene un nombre global configurado. Lo corregimos creando un pequeño fragmento de configuración global.

Comando:

TERMINAL
```
echo 'ServerName mail.ejemplo1.com' | sudo tee /etc/apache2/conf-available/servername.conf
sudo a2enconf servername
sudo apachectl configtest
sudo systemctl reload apache2
```

Análisis del comando: creamos `/etc/apache2/conf-available/servername.conf` con una sola directiva y lo activamos con `a2enconf` (que crea el enlace en `conf-enabled/`). Después comprobamos y recargamos.

Resultado esperado: `configtest` responde solo `Syntax OK`.

Comprobación: `ls -l /etc/apache2/conf-enabled/servername.conf` debe mostrar un enlace (`->`) a `../conf-available/servername.conf`.

Siguiente paso: gestionar módulos.

2.6 Activación y desactivación de módulos, sitios y configuraciones

Las herramientas `a2*` funcionan todas igual: crean o eliminan enlaces y te recuerdan que recargues Apache.

Herramienta	Actúa sobre	Ejemplo
`a2enmod` / `a2dismod`	Módulos (`mods-available`)	`sudo a2enmod rewrite`
`a2ensite` / `a2dissite`	Sitios (`sites-available`)	`sudo a2ensite ejemplo1.com`
`a2enconf` / `a2disconf`	Configuraciones (`conf-available`)	`sudo a2enconf servername`

Algunos módulos que usaremos:

Módulo	Para qué
`ssl`	HTTPS. Lo activará Certbot.
`rewrite`	Reescritura de URL y redirecciones avanzadas.

Módulo	Para qué
`headers`	Añadir o modificar cabeceras HTTP (seguridad).
`proxy` y `proxy_fcgi`	Pasar las peticiones PHP a PHP-FPM.
`setenvif`	Variables según la petición (lo requiere la configuración de PHP-FPM).
`http2`	Protocolo HTTP/2, más eficiente.

PASO 2.6 – Activar los módulos básicos

Objetivo: activar los módulos que necesitaremos en los módulos siguientes y practicar el ciclo activar → comprobar → recargar.

Comando:

TERMINAL
```
sudo a2enmod rewrite headers http2
sudo apachectl configtest
sudo systemctl restart apache2
sudo apachectl -M | grep -E 'rewrite|headers|http2|mpm'
```

Análisis del comando: activamos tres módulos de una vez. Tras activar **módulos** se recomienda `restart` (no basta `reload` para algunos). El último comando filtra la lista de módulos cargados.

Resultado esperado:

SALIDA ESPERADA
```
 mpm_event_module (shared)
 headers_module (shared)
 http2_module (shared)
 rewrite_module (shared)
```

Ahora practica desactivar uno y volver a activarlo:

TERMINAL
```
sudo a2dismod http2
sudo systemctl restart apache2
sudo apachectl -M | grep http2 || echo "http2 desactivado"
sudo a2enmod http2
sudo systemctl restart apache2
```

Errores habituales: `ERROR: Module xyz does not exist!` significa que el módulo no está instalado o que el nombre está mal escrito (lista los disponibles con `ls /etc/apache2/mods-available/`).

Siguiente paso: entender los logs.

2.7 Logs de acceso y de errores

Los logs son la primera herramienta de diagnóstico de un administrador. Apache escribe en `/var/log/apache2/`:

Archivo	Contenido
`access.log`	Una línea por petición atendida por el sitio por defecto.

Archivo	Contenido
`error.log`	Errores y avisos de Apache y de los módulos.
`other_vhosts_access.log`	Accesos a VirtualHosts que no definen su propio log.
`ejemplo1.com-access.log` (lo crearemos)	Accesos al sitio ejemplo1.com.
`ejemplo1.com-error.log` (lo crearemos)	Errores del sitio ejemplo1.com.

Una línea de `access.log` (formato *combined*):

SALIDA ESPERADA

```
198.51.100.7 - - [08/Nov/2026:22:10:31 +0100] "GET /prueba.html HTTP/1.1" 200 289 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
```

Campo	Valor	Significado
IP del cliente	198.51.100.7	Quién hizo la petición.
Fecha	[08/Nov/2026:22:10:31 +0100]	Cuándo.
Petición	"GET /prueba.html HTTP/1.1"	Método, ruta y protocolo.
Código	200	Resultado.
Tamaño	289	Bytes enviados.
Referer	"-"	Página de la que venía el visitante.
User-Agent	"Mozilla/5.0 ..."	Navegador o robot.

PASO 2.7 – Ver los logs en tiempo real

Objetivo: aprender a observar lo que ocurre mientras haces pruebas.

Comando:

TERMINAL

```
sudo tail -f /var/log/apache2/access.log /var/log/apache2/error.log
```

Análisis del comando: `tail` muestra el final de un archivo; `-f` (*follow*) se queda esperando y muestra cada línea nueva al instante. Con dos archivos, los muestra con un encabezado `==> archivo <==`.

Comprobación: con el comando en marcha, recarga `http://TU_IP/prueba.html` y luego visita `http://TU_IP/no-existe`. Verás aparecer una línea con `200` y otra con `404`. Pulsa **Ctrl+C** para salir.

Otros comandos útiles con logs:

TERMINAL

```
# Las 20 últimas líneas de errores
sudo tail -n 20 /var/log/apache2/error.log
# Cuántas peticiones ha hecho cada IP (top 10)
sudo awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head
# Todas las peticiones que dieron error 404
sudo grep '" 404 ' /var/log/apache2/access.log
```

> **CONSEJO:** Los logs rotan automáticamente cada día gracias a **logrotate** (`/etc/logrotate.d/apache2`): el de ayer pasa a llamarse `access.log.1`, los anteriores se comprimen (`access.log.2.gz`) y se conservan 14 días. Para leer uno comprimido usa `zcat` o `zgrep`.

Siguiente paso: Módulo 3, alojar varias webs con VirtualHosts.

2.8 Ejercicios prácticos del Módulo 2

1. Detén Apache con `systemctl stop`, comprueba desde el navegador que la web no responde, consulta `systemctl status` y vuelve a arrancarlo.
2. Introduce a propósito una errata en `/etc/apache2/conf-available/servername.conf` (por ejemplo `ServerNam`), ejecuta `apachectl configtest`, lee el mensaje de error y corrígelo. ¿Qué habría pasado si hubieras hecho `restart` sin comprobar?
3. Crea una carpeta `/var/www/html/sinindice` con un archivo `a.txt` y visítala. ¿Qué ves? ¿Qué directiva lo provoca?
4. Usa `awk` sobre `access.log` para averiguar qué rutas se piden más a menudo (pista: el campo 7).
5. Busca en `error.log` líneas con la palabra `AH` y averigua qué significan sus códigos en la documentación oficial de Apache.

Módulo 3. Configuración de Virtual Hosting

Objetivos del módulo

- Entender qué es un VirtualHost y cómo un solo servidor aloja muchos dominios.
- Crear para cada dominio un usuario Linux, una carpeta y un archivo de configuración independientes.
- Configurar `ServerName`, `ServerAlias`, `DocumentRoot`, permisos y logs propios.
- Habilitar los sitios, probarlos antes y después de configurar el DNS y resolver conflictos.
- Proteger cada dominio con HTTPS mediante un certificado de Let's Encrypt.

3.1 Qué es un VirtualHost

Un **VirtualHost** (anfitrión virtual) es un bloque de configuración que le dice a Apache: «cuando llegue una petición para **este** nombre, sírvela desde **esta** carpeta, con **estas** opciones y guarda los logs **aquí**».

Apache admite dos tipos:

- **Por IP**: cada web tiene una IP distinta. Hoy apenas se usa porque las IPv4 son escasas.
- **Por nombre** (*name-based*): todas las webs comparten IP y Apache las distingue por la cabecera `Host` de la petición. Es lo que usaremos.

Cómo elige Apache el VirtualHost:

DIAGRAMA



En HTTPS existe un detalle adicional: la cabecera `Host` va cifrada, así que Apache no puede leerla hasta después de elegir el certificado. Para resolverlo, el navegador envía el nombre en claro al iniciar la conexión TLS mediante una extensión llamada **SNI** (*Server Name Indication*). Todos los navegadores actuales la usan.

> **ATENCIÓN:** El punto 3 es la fuente de confusión número uno: si un VirtualHost tiene un error en `ServerName` o el visitante llega con un nombre desconocido (o directamente con la IP), verá **el primer VirtualHost** cargado. Por eso dejaremos el sitio por defecto (`000-default`) como «comodín» neutro en primer lugar, para que nadie vea la web de ejemplo1 por accidente al visitar ejemplo2 mal configurado.

3.2 Diseño de carpetas y usuarios

Cada web tendrá su propio usuario de sistema. PHP-FPM (Módulo 5) ejecutará el código de cada web con ese usuario, de modo que una web no pueda leer los archivos de la otra.

DIAGRAMA

```
/var/www/
├── html/                          ← sitio comodín (000-default), propietario root
├── ejemplo1.com/                  ← propietario web1, grupo www-data, 750
│   ├── public_html/               ← DocumentRoot  (web1:www-data, 2750)
│   │   ├── index.html / index.php
│   │   └── .well-known/acme-challenge/   ← validación de Let's Encrypt (temporal)
│   ├── config/                    ← credenciales, FUERA del DocumentRoot (web1:web1, 700)
│   └── tmp/                       ← temporales y sesiones de PHP (web1:web1, 700)
└── ejemplo2.com/                  ← misma estructura con el usuario web2
```

¿Por qué `config/` está **fuera** de `public_html/`? Porque todo lo que está dentro de `public_html` puede pedirse por URL. Si un fallo de configuración hiciera que Apache sirviera el PHP como texto, las contraseñas de la base de datos quedarían a la vista. Fuera del DocumentRoot, eso es imposible.

PASO 3.1 – Crear los usuarios de sistema de cada web

Objetivo: crear `web1` y `web2`, usuarios sin contraseña ni acceso por terminal, que serán dueños de los archivos de cada web.

Comando:

TERMINAL

```
sudo useradd --system --user-group --home-dir /var/www/ejemplo1.com --shell /usr/sbin/nologin
web1
sudo useradd --system --user-group --home-dir /var/www/ejemplo2.com --shell /usr/sbin/nologin
web2
```

Análisis del comando:

- `useradd`: herramienta de bajo nivel para crear usuarios (no hace preguntas, ideal para cuentas de servicio).
- `--system`: cuenta de sistema (UID bajo, sin caducidad de contraseña, no aparece en la pantalla de inicio).
- `--user-group`: crea también un grupo con el mismo nombre.
- `--home-dir`: carpeta personal del usuario (la crearemos en el paso siguiente).
- `--shell /usr/sbin/nologin`: impide iniciar sesión con este usuario. Si alguien lo intentara, recibiría «This account is currently not available».

Comprobación:

TERMINAL

```
id web1
id web2
```

SALIDA ESPERADA

```
uid=998(web1) gid=998(web1) groups=998(web1)
uid=997(web2) gid=997(web2) groups=997(web2)
```

(Los números pueden ser distintos.)

Errores habituales: `useradd: user 'web1' already exists`: ya lo habías creado; continúa.

Siguiente paso: crear la estructura de carpetas.

PASO 3.2 – Crear las carpetas y asignar propietarios y permisos

Objetivo: crear la estructura de cada web con los permisos correctos.

Comando (para ejemplo1.com):

TERMINAL

```
sudo mkdir -p /var/www/ejemplo1.com/{public_html,config,tmp}
sudo chown web1:www-data /var/www/ejemplo1.com /var/www/ejemplo1.com/public_html
sudo chown web1:web1 /var/www/ejemplo1.com/config /var/www/ejemplo1.com/tmp
sudo chmod 750 /var/www/ejemplo1.com
sudo chmod 2750 /var/www/ejemplo1.com/public_html
sudo chmod 700 /var/www/ejemplo1.com/config /var/www/ejemplo1.com/tmp
```

Repite lo mismo para ejemplo2.com con el usuario `web2`:

TERMINAL

```
sudo mkdir -p /var/www/ejemplo2.com/{public_html,config,tmp}
sudo chown web2:www-data /var/www/ejemplo2.com /var/www/ejemplo2.com/public_html
sudo chown web2:web2 /var/www/ejemplo2.com/config /var/www/ejemplo2.com/tmp
sudo chmod 750 /var/www/ejemplo2.com
sudo chmod 2750 /var/www/ejemplo2.com/public_html
sudo chmod 700 /var/www/ejemplo2.com/config /var/www/ejemplo2.com/tmp
```

Análisis del comando:

- `mkdir -p`: crea carpetas, incluidas las intermedias, sin error si ya existen. Las llaves `{a,b,c}` son una abreviatura del shell que genera tres rutas.
- `chown web1:www-data`: el dueño es `web1` (podrá escribir) y el grupo `www-data` (Apache podrá entrar y leer).
- `chmod 750` en la carpeta del sitio: dueño todo, grupo (Apache) entrar y listar, el resto nada. Así `web2` no puede entrar en la carpeta de `web1`.
- `chmod 2750` en `public_html`: el `2` inicial activa el bit **setgid**: todo archivo nuevo creado dentro heredará el grupo `www-data` automáticamente, en vez del grupo de quien lo cree.
- `chmod 700` en `config` y `tmp`: solo `web1` puede entrar. Apache no los necesita.

Comprobación:

TERMINAL

```
sudo ls -la /var/www/ejemplo1.com/
```

SALIDA ESPERADA

```
drwxr-x---  5 web1 www-data 4096 nov  8 22:20 .
drwxr-xr-x  5 root root     4096 nov  8 22:20 ..
drwx------  2 web1 web1     4096 nov  8 22:20 config
drwxr-s---  2 web1 www-data 4096 nov  8 22:20 public_html
drwx------  2 web1 web1     4096 nov  8 22:20 tmp
```

La `s` en `drwxr-s---` es el bit setgid.

Siguiente paso: crear una página de bienvenida distinta para cada dominio.

PASO 3.3 – Crear una página de prueba en cada sitio

Objetivo: tener contenido distinto en cada web para comprobar que cada dominio carga la suya.

Comando: creamos el archivo con el usuario `web1` (para que sea su dueño) usando `sudo -u`:

TERMINAL

```
sudo -u web1 tee /var/www/ejemplo1.com/public_html/index.html > /dev/null <<'EOF'
<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>ejemplo1.com</title></head>
<body style="font-family: sans-serif; background: #e8f1fb;">
  <h1>Bienvenido a ejemplo1.com</h1>
  <p>Este contenido se sirve desde /var/www/ejemplo1.com/public_html</p>
</body>
</html>
EOF
```

TERMINAL

```
sudo -u web2 tee /var/www/ejemplo2.com/public_html/index.html > /dev/null <<'EOF'
<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>ejemplo2.com</title></head>
<body style="font-family: sans-serif; background: #eaf5ea;">
  <h1>Bienvenido a ejemplo2.com</h1>
  <p>Este contenido se sirve desde /var/www/ejemplo2.com/public_html</p>
</body>
</html>
EOF
```

Análisis del comando:

- `sudo -u web1 tee archivo`: escribe el archivo como el usuario `web1`.
- `<<'EOF' ... EOF`: un *here-document*: todo lo que hay entre las dos marcas se envía como entrada al comando. Las comillas en `'EOF'` impiden que el shell interprete caracteres especiales como `$`.
- `> /dev/null`: descarta la copia que `tee` muestra por pantalla.

Ajustamos permisos de los archivos (640: dueño lee y escribe, grupo www-data lee):

TERMINAL

```
sudo chmod 640 /var/www/ejemplo1.com/public_html/index.html
/var/www/ejemplo2.com/public_html/index.html
sudo ls -l /var/www/ejemplo1.com/public_html/
```

SALIDA ESPERADA

```
-rw-r----- 1 web1 www-data 287 nov  8 22:25 index.html
```

Fíjate en que el grupo es `www-data` gracias al bit setgid.

Siguiente paso: preparar el sitio comodín.

PASO 3.4 – Preparar el sitio por defecto como comodín

Objetivo: que quien visite el servidor por IP o con un nombre desconocido vea una página neutra.

Comando:

TERMINAL

```
echo '<!DOCTYPE html><html><head><meta charset="utf-
8"><title>Servidor</title></head><body><p>Servidor en funcionamiento.</p></body></html>' |
sudo tee /var/www/html/index.html > /dev/null
```

Archivo: `/etc/apache2/sites-available/000-default.conf`. Sustituye todo su contenido por:

ARCHIVO DE CONFIGURACIÓN

```
# Sitio comodín: atiende peticiones por IP o con nombres no configurados.
# Debe ser el PRIMER VirtualHost en *:80 (por eso se llama 000-...).
# También sirve los desafíos de Let's Encrypt para mail.ejemplo1.com y mail.ejemplo2.com.
<VirtualHost *:80>
    ServerName mail.ejemplo1.com
    ServerAlias mail.ejemplo2.com
    ServerAdmin administracion@ejemplo1.com
    DocumentRoot /var/www/html

    <Directory /var/www/html>
        Options -Indexes
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Análisis: `${APACHE_LOG_DIR}` es una variable definida en `/etc/apache2/envvars` que vale `/var/log/apache2`. `ServerName mail.ejemplo1.com` y su alias permitirán, en el Módulo 6, obtener el certificado del servidor de correo usando esta misma carpeta.

TERMINAL

```
sudo apachectl configtest && sudo systemctl reload apache2
```

El operador `&&` ejecuta la recarga **solo si** la comprobación termina bien. Acostúmbrate a esta combinación.

Siguiente paso: crear el VirtualHost de cada dominio.

3.3 Los archivos de configuración de los VirtualHosts

PASO 3.5 – Crear el VirtualHost de ejemplo1.com (HTTP)

Objetivo: decirle a Apache que `www.ejemplo1.com` y `ejemplo1.com` se sirven desde `/var/www/ejemplo1.com/public_html`.

Explicación: de momento solo configuramos HTTP (puerto 80). Lo necesitamos así para obtener el certificado; en el PASO 3.11 ampliaremos el archivo con HTTPS.

Comando:

TERMINAL

```
sudo nano /etc/apache2/sites-available/ejemplo1.com.conf
```

Archivo: `/etc/apache2/sites-available/ejemplo1.com.conf` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
<VirtualHost *:80>
    # Nombre principal del sitio
    ServerName www.ejemplo1.com
    # Otros nombres que deben servir este mismo sitio
    ServerAlias ejemplo1.com
    # Correo del responsable (aparece en algunas páginas de error)
    ServerAdmin administracion@ejemplo1.com

    # Carpeta que se publica
    DocumentRoot /var/www/ejemplo1.com/public_html

    <Directory /var/www/ejemplo1.com/public_html>
        # Sin listados de carpetas; se permiten enlaces simbólicos
        Options -Indexes +FollowSymLinks
        # No se procesan archivos .htaccess (más rápido y más seguro)
        AllowOverride None
        # Cualquiera puede ver el contenido
        Require all granted
    </Directory>

    # Logs propios de este sitio
    ErrorLog ${APACHE_LOG_DIR}/ejemplo1.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo1.com-access.log combined
</VirtualHost>
```

Análisis de cada directiva:

Directiva	Significado
`<VirtualHost *:80>`	Este bloque atiende peticiones en cualquier IP (*) del servidor, puerto 80.
`ServerName`	Nombre principal. Debe ser único entre todos los VirtualHosts del mismo puerto.
`ServerAlias`	Nombres adicionales (varios separados por espacio; admite comodines como `*.ejemplo1.com`).
`ServerAdmin`	Dirección de contacto.
`DocumentRoot`	Carpeta publicada. Sin barra final.

Directiva	Significado
`<Directory ruta>`	Opciones que se aplican a esa carpeta y sus subcarpetas.
`Options -Indexes`	Desactiva el listado automático de carpetas.
`AllowOverride None`	Ignora los archivos `.htaccess`. Toda la configuración está en este archivo.
`Require all granted`	Concede acceso a todo el mundo.
`ErrorLog` / `CustomLog`	Rutas de los logs de errores y de accesos; `combined` es el formato.

> **CONSEJO:** Muchas aplicaciones (WordPress, Laravel...) traen un `.htaccess` con reglas de reescritura. Si las usas, cambia `AllowOverride None` por `AllowOverride All` (o mejor, copia esas reglas dentro de este bloque `<Directory>`).

Siguiente paso: el mismo archivo para ejemplo2.com.

PASO 3.6 – Crear el VirtualHost de ejemplo2.com (HTTP)

Archivo: `/etc/apache2/sites-available/ejemplo2.com.conf` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
<VirtualHost *:80>
    ServerName www.ejemplo2.com
    ServerAlias ejemplo2.com
    ServerAdmin soporte@ejemplo2.com

    DocumentRoot /var/www/ejemplo2.com/public_html

    <Directory /var/www/ejemplo2.com/public_html>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/ejemplo2.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo2.com-access.log combined
</VirtualHost>
```

Siguiente paso: habilitar ambos sitios.

PASO 3.7 – Habilitar los sitios

Objetivo: activar los dos VirtualHosts.

Comando:

TERMINAL

```
sudo a2ensite ejemplo1.com ejemplo2.com
sudo apachectl configtest
sudo systemctl reload apache2
```

Análisis del comando: `a2ensite` admite el nombre del archivo sin `.conf`. Crea `/etc/apache2/sites-enabled/ejemplo1.com.conf` como enlace al de `sites-available`.

Resultado esperado:

SALIDA ESPERADA

```
Enabling site ejemplo1.com.
Enabling site ejemplo2.com.
To activate the new configuration, you need to run:
  systemctl reload apache2
Syntax OK
```

Comprobación: la herramienta más importante de este módulo:

TERMINAL

```
sudo apachectl -S
```

SALIDA ESPERADA

```
VirtualHost configuration:
*:80                   is a NameVirtualHost
         default server mail.ejemplo1.com (/etc/apache2/sites-enabled/000-default.conf:4)
         port 80 namevhost mail.ejemplo1.com (/etc/apache2/sites-enabled/000-default.conf:4)
                 alias mail.ejemplo2.com
         port 80 namevhost www.ejemplo1.com (/etc/apache2/sites-enabled/ejemplo1.com.conf:1)
                 alias ejemplo1.com
         port 80 namevhost www.ejemplo2.com (/etc/apache2/sites-enabled/ejemplo2.com.conf:1)
                 alias ejemplo2.com
ServerRoot: "/etc/apache2"
Main DocumentRoot: "/var/www/html"
...
```

Interpretación: en el puerto 80 hay tres VirtualHosts por nombre; el **default server** (el que atiende nombres desconocidos) es el comodín de `000-default.conf`. Cada línea indica el archivo y la línea donde está definido: muy útil cuando algo no cuadra.

Siguiente paso: probar los dominios.

3.4 Probar los dominios

Hay tres formas de probar, de la más rápida a la definitiva.

PASO 3.8 – Probar desde el propio servidor con curl (sin DNS)

Objetivo: verificar que Apache elige el VirtualHost correcto aunque el DNS aún no esté configurado.

Comando:

TERMINAL

```
curl -s -H "Host: www.ejemplo1.com" http://127.0.0.1/ | grep '<h1>'
curl -s -H "Host: ejemplo2.com" http://127.0.0.1/ | grep '<h1>'
curl -s -H "Host: desconocido.com" http://127.0.0.1/
```

Análisis del comando: `-H "Host: ..."` envía la cabecera Host que queramos, simulando que el visitante escribió ese nombre. `-s` (silencioso) oculta la barra de progreso.

Resultado esperado:

SALIDA ESPERADA

```
  <h1>Bienvenido a ejemplo1.com</h1>
  <h1>Bienvenido a ejemplo2.com</h1>
<!DOCTYPE html><html><head><meta charset="utf-
8"><title>Servidor</title></head><body><p>Servidor en funcionamiento.</p></body></html>
```

El tercer caso demuestra que los nombres desconocidos caen en el comodín.

PASO 3.9 – Probar desde tu navegador con el archivo hosts (sin DNS)

Objetivo: ver las webs en tu navegador antes de tocar el DNS (o en un laboratorio sin dominio real).

Explicación: el archivo `hosts` de tu ordenador tiene prioridad sobre el DNS. Añadiendo líneas, tu equipo creerá que esos nombres apuntan a tu servidor.

- En **Windows**: abre el Bloc de notas **como administrador** y edita `C:\Windows\System32\drivers\etc\hosts`.
- En **macOS / Linux**: `sudo nano /etc/hosts`.

Archivo: `hosts` de **tu ordenador** (añade al final):

ARCHIVO DE CONFIGURACIÓN

```
203.0.113.10   ejemplo1.com www.ejemplo1.com mail.ejemplo1.com
203.0.113.10   ejemplo2.com www.ejemplo2.com mail.ejemplo2.com
```

Comprobación: visita `http://www.ejemplo1.com` y `http://www.ejemplo2.com`. Cada uno debe mostrar su página con su color de fondo.

> **ATENCIÓN:** Cuando configures el DNS real, **borra estas líneas**. Si no, tu equipo seguirá usándolas y podrías no darte cuenta de que el DNS está mal.

PASO 3.10 – Crear los registros DNS de las webs

Objetivo: que cualquier persona del mundo llegue a tus webs.

Explicación: entra en el panel de tu proveedor DNS y crea, para **cada** dominio, los registros siguientes. Creamos también ahora los nombres `mail` porque los necesitaremos para el certificado del correo (el resto de registros de correo se crean en el Módulo 9).

Registros de `ejemplo1.com`:

Tipo	Nombre	Valor	TTL
A	@	203.0.113.10	3600
A	www	203.0.113.10	3600
A	mail	203.0.113.10	3600
AAAA (si tienes IPv6)	@, www y mail	2001:db8:10::10	3600

Registros de `ejemplo2.com`:

Tipo	Nombre	Valor	TTL
A	@	203.0.113.10	3600
A	www	203.0.113.10	3600
A	mail	203.0.113.10	3600
AAAA (si tienes IPv6)	@, www y mail	2001:db8:10::10	3600

> **ATENCIÓN:** Crea registros **AAAA** solo si tu servidor tiene IPv6 configurada **y funcionando**. Un AAAA que apunta a una IPv6 que no responde hace que muchos visitantes (y Let's Encrypt) fallen, porque prefieren IPv6. Comprueba la IPv6 del servidor con `ip -6 addr show scope global`.

Comprobación (desde el servidor o desde tu ordenador):

TERMINAL

```
dig +short A www.ejemplo1.com
dig +short A ejemplo2.com
dig +short A mail.ejemplo2.com @1.1.1.1
```

Cada consulta debe devolver `203.0.113.10`. La opción `@1.1.1.1` pregunta directamente al resolvedor de Cloudflare, útil para saltarte cachés locales.

Errores habituales: si no devuelve nada, espera unos minutos (propagación) y revisa que no hayas escrito el dominio completo en el campo «Nombre» (algunos paneles añaden el dominio automáticamente y acabarías creando `www.ejemplo1.com.ejemplo1.com`).

3.5 Solucionar conflictos entre VirtualHosts

Síntoma	Causa probable	Solución
Todos los dominios muestran la misma web.	El navegador cae en el vhost por defecto porque ningún `ServerName`/`ServerAlias` coincide, o los sitios no están habilitados.	`sudo apachectl -S`: comprueba que cada nombre aparece como `namevhost` o `alias`.
`ejemplo2.com` muestra la web de ejemplo1.	Falta `ServerAlias ejemplo2.com` o está en el archivo equivocado.	Corrige el alias y recarga.
Aviso `AH01630` o `AH00548: NameVirtualHost has no effect`.	Directiva obsoleta de Apache 2.2.	Bórrala: en 2.4 no hace falta.
Dos sitios con el mismo `ServerName`.	Gana el primero que se carga (orden alfabético de archivos).	Cada `ServerName` debe ser único; revisa `apachectl -S`.
403 Forbidden en un dominio.	Permisos de carpeta (falta `x` para www-data) o falta el bloque `<Directory>`.	`sudo -u www-data ls /var/www/ejemplo1.com/public_html` debe funcionar.

Síntoma	Causa probable	Solución
El cambio no se aplica.	No recargaste Apache o editaste el archivo en `sites-available` de otro sitio.	`sudo systemctl reload apache2` y revisa el archivo indicado en `apachectl -S`.
`Could not open configuration file ... No such file or directory`.	Un enlace en `sites-enabled` apunta a un archivo borrado.	`sudo a2dissite nombre` o borra el enlace roto.

Una herramienta muy útil para comprobar permisos de toda una ruta de golpe:

TERMINAL

```
namei -l /var/www/ejemplo1.com/public_html/index.html
```

SALIDA ESPERADA

```
f: /var/www/ejemplo1.com/public_html/index.html
drwxr-xr-x root root     /
drwxr-xr-x root root     var
drwxr-xr-x root root     www
drwxr-x--- web1 www-data ejemplo1.com
drwxr-s--- web1 www-data public_html
-rw-r----- web1 www-data index.html
```

3.6 HTTPS para cada dominio

Hasta ahora todo viaja **sin cifrar**: cualquiera en la red podría leer o modificar lo que se envía. HTTPS cifra la conexión con **TLS** y además demuestra, mediante un **certificado**, que el servidor es realmente quien dice ser.

Usaremos **Let's Encrypt**, una autoridad de certificación gratuita y automatizada, a través de su cliente oficial **Certbot**. El funcionamiento interno y la renovación se explican a fondo en el Módulo 10; aquí lo usamos para dejar las webs seguras desde ya.

Cómo demuestra Certbot que el dominio es tuyo (desafío **HTTP-01**):

DIAGRAMA

```
1. Certbot pide un certificado para ejemplo1.com y www.ejemplo1.com
2. Let's Encrypt responde: "publica este token en
       http://ejemplo1.com/.well-known/acme-challenge/<token>"
3. Certbot escribe el archivo en /var/www/ejemplo1.com/public_html/.well-known/acme-challenge/
4. Los servidores de Let's Encrypt descargan esa URL desde Internet (puerto 80)
5. Si coincide → emiten el certificado (válido 90 días). Certbot borra el token.
```

Por eso es imprescindible que el DNS apunte ya a tu servidor y que el puerto 80 esté abierto.

CONSEJO: Usaremos el modo `certonly --webroot`: Certbot solo obtiene el certificado y **no toca** tus archivos de Apache. Así controlas al cien por cien tu configuración y la entiendes línea por línea. (El modo `certbot --apache` modifica tus VirtualHosts automáticamente; es cómodo, pero genera archivos `-le-ssl.conf` duplicados que confunden cuando más adelante añades PHP.)

PASO 3.11 – Instalar Certbot y obtener los certificados de las webs

Objetivo: obtener un certificado para cada dominio.

Comando:

TERMINAL

```
sudo apt install -y certbot
sudo certbot certonly --webroot -w /var/www/ejemplo1.com/public_html -d ejemplo1.com -d
www.ejemplo1.com --email administracion@ejemplo1.com --agree-tos --no-eff-email
```

Análisis del comando:

- `certonly`: solo obtener el certificado, sin instalarlo.
- `--webroot -w carpeta`: método de validación: escribir el token en esa carpeta (el DocumentRoot del sitio).
- `-d ejemplo1.com -d www.ejemplo1.com`: los nombres que cubrirá el certificado. El primero da nombre a la carpeta del certificado.
- `--email`: dirección a la que Let's Encrypt avisará si algo va mal con la renovación.
- `--agree-tos`: aceptas los términos del servicio de Let's Encrypt (léelos en su web).
- `--no-eff-email`: no compartir tu correo con la EFF para boletines.

Resultado esperado:

SALIDA ESPERADA

```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/ejemplo1.com/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/ejemplo1.com/privkey.pem
This certificate expires on 2027-02-06.
Certbot has set up a scheduled task to automatically renew this certificate in the background.
```

Repite para el segundo dominio:

TERMINAL

```
sudo certbot certonly --webroot -w /var/www/ejemplo2.com/public_html -d ejemplo2.com -d
www.ejemplo2.com --email soporte@ejemplo2.com --agree-tos --no-eff-email
```

Comprobación:

TERMINAL

```
sudo certbot certificates
```

Debe listar los dos certificados con sus dominios y la fecha de caducidad.

Errores habituales:

- `Timeout during connect (likely firewall problem)`: el puerto 80 está cerrado en UFW o en el cortafuegos del proveedor.
- `DNS problem: NXDOMAIN looking up A for www.ejemplo1.com`: el registro DNS aún no existe o no se ha propagado.
- `Invalid response from http://.../.well-known/acme-challenge/...: 404`: el DNS apunta a otro servidor, o el `-w` no coincide con el DocumentRoot del VirtualHost que atiende ese nombre.

- `too many certificates already issued`: has superado los límites de Let's Encrypt por hacer muchas pruebas. Para practicar, añade `--test-cert` (entorno de pruebas sin límites) y repite sin él cuando todo funcione.

> **CONSEJO: Laboratorio sin dominio real:** omite este paso y, en el archivo del PASO 3.12, sustituye las rutas del certificado por las del certificado autofirmado de Ubuntu: `SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem` y `SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key`. El navegador mostrará un aviso de seguridad que podrás aceptar.

Siguiente paso: configurar el VirtualHost HTTPS.

PASO 3.12 – Configurar HTTPS en el VirtualHost de ejemplo1.com

Objetivo: servir la web por HTTPS y redirigir automáticamente todo el tráfico HTTP a `https://www.ejemplo1.com`.

Explicación: ampliamos el archivo con un segundo bloque `<VirtualHost *:443>`. El bloque del puerto 80 pasa a tener una única función: redirigir a HTTPS (excepto las peticiones de validación de Let's Encrypt). También redirigimos `ejemplo1.com` a `www.ejemplo1.com` para que la web tenga una única dirección canónica.

Comando: activa los módulos necesarios:

TERMINAL

```
sudo a2enmod ssl rewrite headers
```

Archivo: `/etc/apache2/sites-available/ejemplo1.com.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
# ------------------------------------------------------------
# ejemplo1.com - HTTP: solo redirige a HTTPS
# ------------------------------------------------------------
<VirtualHost *:80>
    ServerName www.ejemplo1.com
    ServerAlias ejemplo1.com
    ServerAdmin administracion@ejemplo1.com
    DocumentRoot /var/www/ejemplo1.com/public_html

    RewriteEngine On
    # Las validaciones de Let's Encrypt se sirven por HTTP sin redirigir
    RewriteCond %{REQUEST_URI} !^/\.well-known/acme-challenge/
    RewriteRule ^ https://www.ejemplo1.com%{REQUEST_URI} [L,R=301]

    ErrorLog ${APACHE_LOG_DIR}/ejemplo1.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo1.com-access.log combined
</VirtualHost>

# ------------------------------------------------------------
# ejemplo1.com - HTTPS: el sitio real
# ------------------------------------------------------------
<VirtualHost *:443>
    ServerName www.ejemplo1.com
    ServerAlias ejemplo1.com
    ServerAdmin administracion@ejemplo1.com
    DocumentRoot /var/www/ejemplo1.com/public_html

    # Cifrado TLS con el certificado de Let's Encrypt
    SSLEngine on
```

```
    SSLCertificateFile    /etc/letsencrypt/live/ejemplo1.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/ejemplo1.com/privkey.pem
    Protocols h2 http/1.1

    # Dirección canónica: ejemplo1.com -> www.ejemplo1.com
    RewriteEngine On
    RewriteCond %{HTTP_HOST} !^www\.ejemplo1\.com$ [NC]
    RewriteRule ^ https://www.ejemplo1.com%{REQUEST_URI} [L,R=301]

    # Indica a los navegadores que usen siempre HTTPS durante 6 meses
    Header always set Strict-Transport-Security "max-age=15552000"

    <Directory /var/www/ejemplo1.com/public_html>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/ejemplo1.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo1.com-access.log combined
</VirtualHost>
```

Análisis de las directivas nuevas:

Directiva	Significado
`RewriteEngine On`	Activa el motor de reescritura en este VirtualHost.
`RewriteCond %{REQUEST_URI} !^/\.well-known/acme-challenge/`	Condición: la ruta pedida **no** (`!`) empieza por la carpeta de validación.
`RewriteRule ^ https://www.ejemplo1.com%{REQUEST_URI} [L,R=301]`	Para cualquier ruta (`^`), redirige a la misma ruta en HTTPS. `R=301`: redirección permanente; `L`: última regla.
`SSLEngine on`	Activa TLS en este VirtualHost.
`SSLCertificateFile`	Certificado más la cadena de certificados intermedios (`fullchain.pem`).
`SSLCertificateKeyFile`	Clave privada (solo root puede leerla).
`Protocols h2 http/1.1`	Ofrece HTTP/2 y, si el cliente no lo soporta, HTTP/1.1.
`RewriteCond %{HTTP_HOST} !^www\.ejemplo1\.com$ [NC]`	Si el nombre pedido no es exactamente `www.ejemplo1.com` (sin distinguir mayúsculas, `NC`)...
`Header always set Strict-Transport-Security`	Cabecera HSTS: el navegador recordará usar solo HTTPS. Actívala cuando estés seguro de que HTTPS funciona bien.

ATENCIÓN: Los protocolos y cifrados TLS globales están definidos en `/etc/apache2/mods-available/ssl.conf` (Ubuntu 24.04 ya desactiva SSLv3, TLS 1.0 y 1.1 con `SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1`). Compruébalo con `grep -E '^\s*SSLProtocol' /etc/apache2/mods-available/ssl.conf`.

Siguiente paso: lo mismo para ejemplo2.com.

PASO 3.13 – Configurar HTTPS en el VirtualHost de ejemplo2.com

Archivo: `/etc/apache2/sites-available/ejemplo2.com.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
<VirtualHost *:80>
    ServerName www.ejemplo2.com
    ServerAlias ejemplo2.com
    ServerAdmin soporte@ejemplo2.com
    DocumentRoot /var/www/ejemplo2.com/public_html

    RewriteEngine On
    RewriteCond %{REQUEST_URI} !^/\.well-known/acme-challenge/
    RewriteRule ^ https://www.ejemplo2.com%{REQUEST_URI} [L,R=301]

    ErrorLog ${APACHE_LOG_DIR}/ejemplo2.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo2.com-access.log combined
</VirtualHost>

<VirtualHost *:443>
    ServerName www.ejemplo2.com
    ServerAlias ejemplo2.com
    ServerAdmin soporte@ejemplo2.com
    DocumentRoot /var/www/ejemplo2.com/public_html

    SSLEngine on
    SSLCertificateFile    /etc/letsencrypt/live/ejemplo2.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/ejemplo2.com/privkey.pem
    Protocols h2 http/1.1

    RewriteEngine On
    RewriteCond %{HTTP_HOST} !^www\.ejemplo2\.com$ [NC]
    RewriteRule ^ https://www.ejemplo2.com%{REQUEST_URI} [L,R=301]

    Header always set Strict-Transport-Security "max-age=15552000"

    <Directory /var/www/ejemplo2.com/public_html>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/ejemplo2.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo2.com-access.log combined
</VirtualHost>
```

PASO 3.14 – Activar el comodín HTTPS y aplicar los cambios

Objetivo: que las peticiones HTTPS con nombres desconocidos no caigan en ejemplo1.com.

Explicación: Ubuntu incluye el sitio `default-ssl`, que usa el certificado autofirmado. Como su nombre empieza por «d», se carga antes que «ejemplo1.com» y será el comodín del puerto 443.

Comando:

TERMINAL

```
sudo a2ensite default-ssl
sudo apachectl configtest
sudo systemctl restart apache2
sudo apachectl -S
```

Resultado esperado (fragmento):

SALIDA ESPERADA

```
*:443                  is a NameVirtualHost
         default server mail.ejemplo1.com (/etc/apache2/sites-enabled/default-ssl.conf:2)
         port 443 namevhost mail.ejemplo1.com (/etc/apache2/sites-enabled/default-ssl.conf:2)
         port 443 namevhost www.ejemplo1.com (/etc/apache2/sites-enabled/ejemplo1.com.conf:22)
                 alias ejemplo1.com
         port 443 namevhost www.ejemplo2.com (/etc/apache2/sites-enabled/ejemplo2.com.conf:16)
                 alias ejemplo2.com
*:80                   is a NameVirtualHost
...
```

Comprobación:

TERMINAL

```
curl -I http://ejemplo1.com
curl -I https://ejemplo1.com
curl -I https://www.ejemplo1.com
curl -sI https://www.ejemplo2.com | head -1
```

Resultado esperado:

SALIDA ESPERADA

```
HTTP/1.1 301 Moved Permanently
Location: https://www.ejemplo1.com/
...
HTTP/2 301
location: https://www.ejemplo1.com/
...
HTTP/2 200
strict-transport-security: max-age=15552000
...
HTTP/2 200
```

En el navegador, `http://ejemplo1.com` debe acabar en `https://www.ejemplo1.com` con el candado cerrado.

Errores habituales:

- `SSLCertificateFile: file '/etc/letsencrypt/live/...' does not exist or is empty`: la ruta no coincide con la que dio Certbot (consulta `sudo certbot certificates`).
- `Invalid command 'RewriteEngine'` o `Invalid command 'Header'`: falta `a2enmod rewrite headers`.
- `Invalid command 'SSLEngine'`: falta `a2enmod ssl`.

- Bucle de redirecciones (`ERR_TOO_MANY_REDIRECTS`): tienes un proxy o CDN (por ejemplo Cloudflare en modo «Flexible») que habla con tu servidor por HTTP. Cambia el modo a «Full (strict)».

Siguiente paso: Módulo 4, la base de datos MariaDB.

3.7 Ejercicios prácticos del Módulo 3

1. Deshabilita `ejemplo2.com` con `a2dissite`, recarga y visita `https://www.ejemplo2.com`. ¿Qué web aparece y por qué? Vuelve a habilitarlo.
2. Crea un tercer sitio `pruebas.ejemplo1.com` (usuario `web3`, carpeta propia, solo HTTP) y verifica con `curl -H "Host: ..."` que funciona. Después elimínalo por completo (sitio, carpeta y usuario).
3. Usa `namei -l` para encontrar por qué un archivo con permisos `600` da un error 403.
4. Revisa `/var/log/apache2/ejemplo1.com-access.log` y localiza las peticiones de validación de Let's Encrypt (`acme-challenge`).
5. Comprueba con `openssl s_client -connect www.ejemplo2.com:443 -servername www.ejemplo2.com </dev/null | head -20` qué certificado se presenta. Repite sin `-servername`. ¿Qué cambia y por qué? (Pista: SNI.)

Módulo 4. Instalación y configuración de MariaDB

Objetivos del módulo

- Comprender qué es un sistema gestor de bases de datos relacional y cómo funciona MariaDB.
- Instalar MariaDB, protegerla y dejarla arrancando automáticamente.
- Trabajar en la consola SQL: crear bases de datos, tablas, usuarios y permisos.
- Crear una base de datos y un usuario independientes para cada VirtualHost.
- Cambiar contraseñas, hacer copias de seguridad, restaurarlas y comprobar conexiones.

4.1 Qué es MariaDB

MariaDB es un sistema gestor de bases de datos relacional (SGBD), nacido en 2009 como continuación libre de MySQL por parte de sus creadores originales. Es compatible con MySQL en la práctica totalidad de casos: los mismos comandos, el mismo protocolo y las mismas librerías de conexión. Ubuntu 24.04 incluye MariaDB 10.11, una versión de soporte a largo plazo.

Conceptos básicos:

Concepto	Qué es	Ejemplo
Servidor	El programa `mariadbd`, que guarda los datos y atiende conexiones.	Servicio `mariadb`.
Base de datos	Un contenedor de tablas, normalmente una por aplicación.	`web1_db`
Tabla	Datos organizados en filas y columnas.	`contactos`
Columna	Un campo con un tipo de dato.	`email VARCHAR(150)`
Fila (registro)	Un elemento concreto.	(1, 'Ana', 'ana@...')
Clave primaria	Columna que identifica de forma única cada fila.	`id`
Usuario	Cuenta que se conecta, identificada por nombre **y** origen.	`'web1_user'@'localhost'`
Privilegio	Lo que un usuario puede hacer sobre qué objetos.	`SELECT` en `web1_db.*`
SQL	Lenguaje con el que se habla con la base de datos.	`SELECT * FROM contactos;`

> **ATENCIÓN:** En MariaDB un usuario se identifica por **nombre y host** a la vez. `'web1_user'@'localhost'` y `'web1_user'@'%'` son dos usuarios distintos, con contraseñas y permisos distintos. `localhost` significa «conectándose desde el propio servidor»; `%` significa «desde cualquier sitio». En este manual solo usaremos `localhost`.

Cómo se conectará cada aplicación:

DIAGRAMA



PASO 4.1 – Instalar MariaDB

Objetivo: instalar el servidor y el cliente de MariaDB.

Comando:

TERMINAL

```
sudo apt install -y mariadb-server mariadb-client
```

Análisis del comando: `mariadb-server` instala el servidor (`mariadbd`) y sus herramientas; `mariadb-client` el cliente de consola `mariadb` y utilidades como `mariadb-dump`.

Resultado esperado: termina con `Created symlink ... mariadb.service` y sin errores.

Comprobación:

TERMINAL

```
mariadb --version
systemctl status mariadb --no-pager
systemctl is-enabled mariadb
```

SALIDA ESPERADA

```
mariadb  Ver 15.2 Distrib 10.11.13-MariaDB, for debian-linux-gnu (x86_64) using  EditLine
wrapper
● mariadb.service - MariaDB 10.11.13 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
     Active: active (running) since ...
enabled
```

Si no estuviera habilitado: `sudo systemctl enable --now mariadb`.

Siguiente paso: comprobar dónde escucha.

PASO 4.2 – Verificar que MariaDB no es accesible desde Internet

Objetivo: confirmar que MariaDB solo escucha en la interfaz local.

Explicación: una base de datos expuesta a Internet es uno de los fallos de seguridad más graves y más explotados. Ubuntu configura MariaDB para escuchar solo en `127.0.0.1`. Lo comprobamos.

Comando:

TERMINAL

```
sudo ss -tlnp | grep -E 'mariadbd|mysqld'
grep -E '^\s*bind-address' /etc/mysql/mariadb.conf.d/50-server.cnf
```

Análisis del comando: `ss -tlnp` lista los sockets TCP (`t`) en escucha (`l`), en formato numérico (`n`) y con el proceso (`p`). El `grep` busca la directiva `bind-address` en el archivo de configuración del servidor.

Resultado esperado:

SALIDA ESPERADA

```
LISTEN 0  80  127.0.0.1:3306  0.0.0.0:*  users:(("mariadbd",pid=4120,fd=21))
bind-address            = 127.0.0.1
```

Comprobación: la dirección debe ser `127.0.0.1:3306`, **nunca** `0.0.0.0:3306` ni `*:3306`.

Errores habituales: si aparece `0.0.0.0`, edita `/etc/mysql/mariadb.conf.d/50-server.cnf`, deja la línea `bind-address = 127.0.0.1` dentro de la sección `[mysqld]` y reinicia con `sudo systemctl restart mariadb`.

> **CONSEJO:** La estructura de configuración de MariaDB en Ubuntu: `/etc/mysql/my.cnf` incluye `/etc/mysql/conf.d/` y `/etc/mysql/mariadb.conf.d/`. El archivo que más tocarás es `50-server.cnf` (sección `[mysqld]`). Para tus ajustes propios es buena práctica crear un archivo nuevo, por ejemplo `/etc/mysql/mariadb.conf.d/99-local.cnf`, en lugar de modificar los del paquete.

Siguiente paso: proteger la instalación.

PASO 4.3 – Proteger la instalación con mariadb-secure-installation

Objetivo: eliminar los elementos inseguros de una instalación nueva.

Explicación: en Ubuntu, el usuario `root` de MariaDB se autentica por **unix_socket**: solo puede entrar quien sea root en Linux (con `sudo`), sin contraseña de MariaDB. Es un método muy seguro y lo mantendremos. El asistente elimina usuarios anónimos, el acceso remoto de root y la base de datos de pruebas.

Comando:

TERMINAL

```
sudo mariadb-secure-installation
```

Responde así:

Pregunta	Respuesta	Por qué
Enter current password for root (enter for none)	Intro	No hay contraseña: root usa unix_socket.

Pregunta	Respuesta	Por qué
Switch to unix_socket authentication [Y/n]	n	Ya lo usa. Responder `n` deja la configuración tal cual.
Change the root password? [Y/n]	n	Con unix_socket no hace falta contraseña.
Remove anonymous users? [Y/n]	Y	Los usuarios anónimos permiten entrar sin nombre.
Disallow root login remotely? [Y/n]	Y	Root solo desde el propio servidor.
Remove test database and access to it? [Y/n]	Y	La base `test` es accesible por cualquiera.
Reload privilege tables now? [Y/n]	Y	Aplica los cambios.

Resultado esperado: termina con `Thanks for using MariaDB!`.

Comprobación: entra como root sin contraseña (gracias a sudo) e intenta entrar como root **sin** sudo:

TERMINAL
```
sudo mariadb -e "SELECT user, host, plugin FROM mysql.user;"
mariadb -u root
```

SALIDA ESPERADA
```
+-------------+-----------+-----------------------+
| User        | Host      | plugin                |
+-------------+-----------+-----------------------+
| mariadb.sys | localhost | mysql_native_password |
| root        | localhost | mysql_native_password |
| mysql       | localhost | mysql_native_password |
+-------------+-----------+-----------------------+
ERROR 1698 (28000): Access denied for user 'root'@'localhost'
```

El error final es **la prueba de que funciona**: sin ser root en Linux, no se puede entrar como root en MariaDB. (La columna `plugin` muestra el método principal; root tiene además `unix_socket` como alternativa, que es la que se usa con sudo.)

Siguiente paso: conocer la consola.

4.2 La consola de MariaDB

PASO 4.4 – Acceder a la consola y orientarse

Objetivo: entrar a la consola SQL y aprender sus comandos básicos.

Comando:

TERMINAL
```
sudo mariadb
```

Verás el indicador `MariaDB [(none)]>`. `(none)` indica que no has seleccionado ninguna base de datos. Prueba:

SQL
```
SELECT VERSION();
SHOW DATABASES;
SELECT CURRENT_USER();
STATUS;
```

Reglas de la consola:

- Toda instrucción SQL termina en **punto y coma** `;`. Si pulsas Intro sin él, verás `->`: MariaDB espera que termines. Escribe `;` e Intro.
- Las palabras clave (`SELECT`, `CREATE`) pueden ir en mayúsculas o minúsculas; por convención se escriben en mayúsculas.
- Los nombres de bases de datos y tablas **distinguen mayúsculas** en Linux.
- Los textos van entre comillas simples: `'texto'`.
- Para salir: `EXIT;` o **Ctrl+D**. Para cancelar una instrucción a medio escribir: **Ctrl+C**.

Resultado esperado de `SHOW DATABASES;`:

SALIDA ESPERADA

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.001 sec)
```

Estas bases son internas: `mysql` guarda usuarios y privilegios; las demás ofrecen información del servidor. No las modifiques directamente.

Siguiente paso: generar contraseñas seguras.

4.3 Bases de datos y usuarios independientes para cada VirtualHost

PASO 4.5 – Generar contraseñas seguras para los usuarios de base de datos

Objetivo: crear dos contraseñas aleatorias y robustas.

Comando (en el terminal de Linux, **fuera de la consola de MariaDB):**

TERMINAL

```
openssl rand -base64 24
openssl rand -base64 24
```

Análisis del comando: `openssl rand` genera bytes aleatorios criptográficamente seguros; `-base64 24` genera 24 bytes y los codifica en 32 caracteres legibles.

Resultado esperado: dos líneas como `p3Q9xV2r...` (cada vez distintas). Guárdalas en un gestor de contraseñas: la primera será la de `web1_user` y la segunda la de `web2_user`.

> **ATENCIÓN:** En este manual las representamos como `<CLAVE_WEB1>` y `<CLAVE_WEB2>`. Sustitúyelas siempre por las tuyas. Evita contraseñas con comillas simples `'` o barras invertidas `\`: complican su uso dentro de SQL y PHP.

Siguiente paso: crear las bases de datos.

PASO 4.6 – Crear las bases de datos web1_db y web2_db

Objetivo: crear una base de datos independiente para cada sitio.

Comando (dentro de `sudo mariadb`):

```sql
CREATE DATABASE web1_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE web2_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
SHOW DATABASES;
```

Análisis del comando:

- `CREATE DATABASE web1_db`: crea la base de datos.
- `CHARACTER SET utf8mb4`: codificación de caracteres. `utf8mb4` admite todos los caracteres Unicode, incluidas tildes, eñes y emojis. (El antiguo `utf8` de MySQL solo admite 3 bytes por carácter y falla con emojis.)
- `COLLATE utf8mb4_unicode_ci`: reglas de comparación y ordenación. `ci` = *case insensitive*: `'Ana'` y `'ana'` se consideran iguales al buscar.

Resultado esperado: `Query OK, 1 row affected` por cada una, y ambas aparecen en `SHOW DATABASES`.

Errores habituales: `ERROR 1007 (HY000): Can't create database 'web1_db'; database exists`. Ya existía; puedes usar `CREATE DATABASE IF NOT EXISTS ...` para evitar el error.

Siguiente paso: crear los usuarios.

PASO 4.7 – Crear los usuarios de base de datos

Objetivo: crear `web1_user` y `web2_user`, que solo podrán conectarse desde el propio servidor.

Comando (dentro de `sudo mariadb`):

```sql
CREATE USER 'web1_user'@'localhost' IDENTIFIED BY '<CLAVE_WEB1>';
CREATE USER 'web2_user'@'localhost' IDENTIFIED BY '<CLAVE_WEB2>';
SELECT user, host FROM mysql.user WHERE user LIKE 'web%';
```

Análisis del comando:

- `CREATE USER 'web1_user'@'localhost'`: crea el usuario `web1_user` que solo puede conectarse desde `localhost` (socket Unix o 127.0.0.1).
- `IDENTIFIED BY '...'`: su contraseña. MariaDB no la guarda en claro, sino un *hash*.

Resultado esperado:

SALIDA ESPERADA

```
+-----------+-----------+
| User      | Host      |
+-----------+-----------+
| web1_user | localhost |
| web2_user | localhost |
+-----------+-----------+
```

Siguiente paso: asignar permisos limitados.

PASO 4.8 – Asignar permisos específicos a cada usuario

Objetivo: que `web1_user` solo pueda trabajar con `web1_db` y `web2_user` solo con `web2_db`, y con los privilegios mínimos necesarios.

Explicación: una aplicación web normal solo necesita leer, insertar, modificar y borrar filas. No necesita borrar tablas enteras, crear usuarios ni tocar otras bases de datos. Si un atacante encontrara una inyección SQL en la web 1, con estos permisos no podría ni ver la base de datos de la web 2.

Distinguimos dos niveles de permisos:

Privilegio	Permite	¿Lo damos?
`SELECT`	Leer filas.	Sí
`INSERT`	Añadir filas.	Sí
`UPDATE`	Modificar filas.	Sí
`DELETE`	Borrar filas.	Sí
`CREATE`, `ALTER`, `INDEX`, `DROP`	Crear, modificar y borrar tablas e índices.	Solo si la aplicación instala o actualiza su propio esquema (WordPress, por ejemplo, los necesita). En nuestro ejemplo, no: creamos las tablas como administradores.
`CREATE TEMPORARY TABLES`, `LOCK TABLES`	Tablas temporales y bloqueos.	Solo si la aplicación lo requiere.
`ALL PRIVILEGES`	Todo sobre la base indicada.	No.
`GRANT OPTION`, `SUPER`, `FILE`, `PROCESS`	Administración del servidor.	**Nunca** a una aplicación.

Comando (dentro de `sudo mariadb`):

```sql
GRANT SELECT, INSERT, UPDATE, DELETE ON web1_db.* TO 'web1_user'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON web2_db.* TO 'web2_user'@'localhost';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'web1_user'@'localhost';
SHOW GRANTS FOR 'web2_user'@'localhost';
```

Análisis del comando:

- `GRANT privilegios ON web1_db.* TO usuario`: concede esos privilegios sobre todas las tablas (`*`) de `web1_db`.
- `FLUSH PRIVILEGES`: recarga las tablas de permisos. Con `GRANT` no es estrictamente necesario, pero no hace daño y es una buena costumbre.
- `SHOW GRANTS`: muestra los permisos efectivos.

Resultado esperado:

```
SALIDA ESPERADA
+-----------------------------------------------------------------------------------------------
---+
| Grants for web1_user@localhost
|
+-----------------------------------------------------------------------------------------------
---+
| GRANT USAGE ON *.* TO `web1_user`@`localhost` IDENTIFIED BY PASSWORD '*A1B2C3...'
|
| GRANT SELECT, INSERT, UPDATE, DELETE ON `web1_db`.* TO `web1_user`@`localhost`
|
+-----------------------------------------------------------------------------------------------
---+
```

`USAGE ON *.*` significa «puede conectarse, pero sin privilegios globales». Es correcto.

Siguiente paso: crear las tablas de las aplicaciones.

PASO 4.9 – Crear la estructura de tablas de cada aplicación

Objetivo: crear la tabla `contactos` que usará la aplicación PHP del Módulo 5.

Explicación: nuestra aplicación de ejemplo será una agenda de contactos con un pequeño formulario. Cada base de datos tendrá la misma estructura, pero datos completamente independientes.

Comando (dentro de `sudo mariadb`):

```sql
SQL
USE web1_db;
CREATE TABLE contactos (
    id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
    nombre      VARCHAR(100) NOT NULL,
    email       VARCHAR(150) NOT NULL,
    telefono    VARCHAR(30)  NULL,
    notas       TEXT         NULL,
    creado_en   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uk_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

INSERT INTO contactos (nombre, email, telefono, notas)
VALUES ('Ana Pérez', 'ana@ejemplo1.com', '600111222', 'Contacto de prueba de la web 1');

USE web2_db;
CREATE TABLE contactos LIKE web1_db.contactos;
INSERT INTO contactos (nombre, email, telefono, notas)
VALUES ('Luis Gómez', 'luis@ejemplo2.com', '600333444', 'Contacto de prueba de la web 2');
```

Análisis del comando:

Elemento	Significado
`USE web1_db;`	Selecciona la base de datos sobre la que trabajarán las instrucciones siguientes.
`INT UNSIGNED`	Número entero sin signo (0 a 4.294.967.295).
`NOT NULL`	El campo es obligatorio.
`AUTO_INCREMENT`	MariaDB asigna automáticamente 1, 2, 3...

Elemento	Significado
`VARCHAR(100)`	Texto de longitud variable, hasta 100 caracteres.
`TEXT`	Texto largo (hasta 64 KB).
`TIMESTAMP ... DEFAULT CURRENT_TIMESTAMP`	Fecha y hora, rellenada automáticamente al insertar.
`PRIMARY KEY (id)`	`id` identifica cada fila de forma única.
`UNIQUE KEY uk_email (email)`	No puede haber dos contactos con el mismo email.
`ENGINE=InnoDB`	Motor de almacenamiento con transacciones y recuperación ante fallos (el predeterminado).
`CREATE TABLE contactos LIKE web1_db.contactos`	Copia la estructura (no los datos) de otra tabla.

Comprobación:

```sql
SELECT * FROM web1_db.contactos;
SELECT * FROM web2_db.contactos;
DESCRIBE web1_db.contactos;
EXIT;
```

Cada consulta debe devolver su propio contacto.

Siguiente paso: comprobar que los permisos funcionan.

PASO 4.10 – Comprobar las conexiones y el aislamiento entre usuarios

Objetivo: demostrar que cada usuario entra en su base de datos y **no** puede acceder a la otra.

Comando (en el terminal de Linux):

```terminal
mariadb -u web1_user -p web1_db -e "SELECT nombre, email FROM contactos;"
mariadb -u web1_user -p web2_db -e "SELECT * FROM contactos;"
mariadb -u web1_user -p web1_db -e "DROP TABLE contactos;"
```

Análisis del comando:

- `-u web1_user`: usuario con el que conectar.
- `-p`: pide la contraseña de forma interactiva (no la escribas en la línea de comandos: quedaría en el historial).
- `web1_db`: base de datos a usar.
- `-e "..."`: ejecuta la instrucción y sale.

Resultado esperado:

```
SALIDA ESPERADA
+-----------+------------------+
| nombre    | email            |
+-----------+------------------+
| Ana Pérez | ana@ejemplo1.com |
+-----------+------------------+
ERROR 1044 (42000): Access denied for user 'web1_user'@'localhost' to database 'web2_db'
ERROR 1142 (42000): DROP command denied to user 'web1_user'@'localhost' for table
`web1_db`.`contactos`
```

Las dos denegaciones son exactamente lo que queremos: aislamiento entre webs y protección contra borrados de tablas.

Errores habituales:

- `ERROR 1045 (28000): Access denied for user 'web1_user'@'localhost' (using password: YES)`: contraseña incorrecta o usuario creado con otro host. Revisa con `SELECT user,host FROM mysql.user;`.
- `ERROR 2002 (HY000): Can't connect to local server through socket '/run/mysqld/mysqld.sock'`: MariaDB no está funcionando (`systemctl status mariadb`).

Siguiente paso: aprender a cambiar contraseñas.

PASO 4.11 – Modificar contraseñas

Objetivo: cambiar la contraseña de un usuario de base de datos (por ejemplo, si sospechas que se ha filtrado).

Comando (dentro de `sudo mariadb`):

```sql
SQL
ALTER USER 'web1_user'@'localhost' IDENTIFIED BY '<NUEVA_CLAVE_WEB1>';
FLUSH PRIVILEGES;
```

Análisis del comando: `ALTER USER ... IDENTIFIED BY` sustituye la contraseña. A partir de ese momento, la aplicación dejará de conectar hasta que actualices su archivo de configuración (Módulo 5, `config.php`).

Comprobación: `mariadb -u web1_user -p web1_db -e "SELECT 1;"` con la nueva clave debe devolver `1`.

Otras operaciones de administración de usuarios:

```sql
SQL
-- Quitar un privilegio concreto
REVOKE DELETE ON web1_db.* FROM 'web1_user'@'localhost';
-- Volver a concederlo
GRANT DELETE ON web1_db.* TO 'web1_user'@'localhost';
-- Bloquear temporalmente una cuenta sin borrarla
ALTER USER 'web1_user'@'localhost' ACCOUNT LOCK;
ALTER USER 'web1_user'@'localhost' ACCOUNT UNLOCK;
-- Borrar un usuario
DROP USER 'usuario_obsoleto'@'localhost';
-- Ver qué usuarios están conectados ahora mismo
SHOW PROCESSLIST;
```

Siguiente paso: copias de seguridad.

4.4 Copias de seguridad y restauración

Una base de datos sin copia de seguridad es una base de datos que vas a perder. La herramienta estándar es **mariadb-dump** (antes `mysqldump`), que genera un archivo de texto con todas las instrucciones SQL necesarias para reconstruir la base de datos.

PASO 4.12 – Hacer una copia de seguridad manual

Objetivo: exportar `web1_db` a un archivo comprimido.

Comando:

TERMINAL

```
sudo mkdir -p /var/backups/mariadb
sudo chmod 700 /var/backups/mariadb
sudo mariadb-dump --single-transaction --routines --triggers --events --databases web1_db |
gzip | sudo tee /var/backups/mariadb/web1_db_$(date +%F_%H%M).sql.gz > /dev/null
ls -lh /var/backups/mariadb/
```

Análisis del comando:

- `mariadb-dump`: exporta la base de datos (lo ejecutamos con sudo para usar la cuenta root por unix_socket).
- `--single-transaction`: hace la copia como una «foto» coherente sin bloquear las tablas InnoDB; la web sigue funcionando mientras tanto.
- `--routines --triggers --events`: incluye procedimientos almacenados, disparadores y eventos programados.
- `--databases web1_db`: incluye en el archivo la instrucción `CREATE DATABASE` y `USE`, de modo que la restauración recrea la base si no existe.
- `| gzip`: comprime (los volcados SQL se reducen mucho).
- `$(date +%F_%H%M)`: inserta la fecha y hora actuales en el nombre, por ejemplo `2026-11-08_2310`.
- `chmod 700` en la carpeta: las copias contienen todos tus datos, solo root debe leerlas.

Resultado esperado:

SALIDA ESPERADA

```
-rw-r--r-- 1 root root 1,2K nov  8 23:10 web1_db_2026-11-08_2310.sql.gz
```

Comprobación: mira el contenido sin descomprimir el archivo:

TERMINAL

```
sudo zcat /var/backups/mariadb/web1_db_*.sql.gz | head -40
```

Verás instrucciones `CREATE DATABASE`, `CREATE TABLE` e `INSERT INTO`.

Para copiar **todas** las bases de datos de una vez:

TERMINAL

```
sudo mariadb-dump --single-transaction --routines --triggers --events --all-databases | gzip |
sudo tee /var/backups/mariadb/todas_$(date +%F).sql.gz > /dev/null
```

Siguiente paso: restaurar.

PASO 4.13 – Restaurar una base de datos

Objetivo: recuperar `web1_db` desde la copia, simulando una pérdida de datos.

Comando: primero simulamos el desastre borrando la base de datos (**solo en tu laboratorio o tras verificar la copia**):

TERMINAL
```
sudo mariadb -e "DROP DATABASE web1_db;"
sudo mariadb -e "SHOW DATABASES;"
```

Ahora restauramos:

TERMINAL
```
sudo sh -c 'zcat /var/backups/mariadb/web1_db_2026-11-08_2310.sql.gz | mariadb'
```

Análisis del comando: `zcat` descomprime y envía el SQL a la consola `mariadb`, que ejecuta las instrucciones una tras otra. Como el volcado se hizo con `--databases`, el propio archivo contiene `CREATE DATABASE` y `USE web1_db`. Usamos `sh -c '...'` para que toda la tubería se ejecute con privilegios de root. Sustituye el nombre del archivo por el tuyo.

Comprobación:

TERMINAL
```
sudo mariadb -e "SELECT * FROM web1_db.contactos;"
mariadb -u web1_user -p web1_db -e "SELECT COUNT(*) FROM contactos;"
```

Los datos han vuelto **y** el usuario `web1_user` sigue pudiendo acceder (los usuarios y permisos viven en la base `mysql`, que no hemos borrado).

> **ATENCIÓN:** Si restauras un volcado hecho **sin** `--databases` (solo con el nombre de la base), el archivo no contiene `CREATE DATABASE`. En ese caso crea primero la base vacía y restaura indicándola: `sudo mariadb -e "CREATE DATABASE web1_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"` y después `zcat archivo.sql.gz | sudo mariadb web1_db`.

> **CONSEJO:** Una copia que nunca has restaurado es solo una esperanza. Programa una prueba de restauración en un servidor de pruebas al menos una vez al mes. La automatización diaria de las copias se configura en el Módulo 12.

Errores habituales: `ERROR 1049 (42000): Unknown database 'web1_db'` al restaurar: el volcado no incluía `CREATE DATABASE` (ver la nota anterior).

Siguiente paso: Módulo 5, conectar PHP con MariaDB.

4.5 Referencia rápida de SQL

Operación	Sintaxis
Crear base de datos	`CREATE DATABASE nombre CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;`
Borrar base de datos	`DROP DATABASE nombre;`
Crear usuario	`CREATE USER 'u'@'localhost' IDENTIFIED BY 'clave';`
Conceder permisos	`GRANT SELECT, INSERT, UPDATE, DELETE ON bd.* TO 'u'@'localhost';`
Ver permisos	`SHOW GRANTS FOR 'u'@'localhost';`
Retirar permisos	`REVOKE DELETE ON bd.* FROM 'u'@'localhost';`
Cambiar contraseña	`ALTER USER 'u'@'localhost' IDENTIFIED BY 'nueva';`
Borrar usuario	`DROP USER 'u'@'localhost';`
Listar tablas	`SHOW TABLES;`
Estructura de una tabla	`DESCRIBE tabla;`
Insertar	`INSERT INTO t (c1, c2) VALUES ('a', 'b');`
Leer	`SELECT c1, c2 FROM t WHERE id = 1 ORDER BY c1 LIMIT 10;`
Modificar	`UPDATE t SET c1 = 'x' WHERE id = 1;`
Borrar filas	`DELETE FROM t WHERE id = 1;`

> **ATENCIÓN:** Un `UPDATE` o `DELETE` **sin** `WHERE` afecta a **todas** las filas de la tabla. Antes de ejecutar uno, ejecuta el `SELECT` equivalente con el mismo `WHERE` para ver qué filas vas a tocar.

4.6 Ejercicios prácticos del Módulo 4

1. Crea una tabla `productos` (id, nombre, precio DECIMAL(10,2), stock INT) en `web2_db`, inserta tres productos y practica `SELECT` con `WHERE`, `ORDER BY` y `LIMIT`.
2. Intenta conectar como `web2_user` a `web1_db` y documenta el error.
3. Quita a `web1_user` el permiso `DELETE`, intenta borrar un contacto y vuelve a concederlo.
4. Haz una copia de `web2_db`, borra un registro, restaura la copia y verifica que el registro ha vuelto.
5. Explica por qué es más seguro `'web1_user'@'localhost'` que `'web1_user'@'%'` y por qué no damos `ALL PRIVILEGES` a las aplicaciones.

Módulo 5. Integración de MariaDB con Apache y PHP

Objetivos del módulo

- Entender cómo colaboran Apache, PHP-FPM y MariaDB para servir una página dinámica.
- Instalar PHP 8.3 con PHP-FPM y los módulos necesarios.
- Crear un *pool* de PHP-FPM independiente para cada web, con su propio usuario Linux.
- Conectar Apache con cada pool desde su VirtualHost.
- Programar una aplicación PHP que se conecta a su propia base de datos con PDO y consultas preparadas.
- Implementar las operaciones de inserción, lectura, actualización y eliminación (CRUD), con gestión de errores y protección de credenciales.

5.1 Cómo funciona una página dinámica

Una página estática (HTML) es un archivo que Apache entrega tal cual. Una página dinámica (PHP) es un **programa** que se ejecuta en cada visita y genera el HTML en ese momento, normalmente consultando una base de datos.

Hay dos formas de que Apache ejecute PHP:

Método	Cómo funciona	Ventajas / inconvenientes
`mod_php` (libapache2-mod-php)	PHP se carga **dentro** de cada proceso de Apache.	Sencillo, pero obliga a usar el MPM `prefork` (más memoria), y todas las webs se ejecutan con el mismo usuario `www-data`.
PHP-FPM (FastCGI Process Manager)	PHP es un servicio aparte. Apache le pasa las peticiones `.php` por un socket.	Compatible con `mpm_event`, cada web puede tener su propio *pool* de procesos con su propio usuario y su propia configuración. **Es el método profesional y el que usaremos.**

El recorrido completo de una petición:

DIAGRAMA



5.2 Instalación de PHP y PHP-FPM

PASO 5.1 – Instalar PHP-FPM y los módulos necesarios

Objetivo: instalar el intérprete PHP como servicio FPM y las extensiones que usan las aplicaciones habituales.

Comando:

TERMINAL

```
sudo apt install -y php-fpm php-mysql php-mbstring php-xml php-curl php-intl php-zip php-gd
```

Análisis del comando: en Ubuntu 24.04, los paquetes `php-*` son «metapaquetes» que instalan la versión del sistema (`php8.3-*`):

Paquete	Para qué sirve
`php-fpm`	El servicio PHP-FPM (`php8.3-fpm`) y el intérprete de línea de comandos.
`php-mysql`	Las extensiones `pdo_mysql` y `mysqli` para hablar con MariaDB/MySQL.
`php-mbstring`	Manejo de textos multibyte (tildes, eñes, UTF-8).
`php-xml`	Procesado de XML (lo usan muchísimas aplicaciones).
`php-curl`	Peticiones HTTP desde PHP (APIs, pasarelas de pago).
`php-intl`	Internacionalización: formatos de fecha, moneda, ordenación por idioma.
`php-zip` y `php-gd`	Archivos comprimidos e imágenes (miniaturas, captchas).

ATENCIÓN: **No** instales el paquete `php` a secas ni `libapache2-mod-php`: arrastran `mod_php` y cambian Apache al MPM `prefork`, deshaciendo el diseño de este módulo. Si ves que se instala `libapache2-mod-php8.3`, cancela.

Resultado esperado: al final aparece un aviso como este:

SALIDA ESPERADA

```
NOTICE: Not enabling PHP 8.3 FPM by default.
NOTICE: To enable PHP 8.3 FPM in Apache2 do:
NOTICE: a2enmod proxy_fcgi setenvif
NOTICE: a2enconf php8.3-fpm
```

Es normal: haremos la integración nosotros, sitio por sitio.

Comprobación:

TERMINAL

```
php -v
systemctl status php8.3-fpm --no-pager
php -m | grep -Ei 'pdo_mysql|mysqli|mbstring|intl'
sudo apachectl -M | grep -i php || echo "Correcto: mod_php no está cargado"
```

SALIDA ESPERADA

```
PHP 8.3.6 (cli) (built: ...) (NTS)
● php8.3-fpm.service - The PHP 8.3 FastCGI Process Manager
     Active: active (running) since ...
intl
mbstring
mysqli
pdo_mysql
Correcto: mod_php no está cargado
```

Errores habituales: si `php -v` muestra otra versión (por ejemplo 8.2 en Debian 12), usa esa versión en **todas** las rutas de este módulo (`/etc/php/8.2/...`, `php8.2-fpm`).

Siguiente paso: conocer la estructura de PHP-FPM.

5.3 Estructura de configuración de PHP-FPM

DIAGRAMA

```
/etc/php/8.3/
├── cli/php.ini              ← configuración de PHP en la línea de comandos
├── fpm/
│   ├── php.ini              ← configuración global de PHP para las webs
│   ├── php-fpm.conf         ← configuración del gestor de procesos
│   └── pool.d/
│       ├── www.conf         ← pool por defecto (usuario www-data) – lo desactivaremos
│       ├── web1.conf        ← pool de ejemplo1.com (usuario web1) – lo crearemos
│       └── web2.conf        ← pool de ejemplo2.com (usuario web2) – lo crearemos
└── mods-available/          ← extensiones (.ini)
/run/php/php8.3-fpm-web1.sock   ← socket del pool web1 (se crea al arrancar)
/var/log/php8.3-fpm.log         ← log del gestor FPM
```

Un **pool** es un grupo de procesos PHP con su propia configuración: con qué usuario se ejecutan, en qué socket escuchan, cuántos procesos pueden lanzar y qué opciones de PHP aplican. Con un pool por web conseguimos:

- **Aislamiento**: el código de ejemplo1.com se ejecuta como `web1` y no puede leer los archivos de `web2`.
- **Límites independientes**: si una web recibe muchísimas visitas, agota sus procesos, no los de la otra.
- **Configuración propia**: límites de memoria, rutas de sesión o remitente del correo distintos por web.

PASO 5.2 – Crear la carpeta de logs de cada web

Objetivo: preparar un lugar donde PHP escriba los errores de cada aplicación.

Comando:

TERMINAL

```
sudo install -d -o web1 -g web1 -m 700 /var/www/ejemplo1.com/logs
sudo install -d -o web2 -g web2 -m 700 /var/www/ejemplo2.com/logs
```

Análisis del comando: `install -d` crea una carpeta y, en la misma orden, fija propietario (`-o`), grupo (`-g`) y permisos (`-m`).

Siguiente paso: crear los pools.

PASO 5.3 – Crear el pool de PHP-FPM de ejemplo1.com

Objetivo: definir el pool `web1`.

Comando:

TERMINAL

```
sudo nano /etc/php/8.3/fpm/pool.d/web1.conf
```

Archivo: `/etc/php/8.3/fpm/pool.d/web1.conf` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
; Pool de PHP-FPM para ejemplo1.com
[web1]
; Usuario y grupo con los que se ejecuta el código PHP de esta web
user = web1
group = web1

; Socket por el que Apache enviará las peticiones
listen = /run/php/php8.3-fpm-web1.sock
; Apache (www-data) debe poder leer y escribir en el socket
listen.owner = www-data
listen.group = www-data
listen.mode = 0660

; Gestión de procesos: se crean bajo demanda y se cierran si están inactivos
pm = ondemand
pm.max_children = 10
pm.process_idle_timeout = 30s
pm.max_requests = 500

; Directorio de trabajo de los procesos
chdir = /

; --- Opciones de PHP fijadas para esta web (no modificables desde el código) ---
; PHP solo puede abrir archivos dentro de la carpeta de la web
php_admin_value[open_basedir] = /var/www/ejemplo1.com/:/usr/share/php/
; Temporales, subidas y sesiones dentro de la carpeta privada tmp/
php_admin_value[sys_temp_dir] = /var/www/ejemplo1.com/tmp
php_admin_value[upload_tmp_dir] = /var/www/ejemplo1.com/tmp
php_admin_value[session.save_path] = /var/www/ejemplo1.com/tmp
php_admin_value[session.gc_probability] = 1
; Errores: nunca se muestran al visitante, se guardan en el log de la web
php_admin_flag[display_errors] = off
php_admin_flag[log_errors] = on
php_admin_value[error_log] = /var/www/ejemplo1.com/logs/php-error.log
; Límites de recursos
php_admin_value[memory_limit] = 128M
php_admin_value[upload_max_filesize] = 16M
php_admin_value[post_max_size] = 16M
php_admin_value[max_execution_time] = 30
; Funciones peligrosas deshabilitadas
php_admin_value[disable_functions] =
exec,passthru,shell_exec,system,proc_open,popen,pcntl_exec
; Remitente del correo enviado con mail() (se usa en el Módulo 11)
php_admin_value[sendmail_path] = /usr/sbin/sendmail -t -i -f noreply@ejemplo1.com
; Cookies de sesión seguras
```

```
php_admin_flag[session.cookie_httponly] = on
php_admin_flag[session.cookie_secure] = on
php_admin_flag[session.use_strict_mode] = on
```

Análisis de las directivas:

Directiva	Significado
`[web1]`	Nombre del pool (aparece en los procesos y en los logs).
`user` / `group`	Identidad con la que se ejecuta el código PHP.
`listen`	Ruta del socket Unix.
`listen.owner/group/mode`	Propietario y permisos del socket: solo Apache (www-data) puede conectarse.
`pm = ondemand`	Los procesos se crean al llegar visitas y se cierran tras `process_idle_timeout`. Ideal para servidores pequeños con varias webs. Alternativas: `dynamic` (mantiene algunos procesos siempre listos) y `static`.
`pm.max_children`	Máximo de procesos simultáneos. Cada uno puede gastar hasta `memory_limit`; 10 × 128 MB es el peor caso.
`pm.max_requests`	Cada proceso se recicla tras 500 peticiones (evita fugas de memoria).
`php_admin_value[...]`	Fija una opción de PHP que el código de la web **no** puede cambiar con `ini_set()`.
`open_basedir`	Jaula: PHP no puede abrir archivos fuera de esas rutas.
`disable_functions`	Impide ejecutar comandos del sistema desde PHP, la vía favorita de los atacantes.
`session.cookie_secure`	La cookie de sesión solo viaja por HTTPS.

Siguiente paso: el pool de ejemplo2.com.

PASO 5.4 – Crear el pool de PHP-FPM de ejemplo2.com

Archivo: `/etc/php/8.3/fpm/pool.d/web2.conf` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
; Pool de PHP-FPM para ejemplo2.com
[web2]
user = web2
group = web2

listen = /run/php/php8.3-fpm-web2.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660

pm = ondemand
pm.max_children = 10
pm.process_idle_timeout = 30s
pm.max_requests = 500

chdir = /

php_admin_value[open_basedir] = /var/www/ejemplo2.com/:/usr/share/php/
php_admin_value[sys_temp_dir] = /var/www/ejemplo2.com/tmp
php_admin_value[upload_tmp_dir] = /var/www/ejemplo2.com/tmp
php_admin_value[session.save_path] = /var/www/ejemplo2.com/tmp
php_admin_value[session.gc_probability] = 1
```

```
php_admin_flag[display_errors] = off
php_admin_flag[log_errors] = on
php_admin_value[error_log] = /var/www/ejemplo2.com/logs/php-error.log
php_admin_value[memory_limit] = 128M
php_admin_value[upload_max_filesize] = 16M
php_admin_value[post_max_size] = 16M
php_admin_value[max_execution_time] = 30
php_admin_value[disable_functions] =
exec,passthru,shell_exec,system,proc_open,popen,pcntl_exec
php_admin_value[sendmail_path] = /usr/sbin/sendmail -t -i -f noreply@ejemplo2.com
php_admin_flag[session.cookie_httponly] = on
php_admin_flag[session.cookie_secure] = on
php_admin_flag[session.use_strict_mode] = on
```

PASO 5.5 – Desactivar el pool por defecto y aplicar la configuración

Objetivo: dejar solo los pools que usamos y comprobar la configuración.

Explicación: el pool `www` ejecuta PHP como `www-data` y no lo necesitamos. Lo desactivamos renombrándolo (PHP-FPM solo carga los archivos que terminan en `.conf`), así podemos recuperarlo si hiciera falta.

Comando:

TERMINAL

```
sudo mv /etc/php/8.3/fpm/pool.d/www.conf /etc/php/8.3/fpm/pool.d/www.conf.desactivado
sudo php-fpm8.3 -t
sudo systemctl restart php8.3-fpm
ls -l /run/php/
```

Análisis del comando: `php-fpm8.3 -t` comprueba la sintaxis de toda la configuración de FPM (equivale a `apachectl configtest`).

Resultado esperado:

SALIDA ESPERADA

```
NOTICE: configuration file /etc/php/8.3/fpm/php-fpm.conf test is successful
srw-rw---- 1 www-data www-data 0 nov  8 23:40 php8.3-fpm-web1.sock
srw-rw---- 1 www-data www-data 0 nov  8 23:40 php8.3-fpm-web2.sock
-rw-r--r-- 1 root     root     5 nov  8 23:40 php8.3-fpm.pid
```

La `s` inicial indica que son sockets.

Errores habituales:

- `ERROR: [pool web1] cannot get uid for user 'web1'`: el usuario no existe (vuelve al PASO 3.1).
- `ERROR: No pool defined`: se desactivaron todos los pools; revisa que `web1.conf` y `web2.conf` terminan en `.conf`.
- Consulta los detalles con `sudo journalctl -u php8.3-fpm -n 30 --no-pager` y en `/var/log/php8.3-fpm.log`.

Siguiente paso: conectar Apache con los pools.

5.4 Conexión de Apache con PHP-FPM

PASO 5.6 – Activar los módulos de proxy FastCGI

Comando:

TERMINAL

```
sudo a2enmod proxy_fcgi setenvif
sudo systemctl restart apache2
```

Análisis del comando: `proxy_fcgi` (que activa también `proxy`) permite a Apache hablar el protocolo FastCGI con PHP-FPM; `setenvif` lo requiere para pasar correctamente ciertas cabeceras (como la de autorización).

> **CONSEJO:** No ejecutamos `a2enconf php8.3-fpm`: esa configuración activaría PHP para **todas** las webs a través de un pool común. Nosotros asignaremos un pool a cada VirtualHost.

PASO 5.7 – VirtualHost definitivo de ejemplo1.com con PHP

Objetivo: indicar en el VirtualHost que los archivos `.php` se envíen al pool `web1`.

Archivo: `/etc/apache2/sites-available/ejemplo1.com.conf` (sustituye todo el contenido; es la versión definitiva):

ARCHIVO DE CONFIGURACIÓN

```
# ------------------------------------------------------------
# ejemplo1.com - HTTP: solo redirige a HTTPS
# ------------------------------------------------------------
<VirtualHost *:80>
    ServerName www.ejemplo1.com
    ServerAlias ejemplo1.com
    ServerAdmin administracion@ejemplo1.com
    DocumentRoot /var/www/ejemplo1.com/public_html

    RewriteEngine On
    RewriteCond %{REQUEST_URI} !^/\.well-known/acme-challenge/
    RewriteRule ^ https://www.ejemplo1.com%{REQUEST_URI} [L,R=301]

    ErrorLog ${APACHE_LOG_DIR}/ejemplo1.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo1.com-access.log combined
</VirtualHost>

# ------------------------------------------------------------
# ejemplo1.com - HTTPS con PHP-FPM (pool web1)
# ------------------------------------------------------------
<VirtualHost *:443>
    ServerName www.ejemplo1.com
    ServerAlias ejemplo1.com
    ServerAdmin administracion@ejemplo1.com
    DocumentRoot /var/www/ejemplo1.com/public_html

    SSLEngine on
    SSLCertificateFile    /etc/letsencrypt/live/ejemplo1.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/ejemplo1.com/privkey.pem
    Protocols h2 http/1.1
```

```
    RewriteEngine On
    RewriteCond %{HTTP_HOST} !^www\.ejemplo1\.com$ [NC]
    RewriteRule ^ https://www.ejemplo1.com%{REQUEST_URI} [L,R=301]

    Header always set Strict-Transport-Security "max-age=15552000"

    # Página inicial: primero index.php, después index.html
    DirectoryIndex index.php index.html

    # Los archivos .php se ejecutan en el pool web1 (usuario web1)
    <FilesMatch "\.php$">
        SetHandler "proxy:unix:/run/php/php8.3-fpm-web1.sock|fcgi://web1"
    </FilesMatch>

    <Directory /var/www/ejemplo1.com/public_html>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/ejemplo1.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo1.com-access.log combined
</VirtualHost>
```

Análisis de las directivas nuevas:

Directiva	Significado
`DirectoryIndex index.php index.html`	Si se pide una carpeta (`/`), Apache busca primero `index.php`.
`<FilesMatch "\.php$">`	Se aplica a los archivos cuyo nombre termina en `.php`.
`SetHandler "proxy:unix:RUTA\`	fcgi://web1"`

PASO 5.8 – VirtualHost definitivo de ejemplo2.com con PHP

Archivo: `/etc/apache2/sites-available/ejemplo2.com.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
<VirtualHost *:80>
    ServerName www.ejemplo2.com
    ServerAlias ejemplo2.com
    ServerAdmin soporte@ejemplo2.com
    DocumentRoot /var/www/ejemplo2.com/public_html

    RewriteEngine On
    RewriteCond %{REQUEST_URI} !^/\.well-known/acme-challenge/
    RewriteRule ^ https://www.ejemplo2.com%{REQUEST_URI} [L,R=301]

    ErrorLog ${APACHE_LOG_DIR}/ejemplo2.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo2.com-access.log combined
</VirtualHost>

<VirtualHost *:443>
    ServerName www.ejemplo2.com
    ServerAlias ejemplo2.com
    ServerAdmin soporte@ejemplo2.com
```

```
    DocumentRoot /var/www/ejemplo2.com/public_html

    SSLEngine on
    SSLCertificateFile    /etc/letsencrypt/live/ejemplo2.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/ejemplo2.com/privkey.pem
    Protocols h2 http/1.1

    RewriteEngine On
    RewriteCond %{HTTP_HOST} !^www\.ejemplo2\.com$ [NC]
    RewriteRule ^ https://www.ejemplo2.com%{REQUEST_URI} [L,R=301]

    Header always set Strict-Transport-Security "max-age=15552000"

    DirectoryIndex index.php index.html

    <FilesMatch "\.php$">
        SetHandler "proxy:unix:/run/php/php8.3-fpm-web2.sock|fcgi://web2"
    </FilesMatch>

    <Directory /var/www/ejemplo2.com/public_html>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/ejemplo2.com-error.log
    CustomLog ${APACHE_LOG_DIR}/ejemplo2.com-access.log combined
</VirtualHost>
```

Aplica los cambios:

TERMINAL

```
sudo apachectl configtest && sudo systemctl reload apache2
```

PASO 5.9 – Comprobar que PHP se ejecuta con el usuario correcto

Objetivo: verificar que cada web ejecuta PHP en su propio pool.

Comando:

TERMINAL

```
echo '<?php echo "PHP " . PHP_VERSION . " ejecutado por: " . get_current_user() . " / proceso:
" . posix_getpwuid(posix_geteuid())["name"];' | sudo -u web1 tee
/var/www/ejemplo1.com/public_html/quien.php > /dev/null
echo '<?php echo "PHP " . PHP_VERSION . " ejecutado por: " . get_current_user() . " / proceso:
" . posix_getpwuid(posix_geteuid())["name"];' | sudo -u web2 tee
/var/www/ejemplo2.com/public_html/quien.php > /dev/null
curl -s https://www.ejemplo1.com/quien.php; echo
curl -s https://www.ejemplo2.com/quien.php; echo
```

Análisis del comando: `posix_geteuid()` devuelve el usuario efectivo del proceso PHP; `posix_getpwuid()` lo traduce a nombre.

Resultado esperado:

SALIDA ESPERADA

```
PHP 8.3.6 ejecutado por: web1 / proceso: web1
PHP 8.3.6 ejecutado por: web2 / proceso: web2
```

Comprobación adicional: borra los archivos de prueba (no dejes nunca scripts de diagnóstico publicados):

TERMINAL

```
sudo rm /var/www/ejemplo1.com/public_html/quien.php
/var/www/ejemplo2.com/public_html/quien.php
```

Errores habituales:

Síntoma	Causa	Solución
El navegador descarga el archivo o muestra el código PHP.	El `<FilesMatch>` no se aplica (falta en el bloque 443, o falta `proxy_fcgi`).	Revisa el VirtualHost y `apachectl -M \`
`503 Service Unavailable`.	El socket no existe: PHP-FPM está parado o la ruta no coincide.	`ls /run/php/` y `systemctl status php8.3-fpm`. En el error.log verás `AH01079: failed to make connection to backend`.
`Permission denied: AH02454` en el log.	Apache no puede usar el socket.	Revisa `listen.owner/group/mode` del pool.
`File not found.` (texto plano).	PHP-FPM no encuentra o no puede leer el script.	Permisos: el usuario del pool debe poder leer el archivo.
`open_basedir restriction in effect`.	El código intenta abrir un archivo fuera de la jaula.	Es la protección funcionando; ajusta la ruta del código, no amplíes la jaula sin motivo.

Siguiente paso: programar la aplicación.

5.5 Conexión de PHP con MariaDB

PHP tiene dos extensiones para MariaDB/MySQL: **mysqli** (específica de MySQL) y **PDO** (*PHP Data Objects*, una interfaz común para muchas bases de datos). Usaremos PDO porque su sintaxis de consultas preparadas es clara y porque el código se adapta fácilmente a otras bases de datos.

Consultas preparadas y SQL injection

La **inyección SQL** es uno de los ataques web más frecuentes. Ocurre cuando el código mete directamente en la consulta lo que escribe el usuario:

PHP

```
// ¡NUNCA HAGAS ESTO!
$email = $_GET['email'];
$sql = "SELECT * FROM contactos WHERE email = '$email'";
```

Si el atacante escribe como email `' OR '1'='1`, la consulta queda `... WHERE email = '' OR '1'='1'` y devuelve **todos** los registros. Con variantes más elaboradas puede leer otras tablas o borrar datos.

Las **consultas preparadas** separan el código SQL de los datos. Primero se envía la consulta con huecos (`:email`) y después, por separado, los valores. MariaDB nunca interpreta los valores como código:

PHP
```
// Correcto: consulta preparada
$stmt = $pdo->prepare('SELECT * FROM contactos WHERE email = :email');
$stmt->execute(['email' => $_GET['email']]);
```

ATENCIÓN: Regla sin excepciones: **todo dato que venga de fuera** (formularios, URL, cookies, cabeceras, APIs) va a la base de datos mediante consulta preparada, y **todo dato que se muestre en HTML** pasa por `htmlspecialchars()` para evitar ataques XSS (inyección de JavaScript).

Estructura de archivos de la aplicación

DIAGRAMA
```
/var/www/ejemplo1.com/
├── config/
│   └── config.php        ← credenciales (600, solo web1). NO accesible por URL
├── app/
│   └── db.php            ← función de conexión reutilizable (fuera de public_html)
├── public_html/
│   ├── index.php         ← agenda de contactos: listar, crear, editar, borrar
│   └── estilo.css        ← apariencia
├── logs/php-error.log    ← errores de PHP de esta web
└── tmp/                  ← sesiones de PHP
```

PASO 5.10 – Crear el archivo de credenciales protegido

Objetivo: guardar los datos de conexión fuera del DocumentRoot y legibles solo por `web1`.

Comando:

TERMINAL
```
sudo -u web1 nano /var/www/ejemplo1.com/config/config.php
```

Archivo: `/var/www/ejemplo1.com/config/config.php` (nuevo, contenido completo):

```php
<?php
// Configuración privada de ejemplo1.com
// Este archivo está FUERA de public_html y solo lo puede leer el usuario web1.
return [
    'db' => [
        'host'    => 'localhost',   // 'localhost' = socket Unix /run/mysqld/mysqld.sock
        'name'    => 'web1_db',
        'user'    => 'web1_user',
        'pass'    => '<CLAVE_WEB1>',
        'charset' => 'utf8mb4',
    ],
    'app' => [
        'nombre'          => 'ejemplo1.com',
        'correo_contacto' => 'info@ejemplo1.com',
    ],
];
```

Protege el archivo:

```terminal
sudo chmod 600 /var/www/ejemplo1.com/config/config.php
sudo ls -l /var/www/ejemplo1.com/config/
```

```
-rw------- 1 web1 web1 512 nov  9 00:05 config.php
```

Análisis: el archivo devuelve un *array* con `return`, de modo que quien lo incluya recibe los datos en una variable sin crear variables globales. Con `host = localhost`, PDO se conecta por el socket Unix, que es más rápido que TCP y no pasa por la red.

Comprobación: ni Apache ni `web2` pueden leerlo:

```terminal
sudo -u www-data cat /var/www/ejemplo1.com/config/config.php
sudo -u web2 cat /var/www/ejemplo1.com/config/config.php
```

Ambos deben responder `Permission denied`.

PASO 5.11 – Crear el archivo de conexión

Objetivo: centralizar la conexión a la base de datos en una función reutilizable.

Comando:

```terminal
sudo install -d -o web1 -g web1 -m 700 /var/www/ejemplo1.com/app
sudo -u web1 nano /var/www/ejemplo1.com/app/db.php
```

Archivo: `/var/www/ejemplo1.com/app/db.php` (nuevo, contenido completo):

```php
<?php
```

```
declare(strict_types=1);

/**
 * Devuelve una conexión PDO a la base de datos de esta web.
 * La conexión se crea una sola vez por petición y se reutiliza.
 */
function obtenerConexion(): PDO
{
    static $pdo = null;
    if ($pdo instanceof PDO) {
        return $pdo;
    }

    $config = require __DIR__ . '/../config/config.php';
    $db = $config['db'];

    $dsn = sprintf('mysql:host=%s;dbname=%s;charset=%s', $db['host'], $db['name'],
$db['charset']);

    $pdo = new PDO($dsn, $db['user'], $db['pass'], [
        // Los errores lanzan excepciones (no fallan en silencio)
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        // Los resultados se devuelven como arrays asociativos
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        // Consultas preparadas reales en el servidor (no emuladas)
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);

    return $pdo;
}

/** Escapa texto para mostrarlo en HTML de forma segura (evita XSS). */
function e(?string $texto): string
{
    return htmlspecialchars($texto ?? '', ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

Análisis del código:

Elemento	Significado
`declare(strict_types=1)`	PHP no convierte tipos automáticamente: detecta errores antes.
`static $pdo`	La variable conserva su valor entre llamadas: una sola conexión por petición.
`__DIR__`	La carpeta del archivo actual; así las rutas funcionan desde cualquier sitio.
DSN	Cadena de conexión: driver `mysql`, host, base de datos y juego de caracteres.
`ERRMODE_EXCEPTION`	Ante un error, PDO lanza una `PDOException` que podemos capturar.
`EMULATE_PREPARES => false`	Las consultas preparadas las hace MariaDB, no una simulación en PHP.
`htmlspecialchars`	Convierte `<`, `>`, `"`, `'` y `&` en entidades HTML.

TERMINAL

```
sudo chmod 600 /var/www/ejemplo1.com/app/db.php
```

PASO 5.12 – Página de prueba de conexión

Objetivo: verificar la conexión PHP → MariaDB antes de programar la aplicación completa.

Comando:

TERMINAL
```
sudo -u web1 nano /var/www/ejemplo1.com/public_html/prueba-bd.php
```

Archivo: `/var/www/ejemplo1.com/public_html/prueba-bd.php` (temporal):

PHP
```
<?php
declare(strict_types=1);
require __DIR__ . '/../app/db.php';

header('Content-Type: text/plain; charset=utf-8');
try {
    $pdo = obtenerConexion();
    $fila = $pdo->query('SELECT DATABASE() AS bd, CURRENT_USER() AS usuario, VERSION() AS
version')->fetch();
    echo "Conexión correcta\n";
    echo "Base de datos: {$fila['bd']}\n";
    echo "Usuario:       {$fila['usuario']}\n";
    echo "Servidor:      {$fila['version']}\n";
    $total = $pdo->query('SELECT COUNT(*) FROM contactos')->fetchColumn();
    echo "Contactos:     {$total}\n";
} catch (PDOException $e) {
    error_log('Prueba BD: ' . $e->getMessage());
    http_response_code(500);
    echo "Error de conexión. Consulta el log de PHP de la web.\n";
}
```

Comprobación:

TERMINAL
```
curl -s https://www.ejemplo1.com/prueba-bd.php
```

SALIDA ESPERADA
```
Conexión correcta
Base de datos: web1_db
Usuario:       web1_user@localhost
Servidor:      10.11.13-MariaDB-0ubuntu0.24.04.1
Contactos:     1
```

Errores habituales (consulta siempre `sudo tail /var/www/ejemplo1.com/logs/php-error.log`):

Mensaje en el log	Causa	Solución
`SQLSTATE[HY000] [1045] Access denied for user 'web1_user'@'localhost'`	Contraseña de `config.php` distinta a la de MariaDB.	Corrige `config.php` o cambia la clave con `ALTER USER` (PASO 4.11).
`SQLSTATE[HY000] [2002] No such file or directory`	MariaDB parado o socket en otra ruta.	`systemctl status mariadb`.
`SQLSTATE[HY000] [1049] Unknown database 'web1_db'`	Nombre de base incorrecto.	`SHOW DATABASES;`

Mensaje en el log	Causa	Solución
`could not find driver`	Falta `php-mysql`.	`sudo apt install php-mysql && sudo systemctl restart php8.3-fpm`.
`Failed opening required '.../config/config.php'` + `Permission denied`	El archivo no es de `web1`.	`sudo chown web1:web1` y `chmod 600`.

Cuando funcione, **bórrala**:

TERMINAL
```
sudo rm /var/www/ejemplo1.com/public_html/prueba-bd.php
```

5.6 Aplicación completa: agenda de contactos (CRUD)

CRUD son las cuatro operaciones básicas sobre datos: *Create* (crear, `INSERT`), *Read* (leer, `SELECT`), *Update* (actualizar, `UPDATE`) y *Delete* (borrar, `DELETE`). Nuestra aplicación las implementa todas con estas medidas de seguridad:

- Consultas preparadas en todas las operaciones.
- Escape HTML de toda la salida.
- Validación de datos en el servidor.
- Token **CSRF**: impide que otra web engañe al navegador del visitante para enviar formularios a la nuestra.
- Las modificaciones solo se aceptan por `POST`, y tras cada una se redirige (patrón *Post/Redirect/Get*) para que recargar la página no repita la acción.
- Los errores técnicos se registran en el log, nunca se muestran al visitante.

PASO 5.13 – Crear la aplicación index.php

Comando:

TERMINAL
```
sudo -u web1 nano /var/www/ejemplo1.com/public_html/index.php
```

Archivo: `/var/www/ejemplo1.com/public_html/index.php` (nuevo, contenido completo):

PHP
```
<?php
declare(strict_types=1);

require __DIR__ . '/../app/db.php';
$config = require __DIR__ . '/../config/config.php';

session_start();
if (empty($_SESSION['csrf'])) {
    $_SESSION['csrf'] = bin2hex(random_bytes(32));
}

$errores = [];
$aviso = '';
```

```
$editar = null;
$contactos = [];
$datos = ['nombre' => '', 'email' => '', 'telefono' => '', 'notas' => ''];

/** Valida los datos del formulario y devuelve la lista de errores. */
function validar(array $d): array
{
    $err = [];
    if ($d['nombre'] === '' || mb_strlen($d['nombre']) > 100) {
        $err[] = 'El nombre es obligatorio y debe tener como máximo 100 caracteres.';
    }
    if (!filter_var($d['email'], FILTER_VALIDATE_EMAIL) || mb_strlen($d['email']) > 150) {
        $err[] = 'El email no es válido.';
    }
    if (mb_strlen($d['telefono']) > 30) {
        $err[] = 'El teléfono debe tener como máximo 30 caracteres.';
    }
    return $err;
}

try {
    $pdo = obtenerConexion();

    // ---------- Operaciones de escritura (solo por POST) ----------
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        if (!hash_equals($_SESSION['csrf'], (string)($_POST['csrf'] ?? ''))) {
            http_response_code(400);
            exit('Petición no válida (token CSRF incorrecto). Recarga la página.');
        }

        $accion = (string)($_POST['accion'] ?? '');
        $id = filter_var($_POST['id'] ?? null, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);
        $datos = [
            'nombre'   => trim((string)($_POST['nombre'] ?? '')),
            'email'    => trim((string)($_POST['email'] ?? '')),
            'telefono' => trim((string)($_POST['telefono'] ?? '')),
            'notas'    => trim((string)($_POST['notas'] ?? '')),
        ];

        if ($accion === 'crear' || $accion === 'actualizar') {
            $errores = validar($datos);
        }

        if (!$errores) {
            switch ($accion) {
                case 'crear':      // CREATE
                    $stmt = $pdo->prepare(
                        'INSERT INTO contactos (nombre, email, telefono, notas)
                         VALUES (:nombre, :email, :telefono, :notas)'
                    );
                    $stmt->execute($datos);
                    header('Location: /?ok=creado');
                    exit;

                case 'actualizar': // UPDATE
                    if ($id === false) { throw new InvalidArgumentException('Id no válido'); }
                    $stmt = $pdo->prepare(
                        'UPDATE contactos
                            SET nombre = :nombre, email = :email, telefono = :telefono, notas = :notas
                          WHERE id = :id'
                    );
                    $stmt->execute($datos + ['id' => $id]);
                    header('Location: /?ok=actualizado');
```

```
                    exit;

                case 'eliminar':   // DELETE
                    if ($id === false) { throw new InvalidArgumentException('Id no válido'); }
                    $stmt = $pdo->prepare('DELETE FROM contactos WHERE id = :id');
                    $stmt->execute(['id' => $id]);
                    header('Location: /?ok=eliminado');
                    exit;

                default:
                    http_response_code(400);
                    exit('Acción desconocida.');
            }
        }
        if ($accion === 'actualizar' && $id !== false) {
            $editar = $datos + ['id' => $id];
        }
    }

    // ---------- Contacto a editar (READ de una fila) ----------
    if ($editar === null && isset($_GET['editar'])) {
        $idEditar = filter_var($_GET['editar'], FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);
        if ($idEditar !== false) {
            $stmt = $pdo->prepare('SELECT id, nombre, email, telefono, notas FROM contactos WHERE id =
:id');
            $stmt->execute(['id' => $idEditar]);
            $editar = $stmt->fetch() ?: null;
        }
    }

} catch (PDOException $ex) {
    // 1062 = clave duplicada (el email ya existe)
    if (($ex->errorInfo[1] ?? null) === 1062) {
        $errores[] = 'Ya existe un contacto con ese email.';
        if (isset($id) && $id !== false && ($accion ?? '') === 'actualizar') {
            $editar = $datos + ['id' => $id];
        }
    } else {
        error_log('Error de base de datos: ' . $ex->getMessage());
        $errores[] = 'Se ha producido un error interno. Inténtalo más tarde.';
    }
} catch (InvalidArgumentException $ex) {
    $errores[] = 'Datos no válidos.';
}

// ---------- Listado (READ de todas las filas) ----------
// Va en un bloque aparte para que el listado se muestre aunque falle una operación anterior.
try {
    $contactos = obtenerConexion()->query(
        'SELECT id, nombre, email, telefono, notas, creado_en FROM contactos ORDER BY creado_en DESC, id
DESC'
    )->fetchAll();
} catch (PDOException $ex) {
    error_log('Error al leer contactos: ' . $ex->getMessage());
    $errores[] = 'No se ha podido cargar el listado de contactos.';
}

$mensajesOk = ['creado' => 'Contacto creado.', 'actualizado' => 'Contacto actualizado.', 'eliminado' =>
'Contacto eliminado.'];
$aviso = $mensajesOk[$_GET['ok'] ?? ''] ?? '';
$form = $editar ?? $datos;
?>
<!DOCTYPE html>
<html lang="es">
```

```
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Agenda · <?= e($config['app']['nombre']) ?></title>
    <link rel="stylesheet" href="/estilo.css">
</head>
<body>
<main>
    <h1>Agenda de contactos · <?= e($config['app']['nombre']) ?></h1>

    <?php if ($aviso): ?><p class="ok"><?= e($aviso) ?></p><?php endif; ?>
    <?php foreach ($errores as $err): ?><p class="error"><?= e($err) ?></p><?php endforeach; ?>

    <h2><?= $editar ? 'Editar contacto' : 'Nuevo contacto' ?></h2>
    <form method="post" action="/">
        <input type="hidden" name="csrf" value="<?= e($_SESSION['csrf']) ?>">
        <input type="hidden" name="accion" value="<?= $editar ? 'actualizar' : 'crear' ?>">
        <?php if ($editar): ?><input type="hidden" name="id" value="<?= (int)$editar['id'] ?>"><?php
endif; ?>
        <label>Nombre <input name="nombre" required maxlength="100" value="<?= e($form['nombre'])
?>"></label>
        <label>Email <input name="email" type="email" required maxlength="150" value="<?=
e($form['email']) ?>"></label>
        <label>Teléfono <input name="telefono" maxlength="30" value="<?= e($form['telefono']) ?>"></label>
        <label>Notas <textarea name="notas" rows="3"><?= e($form['notas']) ?></textarea></label>
        <button type="submit"><?= $editar ? 'Guardar cambios' : 'Añadir contacto' ?></button>
        <?php if ($editar): ?><a href="/">Cancelar</a><?php endif; ?>
    </form>

    <h2>Contactos (<?= count($contactos) ?>)</h2>
    <table>

<thead><tr><th>Nombre</th><th>Email</th><th>Teléfono</th><th>Notas</th><th>Alta</th><th></th></tr></thead>
        <tbody>
        <?php foreach ($contactos as $c): ?>
            <tr>
                <td><?= e($c['nombre']) ?></td>
                <td><?= e($c['email']) ?></td>
                <td><?= e($c['telefono']) ?></td>
                <td><?= e($c['notas']) ?></td>
                <td><?= e($c['creado_en']) ?></td>
                <td class="acciones">
                    <a href="/?editar=<?= (int)$c['id'] ?>">Editar</a>
                    <form method="post" action="/" onsubmit="return confirm('¿Eliminar este contacto?');">
                        <input type="hidden" name="csrf" value="<?= e($_SESSION['csrf']) ?>">
                        <input type="hidden" name="accion" value="eliminar">
                        <input type="hidden" name="id" value="<?= (int)$c['id'] ?>">
                        <button type="submit" class="peligro">Eliminar</button>
                    </form>
                </td>
            </tr>
        <?php endforeach; ?>
        </tbody>
    </table>
</main>
</body>
</html>
```

Análisis del código por bloques:

Bloque	Qué hace
`session_start()` y `$_SESSION['csrf']`	Crea una sesión (guardada en `tmp/`) con un token aleatorio de 64 caracteres. Cada formulario lo envía y el servidor lo compara con `hash_equals()` (comparación resistente a ataques de tiempo).
`validar()`	Comprueba longitudes y formato del email con `filter_var`. Validar en el servidor es obligatorio: los controles del navegador (`required`, `type="email"`) se pueden saltar.
`INSERT ... VALUES (:nombre, ...)` + `execute($datos)`	**Create** con consulta preparada. Las claves del array coinciden con los marcadores.
`UPDATE ... WHERE id = :id`	**Update**. El `WHERE` limita el cambio a un único contacto.
`DELETE ... WHERE id = :id`	**Delete**. Llega por `POST` con token CSRF, nunca por un enlace `GET`.
`SELECT ... WHERE id = :id`	**Read** de una fila para rellenar el formulario de edición.
`SELECT ... ORDER BY creado_en DESC`	**Read** de todas las filas. No tiene datos externos, así que `query()` es seguro.
`header('Location: /?ok=creado')`	Patrón Post/Redirect/Get.
`catch (PDOException $ex)`	El código 1062 (duplicado) se traduce a un mensaje amable; cualquier otro error se registra con `error_log()` y el usuario ve un texto genérico.
`(int)$c['id']` y `e(...)`	Toda salida se escapa o se fuerza a número.

PASO 5.14 – Añadir la hoja de estilos

Archivo: `/var/www/ejemplo1.com/public_html/estilo.css` (nuevo, contenido completo):

```css
body { font-family: system-ui, sans-serif; margin: 0; background: #f4f6f9; color: #1f2933; }
main { max-width: 980px; margin: 2rem auto; background: #fff; padding: 2rem; border-radius:
8px; box-shadow: 0 2px 8px rgba(0,0,0,.08); }
h1 { margin-top: 0; color: #1f3864; }
label { display: block; margin: .6rem 0; font-weight: 600; }
input, textarea { display: block; width: 100%; padding: .5rem; margin-top: .25rem; border: 1px
solid #c3ccd6; border-radius: 4px; box-sizing: border-box; font: inherit; }
button { background: #2e75b6; color: #fff; border: 0; padding: .55rem 1rem; border-radius:
4px; cursor: pointer; }
button.peligro { background: #c0392b; padding: .3rem .6rem; }
table { width: 100%; border-collapse: collapse; margin-top: 1rem; font-size: .92rem; }
th, td { text-align: left; padding: .5rem; border-bottom: 1px solid #e3e8ee; vertical-align:
top; }
th { background: #1f3864; color: #fff; }
.acciones { white-space: nowrap; }
.acciones form { display: inline; }
.ok { background: #eaf5ea; border-left: 4px solid #538135; padding: .6rem; }
.error { background: #fdf0e6; border-left: 4px solid #c55a11; padding: .6rem; }
```

Fija los permisos de los archivos públicos y elimina la página de bienvenida estática (ya no se usa, porque `index.php` tiene prioridad):

TERMINAL

```
sudo chmod 640 /var/www/ejemplo1.com/public_html/index.php
/var/www/ejemplo1.com/public_html/estilo.css
sudo rm -f /var/www/ejemplo1.com/public_html/index.html
sudo ls -l /var/www/ejemplo1.com/public_html/
```

PASO 5.15 – Probar la aplicación desde el navegador

Objetivo: verificar las cuatro operaciones CRUD.

Comprobación: abre `https://www.ejemplo1.com` y realiza esta secuencia:

1. Debe aparecer la agenda con el contacto «Ana Pérez» creado en el Módulo 4 (**Read**).
2. Añade un contacto nuevo (**Create**). Debe aparecer el aviso «Contacto creado.»
3. Intenta añadir otro con el mismo email. Debe aparecer «Ya existe un contacto con ese email.»
4. Pulsa «Editar», cambia el teléfono y guarda (**Update**).
5. Pulsa «Eliminar» y confirma (**Delete**).

Comprueba en la base de datos que los cambios son reales:

TERMINAL

```
sudo mariadb -e "SELECT id, nombre, email, telefono FROM web1_db.contactos;"
```

Prueba también la protección frente a inyección SQL: en el campo email del formulario de edición la validación rechazará `' OR '1'='1`; y aunque un atacante saltara la validación, la consulta preparada trataría ese texto como un valor literal.

Errores habituales: si la página aparece en blanco o con «error interno», mira `sudo tail -n 20 /var/www/ejemplo1.com/logs/php-error.log`. Un error de sintaxis aparecerá como `PHP Parse error: syntax error, unexpected ... in /var/www/ejemplo1.com/public_html/index.php on line N`.

PASO 5.16 – Desplegar la misma aplicación en ejemplo2.com

Objetivo: tener la aplicación en la segunda web, conectada a **su** base de datos.

Comando: copiamos el código y creamos una configuración propia:

TERMINAL

```
sudo install -d -o web2 -g web2 -m 700 /var/www/ejemplo2.com/app
sudo install -o web2 -g web2 -m 600 /var/www/ejemplo1.com/app/db.php
/var/www/ejemplo2.com/app/db.php
sudo install -o web2 -g www-data -m 640 /var/www/ejemplo1.com/public_html/index.php
/var/www/ejemplo2.com/public_html/index.php
sudo install -o web2 -g www-data -m 640 /var/www/ejemplo1.com/public_html/estilo.css
/var/www/ejemplo2.com/public_html/estilo.css
sudo rm -f /var/www/ejemplo2.com/public_html/index.html
sudo -u web2 nano /var/www/ejemplo2.com/config/config.php
```

Análisis del comando: `install` copia el archivo y fija propietario, grupo y permisos en una sola orden (un `cp` normal dejaría a root como propietario).

Archivo: `/var/www/ejemplo2.com/config/config.php` (nuevo, contenido completo):

PHP

```php
<?php
// Configuración privada de ejemplo2.com
return [
    'db' => [
        'host'    => 'localhost',
        'name'    => 'web2_db',
        'user'    => 'web2_user',
        'pass'    => '<CLAVE_WEB2>',
        'charset' => 'utf8mb4',
    ],
    'app' => [
        'nombre'          => 'ejemplo2.com',
        'correo_contacto' => 'soporte@ejemplo2.com',
    ],
];
```

TERMINAL

```
sudo chmod 600 /var/www/ejemplo2.com/config/config.php
```

Comprobación: abre `https://www.ejemplo2.com`. Debe mostrar «Agenda de contactos · ejemplo2.com» con el contacto «Luis Gómez». Los contactos que añadas en una web **no** aparecen en la otra: cada una tiene su propia base de datos, su propio usuario de base de datos y su propio usuario Linux.

5.7 Protección de credenciales: resumen

Medida	Dónde se aplica
Credenciales fuera del DocumentRoot	`config/config.php`
Archivo legible solo por el usuario de la web	`chmod 600`, propietario `web1`
Un usuario de base de datos por web, con privilegios mínimos	`GRANT SELECT, INSERT, UPDATE, DELETE`
Usuario de base de datos limitado a `localhost`	`'web1_user'@'localhost'`
MariaDB no escucha en Internet	`bind-address = 127.0.0.1`
Errores técnicos nunca visibles	`display_errors = off`, `error_log()`
Jaula de archivos de PHP	`open_basedir`
No guardar el archivo de configuración en repositorios públicos (Git)	Añade `config/` a `.gitignore`

5.8 Ejercicios prácticos del Módulo 5

1. Añade a la agenda un buscador por nombre usando `SELECT ... WHERE nombre LIKE :q` con el valor `'%' . $busqueda . '%'` pasado como parámetro.

2. Escribe en `/var/www/ejemplo1.com/public_html/intrusion.php` un script que intente leer `/var/www/ejemplo2.com/config/config.php`. Ejecútalo desde el navegador, observa el error de `open_basedir` en el log y borra el script.
3. Cambia `pm.max_children` del pool `web2` a 2, abre muchas pestañas a la vez y busca en `/var/log/php8.3-fpm.log` el aviso `server reached pm.max_children setting`.
4. Rompe a propósito la contraseña en `config.php` de ejemplo2 y documenta qué ve el visitante y qué aparece en el log. Restáurala.
5. Reescribe la consulta de borrado usando `mysqli` con `prepare()`, `bind_param()` y `execute()` para comparar ambas extensiones.

Módulo 6. Instalación y configuración de Postfix

Objetivos del módulo

- Comprender cómo viaja un correo electrónico por Internet y qué papel cumple cada componente.
- Distinguir entre recepción (puerto 25), envío autenticado (587 y 465) y relay.
- Instalar Postfix y entender sus archivos y parámetros principales: `myhostname`, `mydomain`, `myorigin`, `mydestination`, `mynetworks`, `inet_interfaces`.
- Configurar TLS y restricciones antispam básicas.
- Enviar correos de prueba, leer los logs y gestionar la cola de correo.

6.1 Qué es SMTP y cómo viaja un correo

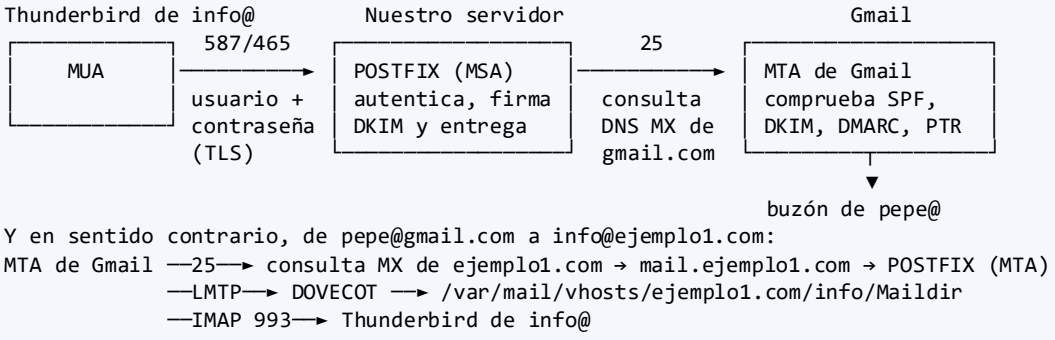
SMTP (*Simple Mail Transfer Protocol*) es el protocolo con el que se **envía** y se **transporta** el correo entre servidores. No sirve para leer el correo: para eso existen IMAP y POP3 (Módulo 7).

En el correo intervienen varios componentes con nombres propios:

Sigla	Nombre	Función	En nuestro servidor
MUA	*Mail User Agent*	Programa del usuario para leer y escribir correo.	Thunderbird, Outlook, el móvil, la app PHP.
MSA	*Mail Submission Agent*	Recibe el correo que envían **tus usuarios autenticados**.	Postfix en los puertos 587 y 465.
MTA	*Mail Transfer Agent*	Transporta el correo de servidor a servidor.	Postfix en el puerto 25.
MDA	*Mail Delivery Agent*	Deposita el correo en el buzón del destinatario.	Dovecot LMTP (Módulo 7).
MAA	*Mail Access Agent*	Permite al usuario consultar su buzón.	Dovecot IMAP/POP3.

El viaje completo de un correo de `info@ejemplo1.com` a `pepe@gmail.com`:

DIAGRAMA



Una conversación SMTP real es texto plano. Así recibe nuestro servidor un mensaje (C = cliente, S = servidor):

TEXTO

```
S: 220 mail.ejemplo1.com ESMTP
C: EHLO mail-ed1-f45.google.com
S: 250-mail.ejemplo1.com
S: 250-STARTTLS
S: 250 SMTPUTF8
C: MAIL FROM:<pepe@gmail.com>
S: 250 2.1.0 Ok
C: RCPT TO:<info@ejemplo1.com>
S: 250 2.1.5 Ok
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: From: Pepe <pepe@gmail.com>
C: To: info@ejemplo1.com
C: Subject: Hola
C:
C: Texto del mensaje.
C: .
S: 250 2.0.0 Ok: queued as 4ZK3t21XyPz5
C: QUIT
```

CONSEJO: Fíjate en que hay dos remitentes: el del **sobre** (`MAIL FROM`, también llamado *envelope sender* o *Return-Path*), que usan los servidores para devolver errores y comprobar SPF; y el de la **cabecera** (`From:`), que es el que ve el usuario y el que comprueba DMARC. Entender esta diferencia es clave en el Módulo 9.

6.2 Los puertos SMTP: recepción, envío y envío autenticado

Puerto	Nombre	Quién se conecta	Autenticación	Cifrado	Uso
25	SMTP	Otros servidores de correo de Internet.	No (los servidores no tienen contraseña en el tuyo).	Opcional con STARTTLS.	**Recepción**: correo que llega a tus dominios. Postfix solo acepta mensajes **para tus dominios**; rechaza todo lo demás (evita ser un *relay abierto*).
587	Submission	Tus usuarios (Thunderbird, móvil, apps).	**Obligatoria** (usuario y contraseña).	Obligatorio con STARTTLS.	**Envío autenticado**: tus usuarios envían correo a cualquier destino.
465	SMTPS (Submissions)	Tus usuarios.	**Obligatoria**.	Obligatorio, TLS implícito desde el primer byte.	Igual que 587, pero con TLS directo. RFC 8314 lo recomienda; ambos son válidos.

Conceptos:

- **STARTTLS**: la conexión empieza en texto plano y el cliente pide pasar a cifrado con el comando `STARTTLS`.
- **TLS implícito**: la conexión es cifrada desde el principio (como HTTPS).
- **Relay abierto**: un servidor que acepta y reenvía correo de cualquiera a cualquiera. Los spammers los buscan constantemente; si tu servidor lo fuera, en horas estaría en todas las listas negras. La regla de Postfix será: «acepto correo **para** mis dominios de cualquiera; acepto correo **hacia** cualquier destino solo de usuarios autenticados o del propio servidor».

> **ATENCIÓN:** Tus usuarios **nunca** deben enviar por el puerto 25. Muchos operadores de acceso a Internet bloquean el 25 saliente de sus clientes domésticos precisamente para frenar el spam de equipos infectados. Los clientes de correo usan siempre 587 o 465.

PASO 6.1 – Comprobar si tu proveedor permite el puerto 25 saliente

Objetivo: saber si tu servidor podrá entregar correo a otros servidores.

Comando:

TERMINAL

```
nc -vz -w 5 gmail-smtp-in.l.google.com 25
```

Análisis del comando: `nc` (netcat) abre una conexión; `-v` muestra detalles, `-z` solo comprueba si el puerto responde sin enviar datos, `-w 5` espera como máximo 5 segundos. `gmail-smtp-in.l.google.com` es uno de los servidores de recepción de Gmail.

Resultado esperado:

SALIDA ESPERADA

```
Connection to gmail-smtp-in.l.google.com (142.250.153.27) 25 port [tcp/smtp] succeeded!
```

Errores habituales: si obtienes `Connection timed out`, tu proveedor bloquea el 25 saliente. Opciones: solicitar el desbloqueo (muchos lo hacen tras verificar tu cuenta), cambiar de proveedor o configurar un *relayhost* (sección 6.7). Podrás seguir el manual igualmente: la recepción no depende de esto.

Siguiente paso: obtener el certificado del servidor de correo.

PASO 6.2 – Obtener el certificado TLS para mail.ejemplo1.com y mail.ejemplo2.com

Objetivo: tener un certificado válido que Postfix y Dovecot presentarán a clientes y servidores.

Explicación: los clientes de correo de ejemplo1 se conectarán a `mail.ejemplo1.com` y los de ejemplo2 a `mail.ejemplo2.com`. Un único certificado con ambos nombres sirve para los dos. Lo validamos a través del sitio comodín de Apache (PASO 3.4), cuyo DocumentRoot es `/var/www/html`.

Comando:

TERMINAL

```
sudo certbot certonly --webroot -w /var/www/html -d mail.ejemplo1.com -d mail.ejemplo2.com --
email administracion@ejemplo1.com --agree-tos --no-eff-email
```

Resultado esperado:

SALIDA ESPERADA

```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/mail.ejemplo1.com/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/mail.ejemplo1.com/privkey.pem
```

Comprobación: `sudo certbot certificates` debe mostrar un tercer certificado `mail.ejemplo1.com` con los dominios `mail.ejemplo1.com mail.ejemplo2.com`.

> CONSEJO: **Laboratorio sin dominio real:** usa el certificado autofirmado. En los archivos de Postfix y Dovecot de este manual sustituye `/etc/letsencrypt/live/mail.ejemplo1.com/fullchain.pem` por `/etc/ssl/certs/ssl-cert-snakeoil.pem` y `/etc/letsencrypt/live/mail.ejemplo1.com/privkey.pem` por `/etc/ssl/private/ssl-cert-snakeoil.key`. Los clientes de correo te pedirán aceptar una excepción de seguridad.

Siguiente paso: instalar Postfix.

6.3 Instalación de Postfix

PASO 6.3 – Instalar Postfix

Objetivo: instalar el servidor SMTP y una utilidad para enviar correos desde la consola.

Comando:

TERMINAL
```
sudo apt install -y postfix bsd-mailx
```

Análisis del comando: `postfix` es el servidor; `bsd-mailx` aporta el comando `mail` para enviar y leer correo desde el terminal.

Durante la instalación aparece un asistente de configuración (debconf):

Pantalla	Respuesta	Explicación
General type of mail configuration	**Internet Site**	El servidor envía y recibe directamente con SMTP.
System mail name	**ejemplo1.com**	Dominio que se añade al correo generado localmente (se guarda en `/etc/mailname`).

Si se abre alguna pantalla más (destinos, redes...), acepta el valor por defecto: sustituiremos la configuración completa en el paso siguiente. Si te equivocas, puedes repetir el asistente con `sudo dpkg-reconfigure postfix`.

Resultado esperado: termina con `Postfix (main.cf) is now set up with a default configuration.`

Comprobación:

TERMINAL
```
postconf mail_version
systemctl status postfix@- --no-pager
```

SALIDA ESPERADA
```
mail_version = 3.8.6
● postfix@-.service - Postfix Mail Transport Agent (instance -)
     Active: active (running) since ...
```

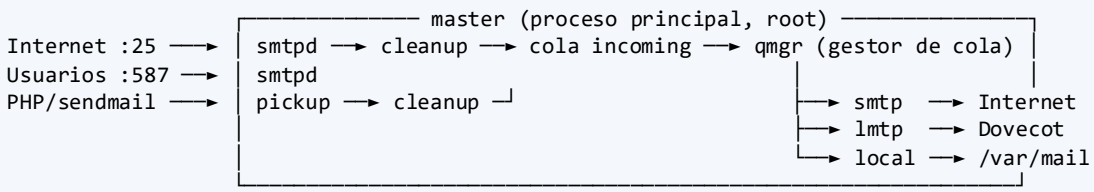
> **CONSEJO:** En Ubuntu, `postfix.service` es solo un «envoltorio» que arranca las instancias. La instancia real se llama `postfix@-.service`. Para reiniciar puedes usar indistintamente `systemctl restart postfix`, pero para ver el estado detallado y los errores usa `postfix@-`.

Siguiente paso: conocer la estructura de Postfix.

6.4 Estructura y arquitectura de Postfix

Postfix no es un único programa, sino un conjunto de pequeños procesos especializados coordinados por un proceso **master**. Este diseño lo hace rápido y seguro: cada proceso tiene los privilegios justos y muchos se ejecutan encerrados (*chroot*) en `/var/spool/postfix`.

DIAGRAMA



Archivo o carpeta	Contenido
`/etc/postfix/main.cf`	Configuración principal: parámetros `nombre = valor`.
`/etc/postfix/master.cf`	Qué servicios arrancan (smtpd en 25, 587, 465...) y con qué opciones.
`/etc/aliases`	Alias de las cuentas locales del sistema (root, postmaster...).
`/etc/mailname`	Dominio de correo del sistema.
`/var/spool/postfix/`	Colas de correo y sockets (`private/`, `public/`).
`/var/log/mail.log`	Log de todo el correo (lo escribe rsyslog).

Herramientas de administración:

Comando	Qué hace
`postconf -n`	Muestra solo los parámetros que **tú** has cambiado respecto a los valores por defecto.
`postconf -d parametro`	Muestra el valor por defecto de un parámetro.
`postconf parametro`	Muestra el valor actual.
`sudo postconf -e 'param = valor'`	Modifica un parámetro en `main.cf`.
`sudo postfix check`	Revisa configuración, permisos y archivos.
`sudo postmap archivo`	Compila un archivo de tabla a formato `.db` (lo verás en el Módulo 8).
`sudo newaliases`	Compila `/etc/aliases`.
`mailq` o `postqueue -p`	Muestra la cola de correo pendiente.
`sudo postqueue -f`	Fuerza el reintento inmediato de toda la cola.
`sudo postsuper -d ID`	Borra un mensaje de la cola (`-d ALL` borra todos).

Comando	Qué hace
`sudo postcat -q ID`	Muestra el contenido de un mensaje en cola.

6.5 Los parámetros fundamentales

Parámetro	Significado	Valor en nuestro servidor
`myhostname`	Nombre completo (FQDN) del servidor. Se usa en el saludo SMTP (`220 mail.ejemplo1.com`) y en el `EHLO` al enviar. **Debe** coincidir con el registro A y con el PTR.	`mail.ejemplo1.com`
`mydomain`	Dominio del servidor. Por defecto es `myhostname` sin la primera parte.	`ejemplo1.com`
`myorigin`	Dominio que se añade a los correos generados localmente sin dominio (por ejemplo, el que envía `cron` como `root`).	`$mydomain` → `root@ejemplo1.com`
`mydestination`	Dominios cuyos destinatarios son **cuentas del sistema Linux** (entrega local en `/var/mail/usuario`).	`$myhostname, localhost.$mydomain, localhost`
`mynetworks`	Redes «de confianza» que pueden enviar a cualquier destino sin autenticarse.	Solo el propio servidor: `127.0.0.0/8` y `[::1]/128`.
`inet_interfaces`	En qué interfaces de red escucha Postfix.	`all` (necesario para recibir de Internet).
`inet_protocols`	IPv4, IPv6 o ambos.	`all` (o `ipv4` si tu servidor no tiene IPv6 funcional).
`relayhost`	Servidor intermediario por el que enviar todo el correo saliente.	Vacío (entrega directa).

> **ATENCIÓN: Muy importante:** `ejemplo1.com` y `ejemplo2.com` **no** se ponen en `mydestination`. Esos dominios tendrán buzones **virtuales** (Módulo 8), gestionados por Dovecot y desvinculados de las cuentas Linux. Un dominio nunca debe aparecer a la vez en `mydestination` y en `virtual_mailbox_domains`. En `mydestination` solo dejamos el nombre del propio servidor, para el correo administrativo del sistema.

Tipos de dominio que Postfix sabe gestionar:

Clase de dominio	Parámetro	Buzones	Uso
Local	`mydestination`	Cuentas Linux (`/var/mail/usuario` o `~/Maildir`).	Correo del sistema (root, cron).
Alias virtual	`virtual_alias_domains`	Ninguno: todo se reenvía a otras direcciones.	Dominios que solo redirigen.
Buzón virtual	`virtual_mailbox_domains`	Buzones independientes de las cuentas Linux.	**Nuestros dominios de clientes.**
Relay	`relay_domains`	En otro servidor.	Servidor de respaldo (MX secundario).

6.6 Configuración de Postfix

PASO 6.4 – Escribir la configuración principal main.cf

Objetivo: dejar `main.cf` con una configuración limpia, comentada y segura.

Explicación: en lugar de modificar parámetro a parámetro, sustituimos el archivo por una versión completa que entiendes línea por línea. Primero guardamos una copia del original.

Comando:

TERMINAL

```
sudo cp /etc/postfix/main.cf /etc/postfix/main.cf.original
sudo nano /etc/postfix/main.cf
```

Archivo: `/etc/postfix/main.cf` (sustituye todo el contenido; versión del Módulo 6):

ARCHIVO DE CONFIGURACIÓN

```
# ================================================================
#  /etc/postfix/main.cf  -  mail.ejemplo1.com
#  Versión del Módulo 6: SMTP básico con TLS.
#  (En los módulos 7, 8 y 9 se añadirán Dovecot, buzones virtuales y DKIM)
# ================================================================

# --- Generales ------------------------------------------------------
compatibility_level = 3.6
smtpd_banner = $myhostname ESMTP
biff = no
append_dot_mydomain = no
readme_directory = no

# --- Identidad del servidor -----------------------------------------
myhostname = mail.ejemplo1.com
mydomain = ejemplo1.com
myorigin = $mydomain
# Solo el propio servidor es destino LOCAL (cuentas del sistema)
mydestination = $myhostname, localhost.$mydomain, localhost
# Redes de confianza: solo el propio servidor
mynetworks = 127.0.0.0/8 [::ffff:127.0.0.0]/104 [::1]/128
inet_interfaces = all
inet_protocols = all
relayhost =

# --- Alias y límites ------------------------------------------------
alias_maps = hash:/etc/aliases
alias_database = hash:/etc/aliases
recipient_delimiter = +
mailbox_size_limit = 0
# Tamaño máximo de un mensaje: 50 MB
message_size_limit = 52428800

# --- TLS cuando otros se conectan a nosotros (smtpd) ----------------
smtpd_tls_cert_file = /etc/letsencrypt/live/mail.ejemplo1.com/fullchain.pem
smtpd_tls_key_file = /etc/letsencrypt/live/mail.ejemplo1.com/privkey.pem
smtpd_tls_security_level = may
smtpd_tls_auth_only = yes
smtpd_tls_protocols = >=TLSv1.2
smtpd_tls_mandatory_protocols = >=TLSv1.2
```

```
smtpd_tls_loglevel = 1
smtpd_tls_session_cache_database = btree:${data_directory}/smtpd_scache

# --- TLS cuando nosotros enviamos a otros (smtp) ----------------------
smtp_tls_security_level = may
smtp_tls_CApath = /etc/ssl/certs
smtp_tls_loglevel = 1
smtp_tls_session_cache_database = btree:${data_directory}/smtp_scache

# --- Restricciones (antispam y anti relay abierto) --------------------
smtpd_helo_required = yes
disable_vrfy_command = yes
smtpd_helo_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_invalid_helo_hostname,
    reject_non_fqdn_helo_hostname
smtpd_sender_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_non_fqdn_sender,
    reject_unknown_sender_domain
smtpd_relay_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    defer_unauth_destination
smtpd_recipient_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_unauth_destination,
    reject_non_fqdn_recipient,
    reject_unknown_recipient_domain
```

Análisis por bloques:

Parámetro	Significado
`compatibility_level = 3.6`	Activa los valores por defecto seguros de Postfix 3.6 en adelante.
`smtpd_banner = $myhostname ESMTP`	Saludo del servidor. No revela la versión de Postfix ni el sistema operativo.
`biff = no`	Desactiva un antiguo servicio de notificación local.
`append_dot_mydomain = no`	No completa direcciones incompletas (es tarea del cliente).
`recipient_delimiter = +`	Permite direcciones como `info+facturas@ejemplo1.com`, que se entregan a `info`.
`smtpd_tls_security_level = may`	En el puerto 25, TLS se ofrece pero no se exige (si lo exigiéramos, rechazaríamos correo de servidores antiguos).
`smtpd_tls_auth_only = yes`	Nunca se aceptan contraseñas sin cifrar.
`smtpd_tls_protocols = >=TLSv1.2`	Solo TLS 1.2 y 1.3.
`smtp_tls_security_level = may`	Al enviar, cifra si el destino lo admite.
`smtp_tls_CApath`	Autoridades de confianza para verificar los certificados de otros servidores.
`smtpd_helo_required`	El cliente debe presentarse con HELO/EHLO.

Parámetro	Significado
`disable_vrfy_command`	Desactiva VRFY, que permitiría a un spammer averiguar qué direcciones existen.
`reject_invalid_helo_hostname` / `reject_non_fqdn_helo_hostname`	Rechaza clientes que se presentan con nombres mal formados. Los servidores legítimos usan un FQDN.
`reject_non_fqdn_sender` / `reject_unknown_sender_domain`	Rechaza remitentes con dominios incompletos o inexistentes en el DNS.
`smtpd_relay_restrictions`	La regla anti relay abierto: permite enviar a cualquier destino solo a `mynetworks` y a usuarios autenticados (SASL); al resto solo les deja entregar a nuestros dominios.
`reject_unauth_destination`	Rechaza destinatarios que no son nuestros si el cliente no está autorizado.
`permit_sasl_authenticated`	Autoriza a usuarios autenticados. Aún no tenemos SASL (llega en el Módulo 7), pero dejarlo no hace daño.

> **CONSEJO:** Las restricciones se evalúan **en orden** y la primera que da un veredicto (`permit` o `reject`) gana. Por eso `permit_mynetworks` y `permit_sasl_authenticated` van siempre primero: tus usuarios no deben sufrir los filtros antispam pensados para desconocidos.

Comprobación:

TERMINAL
```
sudo postfix check
postconf -n | head -20
```

`postfix check` no debe mostrar nada. `postconf -n` debe reflejar tus valores.

Errores habituales: `postconf: warning: /etc/postfix/main.cf, line N: missing '=' after attribute name` indica una línea mal escrita. Recuerda: las líneas de continuación (como los elementos de las restricciones) **deben empezar con espacios**; una línea que empieza sin espacio se interpreta como un parámetro nuevo.

Siguiente paso: los alias del sistema.

PASO 6.5 – Configurar los alias del sistema

Objetivo: que el correo dirigido a `root` y a `postmaster` del servidor llegue al administrador.

Explicación: los servicios del sistema (cron, actualizaciones automáticas, Fail2ban, mdadm...) envían avisos a `root`. Nadie lee el buzón de root: los redirigimos al usuario `sysadmin`. En el Módulo 8 cambiaremos este destino por una dirección virtual real.

Archivo: `/etc/aliases` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN
```
# Alias del sistema: formato "alias: destino"
# Después de modificar este archivo ejecuta: sudo newaliases
postmaster:    root
abuse:         root
webmaster:     root
www-data:      root
root:          sysadmin
```

Comando:

```
TERMINAL
sudo newaliases
sudo systemctl restart postfix
```

Análisis del comando: `newaliases` compila `/etc/aliases` en `/etc/aliases.db`, el formato que lee Postfix. Si editas el archivo y olvidas este comando, los cambios no se aplican.

Siguiente paso: abrir el puerto 25 y probar.

PASO 6.6 – Abrir el puerto 25 en el cortafuegos

Comando:

```
TERMINAL
sudo ufw allow 25/tcp comment 'SMTP recepcion'
sudo ss -tlnp | grep ':25 '
```

Resultado esperado:

```
SALIDA ESPERADA
Rule added
Rule added (v6)
LISTEN 0  100   0.0.0.0:25   0.0.0.0:*  users:(("master",pid=6012,fd=13))
LISTEN 0  100      [::]:25      [::]:*  users:(("master",pid=6012,fd=14))
```

Postfix escucha en todas las interfaces (`0.0.0.0` y `[::]`) en el puerto 25.

6.7 Pruebas, logs y cola de correo

PASO 6.7 – Enviar un correo local de prueba

Objetivo: comprobar la entrega local.

Comando:

```
TERMINAL
echo "Este es el cuerpo del mensaje de prueba." | mail -s "Prueba local 1" root
sudo tail -n 15 /var/log/mail.log
```

Análisis del comando: `mail -s "asunto" destinatario` envía lo que recibe por la entrada estándar. Como `root` es alias de `sysadmin`, el mensaje debe llegar al buzón de `sysadmin`.

Resultado esperado en el log (cada línea comparte un **ID de cola**, aquí `5B2A13E0F1`, que te permite seguir el mensaje):

SALIDA ESPERADA

```
postfix/pickup[6100]: 5B2A13E0F1: uid=1000 from=<sysadmin>
postfix/cleanup[6170]: 5B2A13E0F1: message-id=<20261109001501.5B2A13E0F1@mail.ejemplo1.com>
postfix/qmgr[6101]: 5B2A13E0F1: from=<sysadmin@ejemplo1.com>, size=412, nrcpt=1 (queue active)
postfix/local[6172]: 5B2A13E0F1: to=<sysadmin@mail.ejemplo1.com>, orig_to=<root>, relay=local,
delay=0.03, status=sent (delivered to mailbox)
postfix/qmgr[6101]: 5B2A13E0F1: removed
```

Comprobación: lee el correo con el comando `mail` (como `sysadmin`, sin sudo):

TERMINAL

```
mail
```

Verás la lista de mensajes; escribe el número para leerlo y `q` para salir. El buzón local está en `/var/mail/sysadmin`.

Errores habituales: `status=bounced (unknown user: "root")` indica que no ejecutaste `newaliases`.

PASO 6.8 – Enviar un correo a una dirección externa

Objetivo: comprobar que el servidor entrega correo en Internet.

Comando: usa una dirección tuya de Gmail, Outlook u otro proveedor:

TERMINAL

```
echo "Prueba de envío desde mail.ejemplo1.com" | mail -s "Prueba externa" -r
sysadmin@mail.ejemplo1.com tu-direccion@gmail.com
sudo tail -f /var/log/mail.log
```

Análisis del comando: `-r` fija el remitente del sobre. Usamos `sysadmin@mail.ejemplo1.com` porque es una dirección local que sí existe y puede recibir respuestas.

Resultado esperado en el log:

SALIDA ESPERADA

```
postfix/smtp[6230]: Trusted TLS connection established to gmail-smtp-
in.l.google.com[142.250.153.27]:25: TLSv1.3 with cipher TLS_AES_256_GCM_SHA384 (256/256 bits)
postfix/smtp[6230]: 8C1D23E0F4: to=<tu-direccion@gmail.com>, relay=gmail-smtp-
in.l.google.com[142.250.153.27]:25, delay=1.2, dsn=2.0.0, status=sent (250 2.0.0 OK
1762647301 ...)
```

`status=sent` con `250 2.0.0 OK` significa que Gmail **aceptó** el mensaje. Pulsa Ctrl+C para salir.

> **ATENCIÓN:** Que Gmail lo acepte no significa que llegue a la bandeja de entrada. Sin SPF, DKIM, DMARC y PTR (Módulo 9) es muy probable que acabe en spam o que sea rechazado con un mensaje como `550-5.7.1 ... does not meet IPv6 sending guidelines regarding PTR records and authentication`. Es normal en este punto.

Errores habituales:

Mensaje en mail.log	Significado	Solución
`connect to ...:25: Connection timed out` + `status=deferred`	El puerto 25 saliente está bloqueado.	PASO 6.1 y sección 6.8 (relayhost).

Mensaje en mail.log	Significado	Solución
`status=bounced ... 550-5.7.1 ... PTR records`	Falta el DNS inverso.	Módulo 9.
`status=deferred ... 421-4.7.0 ... unusual rate of unsolicited mail`	La reputación de tu IP es mala o nueva.	Configura todo el Módulo 9 y envía poco volumen al principio.
`Network is unreachable` en direcciones IPv6	El servidor no tiene IPv6 funcional.	`sudo postconf -e 'inet_protocols = ipv4'` y `sudo systemctl restart postfix`.

PASO 6.9 – Probar la recepción con una conversación SMTP manual

Objetivo: comprobar que el servidor recibe correo desde Internet y ver el protocolo con tus propios ojos.

Explicación: aún no hemos creado los registros MX, pero cuando un dominio no tiene MX, los servidores usan su registro A. Como `mail.ejemplo1.com` tiene registro A, podemos enviar a `sysadmin@mail.ejemplo1.com`. Haremos la conversación a mano desde **tu ordenador** (si tu conexión doméstica bloquea el 25, hazla desde otro servidor o desde el propio servidor contra su IP pública).

Comando (en tu ordenador con Linux o macOS; en Windows instala `ncat` del paquete Nmap o usa `telnet`):

TERMINAL
```
nc mail.ejemplo1.com 25
```

Escribe las líneas marcadas con C (sin la C) y observa las respuestas:

TEXTO
```
S: 220 mail.ejemplo1.com ESMTP
C: EHLO prueba.ejemplo.org
S: 250-mail.ejemplo1.com
S: 250-PIPELINING
S: 250-SIZE 52428800
S: 250-ETRN
S: 250-STARTTLS
S: 250-ENHANCEDSTATUSCODES
S: 250-8BITMIME
S: 250-DSN
S: 250-SMTPUTF8
S: 250 CHUNKING
C: MAIL FROM:<prueba@gmail.com>
S: 250 2.1.0 Ok
C: RCPT TO:<sysadmin@mail.ejemplo1.com>
S: 250 2.1.5 Ok
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Subject: Prueba manual
C:
C: Hola desde una conversacion SMTP manual.
C: .
S: 250 2.0.0 Ok: queued as 1F0A43E0F7
C: QUIT
S: 221 2.0.0 Bye
```

Comprobación: ejecuta `mail` como `sysadmin` en el servidor; el mensaje debe estar ahí.

Ahora comprueba que **no** eres un relay abierto. Repite la conversación pero con un destinatario externo:

```
TEXTO
C: RCPT TO:<alguien@gmail.com>
S: 454 4.7.1 <alguien@gmail.com>: Relay access denied
```

Ese rechazo es la prueba de que el servidor solo acepta correo para sus propios destinos cuando el cliente no está autenticado.

> **CONSEJO:** También puedes enviar un correo desde tu Gmail a `sysadmin@mail.ejemplo1.com` y verlo llegar con `mail`.

PASO 6.10 – Leer los logs y gestionar la cola

Objetivo: dominar las herramientas de diagnóstico del correo.

Comandos más útiles:

```
TERMINAL
# Seguir el log en tiempo real
sudo tail -f /var/log/mail.log
# Buscar todo lo relacionado con un mensaje por su ID de cola
sudo grep 8C1D23E0F4 /var/log/mail.log
# Buscar los envíos a una dirección
sudo grep 'to=<tu-direccion@gmail.com>' /var/log/mail.log
# Ver sólo errores y avisos
sudo grep -E 'warning|error|fatal|reject' /var/log/mail.log | tail -20
# También desde systemd
sudo journalctl -u postfix@- --since "1 hour ago" --no-pager
```

Cola de correo:

```
TERMINAL
mailq
sudo postcat -q 8C1D23E0F4 | less
sudo postqueue -f
sudo postsuper -d 8C1D23E0F4
sudo postsuper -d ALL deferred
```

Análisis: `mailq` lista los mensajes pendientes con el motivo del retraso; `postcat -q` muestra un mensaje concreto; `postqueue -f` fuerza el reintento; `postsuper -d` borra un mensaje (o todos los diferidos con `ALL deferred`). Postfix reintenta automáticamente los mensajes diferidos durante 5 días (`maximal_queue_lifetime`) antes de devolverlos al remitente.

Estados de entrega en el log:

Estado	Significado
`status=sent`	Entregado (al buzón local o aceptado por el servidor remoto).
`status=deferred`	Error temporal (código 4xx): se reintentará.
`status=bounced`	Error permanente (código 5xx): se devuelve al remitente.
`status=expired`	Se agotó el tiempo de reintentos.

6.8 Si tu proveedor bloquea el puerto 25: relayhost

Si no puedes enviar directamente, puedes entregar todo el correo saliente a un servicio de envío (por ejemplo, el *smarthost* de tu proveedor o un servicio transaccional) que lo reenvía por ti. La recepción por el puerto 25 no cambia. La configuración general es:

Archivo: `/etc/postfix/sasl_passwd` (nuevo; los datos te los da el servicio de relay):

ARCHIVO DE CONFIGURACIÓN

```
[smtp.servicio-relay.example]:587 usuario_relay:<CONTRASEÑA_RELAY>
```

Comando:

TERMINAL

```
sudo chmod 600 /etc/postfix/sasl_passwd
sudo postmap /etc/postfix/sasl_passwd
sudo postconf -e 'relayhost = [smtp.servicio-relay.example]:587'
sudo postconf -e 'smtp_sasl_auth_enable = yes'
sudo postconf -e 'smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd'
sudo postconf -e 'smtp_sasl_security_options = noanonymous'
sudo postconf -e 'smtp_tls_security_level = encrypt'
sudo systemctl restart postfix
```

Análisis: los corchetes `[...]` indican a Postfix que no busque registros MX de ese nombre. `postmap` compila el archivo de contraseñas. `smtp_tls_security_level = encrypt` obliga a cifrar la conexión con el relay, porque por ella viaja tu contraseña. Con un relay, el registro SPF (Módulo 9) debe incluir también los servidores del servicio de relay, según indique su documentación.

6.9 Ejercicios prácticos del Módulo 6

1. Ejecuta `postconf -d myorigin` y `postconf myorigin`. Explica la diferencia.
2. Envía un correo a una dirección inexistente de tu propio servidor (`noexiste@mail.ejemplo1.com`) y sigue en el log el rebote (`status=bounced`). ¿Dónde llega la notificación de error?
3. Repite la conversación SMTP manual pero usando `EHLO prueba` (sin dominio). ¿Qué restricción la rechaza y con qué código?
4. Detén temporalmente la salida al puerto 25 con `sudo ufw deny out 25/tcp`, envía un correo externo, observa la cola con `mailq` y vuelve a permitirlo con `sudo ufw delete deny out 25/tcp`. Fuerza la entrega con `postqueue -f`.
5. Dibuja el recorrido de un correo enviado por la aplicación PHP de ejemplo1.com indicando qué procesos de Postfix intervienen.

Módulo 7. Instalación y configuración de Dovecot

Objetivos del módulo

- Comprender IMAP y POP3 y el papel de Dovecot en la infraestructura de correo.
- Instalar Dovecot 2.3 y conocer su estructura de configuración.
- Preparar el almacenamiento de buzones en formato Maildir con un usuario técnico `vmail`.
- Configurar la autenticación de usuarios virtuales, los protocolos, los sockets y TLS.
- Integrar Dovecot con Postfix en ambos sentidos: entrega de correo (LMTP) y autenticación de envío (SASL).
- Activar el envío autenticado en los puertos 587 y 465 y probar el acceso a los buzones.

7.1 Qué son IMAP y POP3

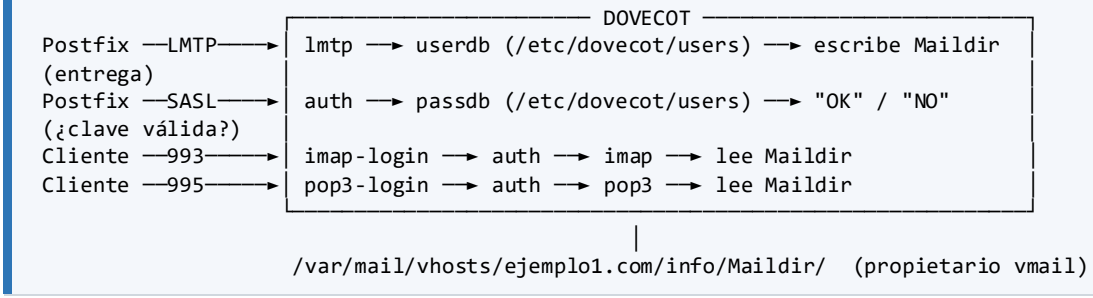
Postfix sabe recibir y enviar correo, pero no permite a los usuarios **leerlo**. Para eso existen dos protocolos:

Característica	IMAP	POP3
Dónde quedan los mensajes	En el servidor. El cliente muestra una vista sincronizada.	Se descargan al dispositivo y, normalmente, se borran del servidor.
Carpetas	Sí (Enviados, Borradores, Papelera, carpetas propias).	No, solo la bandeja de entrada.
Varios dispositivos	Perfecto: móvil, portátil y webmail ven lo mismo.	Problemático: cada dispositivo descarga su copia.
Estado (leído, marcado)	Sincronizado.	No.
Recomendación	**Usar IMAP.**	Solo para casos concretos (un único equipo, poco espacio en el servidor).

Dovecot es el servidor IMAP/POP3 más utilizado. En nuestra infraestructura tiene **tres** funciones:

1. **MAA**: atiende a los clientes de correo por IMAP y POP3.
2. **MDA**: recibe de Postfix los mensajes por **LMTP** y los guarda en el buzón correcto.
3. **Autenticador**: Postfix le pregunta, a través de un socket **SASL**, si el usuario y la contraseña de quien quiere enviar son correctos. Así hay una única base de usuarios para todo.

DIAGRAMA



7.2 Puertos de IMAP y POP3

Puerto	Protocolo	Cifrado	Estado en nuestro servidor
143	IMAP	Texto plano con posibilidad de STARTTLS.	**Deshabilitado.**
993	IMAPS	TLS implícito desde el inicio.	**Habilitado y abierto.**
110	POP3	Texto plano con posibilidad de STLS.	**Deshabilitado.**
995	POP3S	TLS implícito desde el inicio.	**Habilitado y abierto.**

Los puertos 143 y 110 podrían usarse de forma segura obligando a STARTTLS, pero no aportan nada que no den 993 y 995, y un error de configuración podría hacer que alguien enviara su contraseña sin cifrar. La configuración profesional más simple y robusta es: **solo 993 y 995**, y si no necesitas POP3, ni siquiera eso.

7.3 Maildir: dónde y cómo se guardan los mensajes

Hay dos formatos clásicos de buzón:

- **mbox**: todos los mensajes de una carpeta en un único archivo. Simple, pero lento y frágil (un fallo puede corromper el buzón entero).
- **Maildir**: cada mensaje es un archivo independiente. Rápido, seguro ante fallos y fácil de copiar. **Es el que usaremos.**

Estructura de un Maildir:

DIAGRAMA

```
/var/mail/vhosts/                          ← raíz de todos los buzones (vmail:vmail)
└── ejemplo1.com/                          ← un directorio por dominio
    └── info/                              ← "home" del usuario virtual info@ejemplo1.com
        └── Maildir/
            ├── new/                       ← mensajes recién llegados, no vistos por el cliente
            ├── cur/                       ← mensajes ya vistos por el cliente
            ├── tmp/                       ← escritura temporal durante la entrega
            ├── .Sent/   (cur/ new/ tmp/)  ← carpeta Enviados
            ├── .Drafts/                   ← Borradores
            ├── .Trash/                    ← Papelera
            ├── .Junk/                     ← Correo no deseado
            └── dovecot.index*             ← índices de Dovecot (aceleran las búsquedas)
```

Todos los buzones pertenecen a un único usuario técnico de Linux, **vmail**. Las cuentas de correo **no** son cuentas de Linux: no pueden iniciar sesión en el servidor, no tienen contraseña de sistema y solo existen para Dovecot y Postfix. Lo trataremos en detalle en el Módulo 8.

PASO 7.1 – Instalar Dovecot

Objetivo: instalar Dovecot con IMAP, POP3 y LMTP.

Comando:

TERMINAL

```
sudo apt install -y dovecot-core dovecot-imapd dovecot-pop3d dovecot-lmtpd
```

Análisis del comando:

Paquete	Aporta
`dovecot-core`	El núcleo: autenticación, gestión de buzones, `doveadm`.
`dovecot-imapd`	El servicio IMAP.
`dovecot-pop3d`	El servicio POP3 (omítelo si no lo vas a usar).
`dovecot-lmtpd`	El servicio LMTP, por el que Postfix entrega el correo.

Comprobación:

TERMINAL

```
dovecot --version
systemctl status dovecot --no-pager
```

SALIDA ESPERADA

```
2.3.21 (47349e2482)
● dovecot.service - Dovecot IMAP/POP3 email server
     Active: active (running) since ...
```

ATENCIÓN: Si la versión empieza por **2.4**, la configuración de este módulo no es válida tal cual: estás en Debian 13, Ubuntu 26.04 o posterior. Consulta la documentación oficial de Dovecot 2.4 («Upgrading Dovecot CE from 2.3 to 2.4») antes de continuar.

Siguiente paso: conocer la estructura de configuración.

7.4 Estructura de configuración de Dovecot

DIAGRAMA

```
/etc/dovecot/
├── dovecot.conf                ← archivo principal; incluye conf.d/*.conf
├── conf.d/
│   ├── 10-auth.conf            ← mecanismos de autenticación y qué backend se usa
│   ├── auth-system.conf.ext    ← backend de usuarios del sistema (PAM) – lo desactivaremos
│   ├── auth-passwdfile.conf.ext← backend de archivo de usuarios – lo activaremos
│   ├── 10-mail.conf            ← ubicación y formato de los buzones
│   ├── 10-master.conf          ← servicios, puertos y sockets
│   ├── 10-ssl.conf             ← certificados y TLS
│   ├── 15-mailboxes.conf       ← carpetas especiales (Enviados, Papelera...)
│   ├── 20-imap.conf / 20-pop3.conf / 20-lmtp.conf ← opciones por protocolo
│   └── ...
└── users                       ← (lo crearemos) usuarios virtuales y contraseñas
/usr/share/dovecot/protocols.d/ ← un archivo por protocolo instalado (imap, pop3, lmtp)
```

Comandos de administración:

Comando	Qué hace
`sudo doveconf -n`	Muestra la configuración efectiva que difiere de los valores por defecto. Es lo primero que se pide en cualquier foro de ayuda.
`sudo doveconf protocols`	Muestra un parámetro concreto.

Comando	Qué hace
`sudo doveadm reload`	Recarga la configuración.
`sudo doveadm user usuario`	Muestra cómo ve Dovecot a un usuario (uid, home, buzón).
`sudo doveadm auth test usuario`	Prueba una contraseña.
`sudo doveadm pw -s SHA512-CRYPT`	Genera el hash de una contraseña.
`sudo doveadm mailbox list -u usuario`	Lista las carpetas de un buzón.
`sudo doveadm who`	Muestra quién está conectado.

Comprueba qué protocolos están activos (los activa automáticamente cada paquete instalado):

TERMINAL
```
sudo doveconf protocols
```

SALIDA ESPERADA
```
protocols = imap pop3 lmtp
```

PASO 7.2 – Crear el usuario técnico vmail y la carpeta de buzones

Objetivo: crear el usuario Linux que será propietario de todos los buzones virtuales.

Comando:

TERMINAL
```
sudo groupadd --system --gid 5000 vmail
sudo useradd --system --uid 5000 --gid vmail --home-dir /var/mail/vhosts --shell
/usr/sbin/nologin vmail
sudo install -d -o vmail -g vmail -m 750 /var/mail/vhosts
id vmail
ls -ld /var/mail/vhosts
```

Análisis del comando:

- `groupadd --gid 5000 vmail`: crea el grupo con el identificador numérico fijo 5000. Usar un número fijo facilita restaurar copias en otro servidor.
- `useradd --uid 5000 ... --shell /usr/sbin/nologin`: crea el usuario sin posibilidad de iniciar sesión.
- `install -d -o vmail -g vmail -m 750`: crea la carpeta raíz de buzones; solo `vmail` (y su grupo) puede entrar.

Resultado esperado:

SALIDA ESPERADA
```
uid=5000(vmail) gid=5000(vmail) groups=5000(vmail)
drwxr-x--- 2 vmail vmail 4096 nov  9 01:10 /var/mail/vhosts
```

Errores habituales: `GID '5000' already exists` o `UID 5000 is not unique`: otro usuario usa ya ese número. Elige otro libre (por ejemplo 5001) y úsalo de forma coherente en todo el manual.

Siguiente paso: crear el archivo de usuarios.

PASO 7.3 – Crear el archivo de usuarios con la primera cuenta

Objetivo: crear `/etc/dovecot/users` con la cuenta `info@ejemplo1.com`, que usaremos para probar todo el módulo.

Explicación: Dovecot puede leer los usuarios de muchas fuentes (MariaDB, LDAP, archivos). Usaremos un archivo con el formato `passwd-file`, que es sencillo, rápido y suficiente para decenas o cientos de cuentas. Las contraseñas no se guardan en claro, sino como *hash* **SHA512-CRYPT** con sal aleatoria.

Primero genera el hash de la contraseña de la cuenta:

TERMINAL

```
sudo doveadm pw -s SHA512-CRYPT
```

Escribe la contraseña dos veces. El resultado es algo así (cada vez diferente, aunque la contraseña sea la misma, por la sal aleatoria):

SALIDA ESPERADA

```
Enter new password:
Retype new password:
{SHA512-CRYPT}$6$Qm2xY7bQe0nU5d1T$gq1q9...resto del hash...Xh0
```

Copia toda la línea, desde `{SHA512-CRYPT}` hasta el final.

Archivo: `/etc/dovecot/users` (nuevo):

ARCHIVO DE CONFIGURACIÓN

```
info@ejemplo1.com:{SHA512-CRYPT}$6$...PEGA-AQUI-EL-
HASH...:5000:5000::/var/mail/vhosts/ejemplo1.com/info::
```

Formato de cada línea (campos separados por `:`):

Posición	Campo	Valor
1	Usuario	`info@ejemplo1.com` (la dirección completa)
2	Contraseña	El hash con su prefijo de esquema.
3	UID	`5000` (vmail)
4	GID	`5000` (vmail)
5	(GECOS)	Vacío.
6	Home	Carpeta del usuario: `/var/mail/vhosts/dominio/usuario`
7	(Shell)	Vacío.
8	Campos extra	Vacío (aquí se pueden añadir cuotas, por ejemplo).

Protege el archivo (contiene hashes de contraseñas):

TERMINAL
```
sudo chown root:dovecot /etc/dovecot/users
sudo chmod 640 /etc/dovecot/users
```

CONSEJO: Para no tener que hacer esto a mano con cada cuenta, en el Módulo 8 crearemos un script que genera el hash, añade la línea en Dovecot y registra la dirección en Postfix en un único paso.

Siguiente paso: configurar la autenticación.

PASO 7.4 – Configurar la autenticación (10-auth.conf)

Objetivo: que Dovecot autentique solo a los usuarios virtuales del archivo, nunca a las cuentas Linux, y solo por canales cifrados.

Explicación: por defecto Dovecot autentica a los usuarios del sistema mediante PAM (`auth-system.conf.ext`). Lo desactivamos y activamos el archivo de usuarios (`auth-passwdfile.conf.ext`). Haremos los cambios con `sed` para que sean exactos, y luego los verificaremos.

Comando:

TERMINAL
```
sudo cp /etc/dovecot/conf.d/10-auth.conf /etc/dovecot/conf.d/10-auth.conf.original
sudo sed -i 's/^#\?disable_plaintext_auth = .*/disable_plaintext_auth = yes/'
/etc/dovecot/conf.d/10-auth.conf
sudo sed -i 's/^auth_mechanisms = .*/auth_mechanisms = plain login/' /etc/dovecot/conf.d/10-
auth.conf
sudo sed -i 's/^!include auth-system.conf.ext/#!include auth-system.conf.ext/'
/etc/dovecot/conf.d/10-auth.conf
sudo sed -i 's/^#!include auth-passwdfile.conf.ext/!include auth-passwdfile.conf.ext/'
/etc/dovecot/conf.d/10-auth.conf
grep -Ev '^\s*(#|$)' /etc/dovecot/conf.d/10-auth.conf
```

Análisis del comando:

- `sed -i 's/buscar/reemplazar/' archivo`: edita el archivo en el sitio (`-i`) sustituyendo el texto. `^` significa «al principio de la línea»; `#\?` significa «una almohadilla opcional».
- El último `grep -Ev '^\s*(#|$)'` muestra solo las líneas activas (ni comentarios ni líneas vacías).

Resultado esperado (las líneas activas del archivo):

SALIDA ESPERADA
```
disable_plaintext_auth = yes
auth_mechanisms = plain login
!include auth-passwdfile.conf.ext
```

Directiva	Significado
`disable_plaintext_auth = yes`	Rechaza contraseñas si la conexión no está cifrada.
`auth_mechanisms = plain login`	Mecanismos aceptados. `PLAIN` es el estándar; `LOGIN` lo necesitan algunos clientes antiguos (Outlook). Ambos envían la contraseña tal cual, por eso solo se permiten **dentro** de TLS.
`#!include auth-system.conf.ext`	Comentado: las cuentas Linux no pueden usar el correo.

Directiva	Significado
`!include auth-passwdfile.conf.ext`	Activa el backend de archivo.

Errores habituales: si el `grep` final no muestra exactamente esas tres líneas, abre el archivo con `sudo nano /etc/dovecot/conf.d/10-auth.conf` y corrígelo a mano.

Siguiente paso: configurar el backend de archivo.

PASO 7.5 – Configurar el backend de usuarios (auth-passwdfile.conf.ext)

Archivo: `/etc/dovecot/conf.d/auth-passwdfile.conf.ext` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
# Usuarios virtuales definidos en /etc/dovecot/users
# passdb: comprueba la contraseña
passdb {
  driver = passwd-file
  args = scheme=SHA512-CRYPT username_format=%Lu /etc/dovecot/users
}

# userdb: obtiene uid, gid y carpeta (home) del usuario
userdb {
  driver = passwd-file
  args = username_format=%Lu /etc/dovecot/users
}
```

Análisis:

Elemento	Significado
`passdb`	Base de datos de **contraseñas**: se consulta cuando alguien inicia sesión.
`userdb`	Base de datos de **usuarios**: indica uid, gid y home. La consultan IMAP, POP3 y LMTP.
`driver = passwd-file`	Formato del archivo (similar a `/etc/passwd`).
`scheme=SHA512-CRYPT`	Esquema por defecto de las contraseñas (el prefijo `{SHA512-CRYPT}` lo confirma).
`username_format=%Lu`	Busca al usuario por su nombre completo (`%u` = `usuario@dominio`) convertido a minúsculas (`L`). Así `Info@Ejemplo1.com` funciona igual.

PASO 7.6 – Configurar la ubicación de los buzones (10-mail.conf)

Objetivo: indicar que los buzones son Maildir dentro del home de cada usuario.

Comando:

TERMINAL

```
sudo cp /etc/dovecot/conf.d/10-mail.conf /etc/dovecot/conf.d/10-mail.conf.original
sudo sed -i 's|^mail_location = .*|mail_location = maildir:~/Maildir|' /etc/dovecot/conf.d/10-mail.conf
grep -E '^(mail_location|mail_privileged_group|first_valid_uid)' /etc/dovecot/conf.d/10-mail.conf
```

Resultado esperado:

SALIDA ESPERADA

```
mail_location = maildir:~/Maildir
mail_privileged_group = mail
```

Análisis: `maildir:~/Maildir` significa «formato Maildir, en la subcarpeta `Maildir` del home del usuario». Con el home `/var/mail/vhosts/ejemplo1.com/info`, el buzón queda en `/var/mail/vhosts/ejemplo1.com/info/Maildir`. Dovecot crea las carpetas automáticamente con el usuario `vmail` la primera vez que llega un correo o que el usuario inicia sesión. El parámetro `first_valid_uid` (por defecto 500) impide usar uid inferiores; nuestro 5000 cumple.

PASO 7.7 – Configurar las carpetas especiales (15-mailboxes.conf)

Objetivo: que cada buzón nuevo tenga automáticamente las carpetas Enviados, Borradores, Papelera y Spam.

Archivo: `/etc/dovecot/conf.d/15-mailboxes.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
# Carpetas especiales creadas y suscritas automáticamente en cada buzón
namespace inbox {
  mailbox Drafts {
    auto = subscribe
    special_use = \Drafts
  }
  mailbox Junk {
    auto = subscribe
    special_use = \Junk
  }
  mailbox Trash {
    auto = subscribe
    special_use = \Trash
  }
  mailbox Sent {
    auto = subscribe
    special_use = \Sent
  }
  mailbox "Sent Messages" {
    special_use = \Sent
  }
}
```

Análisis: `special_use` comunica a los clientes (RFC 6154) qué carpeta es cada una, de modo que Thunderbird o el móvil guarden los enviados en `Sent` sin configurar nada. `auto = subscribe` crea la carpeta y la suscribe automáticamente.

PASO 7.8 – Configurar los servicios, puertos y sockets (10-master.conf)

Objetivo: definir en qué puertos escucha Dovecot y crear los dos sockets que usará Postfix.

Explicación: este es el archivo de integración más importante. Creamos:

- El socket **LMTP** dentro del *chroot* de Postfix (`/var/spool/postfix/private/dovecot-lmtp`), para que Postfix entregue el correo.
- El socket de **autenticación** (`/var/spool/postfix/private/auth`), para que Postfix compruebe contraseñas.

Ambos están en `/var/spool/postfix/private/` porque los procesos de Postfix se ejecutan encerrados en `/var/spool/postfix` y no pueden ver nada fuera de esa carpeta.

Comando:

TERMINAL

```
sudo cp /etc/dovecot/conf.d/10-master.conf /etc/dovecot/conf.d/10-master.conf.original
sudo nano /etc/dovecot/conf.d/10-master.conf
```

Archivo: `/etc/dovecot/conf.d/10-master.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
# ======================================================================
#  Servicios de Dovecot - mail.ejemplo1.com
# ======================================================================

# IMAP: solo el puerto cifrado 993. El 143 queda deshabilitado (port = 0)
service imap-login {
  inet_listener imap {
    port = 0
  }
  inet_listener imaps {
    port = 993
    ssl = yes
  }
}

# POP3: solo el puerto cifrado 995. El 110 queda deshabilitado
service pop3-login {
  inet_listener pop3 {
    port = 0
  }
  inet_listener pop3s {
    port = 995
    ssl = yes
  }
}

# LMTP: Postfix entrega aquí el correo entrante (socket dentro del chroot de Postfix)
service lmtp {
  unix_listener /var/spool/postfix/private/dovecot-lmtp {
    mode = 0600
    user = postfix
    group = postfix
  }
}

service imap {
}

service pop3 {
}

# Autenticación
service auth {
  # Postfix pregunta aquí si un usuario y contraseña son válidos (SASL)
  unix_listener /var/spool/postfix/private/auth {
    mode = 0660
    user = postfix
    group = postfix
  }
  # Socket interno para las consultas de usuarios (lo usa LMTP y doveadm)
```

```
  unix_listener auth-userdb {
    mode = 0600
    user = vmail
    group = vmail
  }
  # El proceso de autenticación se ejecuta sin privilegios de root
  user = dovecot
}

service auth-worker {
  user = $default_internal_user
}
```

Análisis:

Bloque	Significado
`inet_listener imap { port = 0 }`	Desactiva el puerto 143.
`inet_listener imaps { port = 993; ssl = yes }`	IMAP con TLS implícito.
`unix_listener /var/spool/postfix/private/dovecot-lmtp`	Socket LMTP. Propietario `postfix`, permisos `0600`: solo Postfix puede usarlo.
`unix_listener /var/spool/postfix/private/auth`	Socket SASL para Postfix.
`unix_listener auth-userdb`	Socket interno (en `/run/dovecot/`) para consultas de usuarios.
`user = dovecot` (en `service auth`)	El proceso que lee `/etc/dovecot/users` es `dovecot`, no root. Por eso el archivo es `root:dovecot 640`.
`service auth-worker { user = $default_internal_user }`	Los procesos auxiliares de autenticación tampoco se ejecutan como root (solo serían necesarios como root para leer `/etc/shadow`, que no usamos).

PASO 7.9 – Configurar TLS (10-ssl.conf)

Objetivo: que Dovecot use el certificado de Let's Encrypt y exija cifrado.

Comando: primero comprueba si existe el archivo de parámetros Diffie-Hellman que trae el paquete:

TERMINAL

```
ls -l /usr/share/dovecot/dh.pem
sudo cp /etc/dovecot/conf.d/10-ssl.conf /etc/dovecot/conf.d/10-ssl.conf.original
sudo nano /etc/dovecot/conf.d/10-ssl.conf
```

Archivo: `/etc/dovecot/conf.d/10-ssl.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
# TLS obligatorio para todas las conexiones de clientes
ssl = required

# Certificado de Let's Encrypt (el símbolo < indica "leer el contenido del archivo")
ssl_cert = </etc/letsencrypt/live/mail.ejemplo1.com/fullchain.pem
ssl_key = </etc/letsencrypt/live/mail.ejemplo1.com/privkey.pem

# Autoridades de confianza del sistema
ssl_client_ca_dir = /etc/ssl/certs

# Solo TLS 1.2 o superior; el servidor elige el cifrado
ssl_min_protocol = TLSv1.2
ssl_prefer_server_ciphers = yes

# Parámetros Diffie-Hellman (borra esta línea si el ls anterior dio error)
ssl_dh = </usr/share/dovecot/dh.pem
```

Análisis:

Directiva	Significado
`ssl = required`	Ningún cliente puede autenticarse sin TLS.
`ssl_cert = <ruta`	El `<` es imprescindible: indica que el valor es el contenido del archivo. Sin él, Dovecot da error.
`ssl_min_protocol`	Rechaza TLS 1.0 y 1.1, obsoletos.
`ssl_dh`	Parámetros para el intercambio de claves Diffie-Hellman clásico.

CONSEJO: Dovecot lee los certificados al arrancar, con privilegios de root, así que puede leer `privkey.pem` aunque solo sea legible por root. Cuando Certbot renueve el certificado habrá que recargar Dovecot: lo automatizaremos en el Módulo 10.

PASO 7.10 – Configurar LMTP (20-lmtp.conf)

Archivo: `/etc/dovecot/conf.d/20-lmtp.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
protocol lmtp {
  # Dirección que aparece como remitente en los rechazos generados por LMTP
  postmaster_address = postmaster@ejemplo1.com
  mail_plugins = $mail_plugins
}
```

PASO 7.11 – Comprobar la configuración y reiniciar Dovecot

Comando:

TERMINAL

```
sudo doveconf -n > /dev/null && echo "Configuración correcta"
sudo systemctl restart dovecot
systemctl status dovecot --no-pager
sudo ss -tlnp | grep dovecot
ls -l /var/spool/postfix/private/ | grep -E 'auth|dovecot'
```

Análisis: `doveconf -n` analiza toda la configuración; si hay un error, lo muestra con el archivo y la línea.

Resultado esperado:

SALIDA ESPERADA

```
Configuración correcta
● dovecot.service - Dovecot IMAP/POP3 email server
     Active: active (running) since ...
LISTEN 0  100  0.0.0.0:993  0.0.0.0:*  users:(("dovecot",pid=7120,fd=36))
LISTEN 0  100  0.0.0.0:995  0.0.0.0:*  users:(("dovecot",pid=7120,fd=38))
LISTEN 0  100     [::]:993     [::]:*  users:(("dovecot",pid=7120,fd=37))
LISTEN 0  100     [::]:995     [::]:*  users:(("dovecot",pid=7120,fd=39))
srw-rw---- 1 postfix postfix 0 nov  9 01:30 auth
srw------- 1 postfix postfix 0 nov  9 01:30 dovecot-lmtp
```

No deben aparecer los puertos 143 ni 110.

Comprobación: consulta cómo ve Dovecot al usuario y prueba su contraseña:

TERMINAL

```
sudo doveadm user info@ejemplo1.com
sudo doveadm auth test info@ejemplo1.com
```

SALIDA ESPERADA

```
field   value
uid     5000
gid     5000
home    /var/mail/vhosts/ejemplo1.com/info
mail    maildir:~/Maildir
Password:
passdb: info@ejemplo1.com auth succeeded
extra fields:
  user=info@ejemplo1.com
```

Errores habituales:

Mensaje	Causa	Solución
`doveconf: Fatal: Error in configuration file ... line N`	Errata (llave sin cerrar, directiva mal escrita).	Revisa el archivo y la línea indicados.
`ssl_cert: Can't open file ...`	Ruta del certificado incorrecta o falta el `<`.	`sudo ls -l /etc/letsencrypt/live/mail.ejemplo1.com/`.
`auth failed` en `doveadm auth test`	Hash mal copiado o archivo con permisos incorrectos.	Regenera el hash; comprueba `root:dovecot 640`.

Mensaje	Causa	Solución
`userdb lookup: user info@ejemplo1.com doesn't exist`	La línea del usuario está mal formada (faltan `:`).	Cuenta los campos: deben ser 8 separados por 7 `:` hasta el final.
`Error: passwd-file /etc/dovecot/users: open() failed: Permission denied`	El proceso auth (usuario dovecot) no puede leer el archivo.	`sudo chown root:dovecot /etc/dovecot/users && sudo chmod 640 /etc/dovecot/users`.

Siguiente paso: integrar Postfix con Dovecot.

7.5 Integración de Dovecot con Postfix

Ahora Postfix debe saber tres cosas:

1. Que `ejemplo1.com` y `ejemplo2.com` son dominios de **buzones virtuales**.
2. Qué direcciones existen (para rechazar en el acto el correo a direcciones inexistentes, en lugar de aceptarlo y devolverlo después, lo que generaría *backscatter*).
3. Que la entrega se hace por **LMTP** a Dovecot y que la autenticación se pregunta a Dovecot por **SASL**.

PASO 7.12 – Crear las tablas de dominios y buzones de Postfix

Archivo: `/etc/postfix/virtual_domains` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
# Dominios con buzones virtuales. Formato: dominio  OK
ejemplo1.com    OK
ejemplo2.com    OK
```

Archivo: `/etc/postfix/vmailbox` (nuevo; de momento, solo la primera cuenta):

ARCHIVO DE CONFIGURACIÓN

```
# Direcciones que tienen buzón. Formato: dirección  dominio/usuario/
info@ejemplo1.com           ejemplo1.com/info/
```

Archivo: `/etc/postfix/virtual` (nuevo; alias de direcciones virtuales):

ARCHIVO DE CONFIGURACIÓN

```
# Alias virtuales. Formato: dirección_alias  destino[, destino2...]
# Direcciones obligatorias por RFC (se completarán en el Módulo 8)
postmaster@ejemplo1.com     info@ejemplo1.com
abuse@ejemplo1.com          info@ejemplo1.com
```

Comando: compila las tablas al formato que lee Postfix:

TERMINAL

```
sudo postmap /etc/postfix/virtual_domains
sudo postmap /etc/postfix/vmailbox
sudo postmap /etc/postfix/virtual
ls /etc/postfix/*.db
```

Análisis: `postmap` crea un archivo `.db` (base de datos indexada) junto a cada archivo de texto. Postfix consulta el `.db`, no el texto. **Cada vez** que modifiques uno de estos archivos debes volver a ejecutar `postmap` sobre él (no hace falta reiniciar Postfix: detecta el cambio).

> **CONSEJO:** En `vmailbox`, el valor de la derecha (`ejemplo1.com/info/`) no se usa para la entrega, porque la ruta real la decide Dovecot. Postfix solo necesita saber que la dirección **existe**. Lo dejamos escrito así por claridad y por convención.

PASO 7.13 – Añadir la integración a main.cf

Objetivo: activar los buzones virtuales, la entrega por LMTP y la autenticación SASL.

Comando:

TERMINAL
```
sudo nano /etc/postfix/main.cf
```

Archivo: `/etc/postfix/main.cf`. **Conserva todo** lo que escribiste en el Módulo 6 y **añade al final** este bloque:

ARCHIVO DE CONFIGURACIÓN
```
# --- Buzones virtuales gestionados por Dovecot (Módulo 7) ---------------
virtual_mailbox_domains = hash:/etc/postfix/virtual_domains
virtual_mailbox_maps = hash:/etc/postfix/vmailbox
virtual_alias_maps = hash:/etc/postfix/virtual
virtual_transport = lmtp:unix:private/dovecot-lmtp

# --- Autenticación SASL a través de Dovecot (Módulo 7) -----------------
smtpd_sasl_type = dovecot
smtpd_sasl_path = private/auth
# Desactivada en el puerto 25; se activa solo en 587 y 465 (master.cf)
smtpd_sasl_auth_enable = no
smtpd_sasl_security_options = noanonymous, noplaintext
smtpd_sasl_tls_security_options = noanonymous
```

Análisis:

Parámetro	Significado
`virtual_mailbox_domains`	Lista de dominios virtuales (tabla `virtual_domains`).
`virtual_mailbox_maps`	Direcciones válidas de esos dominios. Las demás se rechazan con `550 User unknown`.
`virtual_alias_maps`	Alias: reescriben el destinatario antes de entregar.
`virtual_transport = lmtp:unix:private/dovecot-lmtp`	Entrega por LMTP en el socket que creó Dovecot (ruta relativa a `/var/spool/postfix`).
`smtpd_sasl_type = dovecot`	El autenticador es Dovecot.
`smtpd_sasl_path = private/auth`	Socket de autenticación (relativo a `/var/spool/postfix`).
`smtpd_sasl_auth_enable = no`	En el puerto 25 nadie puede autenticarse: es solo para recibir de otros servidores.
`smtpd_sasl_security_options = noanonymous, noplaintext`	Sin TLS no se aceptan mecanismos que envíen la contraseña en claro.

Parámetro	Significado
`smtpd_sasl_tls_security_options = noanonymous`	Con TLS sí se aceptan PLAIN y LOGIN (la contraseña va cifrada por TLS).

PASO 7.14 – Activar los puertos de envío autenticado 587 y 465 (master.cf)

Objetivo: abrir los servicios *submission* (587, STARTTLS) y *submissions* (465, TLS implícito), ambos con autenticación obligatoria.

Explicación: `master.cf` define los procesos que arranca Postfix. Cada servicio es una línea con 8 columnas, seguida opcionalmente de líneas `-o parametro=valor` (que **deben empezar por espacios**) que sobrescriben `main.cf` solo para ese servicio. Ubuntu trae estos servicios comentados.

Comando:

TERMINAL

```
sudo cp /etc/postfix/master.cf /etc/postfix/master.cf.original
sudo nano /etc/postfix/master.cf
```

Archivo: `/etc/postfix/master.cf`. Localiza las líneas que empiezan por `#submission inet` y `#smtps inet` (están comentadas, cerca del principio, justo después de la línea `smtp inet ... smtpd`). **Bórralas junto con sus líneas `# -o ...`** y escribe en su lugar este bloque. Conserva el resto del archivo sin cambios.

ARCHIVO DE CONFIGURACIÓN

```
submission inet n       -       y       -       -       smtpd
  -o syslog_name=postfix/submission
  -o smtpd_tls_security_level=encrypt
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_tls_auth_only=yes
  -o smtpd_client_restrictions=permit_sasl_authenticated,reject
  -o smtpd_relay_restrictions=permit_sasl_authenticated,reject
  -o milter_macro_daemon_name=ORIGINATING
smtps     inet  n       -       y       -       -       smtpd
  -o syslog_name=postfix/smtps
  -o smtpd_tls_wrappermode=yes
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_client_restrictions=permit_sasl_authenticated,reject
  -o smtpd_relay_restrictions=permit_sasl_authenticated,reject
  -o milter_macro_daemon_name=ORIGINATING
```

Análisis de las columnas de la primera línea (`submission inet n - y - - smtpd`):

Columna	Valor	Significado
Servicio	`submission`	Nombre del puerto en `/etc/services` (587). `smtps` es el 465.
Tipo	`inet`	Escucha en red TCP.
Privado	`n`	No es un servicio interno.
Sin privilegios	`-`	Se ejecuta con el usuario sin privilegios de Postfix.
Chroot	`y`	Se ejecuta encerrado en `/var/spool/postfix` (por defecto en Ubuntu).
Despertar / Máx. procesos	`- -`	Valores por defecto.
Comando	`smtpd`	El demonio SMTP.

Y de las opciones:

Opción	Significado
`syslog_name=postfix/submission`	En el log aparecerá `postfix/submission/smtpd`, así distingues qué puerto usó cada cliente.
`smtpd_tls_security_level=encrypt`	En el 587 el cifrado STARTTLS es obligatorio.
`smtpd_tls_wrappermode=yes`	En el 465 TLS empieza desde el primer byte.
`smtpd_sasl_auth_enable=yes`	Aquí sí se permite autenticarse.
`smtpd_client_restrictions=permit_sasl_authenticated,reject`	Solo pueden enviar los usuarios autenticados.
`milter_macro_daemon_name=ORIGINATING`	Marca el correo como saliente para que OpenDKIM lo firme (Módulo 9).

Comprueba y aplica:

TERMINAL
```
sudo postfix check
sudo systemctl restart postfix
sudo ss -tlnp | grep master
```

SALIDA ESPERADA
```
LISTEN 0  100  0.0.0.0:25   0.0.0.0:*  users:(("master",pid=7301,fd=13))
LISTEN 0  100  0.0.0.0:465  0.0.0.0:*  users:(("master",pid=7301,fd=21))
LISTEN 0  100  0.0.0.0:587  0.0.0.0:*  users:(("master",pid=7301,fd=17))
...
```

Errores habituales: `postfix/master: fatal: /etc/postfix/master.cf: line N: bad field count` indica que una línea `-o` no empieza con espacios o que falta una columna.

PASO 7.15 – Abrir los puertos de correo de los clientes

Comando:

TERMINAL
```
sudo ufw allow 587/tcp comment 'SMTP submission'
sudo ufw allow 465/tcp comment 'SMTPS'
sudo ufw allow 993/tcp comment 'IMAPS'
sudo ufw allow 995/tcp comment 'POP3S'
sudo ufw status numbered
```

7.6 Pruebas de funcionamiento

PASO 7.16 – Probar la entrega en el buzón (Postfix → LMTP → Dovecot)

Comando:

TERMINAL

```
echo "Primer correo para info." | mail -s "Prueba de buzón virtual" info@ejemplo1.com
sudo grep 'info@ejemplo1.com' /var/log/mail.log | tail -5
sudo ls -R /var/mail/vhosts/ejemplo1.com/info/Maildir | head -20
```

Resultado esperado en el log:

SALIDA ESPERADA

```
postfix/lmtp[7420]: 2D9F03E0FA: to=<info@ejemplo1.com>,
relay=mail.ejemplo1.com[private/dovecot-lmtp], delay=0.05, dsn=2.0.0, status=sent (250 2.0.0
<info@ejemplo1.com> 8Kx0BQ5WD2fjHAAA3sKhVQ Saved)
dovecot: lmtp(info@ejemplo1.com)<7421><8Kx0BQ5WD2fjHAAA3sKhVQ>: msgid=<...@mail.ejemplo1.com>:
saved mail to INBOX
```

Y el mensaje aparecerá como archivo en `Maildir/new/`.

Prueba también el alias y el rechazo de direcciones inexistentes:

TERMINAL

```
echo "Correo al postmaster" | mail -s "Prueba alias" postmaster@ejemplo1.com
echo "No debería llegar" | mail -s "Prueba inexistente" noexiste@ejemplo1.com
sudo tail -n 20 /var/log/mail.log | grep -E 'postmaster|noexiste'
```

El primero se entregará en el buzón de `info@` (`orig_to=<postmaster@ejemplo1.com>`); el segundo será rechazado con `unknown user`.

Errores habituales:

Log	Causa	Solución
`connect to private/dovecot-lmtp: No such file or directory`	El socket no existe: Dovecot parado o error en `10-master.conf`.	`ls -l /var/spool/postfix/private/` y `systemctl status dovecot`.
`status=bounced (... User doesn't exist: info@ejemplo1.com)`	La dirección está en `vmailbox` de Postfix pero no en `/etc/dovecot/users`.	Ambos archivos deben coincidir.
`Recipient address rejected: User unknown in virtual mailbox table`	La dirección no está en `vmailbox` o no ejecutaste `postmap`.	Añádela y `sudo postmap /etc/postfix/vmailbox`.
`do not list domain ejemplo1.com in BOTH mydestination and virtual_mailbox_domains`	El dominio aparece en ambos sitios.	Quítalo de `mydestination`.
`Permission denied` al crear el Maildir	`/var/mail/vhosts` no pertenece a vmail.	`sudo chown vmail:vmail /var/mail/vhosts`.

PASO 7.17 – Probar IMAP por TLS manualmente (puerto 993)

Objetivo: iniciar sesión en el buzón «a mano», igual que haría un cliente de correo.

Comando:

TERMINAL

```
openssl s_client -connect mail.ejemplo1.com:993 -quiet
```

Análisis del comando: `openssl s_client` abre una conexión TLS; `-quiet` oculta los detalles del certificado. Cada comando IMAP empieza con una etiqueta inventada (`a1`, `a2`...) que el servidor repite en su respuesta.

Escribe las líneas que empiezan por `a` (sustituye la contraseña):

TEXTO

```
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ AUTH=PLAIN
AUTH=LOGIN] Dovecot (Ubuntu) ready.
a1 LOGIN info@ejemplo1.com <CONTRASEÑA_INFO>
a1 OK [CAPABILITY ...] Logged in
a2 LIST "" "*"
* LIST (\HasNoChildren \Drafts) "." Drafts
* LIST (\HasNoChildren \Junk) "." Junk
* LIST (\HasNoChildren \Trash) "." Trash
* LIST (\HasNoChildren \Sent) "." Sent
* LIST (\HasNoChildren) "." INBOX
a2 OK List completed
a3 SELECT INBOX
* 2 EXISTS
a3 OK [READ-WRITE] Select completed
a4 FETCH 1 BODY[HEADER.FIELDS (FROM SUBJECT DATE)]
* 1 FETCH (BODY[HEADER.FIELDS (FROM SUBJECT DATE)] {...}
Subject: Prueba de buzón virtual
From: sysadmin@ejemplo1.com
...
a4 OK Fetch completed
a5 LOGOUT
```

Comprobación: también puedes ver las sesiones activas mientras estás conectado con `sudo doveadm who` desde otro terminal.

> **ATENCIÓN:** La contraseña quedará en el historial del terminal de tu ordenador si la escribes en la línea de comandos de otros programas. Aquí se escribe dentro de la sesión TLS, así que no queda en el historial del shell.

PASO 7.18 – Probar POP3 por TLS (puerto 995)

Comando:

TERMINAL

```
openssl s_client -connect mail.ejemplo1.com:995 -quiet
```

```
TEXTO
+OK Dovecot (Ubuntu) ready.
USER info@ejemplo1.com
+OK
PASS <CONTRASEÑA_INFO>
+OK Logged in.
STAT
+OK 2 1024
LIST
+OK 2 messages:
1 498
2 526
.
QUIT
+OK Logging out.
```

PASO 7.19 – Probar el envío autenticado (puertos 587 y 465)

Objetivo: verificar que un usuario autenticado puede enviar a cualquier destino y que sin autenticación no puede.

Explicación: usaremos **swaks** (*Swiss Army Knife for SMTP*), la herramienta estándar para probar servidores SMTP.

Comando:

```
TERMINAL
sudo apt install -y swaks
swaks --server mail.ejemplo1.com --port 587 --tls --auth LOGIN --auth-user info@ejemplo1.com --from info@ejemplo1.com --to tu-direccion@gmail.com --header "Subject: Prueba por el 587"
```

Análisis del comando:

Opción	Significado
`--server` / `--port`	Servidor y puerto.
`--tls`	Usa STARTTLS. Para el puerto 465 se usa `--tlsc` (TLS al conectar).
`--auth LOGIN`	Mecanismo de autenticación.
`--auth-user`	Usuario. Como no indicamos `--auth-password`, swaks la pide de forma interactiva y no queda en el historial.
`--from` / `--to`	Remitente y destinatario.
`--header`	Cabecera adicional (el asunto).

Resultado esperado (fragmento):

SALIDA ESPERADA

```
<~  220 2.0.0 Ready to start TLS
=== TLS started with cipher TLSv1.3:TLS_AES_256_GCM_SHA384:256
 ~> AUTH LOGIN
<~* 235 2.7.0 Authentication successful
 ~> MAIL FROM:<info@ejemplo1.com>
<~* 250 2.1.0 Ok
 ~> RCPT TO:<tu-direccion@gmail.com>
<~* 250 2.1.5 Ok
<~* 250 2.0.0 Ok: queued as 6A1F23E0FD
```

Repite por el puerto 465:

TERMINAL

```
swaks --server mail.ejemplo1.com --port 465 --tlsc --auth LOGIN --auth-user info@ejemplo1.com
--from info@ejemplo1.com --to tu-direccion@gmail.com --header "Subject: Prueba por el 465"
```

Y comprueba que sin autenticación el 587 no deja enviar:

TERMINAL

```
swaks --server mail.ejemplo1.com --port 587 --tls --from info@ejemplo1.com --to tu-
direccion@gmail.com
```

SALIDA ESPERADA

```
<** 554 5.7.1 <unknown[203.0.113.10]>: Client host rejected: Access denied
```

Errores habituales:

Síntoma	Causa	Solución
`535 5.7.8 Error: authentication failed`	Contraseña incorrecta.	`sudo doveadm auth test info@ejemplo1.com`; mira `mail.log` para ver el motivo exacto.
`warning: SASL: Connect to private/auth failed: No such file or directory`	El socket de autenticación no existe.	Revisa el bloque `service auth` de `10-master.conf` y reinicia Dovecot.
`503 5.5.1 Error: authentication not enabled`	Te conectaste al 25, o falta `smtpd_sasl_auth_enable=yes` en master.cf.	Revisa puerto y `master.cf`.
La conexión se corta en el 465	Usaste `--tls` en vez de `--tlsc`.	En el 465 el TLS es implícito.

Siguiente paso: Módulo 8, creación y gestión de las cuentas de correo.

7.7 Ejercicios prácticos del Módulo 7

1. Ejecuta `sudo doveconf -n` y explica cada línea de la salida.
2. Intenta conectar por IMAP sin cifrar con `nc mail.ejemplo1.com 143`. ¿Qué ocurre y por qué?
3. Desde la sesión IMAP manual, crea una carpeta con `a6 CREATE Facturas` y comprueba en el servidor que aparece `/var/mail/vhosts/ejemplo1.com/info/Maildir/.Facturas`.

4. Envía un correo a `info+pruebas@ejemplo1.com`. ¿Llega? ¿A qué buzón? ¿Qué parámetro lo hace posible?
5. Introduce a propósito una contraseña errónea en swaks tres veces y busca los intentos en `/var/log/mail.log`. Esos mensajes son los que vigilará Fail2ban en el Módulo 12.

Módulo 8. Creación de cuentas de correo

Objetivos del módulo

- Diferenciar con claridad las cuentas del sistema Linux de las cuentas virtuales de correo.
- Entender qué archivos intervienen en la existencia de una dirección de correo.
- Crear las cuentas `info@ejemplo1.com`, `administracion@ejemplo1.com` y `soporte@ejemplo2.com`, y los alias obligatorios.
- Automatizar el alta, la baja y el cambio de contraseña con scripts de administración.
- Impedir que un usuario envíe correo haciéndose pasar por otro.
- Configurar un cliente de correo real.

8.1 Cuentas del sistema frente a cuentas virtuales

Es un error muy frecuente mezclar estos dos conceptos. En nuestra infraestructura están completamente separados:

Característica	Cuenta del sistema Linux	Cuenta virtual de correo
Ejemplos	`sysadmin`, `web1`, `vmail`, `root`	`info@ejemplo1.com`, `soporte@ejemplo2.com`
Dónde se define	`/etc/passwd` y `/etc/shadow`	`/etc/dovecot/users` (Dovecot) y `/etc/postfix/vmailbox` (Postfix)
Se crea con	`adduser` / `useradd`	El script `correo-alta` (este módulo)
¿Puede entrar por SSH?	Sí (si se permite)	**Nunca**
¿Tiene carpeta en /home?	Normalmente sí	No. Su buzón está en `/var/mail/vhosts/dominio/usuario/`
Propietario de sus archivos	Ella misma	El usuario técnico `vmail` (uid 5000)
Nombre de usuario al autenticarse	`sysadmin`	La dirección completa: `info@ejemplo1.com`
Correo que recibe	El de `@mail.ejemplo1.com` (dominio local, `mydestination`)	El de `@ejemplo1.com` y `@ejemplo2.com` (dominios virtuales)

Ventajas de las cuentas virtuales:

- **Seguridad**: robar la contraseña de un buzón no da acceso al servidor.
- **Multidominio**: `info@ejemplo1.com` e `info@ejemplo2.com` pueden coexistir sin conflicto (en Linux solo puede haber un usuario `info`).
- **Gestión sencilla**: crear 100 buzones no llena el sistema de usuarios Linux.

8.2 Qué hace falta para que una dirección exista

DIAGRAMA

```
Llega un correo para soporte@ejemplo2.com
       │
       ▼
POSTFIX ── ¿ejemplo2.com está en virtual_domains? ────────── no → "Relay access denied"
       │ sí
       ├── ¿hay alias en /etc/postfix/virtual? ── sí → se reescribe el destinatario
       │
       ├── ¿la dirección está en /etc/postfix/vmailbox? ─── no → "User unknown" (550)
       │ sí
       ▼ LMTP
DOVECOT ── ¿la dirección está en /etc/dovecot/users? ─────── no → rebote
       │ sí: uid 5000, home /var/mail/vhosts/ejemplo2.com/soporte
       ▼
Guarda el mensaje en /var/mail/vhosts/ejemplo2.com/soporte/Maildir/new/

Cuando soporte@ejemplo2.com envía por el 587:
POSTFIX ──SASL──► DOVECOT (/etc/dovecot/users: ¿contraseña correcta?)
       └── /etc/postfix/sender_login: ¿soporte@ puede usar este remitente?
```

Resumen de archivos:

Archivo	Propietario del dato	Qué contiene	Tras modificarlo
`/etc/postfix/virtual_domains`	Postfix	Dominios de correo.	`postmap`
`/etc/postfix/vmailbox`	Postfix	Direcciones con buzón.	`postmap`
`/etc/postfix/virtual`	Postfix	Alias (reenvíos).	`postmap`
`/etc/postfix/sender_login`	Postfix	Qué usuario autenticado puede usar cada remitente.	`postmap`
`/etc/dovecot/users`	Dovecot	Contraseñas, uid, gid y carpeta de cada cuenta.	Nada (Dovecot lo relee solo)

8.3 Proteger la identidad del remitente

Ahora mismo, cualquier usuario autenticado podría enviar un correo con `MAIL FROM:<administracion@ejemplo1.com>` aunque haya iniciado sesión como `info@`. Lo impedimos con una tabla que indica qué usuario puede usar cada dirección como remitente.

PASO 8.1 – Crear la tabla de remitentes autorizados

Archivo: `/etc/postfix/sender_login` (nuevo):

ARCHIVO DE CONFIGURACIÓN

```
# Remitente (MAIL FROM)            Usuarios SASL autorizados a usarlo (separados por comas)
info@ejemplo1.com                  info@ejemplo1.com
```

Comando:

TERMINAL

```
sudo postmap /etc/postfix/sender_login
```

Añade a `/etc/postfix/main.cf` (conserva todo lo anterior y añade al final):

ARCHIVO DE CONFIGURACIÓN

```
# --- Remitentes autorizados por usuario (Módulo 8) ----------------------
smtpd_sender_login_maps = hash:/etc/postfix/sender_login
```

PASO 8.2 – Exigir la coincidencia en los puertos de envío

Objetivo: que en los puertos 587 y 465 se rechace un remitente que no pertenezca al usuario autenticado.

Archivo: `/etc/postfix/master.cf`. En los bloques `submission` y `smtps` que escribiste en el PASO 7.14, añade la línea `-o smtpd_sender_restrictions=...` justo debajo de la línea `-o smtpd_client_restrictions=...`. Los dos bloques quedan así (el resto del archivo no cambia):

ARCHIVO DE CONFIGURACIÓN

```
submission inet n       -       y       -       -       smtpd
  -o syslog_name=postfix/submission
  -o smtpd_tls_security_level=encrypt
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_tls_auth_only=yes
  -o smtpd_client_restrictions=permit_sasl_authenticated,reject
  -o smtpd_sender_restrictions=reject_sender_login_mismatch,permit_sasl_authenticated,reject
  -o smtpd_relay_restrictions=permit_sasl_authenticated,reject
  -o milter_macro_daemon_name=ORIGINATING
smtps     inet  n       -       y       -       -       smtpd
  -o syslog_name=postfix/smtps
  -o smtpd_tls_wrappermode=yes
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_client_restrictions=permit_sasl_authenticated,reject
  -o smtpd_sender_restrictions=reject_sender_login_mismatch,permit_sasl_authenticated,reject
  -o smtpd_relay_restrictions=permit_sasl_authenticated,reject
  -o milter_macro_daemon_name=ORIGINATING
```

Análisis: `reject_sender_login_mismatch` rechaza el mensaje si el usuario autenticado no figura en `sender_login` como propietario del remitente indicado en `MAIL FROM`.

TERMINAL

```
sudo postfix check && sudo systemctl restart postfix
```

Comprobación: más adelante, cuando exista `administracion@`, intenta enviar como ella autenticado como `info@`:

TERMINAL

```
swaks --server mail.ejemplo1.com --port 587 --tls --auth LOGIN --auth-user info@ejemplo1.com --from administracion@ejemplo1.com --to tu-direccion@gmail.com
```

SALIDA ESPERADA

```
<** 553 5.7.1 <administracion@ejemplo1.com>: Sender address rejected: not owned by user info@ejemplo1.com
```

8.4 Scripts de administración de cuentas

Hacer las altas a mano en cuatro archivos es lento y propenso a errores (una cuenta en Postfix pero no en Dovecot provoca rebotes). Un administrador automatiza las tareas repetitivas. Crearemos tres scripts en `/usr/local/sbin/`, la carpeta estándar para herramientas de administración propias (solo root puede ejecutarlas con sentido, porque modifican archivos protegidos).

PASO 8.3 – Script de alta de cuentas: correo-alta

Comando:

TERMINAL
```
sudo nano /usr/local/sbin/correo-alta
```

Archivo: `/usr/local/sbin/correo-alta` (nuevo, contenido completo):

TERMINAL
```
#!/bin/bash
# ============================================================================
#  correo-alta  -  Crea una cuenta de correo virtual
#  Uso: sudo correo-alta usuario@dominio
# ============================================================================
set -euo pipefail

USERS=/etc/dovecot/users
VMAILBOX=/etc/postfix/vmailbox
DOMINIOS=/etc/postfix/virtual_domains
SENDERS=/etc/postfix/sender_login
VMAIL_UID=5000
VMAIL_GID=5000
BASE=/var/mail/vhosts

[ "$(id -u)" -eq 0 ] || { echo "Error: ejecuta este script con sudo." >&2; exit 1; }
[ $# -eq 1 ] || { echo "Uso: $0 usuario@dominio" >&2; exit 1; }

DIRECCION="${1,,}"                      # a minúsculas
if ! [[ "$DIRECCION" =~ ^[a-z0-9._-]+@[a-z0-9.-]+\.[a-z]{2,}$ ]]; then
    echo "Error: '$DIRECCION' no es una dirección válida." >&2; exit 1
fi
USUARIO="${DIRECCION%@*}"
DOMINIO="${DIRECCION#*@}"

# El dominio debe estar dado de alta en Postfix
if ! awk -v d="$DOMINIO" '$1==d {f=1} END {exit !f}' "$DOMINIOS"; then
    echo "Error: el dominio $DOMINIO no está en $DOMINIOS." >&2; exit 1
fi
# La cuenta no debe existir
if awk -F: -v a="$DIRECCION" '$1==a {f=1} END {exit !f}' "$USERS"; then
    echo "Error: la cuenta $DIRECCION ya existe." >&2; exit 1
fi

echo "Introduce la contraseña para $DIRECCION (mínimo 12 caracteres):"
HASH=$(doveadm pw -s SHA512-CRYPT)

# Copias de seguridad antes de modificar
cp -p "$USERS" "$USERS.bak"; cp -p "$VMAILBOX" "$VMAILBOX.bak"; cp -p "$SENDERS"
"$SENDERS.bak"
```

```
echo "${DIRECCION}:${HASH}:${VMAIL_UID}:${VMAIL_GID}::${BASE}/${DOMINIO}/${USUARIO}::" >>
"$USERS"
printf '%-35s %s\n' "$DIRECCION" "${DOMINIO}/${USUARIO}/" >> "$VMAILBOX"
printf '%-35s %s\n' "$DIRECCION" "$DIRECCION" >> "$SENDERS"

postmap "$VMAILBOX"
postmap "$SENDERS"

echo "Cuenta $DIRECCION creada correctamente."
echo "El buzón se creará en ${BASE}/${DOMINIO}/${USUARIO}/Maildir al recibir el primer correo
o iniciar sesión."
```

Análisis del script:

Línea	Qué hace
`#!/bin/bash`	Indica que el script se ejecuta con bash.
`set -euo pipefail`	Detiene el script ante cualquier error (`-e`), variable no definida (`-u`) o fallo en una tubería (`pipefail`). Evita dejar las cosas a medias.
`${1,,}`	Convierte el argumento a minúsculas.
`[[ ... =~ ... ]]`	Comprueba con una expresión regular que la dirección tiene un formato válido.
`${DIRECCION%@*}` / `${DIRECCION#*@}`	Extrae la parte antes y después de la `@`.
`awk -v d=... '$1==d'`	Busca coincidencias exactas en la primera columna (más seguro que `grep`, donde el punto significa «cualquier carácter»).
`doveadm pw`	Pide la contraseña dos veces y devuelve su hash. La contraseña nunca se escribe en disco ni en el historial.
`cp -p ... .bak`	Copia de seguridad conservando permisos.
`>>`	Añade una línea al final del archivo (nunca uses `>` aquí: borraría el contenido).
`postmap`	Recompila las tablas de Postfix.

Hazlo ejecutable:

TERMINAL

```
sudo chmod 750 /usr/local/sbin/correo-alta
```

PASO 8.4 – Script de cambio de contraseña: correo-clave

Archivo: `/usr/local/sbin/correo-clave` (nuevo, contenido completo):

TERMINAL

```
#!/bin/bash
# ======================================================================
#  correo-clave  -  Cambia la contraseña de una cuenta de correo virtual
#  Uso: sudo correo-clave usuario@dominio
# ======================================================================
set -euo pipefail
USERS=/etc/dovecot/users
```

```
[ "$(id -u)" -eq 0 ] || { echo "Error: ejecuta este script con sudo." >&2; exit 1; }
[ $# -eq 1 ] || { echo "Uso: $0 usuario@dominio" >&2; exit 1; }
DIRECCION="${1,,}"

if ! awk -F: -v a="$DIRECCION" '$1==a {f=1} END {exit !f}' "$USERS"; then
    echo "Error: la cuenta $DIRECCION no existe." >&2; exit 1
fi

echo "Nueva contraseña para $DIRECCION:"
HASH=$(doveadm pw -s SHA512-CRYPT)

cp -p "$USERS" "$USERS.bak"
TMP=$(mktemp)
awk -F: -v OFS=: -v a="$DIRECCION" -v h="$HASH" '$1==a {$2=h} {print}' "$USERS" > "$TMP"
cat "$TMP" > "$USERS"      # conserva propietario y permisos del archivo original
rm -f "$TMP"

# Cierra las sesiones abiertas para que vuelvan a autenticarse
doveadm kick "$DIRECCION" >/dev/null 2>&1 || true
echo "Contraseña de $DIRECCION actualizada."
```

Análisis: `awk -F: -v OFS=:` lee y escribe el archivo con `:` como separador y sustituye solo el segundo campo de la línea de esa cuenta. Escribir con `cat "$TMP" > "$USERS"` (en lugar de mover el archivo) conserva el propietario `root:dovecot` y los permisos `640`. `doveadm kick` desconecta las sesiones IMAP/POP3 activas de ese usuario.

PASO 8.5 – Script de baja de cuentas: correo-baja

Archivo: `/usr/local/sbin/correo-baja` (nuevo, contenido completo):

TERMINAL

```
#!/bin/bash
# ======================================================================
#  correo-baja  -  Elimina una cuenta de correo virtual
#  Uso: sudo correo-baja usuario@dominio
#  El buzón NO se borra: se mueve a /var/mail/bajas para poder recuperarlo.
# ======================================================================
set -euo pipefail
USERS=/etc/dovecot/users
VMAILBOX=/etc/postfix/vmailbox
SENDERS=/etc/postfix/sender_login
BASE=/var/mail/vhosts
BAJAS=/var/mail/bajas

[ "$(id -u)" -eq 0 ] || { echo "Error: ejecuta este script con sudo." >&2; exit 1; }
[ $# -eq 1 ] || { echo "Uso: $0 usuario@dominio" >&2; exit 1; }
DIRECCION="${1,,}"
USUARIO="${DIRECCION%@*}"
DOMINIO="${DIRECCION#*@}"

if ! awk -F: -v a="$DIRECCION" '$1==a {f=1} END {exit !f}' "$USERS"; then
    echo "Error: la cuenta $DIRECCION no existe." >&2; exit 1
fi

read -r -p "¿Seguro que quieres dar de baja $DIRECCION? Escribe SI para confirmar: " RESP
[ "$RESP" = "SI" ] || { echo "Cancelado."; exit 0; }

for F in "$USERS" "$VMAILBOX" "$SENDERS"; do
    cp -p "$F" "$F.bak"
    TMP=$(mktemp)
    if [ "$F" = "$USERS" ]; then
```

```
        awk -F: -v a="$DIRECCION" '$1!=a' "$F" > "$TMP"
    else
        awk -v a="$DIRECCION" '$1!=a' "$F" > "$TMP"
    fi
    cat "$TMP" > "$F"; rm -f "$TMP"
done
postmap "$VMAILBOX"
postmap "$SENDERS"
doveadm kick "$DIRECCION" >/dev/null 2>&1 || true

if [ -d "$BASE/$DOMINIO/$USUARIO" ]; then
    install -d -o vmail -g vmail -m 750 "$BAJAS"
    mv "$BASE/$DOMINIO/$USUARIO" "$BAJAS/${DIRECCION}_$(date +%F)"
    echo "Buzón movido a $BAJAS/${DIRECCION}_$(date +%F)"
fi
echo "Cuenta $DIRECCION dada de baja. Revisa si tenía alias en /etc/postfix/virtual."
```

TERMINAL

```
sudo chmod 750 /usr/local/sbin/correo-clave /usr/local/sbin/correo-baja
ls -l /usr/local/sbin/correo-*
```

ATENCIÓN: Los scripts guardan una copia `.bak` de cada archivo antes de modificarlo. Si algo sale mal, puedes restaurarla con `sudo cp -p /etc/dovecot/users.bak /etc/dovecot/users` (y lo mismo con los demás archivos, seguido de `postmap`).

8.5 Crear las cuentas del proyecto

PASO 8.6 – Registrar la cuenta info@ creada a mano

Objetivo: añadir la cuenta del Módulo 7 a la tabla de remitentes (se creó antes de existir el script).

Comando:

TERMINAL

```
grep -q '^info@ejemplo1.com' /etc/postfix/sender_login || echo 'info@ejemplo1.com
info@ejemplo1.com' | sudo tee -a /etc/postfix/sender_login
sudo postmap /etc/postfix/sender_login
```

PASO 8.7 – Crear administracion@ejemplo1.com y soporte@ejemplo2.com

Comando:

TERMINAL

```
sudo correo-alta administracion@ejemplo1.com
sudo correo-alta soporte@ejemplo2.com
```

Resultado esperado:

```
SALIDA ESPERADA
Introduce la contraseña para administracion@ejemplo1.com (mínimo 12 caracteres):
Enter new password:
Retype new password:
Cuenta administracion@ejemplo1.com creada correctamente.
El buzón se creará en /var/mail/vhosts/ejemplo1.com/administracion/Maildir al recibir el
primer correo o iniciar sesión.
```

Comprobación:

```
TERMINAL
sudo doveadm user '*'
sudo doveadm auth test soporte@ejemplo2.com
postmap -q soporte@ejemplo2.com hash:/etc/postfix/vmailbox
```

```
SALIDA ESPERADA
administracion@ejemplo1.com
info@ejemplo1.com
soporte@ejemplo2.com
Password:
passdb: soporte@ejemplo2.com auth succeeded
ejemplo2.com/soporte/
```

Análisis: `doveadm user '*'` lista todos los usuarios que conoce Dovecot. `postmap -q clave tabla` consulta una tabla de Postfix exactamente igual que lo haría Postfix al recibir un correo: es la mejor forma de comprobar las tablas.

PASO 8.8 – Crear los alias obligatorios y de servicio

Objetivo: crear las direcciones que todo dominio debe tener y las que usaremos en módulos siguientes.

Explicación: los estándares (RFC 2142 y RFC 5321) exigen que exista `postmaster@` en todo dominio que recibe correo, y es muy recomendable `abuse@`. También crearemos `dmarc@` para los informes DMARC (Módulo 9) y

`noreply@`, el remitente que usarán las aplicaciones web (Módulo 11), para que cualquier rebote o respuesta llegue a una persona.

Archivo: `/etc/postfix/virtual` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
# ================================================================
#  Alias virtuales:  dirección_alias        destino[, destino2...]
#  Tras modificar:   sudo postmap /etc/postfix/virtual
# ================================================================

# ---- ejemplo1.com ----
postmaster@ejemplo1.com      administracion@ejemplo1.com
abuse@ejemplo1.com           administracion@ejemplo1.com
dmarc@ejemplo1.com           administracion@ejemplo1.com
noreply@ejemplo1.com         administracion@ejemplo1.com
root@ejemplo1.com            administracion@ejemplo1.com
# Ejemplo de lista de distribución: llega a dos buzones
equipo@ejemplo1.com          info@ejemplo1.com, administracion@ejemplo1.com

# ---- ejemplo2.com ----
postmaster@ejemplo2.com      soporte@ejemplo2.com
abuse@ejemplo2.com           soporte@ejemplo2.com
dmarc@ejemplo2.com           soporte@ejemplo2.com
noreply@ejemplo2.com         soporte@ejemplo2.com
```

Comando:

TERMINAL

```
sudo postmap /etc/postfix/virtual
postmap -q equipo@ejemplo1.com hash:/etc/postfix/virtual
```

SALIDA ESPERADA

```
info@ejemplo1.com, administracion@ejemplo1.com
```

CONSEJO: Un alias **no** es una cuenta: no tiene contraseña ni buzón propio. Si quieres que `administracion@` pueda **enviar** como `postmaster@ejemplo1.com` o `noreply@ejemplo1.com`, añade esa relación a `/etc/postfix/sender_login`, por ejemplo: `postmaster@ejemplo1.com  administracion@ejemplo1.com` (y `postmap`).

ATENCIÓN: Evita los alias *catch-all* (`@ejemplo1.com  info@ejemplo1.com`), que recogen el correo de cualquier dirección inventada. Atraen enormes cantidades de spam y anulan el rechazo de direcciones inexistentes.

PASO 8.9 – Redirigir el correo del sistema a una cuenta real

Objetivo: que los avisos del servidor (cron, Fail2ban, actualizaciones) lleguen a `administracion@ejemplo1.com`.

Archivo: `/etc/aliases` (modifica solo la última línea; el resto se conserva):

ARCHIVO DE CONFIGURACIÓN

```
postmaster:    root
abuse:         root
webmaster:     root
www-data:      root
root:          administracion@ejemplo1.com
```

Comando:

TERMINAL

```
sudo newaliases
echo "Aviso de prueba del sistema" | mail -s "Prueba correo de root" root
sudo grep 'orig_to=<root>' /var/log/mail.log | tail -1
```

Resultado esperado: una línea con `to=<administracion@ejemplo1.com>, orig_to=<root>, relay=...dovecot-lmtp..., status=sent`.

8.6 Gestión de buzones y permisos

PASO 8.10 – Verificar la estructura y los permisos de los buzones

Comando:

TERMINAL

```
for d in info@ejemplo1.com administracion@ejemplo1.com soporte@ejemplo2.com; do
  echo "Bienvenida a $d" | mail -s "Bienvenida" "$d"
done
sleep 2
sudo ls -l /var/mail/vhosts/ /var/mail/vhosts/ejemplo1.com/
sudo doveadm mailbox status -u soporte@ejemplo2.com messages INBOX
```

Análisis: el bucle `for` envía un correo a cada cuenta, lo que provoca la creación de su buzón. `doveadm mailbox status` muestra cuántos mensajes hay en una carpeta.

Resultado esperado:

SALIDA ESPERADA

```
drwx------ 3 vmail vmail 4096 nov  9 02:10 ejemplo1.com
drwx------ 3 vmail vmail 4096 nov  9 02:10 ejemplo2.com
drwx------ 3 vmail vmail 4096 nov  9 02:10 administracion
drwx------ 3 vmail vmail 4096 nov  9 01:40 info
INBOX messages=1
```

Todo pertenece a `vmail` con permisos `700`: ni las webs (`web1`, `web2`) ni Apache pueden leer el correo.

Otras operaciones útiles con buzones:

Tarea	Comando
Espacio usado por cada buzón	`sudo du -sh /var/mail/vhosts/*/*`

Tarea	Comando
Carpetas de un buzón	`sudo doveadm mailbox list -u info@ejemplo1.com`
Buscar mensajes de un remitente	`sudo doveadm search -u info@ejemplo1.com mailbox INBOX from pepe@gmail.com`
Vaciar la papelera	`sudo doveadm expunge -u info@ejemplo1.com mailbox Trash all`
Borrar mensajes de más de 30 días en Spam	`sudo doveadm expunge -u info@ejemplo1.com mailbox Junk savedbefore 30d`
Reconstruir índices dañados	`sudo doveadm force-resync -u info@ejemplo1.com '*'`

> **CONSEJO: Cuotas.** Dovecot puede limitar el tamaño de cada buzón con el plugin `quota`, añadiendo en el campo 8 de `/etc/dovecot/users` algo como `userdb_quota_rule=*:storage=2G` y activando el plugin en la configuración. Es una ampliación recomendable cuando tengas muchos usuarios; consulta la documentación de Dovecot 2.3 sobre «Quota».

8.7 Configurar un cliente de correo

PASO 8.11 – Configurar Thunderbird (o cualquier cliente)

Objetivo: usar la cuenta desde un programa de correo real.

Datos de configuración para los usuarios de **ejemplo1.com**:

Parámetro	Entrante (IMAP)	Saliente (SMTP)
Servidor	`mail.ejemplo1.com`	`mail.ejemplo1.com`
Puerto	993	587 (o 465)
Seguridad de la conexión	SSL/TLS	STARTTLS (con 587) o SSL/TLS (con 465)
Método de autenticación	Contraseña normal	Contraseña normal
Nombre de usuario	La dirección completa: `info@ejemplo1.com`	La dirección completa

Para los usuarios de **ejemplo2.com** los datos son idénticos, pero con el servidor `mail.ejemplo2.com` (el certificado incluye ambos nombres, así que no habrá avisos).

Si prefieres POP3: servidor `mail.ejemplo1.com`, puerto 995, SSL/TLS.

En Thunderbird: «Cuentas» → «Nueva cuenta de correo» → introduce nombre, dirección y contraseña → pulsa «Configurar manualmente» y escribe los valores de la tabla → «Volver a probar» → «Hecho».

Comprobación:

1. Debe aparecer la bandeja de entrada con el mensaje «Bienvenida».
2. Envía un correo a tu dirección de Gmail y comprueba que llega (aunque aún pueda ir a spam).
3. Responde desde Gmail y comprueba que la respuesta llega a Thunderbird. Para que esto funcione con la dirección `@ejemplo1.com` necesitas el registro **MX** (Módulo 9).
4. En el servidor, mira el log mientras envías: `sudo tail -f /var/log/mail.log | grep -E 'submission|smtps|sasl'`. Verás `sasl_method=PLAIN, sasl_username=info@ejemplo1.com`.

Errores habituales:

Síntoma en el cliente	Causa	Solución
«No se puede conectar»	Puerto cerrado en UFW o en el cortafuegos del proveedor.	`sudo ufw status`; prueba con `openssl s_client`.
«Usuario o contraseña incorrectos»	Usaste solo `info` en vez de `info@ejemplo1.com`.	El usuario siempre es la dirección completa.
Aviso de certificado no válido	Usaste la IP o un nombre que no está en el certificado.	Usa `mail.ejemplo1.com` o `mail.ejemplo2.com`.
Recibe pero no envía	El cliente usa el puerto 25.	Cambia a 587/STARTTLS o 465/SSL.
«Sender address rejected: not owned by user»	El remitente configurado en el cliente no coincide con el usuario.	Corrige la dirección del remitente o añádela en `sender_login`.

8.8 Ejercicios prácticos del Módulo 8

1. Crea la cuenta `ventas@ejemplo2.com`, comprueba que recibe correo y después dala de baja con `correo-baja`. Verifica dónde ha quedado su buzón.
2. Cambia la contraseña de `info@ejemplo1.com` con `correo-clave` y comprueba que la antigua ya no funciona con `doveadm auth test`.
3. Crea un alias `facturas@ejemplo1.com` que entregue en `administracion@` y autoriza a `administracion@` a enviar como `facturas@`. Pruébalo con swaks.
4. Intenta crear una cuenta en un dominio no registrado (`prueba@ejemplo3.com`). ¿Qué hace el script? ¿Qué tendrías que modificar para alojar un tercer dominio de correo?
5. Explica por qué la cuenta Linux `web1` no puede leer los correos de `info@ejemplo1.com` (pista: `ls -ld /var/mail/vhosts`).

Módulo 9. Configuración DNS del servidor de correo

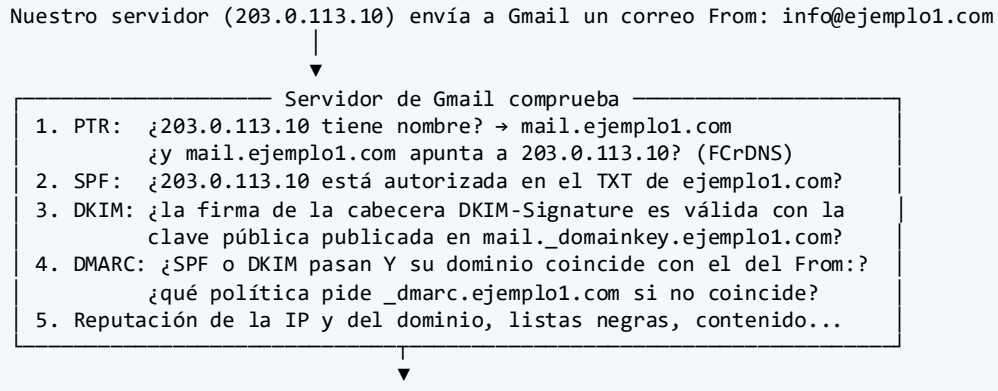
Objetivos del módulo

- Entender por qué el DNS determina si tus correos llegan a la bandeja de entrada o al spam.
- Configurar los registros A, MX, SPF, DKIM y DMARC de cada dominio y el DNS inverso (PTR).
- Instalar y configurar OpenDKIM, generar las claves y conectarlo con Postfix.
- Comprobar la propagación de los registros y verificar la autenticación con servidores reales.
- Conocer los factores de reputación de la IP y las buenas prácticas de entregabilidad.

9.1 Por qué el DNS es crítico para el correo

El protocolo SMTP se diseñó en 1982, cuando Internet era una red de confianza: cualquiera puede escribir `MAIL FROM:<presidente@banco.com>`. Para combatir la suplantación y el spam, los grandes proveedores (Gmail, Outlook, Yahoo…) comprueban varias cosas antes de aceptar un mensaje. Desde 2024, Gmail y Yahoo **exigen** SPF, DKIM, DMARC y DNS inverso a quien envía a sus usuarios.

DIAGRAMA



Mecanismo	Qué demuestra	Dónde se publica
PTR (DNS inverso)	Que la IP pertenece a un servidor de correo identificado y bien configurado.	En la zona inversa de la IP (la gestiona tu proveedor del servidor).
SPF	Qué IP están autorizadas a enviar correo con ese dominio en el sobre (`MAIL FROM`).	TXT en el dominio.
DKIM	Que el mensaje no ha sido alterado y que lo firmó el dominio.	TXT en `selector._domainkey.dominio`.
DMARC	Qué hacer si SPF/DKIM fallan o no coinciden con el `From:` visible, y a dónde enviar informes.	TXT en `_dmarc.dominio`.

9.2 Registros A y MX

PASO 9.1 – Verificar los registros A y crear los registros MX

Objetivo: indicar a Internet qué servidor recibe el correo de cada dominio.

Explicación: los registros A de `mail.ejemplo1.com` y `mail.ejemplo2.com` los creaste en el PASO 3.10. Ahora añade los MX en el panel de tu proveedor DNS. Cada dominio apunta a «su» nombre de servidor de correo, aunque ambos resuelvan a la misma IP: así cada cliente ve un nombre coherente con su dominio.

Registros de `ejemplo1.com`:

Tipo	Nombre	Prioridad	Valor	TTL
MX	@	10	mail.ejemplo1.com	3600

Registros de `ejemplo2.com`:

Tipo	Nombre	Prioridad	Valor	TTL
MX	@	10	mail.ejemplo2.com	3600

En formato de archivo de zona (por si tu proveedor permite importarlo), los registros completos de correo de ejemplo1.com quedarán así al final del módulo:

REGISTROS DNS

```
ejemplo1.com.                  3600 IN MX   10 mail.ejemplo1.com.
mail.ejemplo1.com.             3600 IN A    203.0.113.10
mail.ejemplo1.com.             3600 IN AAAA 2001:db8:10::10
ejemplo1.com.                  3600 IN TXT  "v=spf1 mx ~all"
mail.ejemplo1.com.             3600 IN TXT  "v=spf1 a -all"
mail._domainkey.ejemplo1.com.  3600 IN TXT  "v=DKIM1; h=sha256; k=rsa; p=MIIBIjANBgkq...(clave
pública)..."
_dmarc.ejemplo1.com.           3600 IN TXT  "v=DMARC1; p=none; rua=mailto:dmarc@ejemplo1.com;
adkim=r; aspf=r"
```

ATENCIÓN: El valor del MX es un **nombre**, nunca una IP, y ese nombre debe tener registro A (o AAAA) propio; no puede ser un CNAME. Algunos paneles exigen escribir el punto final (`mail.ejemplo1.com.`); otros lo añaden solos. Si escribes el nombre sin punto en un panel que no lo añade, acabarás con `mail.ejemplo1.com.ejemplo1.com`.

Comprobación:

TERMINAL

```
dig +short MX ejemplo1.com
dig +short MX ejemplo2.com
dig +short A mail.ejemplo2.com
```

SALIDA ESPERADA

```
10 mail.ejemplo1.com.
10 mail.ejemplo2.com.
203.0.113.10
```

Ahora ya puedes enviar desde Gmail a `info@ejemplo1.com` y a `soporte@ejemplo2.com`: Gmail buscará el MX y entregará en tu servidor.

9.3 El DNS inverso (PTR)

El registro **PTR** traduce una IP a un nombre. A diferencia del resto de registros, **no** lo configuras en el panel de tu dominio, porque la «zona inversa» de una IP pertenece a quien la posee: tu proveedor del servidor.

Los servidores receptores comprueban el **FCrDNS** (*Forward-Confirmed reverse DNS*):

DIAGRAMA

```
203.0.113.10 ──PTR──► mail.ejemplo1.com ──A──► 203.0.113.10    ✓ coinciden
```

Además, ese nombre debería coincidir con el `myhostname` de Postfix (que se presenta en el `EHLO`).

PASO 9.2 – Configurar el PTR en el panel de tu proveedor

Objetivo: que la IP del servidor tenga como nombre inverso `mail.ejemplo1.com`.

Explicación: busca en el panel de tu proveedor una opción llamada «Reverse DNS», «rDNS», «PTR» o similar, normalmente en la sección de red o de IPs del servidor. Escribe `mail.ejemplo1.com` para la IPv4 y, si la tienes, también para la IPv6. Algunos proveedores lo configuran automáticamente con el nombre que des al servidor; otros requieren abrir una incidencia.

> **CONSEJO:** Una IP solo puede tener un PTR. No pasa nada por que `ejemplo2.com` use un servidor cuyo PTR es `mail.ejemplo1.com`: es lo normal en cualquier proveedor de alojamiento. Lo que importa es que el PTR exista, resuelva de vuelta a la misma IP y no sea genérico (como `203-0-113-10.static.proveedor.net`).

Comprobación:

TERMINAL

```
dig +short -x 203.0.113.10
dig +short -x 2001:db8:10::10
dig +short A mail.ejemplo1.com
```

SALIDA ESPERADA

```
mail.ejemplo1.com.
mail.ejemplo1.com.
203.0.113.10
```

Errores habituales: si devuelve un nombre genérico del proveedor, el cambio aún no se ha aplicado o no se ha hecho. Si tu proveedor no permite cambiar el PTR (algo habitual en conexiones domésticas), no podrás tener buena entregabilidad desde esa IP: usa un relayhost (sección 6.8).

9.4 SPF (Sender Policy Framework)

SPF es un registro TXT que enumera qué servidores pueden enviar correo con tu dominio en el **remitente del sobre** (`MAIL FROM` / `Return-Path`).

Sintaxis:

Elemento	Significado
`v=spf1`	Versión. Obligatorio al principio.
`mx`	Autoriza las IP de los servidores MX del dominio.
`a`	Autoriza la IP del registro A del dominio.
`a:mail.ejemplo1.com`	Autoriza la IP de ese nombre concreto.
`ip4:203.0.113.10` / `ip6:2001:db8:10::10`	Autoriza IP concretas.
`include:dominio`	Autoriza también los servidores que autorice el SPF de otro dominio (servicios externos).
`-all`	**Fallo**: el resto de IP no están autorizadas; el receptor debería rechazar.
`~all`	**Fallo suave**: no autorizadas, pero se aceptan marcadas como sospechosas.
`?all`	Neutral (no aporta nada).

> **ATENCIÓN:** Solo puede haber **un** registro SPF (TXT que empieza por `v=spf1`) por nombre. Si añades servicios, se combinan en la misma línea. El SPF admite como máximo 10 consultas DNS (`mx`, `a`, `include`...) en total.

PASO 9.3 – Crear los registros SPF

Objetivo: autorizar a nuestro servidor (y solo a él) a enviar correo de cada dominio.

Registros:

Dominio	Tipo	Nombre	Valor
ejemplo1.com	TXT	@	`v=spf1 mx ~all`
ejemplo1.com	TXT	mail	`v=spf1 a -all`
ejemplo2.com	TXT	@	`v=spf1 mx ~all`
ejemplo2.com	TXT	mail	`v=spf1 a -all`

Análisis:

- `v=spf1 mx ~all` en el dominio: autoriza a los servidores MX (`mail.ejemplo1.com` → 203.0.113.10, y su IPv6 si tiene AAAA). Empezamos con `~all` mientras verificamos que todo funciona; cuando DMARC confirme que todo pasa, puedes endurecerlo a `-all`.
- `v=spf1 a -all` en `mail`: protege el nombre del propio servidor, que se usa en el `EHLO` y en los correos del sistema (`@mail.ejemplo1.com`).

Comprobación:

TERMINAL

```
dig +short TXT ejemplo1.com
dig +short TXT mail.ejemplo1.com
dig +short TXT ejemplo2.com
```

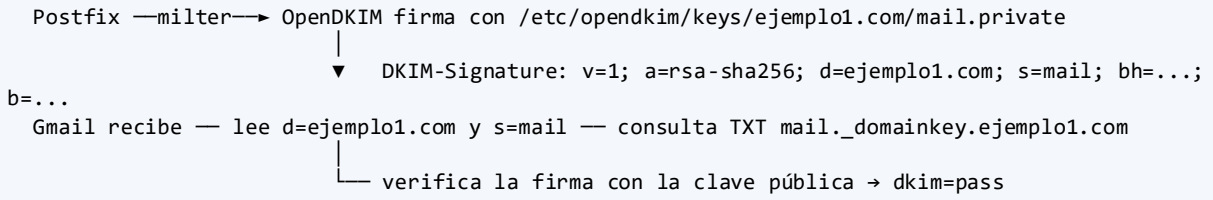
SALIDA ESPERADA

```
"v=spf1 mx ~all"
"v=spf1 a -all"
"v=spf1 mx ~all"
```

9.5 DKIM y OpenDKIM

DKIM (*DomainKeys Identified Mail*) añade a cada correo saliente una **firma digital** en la cabecera `DKIM-Signature`. El servidor firma con una clave **privada** que solo él tiene, y publica la clave **pública** en el DNS. El receptor descarga la clave pública y verifica la firma: si alguien modificó el mensaje por el camino, o lo envió otro servidor, la firma no cuadra.

DIAGRAMA



- **Selector** (`s=mail`): nombre que identifica la clave. Permite tener varias claves (por ejemplo, para rotarlas). Usaremos el selector `mail`.
- **Milter** (*mail filter*): protocolo con el que Postfix pasa cada mensaje a un programa externo (OpenDKIM) antes de enviarlo.

PASO 9.4 – Instalar OpenDKIM

Comando:

TERMINAL

```
sudo apt install -y opendkim opendkim-tools
```

Análisis: `opendkim` es el servicio que firma y verifica; `opendkim-tools` incluye `opendkim-genkey` (generar claves) y `opendkim-testkey` (comprobar el DNS).

PASO 9.5 – Generar las claves de cada dominio

Objetivo: crear un par de claves RSA de 2048 bits para cada dominio.

Comando:

TERMINAL

```
sudo mkdir -p /etc/opendkim/keys/ejemplo1.com /etc/opendkim/keys/ejemplo2.com
sudo opendkim-genkey -b 2048 -d ejemplo1.com -D /etc/opendkim/keys/ejemplo1.com -s mail -v
sudo opendkim-genkey -b 2048 -d ejemplo2.com -D /etc/opendkim/keys/ejemplo2.com -s mail -v
sudo chown -R opendkim:opendkim /etc/opendkim
sudo chmod 700 /etc/opendkim/keys /etc/opendkim/keys/*
sudo chmod 600 /etc/opendkim/keys/*/mail.private
sudo ls -l /etc/opendkim/keys/ejemplo1.com/
```

Análisis del comando:

Opción	Significado
`-b 2048`	Longitud de la clave. 2048 bits es el estándar actual (1024 se considera débil).
`-d ejemplo1.com`	Dominio que firmará.
`-D carpeta`	Dónde guardar los archivos.
`-s mail`	Selector.
`-v`	Muestra lo que hace.

Resultado esperado:

SALIDA ESPERADA

```
-rw------- 1 opendkim opendkim 1679 nov  9 03:00 mail.private
-rw------- 1 opendkim opendkim  503 nov  9 03:00 mail.txt
```

`mail.private` es la clave privada (**nunca** la compartas ni la copies fuera del servidor salvo en copias de seguridad cifradas). `mail.txt` contiene el registro DNS que debes publicar.

PASO 9.6 – Configurar las tablas de OpenDKIM

Objetivo: indicar qué clave usar para cada dominio y qué hosts son de confianza.

Archivo: `/etc/opendkim/key.table` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
# nombre_de_la_clave                dominio:selector:ruta_clave_privada
mail._domainkey.ejemplo1.com
ejemplo1.com:mail:/etc/opendkim/keys/ejemplo1.com/mail.private
mail._domainkey.ejemplo2.com
ejemplo2.com:mail:/etc/opendkim/keys/ejemplo2.com/mail.private
```

Archivo: `/etc/opendkim/signing.table` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
# remitente                         clave con la que se firma
*@ejemplo1.com                      mail._domainkey.ejemplo1.com
*@ejemplo2.com                      mail._domainkey.ejemplo2.com
```

Archivo: `/etc/opendkim/trusted.hosts` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
# Hosts cuyo correo se firma (correo saliente generado en el propio servidor)
127.0.0.1
::1
localhost
mail.ejemplo1.com
```

Análisis:

Tabla	Para qué sirve
`key.table`	Asocia un nombre de clave con su dominio, selector y archivo de clave privada.

Tabla	Para qué sirve
`signing.table`	Decide, según el remitente del mensaje, qué clave usar. `*@ejemplo1.com` = cualquier dirección del dominio.
`trusted.hosts`	El correo procedente de estas direcciones se considera **saliente** y se firma; el resto se considera entrante y se **verifica**. El correo de usuarios autenticados por SMTP (587/465) también se firma.

TERMINAL

```
sudo chown opendkim:opendkim /etc/opendkim/*.table /etc/opendkim/trusted.hosts
```

PASO 9.7 – Configurar OpenDKIM (opendkim.conf)

Comando:

TERMINAL

```
sudo cp /etc/opendkim.conf /etc/opendkim.conf.original
sudo nano /etc/opendkim.conf
```

Archivo: `/etc/opendkim.conf` (sustituye todo el contenido):

ARCHIVO DE CONFIGURACIÓN

```
# ================================================================
#  /etc/opendkim.conf - Firma DKIM para ejemplo1.com y ejemplo2.com
# ================================================================
# Registro en syslog (/var/log/mail.log)
Syslog                  yes
SyslogSuccess           yes
LogWhy                  yes

# Permisos de los archivos que crea y usuario del servicio
UMask                   007
UserID                  opendkim
PidFile                 /run/opendkim/opendkim.pid

# s = firmar el correo saliente, v = verificar el entrante
Mode                    sv
# Normalización de cabeceras/cuerpo antes de firmar (tolerante a cambios menores)
Canonicalization        relaxed/simple
SignatureAlgorithm      rsa-sha256
# Impide que se añada una segunda cabecera From: tras la firma
OversignHeaders         From

# Tablas de claves, de firma y de hosts de confianza
KeyTable                /etc/opendkim/key.table
SigningTable            refile:/etc/opendkim/signing.table
ExternalIgnoreList      refile:/etc/opendkim/trusted.hosts
InternalHosts           refile:/etc/opendkim/trusted.hosts
# El correo que Postfix marca como ORIGINATING (puertos 587/465) también se firma
MTA                     ORIGINATING

# Socket TCP solo local (Postfix funciona en chroot y no ve /run)
Socket                  inet:8891@localhost

# Raíz de confianza DNSSEC para validar las consultas DNS
TrustAnchorFile         /usr/share/dns/root.key
```

Análisis de las directivas principales:

Directiva	Significado
`Mode sv`	Firma (*sign*) y verifica (*verify*).
`Canonicalization relaxed/simple`	`relaxed` en cabeceras (tolera cambios de espacios o mayúsculas que hacen algunos servidores), `simple` en el cuerpo.
`OversignHeaders From`	Protege contra la inserción de un segundo `From:` falso.
`refile:`	El archivo admite expresiones con comodines (`*@ejemplo1.com`).
`MTA ORIGINATING`	Firma el correo que llega por los servicios de Postfix con `milter_macro_daemon_name=ORIGINATING` (los puertos de envío).
`Socket inet:8891@localhost`	Escucha en `127.0.0.1:8891`, inaccesible desde fuera.

PASO 9.8 – Asegurar que el servicio usa el socket configurado

Objetivo: evitar que el arranque del servicio imponga otro socket distinto del de `opendkim.conf`.

Explicación: en Ubuntu, la unidad de systemd de OpenDKIM arranca el programa pasando el socket por línea de comandos (opción `-p`), y la línea de comandos **tiene prioridad** sobre `opendkim.conf`. Lo comprobamos y, para no depender de ello, creamos una configuración adicional (*drop-in*) de systemd que fija nuestro socket.

Comando:

TERMINAL

```
systemctl cat opendkim | grep ExecStart
sudo mkdir -p /etc/systemd/system/opendkim.service.d
sudo tee /etc/systemd/system/opendkim.service.d/override.conf > /dev/null <<'EOF'
[Service]
ExecStart=
ExecStart=/usr/sbin/opendkim -P /run/opendkim/opendkim.pid -p inet:8891@localhost
EOF
sudo systemctl daemon-reload
sudo systemctl restart opendkim
sudo systemctl enable opendkim
sudo ss -tlnp | grep 8891
```

Análisis del comando:

- `systemctl cat opendkim`: muestra la unidad del servicio; verás si incluye `-p local:/run/opendkim/opendkim.sock`.
- En el *drop-in*, la primera línea `ExecStart=` (vacía) **borra** el comando original; la segunda define el nuestro.
- `daemon-reload` hace que systemd relea las unidades.

Resultado esperado:

SALIDA ESPERADA

```
LISTEN 0  4096  127.0.0.1:8891  0.0.0.0:*  users:(("opendkim",pid=8120,fd=3))
```

Errores habituales: si OpenDKIM no arranca, mira `sudo journalctl -u opendkim -n 30 --no-pager`. Mensajes típicos: `opendkim: /etc/opendkim/key.table: ... Permission denied` (revisa propietarios del PASO 9.5) o `can't load key from /etc/opendkim/keys/...: Permission denied`.

PASO 9.9 – Conectar Postfix con OpenDKIM

Archivo: `/etc/postfix/main.cf`. Conserva todo y **añade al final**:

ARCHIVO DE CONFIGURACIÓN

```
# --- Firma DKIM con OpenDKIM (Módulo 9) --------------------------------
milter_default_action = accept
milter_protocol = 6
smtpd_milters = inet:localhost:8891
non_smtpd_milters = $smtpd_milters
```

Análisis:

Parámetro	Significado
`milter_default_action = accept`	Si OpenDKIM no responde, el correo se acepta igualmente (sin firma) en lugar de bloquear todo el servidor.
`milter_protocol = 6`	Versión del protocolo milter.
`smtpd_milters`	Milters para el correo que llega por SMTP (puertos 25, 587, 465).
`non_smtpd_milters`	Milters para el correo generado localmente (`sendmail`, como el de PHP o cron).

Comando:

TERMINAL

```
sudo postfix check && sudo systemctl restart postfix
```

PASO 9.10 – Publicar las claves DKIM en el DNS

Objetivo: publicar la clave pública de cada dominio.

Comando: muestra el contenido de los archivos generados:

TERMINAL

```
sudo cat /etc/opendkim/keys/ejemplo1.com/mail.txt
```

SALIDA ESPERADA

```
mail._domainkey       IN      TXT     ( "v=DKIM1; h=sha256; k=rsa; "
          "p=MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAx3Q...primera parte..."
          "...segunda parte...IDAQAB" )  ; ----- DKIM key mail for ejemplo1.com
```

Explicación: un texto TXT no puede superar 255 caracteres por cadena, y una clave de 2048 bits es más larga, por eso el archivo la divide en varias cadenas entre comillas. En el panel DNS:

- **Nombre**: `mail._domainkey`
- **Tipo**: TXT

- **Valor**: une todas las partes entre comillas en una sola línea, **sin** las comillas ni los paréntesis: `v=DKIM1; h=sha256; k=rsa; p=MIIBIjANBgkq...IDAQAB`. La mayoría de paneles dividen el texto automáticamente. Si tu panel exige las cadenas separadas, pégalas con sus comillas tal como aparecen.

Este comando te muestra el valor ya unido, listo para copiar:

TERMINAL

```
sudo sh -c "tr -d '\n' < /etc/opendkim/keys/ejemplo1.com/mail.txt | sed -e 's/.*( *//' -e 's/*).*//' -e 's/\"[[:space:]]*\"//g' -e 's/\"//g'"; echo
```

Repite para `ejemplo2.com` (archivo `/etc/opendkim/keys/ejemplo2.com/mail.txt`, nombre `mail._domainkey` en la zona de ejemplo2.com).

Comprobación (cuando el registro se haya propagado):

TERMINAL

```
dig +short TXT mail._domainkey.ejemplo1.com
sudo opendkim-testkey -d ejemplo1.com -s mail -vvv
sudo opendkim-testkey -d ejemplo2.com -s mail -vvv
```

Resultado esperado:

SALIDA ESPERADA

```
opendkim-testkey: using default configfile /etc/opendkim.conf
opendkim-testkey: checking key 'mail._domainkey.ejemplo1.com'
opendkim-testkey: key not secure
opendkim-testkey: key OK
```

`key OK` significa que la clave publicada coincide con la privada. `key not secure` solo indica que tu dominio no usa DNSSEC; no es un error.

Errores habituales: `record not found` (aún no propagado o nombre mal escrito); `key data does not match` (copiaste mal el valor o falta un fragmento).

9.6 DMARC

DMARC une SPF y DKIM con el remitente **visible** (`From:`). Un mensaje pasa DMARC si:

- pasa SPF **y** el dominio del `MAIL FROM` coincide (está *alineado*) con el del `From:`, **o**
- pasa DKIM **y** el dominio de la firma (`d=`) coincide con el del `From:`.

Además, el propietario del dominio publica qué debe hacerse con los mensajes que no pasan y a dónde enviar informes.

Etiqueta	Significado	Valores
`v=DMARC1`	Versión (obligatoria, al principio).	
`p=`	Política para el dominio.	`none` (solo informar), `quarantine` (a spam), `reject` (rechazar).
`sp=`	Política para subdominios (si no se indica, usa `p`).	Igual que `p`.
`rua=`	Dirección para informes agregados diarios.	`mailto:dmarc@ejemplo1.com`
`adkim=` / `aspf=`	Alineación estricta (`s`) o relajada (`r`) de DKIM y SPF.	`r` permite subdominios.

Etiqueta	Significado	Valores
`pct=`	Porcentaje de mensajes a los que se aplica la política.	0–100

PASO 9.11 – Crear los registros DMARC

Registros:

Dominio	Tipo	Nombre	Valor
ejemplo1.com	TXT	_dmarc	`v=DMARC1; p=none; rua=mailto:dmarc@ejemplo1.com; adkim=r; aspf=r`
ejemplo2.com	TXT	_dmarc	`v=DMARC1; p=none; rua=mailto:dmarc@ejemplo2.com; adkim=r; aspf=r`

Análisis: empezamos con `p=none` (modo observación). Durante unas semanas recibirás informes XML en `dmarc@` (que es alias de `administracion@`/`soporte@`, PASO 8.8) indicando qué IP envían en nombre de tu dominio y si pasan. Cuando todo pase, sube a `p=quarantine` y después a `p=reject`, que es la protección completa contra la suplantación de tu dominio.

Comprobación:

TERMINAL

```
dig +short TXT _dmarc.ejemplo1.com
dig +short TXT _dmarc.ejemplo2.com
```

9.7 Comprobar la propagación de los registros

Cuando creas o cambias un registro, los resolvedores que tenían en caché la respuesta anterior la mantienen hasta que expira su TTL. Para saber si un cambio ya es visible:

TERMINAL

```
# Preguntar a varios resolvedores públicos
dig +short MX ejemplo1.com @1.1.1.1
dig +short MX ejemplo1.com @8.8.8.8
dig +short MX ejemplo1.com @9.9.9.9
# Preguntar directamente a los servidores autoritativos (la «fuente de verdad»)
dig +short NS ejemplo1.com
dig +short TXT _dmarc.ejemplo1.com @ns1.tuproveedordns.example
# Ver la cadena completa de resolución desde la raíz
dig +trace TXT mail._domainkey.ejemplo1.com
```

Análisis: si los autoritativos ya devuelven el valor nuevo pero `1.1.1.1` no, solo es cuestión de esperar (como máximo el TTL anterior). Si los autoritativos tampoco lo devuelven, el registro está mal creado en el panel. También existen herramientas web que consultan desde muchos países a la vez («DNS propagation checker»).

9.8 Verificación final de la autenticación

PASO 9.12 – Verificar SPF, DKIM y DMARC con un correo real

Objetivo: confirmar que los servidores receptores dan por buenos los tres mecanismos.

Comando: envía un correo desde Thunderbird (o con swaks por el puerto 587) desde `info@ejemplo1.com` a tu cuenta de Gmail. Antes, comprueba en el log que OpenDKIM firma:

TERMINAL
```
sudo tail -f /var/log/mail.log | grep -i dkim
```

SALIDA ESPERADA
```
opendkim[8120]: 7B3C13E101: DKIM-Signature field added (s=mail, d=ejemplo1.com)
```

Comprobación en Gmail: abre el mensaje, pulsa los tres puntos → «Mostrar original». Debes ver:

SALIDA ESPERADA
```
SPF:   PASS con la IP 203.0.113.10
DKIM:  'PASS' con el dominio ejemplo1.com
DMARC: 'PASS'
```

Y en las cabeceras:

SALIDA ESPERADA
```
Authentication-Results: mx.google.com;
       dkim=pass header.i=@ejemplo1.com header.s=mail header.b=Ab3dEfGh;
       spf=pass (google.com: domain of info@ejemplo1.com designates 203.0.113.10 as permitted
sender) smtp.mailfrom=info@ejemplo1.com;
       dmarc=pass (p=NONE sp=NONE dis=NONE) header.from=ejemplo1.com
```

Repite el envío desde `soporte@ejemplo2.com`.

Otras herramientas de verificación:

- **mail-tester.com**: te da una dirección temporal; envías un correo y te devuelve una puntuación sobre 10 con todos los problemas detectados (SPF, DKIM, DMARC, PTR, listas negras, contenido).
- **check-auth@verifier.port25.com**: si le envías un correo, responde con un informe detallado de SPF, DKIM y DMARC.

Errores habituales:

Resultado	Causa	Solución
`dkim=none`	OpenDKIM no firmó.	En el log no aparece `DKIM-Signature field added`: revisa `smtpd_milters`, el socket 8891 y la `signing.table`.
`dkim=fail (bad signature)`	La clave del DNS no corresponde a la privada, o el mensaje se modificó tras firmar.	`opendkim-testkey`; evita reescrituras posteriores (firmas de antivirus, pies añadidos).
`spf=softfail` / `spf=fail`	La IP de envío no está en el SPF (por ejemplo, se envió por IPv6 y el MX no tiene AAAA).	Añade `ip6:` o el registro AAAA, o usa `inet_protocols = ipv4`.

Resultado	Causa	Solución
`dmarc=fail`	SPF y DKIM fallan o no están alineados con el `From:`.	Corrige primero DKIM, que es el mecanismo más robusto.
Gmail rechaza con `550-5.7.25 ... PTR record`	Falta PTR o no coincide (habitual en IPv6).	PASO 9.2 para IPv4 **e** IPv6.

9.9 Reputación de la IP y entregabilidad

Tener los registros perfectos es necesario, pero no suficiente. Los grandes proveedores también valoran la **reputación**:

- **IP limpia**: antes de usar un servidor para correo, comprueba que su IP no está en listas negras (Spamhaus, Barracuda, SpamCop...). Hay comprobadores web que consultan decenas de listas a la vez (busca «multi RBL check»). Si tu IP está listada por un uso anterior, solicita la retirada en la web de la lista o pide otra IP al proveedor.
- **IP residencial**: las IP de conexiones domésticas aparecen en listas como la PBL de Spamhaus y su correo se rechaza casi siempre.
- **Calentamiento** (*warm-up*): una IP nueva no tiene historial. Empieza enviando poco volumen y a destinatarios que interactúan (abren, responden) y aumenta gradualmente.
- **Contenido y comportamiento**: sin listas compradas, con baja tasa de quejas, con enlace de baja en los envíos masivos, sin acortadores de URL sospechosos.
- **Paneles de los proveedores**: Google Postmaster Tools y Microsoft SNDS te muestran cómo ven tu dominio e IP.
- **Coherencia**: `myhostname` = PTR = registro A = nombre en el `EHLO`. Ya lo tenemos.

> **CONSEJO:** Para envíos masivos (boletines con miles de destinatarios), lo recomendable es no usar el mismo servidor e IP que el correo personal: un problema de reputación con el boletín afectaría a todos los buzones.

9.10 Ejercicios prácticos del Módulo 9

1. Envía un correo a `check-auth@verifier.port25.com` desde cada dominio y analiza el informe que recibes.
2. Simula una suplantación: con swaks, conéctate al puerto 25 de Gmail desde otro servidor (no autorizado en tu SPF) usando `--from info@ejemplo1.com`. ¿Qué resultado de SPF y DMARC obtiene? (Solo con direcciones de tu propiedad.)
3. Cambia temporalmente el selector a uno inexistente en la `key.table` y observa qué ocurre con `dkim` en Gmail. Restaura la configuración.
4. Calcula cuántas consultas DNS consume tu registro SPF. ¿Qué pasaría si añadieras 11 `include:`?
5. Tras unos días con `p=none`, abre un informe DMARC recibido en `dmarc@ejemplo1.com` (es un XML comprimido) e identifica las IP y los resultados.

Módulo 10. Certificados SSL/TLS

Objetivos del módulo

- Comprender cómo funciona TLS y qué es exactamente un certificado.
- Conocer Let's Encrypt, el protocolo ACME y los métodos de validación.
- Entender la estructura de archivos de Certbot y qué certificado usa cada servicio.
- Configurar la renovación automática y la recarga de Apache, Postfix y Dovecot tras cada renovación.
- Verificar los certificados de todos los servicios y los parámetros de seguridad TLS.

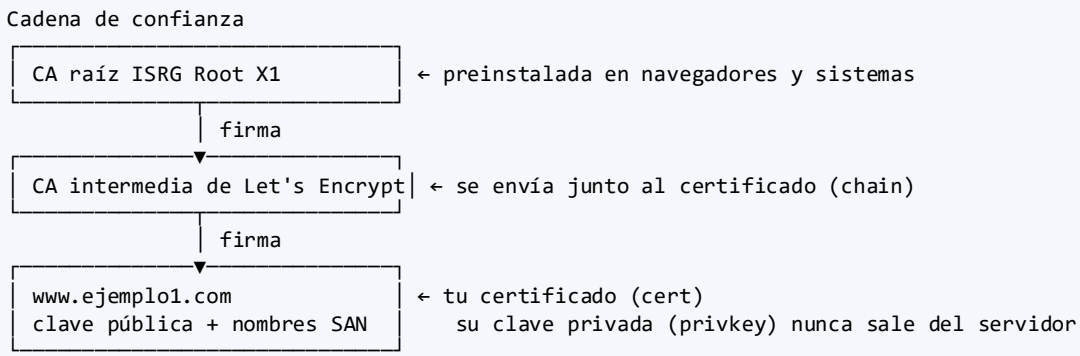
10.1 Cómo funciona TLS

TLS (*Transport Layer Security*, sucesor del antiguo SSL) protege una conexión de tres maneras:

Propiedad	Qué garantiza	Cómo
Confidencialidad	Nadie en el camino puede leer los datos.	Cifrado simétrico (AES, ChaCha20) con una clave de sesión.
Integridad	Nadie puede modificar los datos sin que se detecte.	Códigos de autenticación de mensajes.
Autenticidad	El servidor es quien dice ser.	Certificado firmado por una autoridad de confianza.

Un **certificado** es un documento digital que dice: «la clave pública X pertenece a `www.ejemplo1.com`», firmado por una **Autoridad de Certificación** (CA). Los navegadores y sistemas operativos traen una lista de CA de confianza (en Ubuntu, en `/etc/ssl/certs/`).

DIAGRAMA



El saludo TLS (*handshake*), simplificado:

1. El cliente saluda, indica qué versiones y cifrados admite y el nombre al que quiere conectar (SNI).
2. El servidor responde con su certificado y la cadena intermedia.
3. El cliente verifica la cadena hasta una CA raíz de confianza, que el nombre está en el certificado y que no ha caducado.
4. Ambos acuerdan una clave de sesión mediante un intercambio Diffie-Hellman efímero (ECDHE). El servidor demuestra que posee la clave privada.
5. A partir de ahí, todo viaja cifrado con la clave de sesión.

10.2 Let's Encrypt y ACME

Let's Encrypt es una CA gratuita, automatizada y abierta. Sus certificados:

- Son válidos durante **90 días** (corta duración para limitar los daños si una clave se compromete y para obligar a automatizar).
- Se obtienen con el protocolo **ACME**, que demuestra que controlas el dominio.
- Pueden cubrir varios nombres (campo SAN, *Subject Alternative Name*).

Métodos de validación:

Método	Cómo se demuestra el control	Cuándo usarlo
HTTP-01	Publicando un archivo en `http://dominio/.well-known/acme-challenge/`.	El habitual. Requiere el puerto 80 abierto. **Lo usamos con `--webroot`.**
DNS-01	Creando un registro TXT `_acme-challenge.dominio`.	Certificados comodín (`*.ejemplo1.com`) o servidores sin puerto 80. Requiere un plugin para la API de tu DNS.
TLS-ALPN-01	Por el puerto 443 con un protocolo especial.	Casos muy específicos.

10.3 Los certificados de nuestra infraestructura

Certificado (nombre en Certbot)	Nombres que cubre	Lo usa	Obtenido en
`ejemplo1.com`	ejemplo1.com, www.ejemplo1.com	Apache: VirtualHost 443 de ejemplo1	PASO 3.11
`ejemplo2.com`	ejemplo2.com, www.ejemplo2.com	Apache: VirtualHost 443 de ejemplo2	PASO 3.11
`mail.ejemplo1.com`	mail.ejemplo1.com, mail.ejemplo2.com	Postfix (25, 587, 465) y Dovecot (993, 995)	PASO 6.2

> **CONSEJO: Coherencia entre servicios.** Un cliente solo acepta el certificado si el nombre al que se conecta está dentro de él. Por eso los clientes de correo deben usar `mail.ejemplo1.com` o `mail.ejemplo2.com` (nunca la IP ni `www`), y Postfix y Dovecot comparten el mismo certificado. Apache usa certificados distintos para cada web porque cada VirtualHost presenta el suyo gracias a SNI.

Estructura de archivos de Certbot

DIAGRAMA

```
/etc/letsencrypt/
├── live/                          ← SIEMPRE apunta a la versión vigente (enlaces simbólicos)
│   └── mail.ejemplo1.com/
│       ├── cert.pem      → ../../archive/mail.ejemplo1.com/cert3.pem     certificado
│       ├── chain.pem     → ../../archive/mail.ejemplo1.com/chain3.pem    intermedios
│       ├── fullchain.pem → ../../archive/mail.ejemplo1.com/fullchain3.pem cert + intermedios
│       └── privkey.pem   → ../../archive/mail.ejemplo1.com/privkey3.pem  clave privada
├── archive/                       ← todas las versiones (cert1, cert2, cert3...)
├── renewal/
│   └── mail.ejemplo1.com.conf     ← cómo se obtuvo y cómo renovarlo (método, webroot...)
├── renewal-hooks/
│   ├── pre/                       ← scripts que se ejecutan antes de renovar
│   ├── deploy/                    ← scripts que se ejecutan tras CADA renovación con éxito
│   └── post/                      ← scripts que se ejecutan después de intentar renovar
└── accounts/                      ← tu cuenta ACME
```

ATENCIÓN: Configura siempre los servicios con las rutas de `live/` y con `fullchain.pem` (no `cert.pem`): así, tras cada renovación, solo hay que recargar el servicio. Si usas `cert.pem` sin la cadena, muchos clientes (especialmente de correo y móviles) darán error de certificado.

Consulta tus certificados en cualquier momento:

TERMINAL

```
sudo certbot certificates
```

SALIDA ESPERADA

```
Found the following certs:
  Certificate Name: ejemplo1.com
    Domains: ejemplo1.com www.ejemplo1.com
    Expiry Date: 2027-02-06 22:41:03+00:00 (VALID: 89 days)
    Certificate Path: /etc/letsencrypt/live/ejemplo1.com/fullchain.pem
    Private Key Path: /etc/letsencrypt/live/ejemplo1.com/privkey.pem
  Certificate Name: ejemplo2.com
    ...
  Certificate Name: mail.ejemplo1.com
    Domains: mail.ejemplo1.com mail.ejemplo2.com
    ...
```

10.4 Instalación de Certbot: apt o snap

En el Módulo 3 instalamos Certbot desde los repositorios de Ubuntu (`apt install certbot`). Es una versión estable y se actualiza con el resto del sistema. El proyecto Certbot recomienda oficialmente instalarlo con **snap** para tener siempre la última versión. Ambas opciones son válidas; **no las mezcles**. Si quisieras pasar a snap:

TERMINAL

```
sudo apt remove certbot
sudo snap install --classic certbot
sudo ln -s /snap/bin/certbot /usr/bin/certbot
```

La configuración de `/etc/letsencrypt` se conserva, y la renovación pasa a gestionarla el temporizador del snap (`snap.certbot.renew.timer`).

10.5 Renovación automática

PASO 10.1 – Comprobar el temporizador de renovación

Objetivo: verificar que el sistema intenta renovar los certificados automáticamente.

Explicación: el paquete `certbot` instala un temporizador de systemd que ejecuta `certbot renew` dos veces al día. `renew` solo renueva los certificados a los que les quedan menos de 30 días; el resto los deja intactos.

Comando:

TERMINAL

```
systemctl list-timers | grep -i certbot
systemctl cat certbot.service | grep ExecStart
```

Resultado esperado:

SALIDA ESPERADA

```
Mon 2026-11-09 11:43:00 CET  8h left  Sun 2026-11-08 23:12:01 CET  2h ago  certbot.timer
certbot.service
ExecStart=/usr/bin/certbot -q renew --no-random-sleep-on-renew
```

Errores habituales: si no aparece, actívalo con `sudo systemctl enable --now certbot.timer`.

PASO 10.2 – Simular una renovación

Objetivo: comprobar que la renovación funcionará sin esperar 60 días.

Comando:

TERMINAL

```
sudo certbot renew --dry-run
```

Análisis: `--dry-run` hace todo el proceso contra el entorno de pruebas de Let's Encrypt sin guardar nada. Comprueba que el DNS, el puerto 80 y los webroots siguen bien.

Resultado esperado:

SALIDA ESPERADA

```
Processing /etc/letsencrypt/renewal/ejemplo1.com.conf
Simulating renewal of an existing certificate for ejemplo1.com and www.ejemplo1.com
...
Congratulations, all simulated renewals succeeded:
  /etc/letsencrypt/live/ejemplo1.com/fullchain.pem (success)
  /etc/letsencrypt/live/ejemplo2.com/fullchain.pem (success)
  /etc/letsencrypt/live/mail.ejemplo1.com/fullchain.pem (success)
```

Errores habituales: los mismos del PASO 3.11 (puerto 80 cerrado, DNS que ya no apunta al servidor, webroot que no coincide con el DocumentRoot). Revisa el método guardado en `/etc/letsencrypt/renewal/NOMBRE.conf` (línea `webroot_path`).

PASO 10.3 – Recargar los servicios tras cada renovación (deploy hook)

Objetivo: que Apache, Postfix y Dovecot empiecen a usar el certificado nuevo en cuanto se renueve.

Explicación: los servicios cargan el certificado en memoria al arrancar. Si Certbot renueva el archivo pero nadie recarga el servicio, este seguirá presentando el certificado antiguo hasta que caduque. Los scripts de `renewal-hooks/deploy/` se ejecutan tras cada renovación con éxito y reciben la variable `RENEWED_LINEAGE` con la carpeta del certificado renovado.

Comando:

TERMINAL
```
sudo nano /etc/letsencrypt/renewal-hooks/deploy/recargar-servicios.sh
```

Archivo: `/etc/letsencrypt/renewal-hooks/deploy/recargar-servicios.sh` (nuevo, contenido completo):

TERMINAL
```
#!/bin/bash
# Se ejecuta tras cada renovación correcta de un certificado.
# Certbot define RENEWED_LINEAGE, por ejemplo /etc/letsencrypt/live/mail.ejemplo1.com
set -u

logger -t certbot-deploy "Certificado renovado: ${RENEWED_LINEAGE}"

case "${RENEWED_LINEAGE}" in
    */mail.ejemplo1.com)
        # Certificado del correo: Postfix y Dovecot
        systemctl reload postfix
        systemctl reload dovecot
        logger -t certbot-deploy "Postfix y Dovecot recargados"
        ;;
    *)
        # Certificados de las webs: Apache
        if apachectl configtest >/dev/null 2>&1; then
            systemctl reload apache2
            logger -t certbot-deploy "Apache recargado"
        else
            logger -t certbot-deploy "ERROR: configuración de Apache inválida, no se recarga"
        fi
        ;;
esac
```

Análisis del script:

Línea	Qué hace
`logger -t certbot-deploy`	Escribe un mensaje en el registro del sistema (visible con `journalctl -t certbot-deploy`).
`case ... in */mail.ejemplo1.com)`	Si el certificado renovado es el del correo, recarga Postfix y Dovecot.
`apachectl configtest` antes de recargar	Nunca recarga Apache si su configuración tiene un error (evita tumbar las webs de madrugada).
`systemctl reload`	Recarga sin cortar conexiones.

Hazlo ejecutable y pruébalo simulando la renovación del certificado de correo:

TERMINAL

```
sudo chmod 755 /etc/letsencrypt/renewal-hooks/deploy/recargar-servicios.sh
sudo RENEWED_LINEAGE=/etc/letsencrypt/live/mail.ejemplo1.com /etc/letsencrypt/renewal-
hooks/deploy/recargar-servicios.sh
sudo RENEWED_LINEAGE=/etc/letsencrypt/live/ejemplo1.com /etc/letsencrypt/renewal-
hooks/deploy/recargar-servicios.sh
sudo journalctl -t certbot-deploy -n 10 --no-pager
```

Resultado esperado:

SALIDA ESPERADA

```
nov 09 03:40:11 mail certbot-deploy[9101]: Certificado renovado:
/etc/letsencrypt/live/mail.ejemplo1.com
nov 09 03:40:11 mail certbot-deploy[9105]: Postfix y Dovecot recargados
nov 09 03:40:15 mail certbot-deploy[9112]: Certificado renovado:
/etc/letsencrypt/live/ejemplo1.com
nov 09 03:40:15 mail certbot-deploy[9116]: Apache recargado
```

CONSEJO: `certbot renew --dry-run` **no** ejecuta los *deploy hooks* (porque no se instala ningún certificado real). Por eso los probamos llamando al script directamente, como acabamos de hacer.

10.6 Verificar los certificados de todos los servicios

PASO 10.4 – Comprobar el certificado que presenta cada puerto

Objetivo: confirmar que cada servicio presenta el certificado correcto, vigente y con la cadena completa.

Comandos:

TERMINAL

```
# Web (HTTPS, puerto 443) - con SNI
echo | openssl s_client -connect www.ejemplo1.com:443 -servername www.ejemplo1.com 2>/dev/null
| openssl x509 -noout -subject -issuer -dates -ext subjectAltName
# SMTP con STARTTLS (puertos 25 y 587)
echo | openssl s_client -connect mail.ejemplo1.com:587 -starttls smtp 2>/dev/null | openssl
x509 -noout -subject -dates -ext subjectAltName
# SMTPS (puerto 465, TLS implícito)
echo | openssl s_client -connect mail.ejemplo1.com:465 2>/dev/null | openssl x509 -noout -
subject -dates
# IMAPS (993) y POP3S (995)
echo | openssl s_client -connect mail.ejemplo2.com:993 2>/dev/null | openssl x509 -noout -
subject -dates -ext subjectAltName
echo | openssl s_client -connect mail.ejemplo1.com:995 2>/dev/null | openssl x509 -noout -
subject -dates
```

Análisis del comando:

- `echo |` envía una línea vacía para que `s_client` cierre la conexión tras el saludo.
- `-servername` activa SNI (necesario en Apache para obtener el certificado del VirtualHost correcto).
- `-starttls smtp` inicia la conexión en texto plano y la pasa a TLS con STARTTLS (también existen `-starttls imap` y `-starttls pop3`).

- `openssl x509 -noout -subject -issuer -dates -ext subjectAltName` extrae del certificado el titular, el emisor, las fechas y los nombres que cubre.

Resultado esperado (para el 993 de mail.ejemplo2.com):

SALIDA ESPERADA

```
subject=CN = mail.ejemplo1.com
notBefore=Nov  9 00:41:03 2026 GMT
notAfter=Feb  7 00:41:02 2027 GMT
X509v3 Subject Alternative Name:
    DNS:mail.ejemplo1.com, DNS:mail.ejemplo2.com
```

Para comprobar que la cadena es válida (el cliente puede verificarla):

TERMINAL

```
echo | openssl s_client -connect mail.ejemplo1.com:993 -verify_return_error 2>&1 | grep -E
'Verify return code|Protocol|Cipher'
```

SALIDA ESPERADA

```
Protocol  : TLSv1.3
Cipher    : TLS_AES_256_GCM_SHA384
Verify return code: 0 (ok)
```

`Verify return code: 0 (ok)` significa cadena completa y válida.

PASO 10.5 – Comprobar que los protocolos obsoletos están desactivados

Comando:

TERMINAL

```
openssl s_client -connect www.ejemplo1.com:443 -servername www.ejemplo1.com -tls1_1 </dev/null
2>&1 | grep -E 'alert|Cipher is|error' | head -3
openssl s_client -connect mail.ejemplo1.com:993 -tls1_1 </dev/null 2>&1 | grep -E
'alert|Cipher is|error' | head -3
```

Resultado esperado: un error del tipo `alert protocol version` o `no protocols available` y `Cipher is (NONE)`. Significa que TLS 1.1 es rechazado.

> **CONSEJO:** Para una auditoría completa, usa la herramienta libre **testssl.sh** (`./testssl.sh mail.ejemplo1.com:993`) o, para las webs, el test de **SSL Labs** (en su web). Ambas califican protocolos, cifrados y configuración.

Resumen de la configuración TLS en cada servicio:

Servicio	Archivo	Directivas TLS clave
Apache	`/etc/apache2/sites-available/*.conf` y `mods-available/ssl.conf`	`SSLCertificateFile`, `SSLCertificateKeyFile`, `SSLProtocol`
Postfix entrante	`/etc/postfix/main.cf`	`smtpd_tls_cert_file`, `smtpd_tls_key_file`, `smtpd_tls_protocols`, `smtpd_tls_auth_only`

Servicio	Archivo	Directivas TLS clave
Postfix saliente	`/etc/postfix/main.cf`	`smtp_tls_security_level`, `smtp_tls_CApath`
Postfix 587 / 465	`/etc/postfix/master.cf`	`smtpd_tls_security_level=encrypt` / `smtpd_tls_wrappermode=yes`
Dovecot	`/etc/dovecot/conf.d/10-ssl.conf`	`ssl = required`, `ssl_cert`, `ssl_key`, `ssl_min_protocol`

10.7 Protección de las conexiones de autenticación

Las contraseñas solo deben viajar dentro de TLS. Esta es la lista de comprobación que ya cumple nuestra configuración:

Conexión	Protección	Directiva
Inicio de sesión IMAP/POP3	Solo por 993/995 (TLS implícito); 143/110 deshabilitados.	`port = 0` en `10-master.conf`
Cualquier autenticación en Dovecot	Rechazada sin TLS.	`ssl = required`, `disable_plaintext_auth = yes`
Autenticación SMTP en el puerto 25	Desactivada.	`smtpd_sasl_auth_enable = no`
Autenticación SMTP en 587	Solo tras STARTTLS.	`smtpd_tls_security_level=encrypt`, `smtpd_tls_auth_only=yes`
Autenticación SMTP en 465	TLS desde el inicio.	`smtpd_tls_wrappermode=yes`
Paneles web (si los instalas)	Solo HTTPS + HSTS.	Redirección 80→443 y `Strict-Transport-Security`

Comprobación: intenta autenticarte en el 587 sin STARTTLS:

TERMINAL

```
swaks --server mail.ejemplo1.com --port 587 --auth LOGIN --auth-user info@ejemplo1.com --auth-password prueba --quit-after AUTH
```

SALIDA ESPERADA

```
<** 530 5.7.0 Must issue a STARTTLS command first
```

10.8 Operaciones habituales con certificados

Tarea	Comando
Añadir un nombre a un certificado existente	`sudo certbot certonly --webroot -w /var/www/ejemplo1.com/public_html --cert-name ejemplo1.com -d ejemplo1.com -d www.ejemplo1.com -d tienda.ejemplo1.com`

Tarea	Comando
Forzar la renovación de uno	`sudo certbot renew --cert-name mail.ejemplo1.com --force-renewal` (úsalo con moderación: hay límites)
Revocar (clave comprometida)	`sudo certbot revoke --cert-name ejemplo2.com`
Borrar un certificado que ya no usas	`sudo certbot delete --cert-name ejemplo2.com` (antes quítalo de la configuración de Apache)
Ver los días que le quedan a un certificado	`sudo openssl x509 -enddate -noout -in /etc/letsencrypt/live/ejemplo1.com/cert.pem`

> **CONSEJO: Registro CAA (opcional).** Puedes publicar `ejemplo1.com. CAA 0 issue "letsencrypt.org"` para indicar que solo Let's Encrypt puede emitir certificados para tu dominio. Es una capa adicional de protección frente a emisiones fraudulentas.

> **CONSEJO: Laboratorio.** Si usas el certificado autofirmado y cambias el nombre del servidor, regenera el certificado con `sudo make-ssl-cert generate-default-snakeoil --force-overwrite` y reinicia Apache, Postfix y Dovecot.

10.9 Ejercicios prácticos del Módulo 10

1. Ejecuta `sudo ls -l /etc/letsencrypt/live/ejemplo1.com/` y explica a qué apunta cada enlace.
2. Conéctate al puerto 443 **sin** `-servername` y observa qué certificado recibes. ¿Por qué es el del comodín `default-ssl`?
3. Configura Thunderbird con el servidor `203.0.113.10` (la IP) en lugar de `mail.ejemplo1.com`. ¿Qué aviso aparece y por qué?
4. Escribe un pequeño script que muestre los días que quedan para que caduque cada certificado de `/etc/letsencrypt/live/*/cert.pem` (pista: `openssl x509 -checkend`).
5. Simula una renovación forzada del certificado de correo y verifica en `journalctl -t certbot-deploy` que se recargaron Postfix y Dovecot.

Módulo 11. Integración completa de los servicios

Objetivos del módulo

- Ver la infraestructura como un todo: qué recursos tiene cada dominio y cómo se relacionan.
- Seguir paso a paso el recorrido de una visita web y el de un correo electrónico.
- Conectar el sistema web con el sistema de correo: una aplicación PHP que envía correos firmados con DKIM.
- Disponer de un inventario completo y coherente de la infraestructura.

11.1 Inventario de la infraestructura

Cada dominio es un «inquilino» independiente que comparte el servidor, pero no sus recursos:

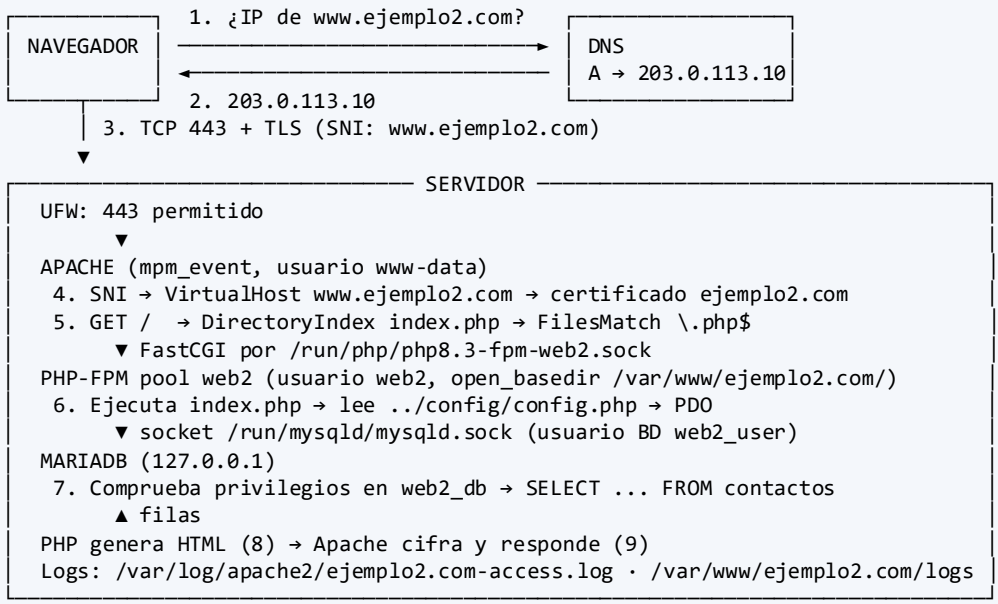
Recurso	ejemplo1.com	ejemplo2.com
Registros DNS web	A `@`, A `www` → 203.0.113.10	A `@`, A `www` → 203.0.113.10
Registros DNS correo	MX → mail.ejemplo1.com; SPF; DKIM `mail._domainkey`; DMARC	MX → mail.ejemplo2.com; SPF; DKIM `mail._domainkey`; DMARC
VirtualHost	`/etc/apache2/sites-available/ejemplo1.com.conf`	`/etc/apache2/sites-available/ejemplo2.com.conf`
Certificado web	`/etc/letsencrypt/live/ejemplo1.com/`	`/etc/letsencrypt/live/ejemplo2.com/`
Usuario Linux de la web	`web1`	`web2`
Carpeta web	`/var/www/ejemplo1.com/public_html`	`/var/www/ejemplo2.com/public_html`
Pool PHP-FPM	`web1` → `/run/php/php8.3-fpm-web1.sock`	`web2` → `/run/php/php8.3-fpm-web2.sock`
Aplicación PHP	Agenda de contactos + formulario de contacto	Agenda de contactos + formulario de contacto
Credenciales	`/var/www/ejemplo1.com/config/config.php`	`/var/www/ejemplo2.com/config/config.php`
Base de datos	`web1_db`	`web2_db`
Usuario de base de datos	`'web1_user'@'localhost'`	`'web2_user'@'localhost'`
Cuentas de correo	info@, administracion@	soporte@
Alias	postmaster@, abuse@, dmarc@, noreply@, root@, equipo@	postmaster@, abuse@, dmarc@, noreply@
Buzones	`/var/mail/vhosts/ejemplo1.com/`	`/var/mail/vhosts/ejemplo2.com/`
Clave DKIM	`/etc/opendkim/keys/ejemplo1.com/mail.private`	`/etc/opendkim/keys/ejemplo2.com/mail.private`

Recurso	ejemplo1.com	ejemplo2.com
Servidor para los clientes de correo	mail.ejemplo1.com	mail.ejemplo2.com

Recursos compartidos: la IP, el nombre `mail.ejemplo1.com` (PTR y `myhostname`), el certificado del correo, los servicios Apache, PHP-FPM, MariaDB, Postfix, Dovecot y OpenDKIM, el cortafuegos y Fail2ban.

11.2 El sistema web: INTERNET → DNS → APACHE → PHP → MARIADB

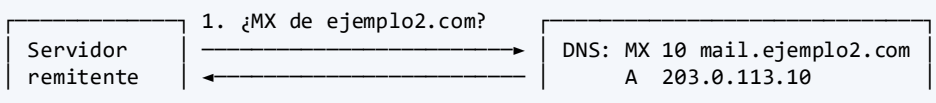
DIAGRAMA



Etapa	Componente	Si falla aquí, verás…
1-2	DNS	`ERR_NAME_NOT_RESOLVED` en el navegador.
3	Red / UFW	El navegador se queda esperando (timeout).
4	Apache / TLS	Aviso de certificado, o la web de otro dominio.
5	Apache → PHP-FPM	503 o el código PHP en texto plano.
6	PHP	500, página en blanco; error en el log de PHP de la web.
7	MariaDB	Mensaje de error de la aplicación; `SQLSTATE` en el log de PHP.

11.3 El sistema de correo: INTERNET → DNS MX → POSTFIX → DOVECOT → BUZÓN → CLIENTE

DIAGRAMA



```
│ (Gmail)     │                                 └──────────────────────────────┘
└─────────────┘
       │ 2. TCP 25 (STARTTLS) · MAIL FROM / RCPT TO:<soporte@ejemplo2.com>
       ▼
┌──────────────────────────────── SERVIDOR ──────────────────────────────────────┐
│  POSTFIX smtpd (25)                                                           │
│   3. Restricciones: HELO válido, remitente con dominio real, ¿destino mío?    │
│   4. virtual_domains: ejemplo2.com ✓  · vmailbox: soporte@ ✓                  │
│   5. Milter OpenDKIM: verifica la firma DKIM del remitente (entrante)         │
│   6. Cola → qmgr → lmtp                                                       │
│        ▼ LMTP por /var/spool/postfix/private/dovecot-lmtp                     │
│  DOVECOT lmtp                                                                 │
│   7. userdb (/etc/dovecot/users): uid 5000, home /var/mail/vhosts/...         │
│   8. Guarda en /var/mail/vhosts/ejemplo2.com/soporte/Maildir/new/             │
│        ▲                                                                      │
│  DOVECOT imap (993)                                                           │
│   10. Autentica a soporte@ejemplo2.com (passdb) y sirve los mensajes          │
└─────────▲─────────────────────────────────────────────────────────────────────┘
          │ 9. IMAP 993 (TLS) usuario soporte@ejemplo2.com
┌─────────┴──────────┐
│ CLIENTE DE CORREO  │  (Thunderbird, móvil)
└────────────────────┘
```

Y en sentido inverso, cuando `soporte@ejemplo2.com` envía:

DIAGRAMA

```
CLIENTE ──587 STARTTLS / 465 TLS──► POSTFIX submission/smtps
    1. AUTH PLAIN → pregunta a DOVECOT por /var/spool/postfix/private/auth → OK
    2. sender_login: ¿soporte@ puede usar MAIL FROM soporte@ejemplo2.com? → sí
    3. Milter OpenDKIM (ORIGINATING): firma con la clave de ejemplo2.com
    4. Cola → smtp → consulta MX del destino → TCP 25 → servidor de destino
       (el destino comprueba PTR, SPF, DKIM y DMARC)
```

11.4 Cómo se relacionan ambos sistemas: la web que envía correos

Las aplicaciones web necesitan enviar correo constantemente: formularios de contacto, confirmaciones de registro, recuperación de contraseñas, avisos de pedidos. Hay dos formas de hacerlo en nuestra infraestructura:

Método	Cómo funciona	Ventajas	Inconvenientes
`sendmail` local (función `mail()` de PHP)	PHP ejecuta `/usr/sbin/sendmail` (que pertenece a Postfix); el mensaje entra por `pickup`, sin red ni contraseñas.	Sencillo, sin credenciales en el código, firmado por DKIM (`non_smtpd_milters`).	Solo funciona en el mismo servidor.
SMTP autenticado (librerías como PHPMailer o Symfony Mailer)	La aplicación se conecta al puerto 587 con usuario y contraseña de una cuenta de correo.	Funciona desde cualquier servidor; control fino.	Hay que guardar otra contraseña en la configuración.

Usaremos el primero, que ya dejamos preparado: en el pool de cada web (PASO 5.3) fijamos `sendmail_path = /usr/sbin/sendmail -t -i -f noreply@ejemplo1.com`, de modo que el **remitente del sobre** de todo lo que envíe la web 1 es `noreply@ejemplo1.com` (alias de `administracion@`, que recibirá rebotes), aunque el código PHP intente otra cosa.

DIAGRAMA



PASO 11.1 – Crear el formulario de contacto de ejemplo1.com

Objetivo: una página que guarda los mensajes de los visitantes en la base de datos y los envía por correo a `info@ejemplo1.com`.

Primero crea la tabla para guardar los mensajes (así no se pierden aunque falle el correo):

TERMINAL

```
sudo mariadb -e "CREATE TABLE web1_db.mensajes (
  id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  mensaje TEXT NOT NULL,
  enviado TINYINT(1) NOT NULL DEFAULT 0,
  creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;"
sudo mariadb -e "CREATE TABLE web2_db.mensajes LIKE web1_db.mensajes;"
```

Como `web1_user` tiene `SELECT, INSERT, UPDATE, DELETE` sobre todas las tablas de `web1_db`, ya puede usar la nueva tabla sin cambiar permisos.

Comando:

TERMINAL

```
sudo -u web1 nano /var/www/ejemplo1.com/public_html/contacto.php
```

Archivo: `/var/www/ejemplo1.com/public_html/contacto.php` (nuevo, contenido completo):

PHP

```
<?php
declare(strict_types=1);

require __DIR__ . '/../app/db.php';
$config = require __DIR__ . '/../config/config.php';

session_start();
if (empty($_SESSION['csrf'])) {
    $_SESSION['csrf'] = bin2hex(random_bytes(32));
}

$errores = [];
$enviado = false;
$datos = ['nombre' => '', 'email' => '', 'mensaje' => ''];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!hash_equals($_SESSION['csrf'], (string)($_POST['csrf'] ?? ''))) {
```

```
        http_response_code(400);
        exit('Petición no válida.');
    }
    // Campo trampa invisible: los robots lo rellenan, las personas no
    if (($_POST['web'] ?? '') !== '') {
        exit('Gracias.');
    }

    $datos = [
        'nombre'  => trim((string)($_POST['nombre'] ?? '')),
        'email'   => trim((string)($_POST['email'] ?? '')),
        'mensaje' => trim((string)($_POST['mensaje'] ?? '')),
    ];

    if ($datos['nombre'] === '' || mb_strlen($datos['nombre']) > 100 || preg_match('/[\r\n]/',
$datos['nombre'])) {
        $errores[] = 'Indica un nombre válido (máximo 100 caracteres).';
    }
    if (!filter_var($datos['email'], FILTER_VALIDATE_EMAIL)) {
        $errores[] = 'Indica un email válido.';
    }
    if ($datos['mensaje'] === '' || mb_strlen($datos['mensaje']) > 5000) {
        $errores[] = 'Escribe un mensaje (máximo 5000 caracteres).';
    }

    if (!$errores) {
        try {
            $pdo = obtenerConexion();
            // 1. Guardar en la base de datos (consulta preparada)
            $stmt = $pdo->prepare('INSERT INTO mensajes (nombre, email, mensaje) VALUES
(:nombre, :email, :mensaje)');
            $stmt->execute($datos);
            $id = (int)$pdo->lastInsertId();

            // 2. Enviar por correo al buzón de la empresa
            $destino = $config['app']['correo_contacto'];
            $dominio = $config['app']['nombre'];
            $asunto  = '=?UTF-8?B?' . base64_encode("Nuevo mensaje web #$id de
{$datos['nombre']}") . '?=';
            $cuerpo  = "Mensaje recibido desde el formulario de $dominio\n\n"
                     . "Nombre: {$datos['nombre']}\n"
                     . "Email:  {$datos['email']}\n"
                     . "Fecha:  " . date('Y-m-d H:i:s') . "\n\n"
                     . $datos['mensaje'] . "\n";
            $cabeceras = [
                'From'                      => "Formulario web <noreply@$dominio>",
                'Reply-To'                  => $datos['email'],
                'MIME-Version'              => '1.0',
                'Content-Type'              => 'text/plain; charset=UTF-8',
                'Content-Transfer-Encoding' => '8bit',
                'X-Mailer'                  => 'Formulario ' . $dominio,
            ];

            if (mail($destino, $asunto, $cuerpo, $cabeceras)) {
                $pdo->prepare('UPDATE mensajes SET enviado = 1 WHERE id = :id')->execute(['id'
=> $id]);
                $enviado = true;
            } else {
                error_log("Contacto: mail() devolvió false para el mensaje #$id");
                $errores[] = 'Hemos guardado tu mensaje, pero no se ha podido enviar el aviso.
Lo revisaremos.';
            }
        } catch (PDOException $ex) {
            error_log('Contacto: error de base de datos: ' . $ex->getMessage());
```

```
            $errores[] = 'Se ha producido un error interno. Inténtalo más tarde.';
        }
    }
}
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Contacto · <?= e($config['app']['nombre']) ?></title>
    <link rel="stylesheet" href="/estilo.css">
</head>
<body>
<main>
    <h1>Contacto · <?= e($config['app']['nombre']) ?></h1>
    <?php if ($enviado): ?>
        <p class="ok">Gracias, <?= e($datos['nombre']) ?>. Hemos recibido tu mensaje y te
responderemos pronto.</p>
    <?php else: ?>
        <?php foreach ($errores as $err): ?><p class="error"><?= e($err) ?></p><?php
endforeach; ?>
        <form method="post" action="/contacto.php">
            <input type="hidden" name="csrf" value="<?= e($_SESSION['csrf']) ?>">
            <div style="position:absolute;left:-9999px" aria-hidden="true">
                <label>No rellenar <input name="web" tabindex="-1" autocomplete="off"></label>
            </div>
            <label>Nombre <input name="nombre" required maxlength="100" value="<?=
e($datos['nombre']) ?>"></label>
            <label>Email <input name="email" type="email" required maxlength="150" value="<?=
e($datos['email']) ?>"></label>
            <label>Mensaje <textarea name="mensaje" rows="6" required maxlength="5000"><?=
e($datos['mensaje']) ?></textarea></label>
            <button type="submit">Enviar</button>
        </form>
    <?php endif; ?>
    <p><a href="/">Volver a la agenda</a></p>
</main>
</body>
</html>
```

Análisis de las decisiones de seguridad:

Elemento	Por qué
`preg_match('/[\r\n]/', nombre)`	Impide la **inyección de cabeceras**: si un atacante pudiera meter saltos de línea en un dato que acaba en una cabecera, podría añadir destinatarios (`Bcc:`) y usar tu formulario para enviar spam.
`filter_var(FILTER_VALIDATE_EMAIL)`	El email del visitante va en `Reply-To`; debe ser una dirección válida y sin saltos de línea.
`From: noreply@ejemplo1.com`	El remitente visible es siempre **tu** dominio, nunca el email del visitante. Si pusieras el del visitante (por ejemplo `@gmail.com`), fallaría DMARC porque tu servidor no está autorizado a enviar en nombre de gmail.com.
`Reply-To: email del visitante`	Al pulsar «Responder» en el buzón, la respuesta va al visitante.
Asunto codificado `=?UTF-8?B?...?=`	Permite tildes y eñes en el asunto de forma estándar.
Guardar antes de enviar	Si falla el correo, el mensaje no se pierde.
Campo trampa `web`	Filtro sencillo contra robots de spam.

Elemento	Por qué
`mail()` con array de cabeceras	Desde PHP 7.2 se aceptan como array, más limpio y seguro que construir el texto a mano.

TERMINAL
```
sudo chmod 640 /var/www/ejemplo1.com/public_html/contacto.php
```

Despliega lo mismo en ejemplo2.com (su `config.php` ya indica `soporte@ejemplo2.com` como destino y su pool usa `noreply@ejemplo2.com` como remitente del sobre):

TERMINAL
```
sudo install -o web2 -g www-data -m 640 /var/www/ejemplo1.com/public_html/contacto.php
/var/www/ejemplo2.com/public_html/contacto.php
```

PASO 11.2 – Probar el envío desde la web y seguir el correo de extremo a extremo

Objetivo: verificar la integración completa web → base de datos → correo → buzón.

Comando: abre el log en un terminal:

TERMINAL
```
sudo tail -f /var/log/mail.log
```

Y en el navegador, entra en `https://www.ejemplo1.com/contacto.php`, rellena el formulario y envíalo.

Resultado esperado en el log:

SALIDA ESPERADA
```
postfix/pickup[9301]: A1B2C3E110: uid=998 from=<noreply@ejemplo1.com>
postfix/cleanup[9310]: A1B2C3E110: message-id=<20261109041201.A1B2C3E110@mail.ejemplo1.com>
opendkim[8120]: A1B2C3E110: DKIM-Signature field added (s=mail, d=ejemplo1.com)
postfix/qmgr[9302]: A1B2C3E110: from=<noreply@ejemplo1.com>, size=1180, nrcpt=1 (queue active)
postfix/lmtp[9312]: A1B2C3E110: to=<info@ejemplo1.com>,
relay=mail.ejemplo1.com[private/dovecot-lmtp], status=sent (250 2.0.0 <info@ejemplo1.com> ...
Saved)
```

Interpretación línea a línea:

1. `pickup ... uid=998 from=<noreply@ejemplo1.com>`: el correo lo generó el usuario Linux 998 (`web1`) con el remitente de sobre que fijamos en su pool.
2. `opendkim ... DKIM-Signature field added (s=mail, d=ejemplo1.com)`: OpenDKIM lo firmó (el correo local pasa por `non_smtpd_milters`).
3. `lmtp ... to=<info@ejemplo1.com> ... Saved`: Dovecot lo guardó en el buzón.

Comprobación final:

TERMINAL
```
sudo mariadb -e "SELECT id, nombre, email, enviado, creado_en FROM web1_db.mensajes;"
sudo doveadm search -u info@ejemplo1.com mailbox INBOX header X-Mailer "Formulario"
```

En Thunderbird, el mensaje aparece en la bandeja de `info@ejemplo1.com`; al pulsar «Responder», el destinatario es el email del visitante.

Errores habituales:

Síntoma	Causa	Solución
`mail() devolvió false` en el log de PHP	`sendmail_path` incorrecto o Postfix detenido.	`ls -l /usr/sbin/sendmail`, `systemctl status postfix@-`.
`warning: mail_queue_enter: create file maildrop/...: Permission denied`	Permisos de la cola de Postfix alterados.	`sudo postfix set-permissions` y `sudo postfix check`.
El correo no aparece en el log	`disable_functions` incluye `mail` o hay un error de PHP antes.	Revisa el log de PHP de la web.
Llega sin firma DKIM	Falta `non_smtpd_milters`.	PASO 9.9.

11.5 Dependencias entre servicios

Saber qué depende de qué te permite diagnosticar más rápido y reiniciar en el orden correcto:

Servicio	Depende de	Si se detiene…
MariaDB	Disco	Las webs muestran errores de base de datos; lo estático sigue funcionando.
PHP-FPM	MariaDB (para las apps), Postfix (para `mail()`)	Las páginas `.php` dan 503; lo estático funciona.
Apache	PHP-FPM, certificados	Ninguna web responde. Certbot no puede renovar por webroot.
Dovecot	Certificado, `/etc/dovecot/users`	Los usuarios no leen correo; Postfix no puede entregar (los mensajes esperan en cola) ni autenticar envíos.
OpenDKIM	Claves	Con `milter_default_action = accept` el correo sale **sin firmar** (puede ir a spam).
Postfix	Dovecot (entrega y SASL), OpenDKIM	No se recibe ni envía correo; los servidores remitentes reintentarán durante días.
Certbot	Apache (webroot en el puerto 80), DNS	Los certificados caducan a los 90 días.

11.6 Ejercicios prácticos del Módulo 11

1. Detén Dovecot, envía el formulario de contacto, observa con `mailq` que el correo queda en cola y arranca Dovecot de nuevo. ¿Se pierde el mensaje?
2. Modifica `contacto.php` para enviar además un acuse de recibo al visitante. ¿Qué remitente y qué `Reply-To` usarías? ¿Qué riesgo tiene enviar correos a direcciones que escribe cualquiera?
3. Comprueba en Gmail que un correo enviado por el formulario de ejemplo2.com pasa SPF, DKIM y DMARC con el dominio ejemplo2.com.
4. Dibuja de memoria los dos diagramas de este módulo e indica en qué archivo se configura cada flecha.

5. Instala PHPMailer en ejemplo2.com (con Composer, en una carpeta fuera de `public_html`) y envía el formulario por SMTP autenticado al puerto 587 usando una cuenta de correo. Compara las dos formas de envío en el log.

Módulo 12. Seguridad y mantenimiento

Objetivos del módulo

- Dejar el cortafuegos UFW con solo los puertos imprescindibles.
- Repasar y reforzar la seguridad de SSH, MariaDB, Apache, Postfix y Dovecot.
- Proteger el servidor contra ataques de fuerza bruta con Fail2ban.
- Automatizar las actualizaciones de seguridad y las copias de seguridad.
- Monitorizar los servicios, revisar los logs y saber recuperarse de los fallos más habituales.

12.1 El modelo de seguridad por capas

Ninguna medida es infalible. La seguridad profesional se construye en capas: si una falla, la siguiente sigue protegiendo.

DIAGRAMA

```
Internet
   |
[1] Cortafuegos UFW ............ solo 22, 25, 80, 443, 465, 587, 993, 995
   |
[2] Fail2ban ................... bloquea IP que fallan contraseñas repetidamente
   |
[3] Servicios endurecidos ...... SSH solo con clave, TLS obligatorio, sin relay abierto
   |
[4] Aislamiento ................ un usuario Linux, un pool PHP y un usuario BD por web
   |
[5] Mínimo privilegio .......... MariaDB solo local, permisos 640/750/600, open_basedir
   |
[6] Actualizaciones ............ parches de seguridad automáticos
   |
[7] Copias de seguridad ........ si todo lo demás falla, se puede reconstruir
   |
[8] Monitorización y logs ...... para enterarte antes que tus usuarios
```

12.2 Configuración definitiva de UFW

PASO 12.1 – Revisar y ordenar las reglas del cortafuegos

Objetivo: comprobar que solo están abiertos los puertos necesarios.

Comando:

TERMINAL

```
sudo ufw status numbered
```

Resultado esperado (IPv4; cada regla aparece también con `(v6)`):

SALIDA ESPERADA

```
Status: active
     To                         Action      From
     --                         ------      ----
[ 1] OpenSSH                    ALLOW IN    Anywhere
[ 2] Apache Full                ALLOW IN    Anywhere
[ 3] 25/tcp                     ALLOW IN    Anywhere      # SMTP recepcion
[ 4] 587/tcp                    ALLOW IN    Anywhere      # SMTP submission
[ 5] 465/tcp                    ALLOW IN    Anywhere      # SMTPS
[ 6] 993/tcp                    ALLOW IN    Anywhere      # IMAPS
[ 7] 995/tcp                    ALLOW IN    Anywhere      # POP3S
```

Tabla de referencia de puertos del servidor:

Puerto	Servicio	Abierto en UFW	Escucha en	Motivo
21/tcp	vsftpd (FTPS)	Solo con el Módulo 15	Todas	Subida de archivos de las webs, cifrada.
22/tcp	SSH	Sí	Todas	Administración (solo con clave).
25/tcp	Postfix smtpd	Sí	Todas	Recepción de correo desde Internet.
80/tcp	Apache	Sí	Todas	Redirección a HTTPS y validación de Let's Encrypt.
443/tcp	Apache	Sí	Todas	Webs HTTPS.
465/tcp	Postfix smtps	Sí	Todas	Envío autenticado (TLS implícito).
587/tcp	Postfix submission	Sí	Todas	Envío autenticado (STARTTLS).
993/tcp	Dovecot IMAPS	Sí	Todas	Lectura de correo.
995/tcp	Dovecot POP3S	Sí (si usas POP3)	Todas	Lectura de correo.
3306/tcp	MariaDB	**No**	127.0.0.1	Solo uso interno.
8891/tcp	OpenDKIM	**No**	127.0.0.1	Solo uso interno.
40000–40100/tcp	vsftpd (pasivo)	Solo con el Módulo 15	Todas	Conexiones de datos FTP.
110, 143	POP3/IMAP sin TLS	**No**	(deshabilitados)	Inseguros.

Si no usas POP3, cierra el 995 en Dovecot (pon `port = 0` también en `pop3s`) y en UFW:

TERMINAL

```
sudo ufw delete allow 995/tcp
```

CONSEJO: Restringir SSH a tu IP. Si administras siempre desde una IP fija (la de tu oficina o una VPN), puedes limitar SSH a ella: `sudo ufw allow from 198.51.100.7 to any port 22 proto tcp` y después `sudo ufw delete allow OpenSSH`. Asegúrate antes de que tienes acceso a la consola web del proveedor por si cambia tu IP.

Comprueba también qué escucha realmente el servidor (lo que no aparezca aquí no es accesible aunque el cortafuegos lo permita):

TERMINAL
```
sudo ss -tlnp | awk 'NR==1 || /LISTEN/' | sort -k4
```

12.3 Protección de SSH

Ya en el Módulo 1 desactivamos el acceso de root y por contraseña. Repasemos el estado y añadamos Fail2ban más adelante.

PASO 12.2 – Verificar el endurecimiento de SSH

Comando:

TERMINAL
```
sudo sshd -T | grep -Ei
'^(permitrootlogin|passwordauthentication|kbdinteractiveauthentication|pubkeyauthentication|maxauthtries|al
```

Resultado esperado:

SALIDA ESPERADA
```
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
maxauthtries 3
x11forwarding no
allowusers sysadmin
```

Buenas prácticas adicionales:

- Protege tu clave privada con frase de paso y guarda una copia en lugar seguro.
- Si varias personas administran el servidor, cada una con su propio usuario y su propia clave (nunca compartidas). Así sabrás quién hizo qué (`/var/log/auth.log`) y podrás revocar a una persona sin afectar al resto.
- Revisa periódicamente `~/.ssh/authorized_keys` y elimina las claves que ya no se usen.
- Cambiar el puerto 22 reduce el ruido en los logs, pero no es una medida de seguridad real. En Ubuntu 24.04 SSH se activa por socket (`ssh.socket`): si cambias el puerto debes hacerlo con un *drop-in* de `ssh.socket` (`ListenStream=`), no solo en `sshd_config`.

12.4 Protección de MariaDB

Medida	Comprobación
Solo escucha en local	`sudo ss -tlnp \`
Puerto cerrado en UFW	`sudo ufw status \`
root solo por unix_socket	`mariadb -u root` sin sudo → `Access denied`
Sin usuarios anónimos ni base `test`	`sudo mariadb -e "SELECT user,host FROM mysql.user;"`

Medida	Comprobación
Sin usuarios con host `%`	En la consulta anterior no debe haber `%`
Privilegios mínimos por aplicación	`SHOW GRANTS FOR 'web1_user'@'localhost';`
Copias diarias	PASO 12.9

> **ATENCIÓN:** Si algún día necesitas administrar MariaDB con una herramienta gráfica desde tu ordenador, **no** abras el puerto 3306. Usa un túnel SSH: `ssh -L 3307:127.0.0.1:3306 sysadmin@mail.ejemplo1.com` y conecta tu herramienta a `127.0.0.1:3307`. El tráfico viaja cifrado por SSH y MariaDB sigue siendo invisible desde Internet.

12.5 Seguridad de Apache

PASO 12.3 – Ocultar información y añadir cabeceras de seguridad

Objetivo: que Apache no revele su versión ni la del sistema, y que los navegadores apliquen protecciones adicionales.

Comando:

TERMINAL
```
sudo nano /etc/apache2/conf-available/security.conf
```

Archivo: `/etc/apache2/conf-available/security.conf`. Localiza y deja así estas directivas (están en el archivo; cambia su valor y conserva el resto):

ARCHIVO DE CONFIGURACIÓN
```
ServerTokens Prod
ServerSignature Off
TraceEnable Off
```

Después crea un archivo propio con las cabeceras de seguridad:

Archivo: `/etc/apache2/conf-available/cabeceras-seguridad.conf` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN
```
# Cabeceras de seguridad para todas las webs
<IfModule mod_headers.c>
    # Impide que el navegador "adivine" tipos de archivo
    Header always set X-Content-Type-Options "nosniff"
    # Impide que otras webs muestren las nuestras dentro de un marco (clickjacking)
    Header always set X-Frame-Options "SAMEORIGIN"
    # Limita la información de procedencia enviada a otras webs
    Header always set Referrer-Policy "strict-origin-when-cross-origin"
    # Desactiva funciones del navegador que nuestras webs no usan
    Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
</IfModule>

# Bloquea el acceso a archivos ocultos y de copia que puedan quedar en las webs
<FilesMatch "(^\.(?!well-known)|\.(bak|old|orig|swp|sql|ini|log)$|~$)">
    Require all denied
</FilesMatch>
```

Análisis:

Directiva	Efecto
`ServerTokens Prod`	La cabecera `Server` muestra solo `Apache`, sin versión ni sistema.
`ServerSignature Off`	Las páginas de error no muestran la versión.
`TraceEnable Off`	Desactiva el método HTTP TRACE (útil en algunos ataques).
`<FilesMatch ...>`	Deniega archivos que empiezan por punto (`.env`, `.git`...) salvo `.well-known`, y copias como `index.php.bak` o volcados `.sql` olvidados.

Comando:

TERMINAL

```
sudo a2enconf security cabeceras-seguridad
sudo apachectl configtest && sudo systemctl reload apache2
curl -sI https://www.ejemplo1.com | grep -Ei '^(server|x-content|x-frame|referrer|strict)'
```

Resultado esperado:

SALIDA ESPERADA

```
server: Apache
strict-transport-security: max-age=15552000
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
referrer-policy: strict-origin-when-cross-origin
```

Más medidas para Apache:

- Desactiva los módulos que no uses: `sudo apachectl -M` y `sudo a2dismod autoindex status` (el módulo `status` expone información del servidor; `autoindex` genera listados de carpetas).
- Mantén `Options -Indexes` y `AllowOverride None` en todos los VirtualHosts.
- No dejes scripts de prueba (`phpinfo()`, `prueba-bd.php`) en producción.
- Mantén actualizadas las aplicaciones web (WordPress, plugins...): son la vía de entrada más frecuente.

12.6 Seguridad de Postfix y Dovecot

Lista de comprobación de lo ya configurado:

Medida	Dónde	Cómo comprobarla
Sin relay abierto	`smtpd_relay_restrictions`	Conversación manual del PASO 6.9 → `Relay access denied`.
Autenticación solo con TLS y solo en 587/465	`master.cf`, `smtpd_tls_auth_only`	PASO 10.7.
Remitente ligado al usuario	`reject_sender_login_mismatch`	PASO 8.2.
Direcciones inexistentes rechazadas en el acto	`virtual_mailbox_maps`	`noexiste@ejemplo1.com` → 550.
Sin `VRFY`	`disable_vrfy_command`	`nc mail.ejemplo1.com 25` → `VRFY info` → `502`.

Medida	Dónde	Cómo comprobarla
IMAP/POP3 solo cifrados	`10-master.conf`	`ss -tlnp` no muestra 143 ni 110.
Contraseñas con hash SHA512-CRYPT	`/etc/dovecot/users`	`sudo cut -d: -f2 /etc/dovecot/users \`
Buzones inaccesibles para otros usuarios	`/var/mail/vhosts` (vmail, 700)	`sudo -u web1 ls /var/mail/vhosts` → `Permission denied`.

PASO 12.4 – Limitar la velocidad de los clientes SMTP

Objetivo: dificultar que una cuenta robada o un atacante envíe miles de correos o pruebe contraseñas sin control.

Archivo: `/etc/postfix/main.cf`. Conserva todo y añade al final:

ARCHIVO DE CONFIGURACIÓN

```
# --- Límites de conexión y envío (Módulo 12) ------------------------------
smtpd_client_connection_rate_limit = 30
smtpd_client_message_rate_limit = 100
smtpd_client_recipient_rate_limit = 200
anvil_rate_time_unit = 60s
smtpd_error_sleep_time = 5s
smtpd_soft_error_limit = 5
smtpd_hard_error_limit = 10
```

Análisis: cada cliente (IP) puede abrir como máximo 30 conexiones, enviar 100 mensajes y 200 destinatarios por minuto (`anvil_rate_time_unit`). Tras 5 errores, Postfix espera 5 segundos entre respuestas; tras 10, corta la conexión. Los clientes de `mynetworks` (el propio servidor) están exentos.

TERMINAL

```
sudo postfix check && sudo systemctl reload postfix
```

12.7 Protección contra fuerza bruta con Fail2ban

Fail2ban lee los logs, detecta patrones de fallos de autenticación (contraseñas incorrectas en SSH, SMTP, IMAP…) y, cuando una IP supera un número de intentos en un tiempo dado, la bloquea en el cortafuegos durante un periodo.

DIAGRAMA

```
/var/log/auth.log, /var/log/mail.log, journal
         │ (lee continuamente)
         ▼
FAIL2BAN ── filtro (expresiones regulares) ── ¿5 fallos de la IP X en 10 min?
         │ sí
         ▼
Regla de cortafuegos: DROP desde IP X durante 1 hora  →  el atacante deja de llegar
```

Conceptos: un **jail** (cárcel) combina un **filtro** (qué buscar en qué log) con una **acción** (cómo bloquear) y unos parámetros: `maxretry` (intentos), `findtime` (ventana de tiempo) y `bantime` (duración del bloqueo).

PASO 12.5 – Instalar y configurar Fail2ban

Comando:

TERMINAL

```
sudo apt install -y fail2ban
```

ATENCIÓN: Nunca edites `/etc/fail2ban/jail.conf`: se sobrescribe al actualizar. Las configuraciones propias van en `/etc/fail2ban/jail.local` o en `/etc/fail2ban/jail.d/*.local`.

Archivo: `/etc/fail2ban/jail.local` (nuevo, contenido completo):

ARCHIVO DE CONFIGURACIÓN

```
[DEFAULT]
# IP que nunca se bloquean (el propio servidor; añade tu IP fija si la tienes)
ignoreip = 127.0.0.1/8 ::1
# Bloqueo de 1 hora tras 5 fallos en 10 minutos
bantime  = 1h
findtime = 10m
maxretry = 5

# --- SSH ---
[sshd]
enabled  = true
backend  = systemd
maxretry = 3

# --- Postfix: clientes que provocan rechazos repetidos ---
[postfix]
enabled  = true
mode     = more
logpath  = /var/log/mail.log
backend  = auto

# --- Postfix: contraseñas SMTP incorrectas (puertos 587/465) ---
[postfix-sasl]
enabled  = true
logpath  = /var/log/mail.log
backend  = auto

# --- Dovecot: contraseñas IMAP/POP3 incorrectas ---
[dovecot]
enabled  = true
logpath  = /var/log/mail.log
backend  = auto

# --- Reincidentes: quien es bloqueado 3 veces en un día, 1 semana fuera ---
[recidive]
enabled  = true
logpath  = /var/log/fail2ban.log
bantime  = 1w
findtime = 1d
maxretry = 3
```

Análisis:

Jail	Qué vigila
`sshd`	Fallos de inicio de sesión SSH (lee el journal de systemd).

Jail	Qué vigila
`postfix` (`mode = more`)	Clientes SMTP rechazados repetidamente (relay denegado, destinatarios inexistentes, HELO inválido...).
`postfix-sasl`	`SASL LOGIN authentication failed` en los puertos de envío.
`dovecot`	`auth failed` en IMAP/POP3.
`recidive`	El propio log de Fail2ban: castiga a los reincidentes con un bloqueo largo.

Comando:

TERMINAL

```
sudo fail2ban-client -t
sudo systemctl enable --now fail2ban
sudo systemctl restart fail2ban
sudo fail2ban-client status
```

Resultado esperado:

SALIDA ESPERADA

```
OK: configuration test is successful
Status
|- Number of jail:      5
`- Jail list:   dovecot, postfix, postfix-sasl, recidive, sshd
```

Comprobación: consulta un jail concreto:

TERMINAL

```
sudo fail2ban-client status postfix-sasl
```

SALIDA ESPERADA

```
Status for the jail: postfix-sasl
|- Filter
|  |- Currently failed: 1
|  |- Total failed:     14
|  `- File list:        /var/log/mail.log
`- Actions
   |- Currently banned: 2
   |- Total banned:     6
   `- Banned IP list:   192.0.2.45 198.51.100.200
```

Operaciones habituales:

Tarea	Comando
Ver todos los jails	`sudo fail2ban-client status`
Desbloquear una IP	`sudo fail2ban-client set sshd unbanip 198.51.100.7`
Bloquear manualmente	`sudo fail2ban-client set sshd banip 192.0.2.45`
Probar un filtro contra un log	`sudo fail2ban-regex /var/log/mail.log /etc/fail2ban/filter.d/dovecot.conf`
Ver el log de Fail2ban	`sudo tail -f /var/log/fail2ban.log`

Errores habituales:

- `Have not found any log file for ... jail`: el archivo indicado en `logpath` no existe. Comprueba que `rsyslog` está instalado y activo (`systemctl status rsyslog`); en Debian 12, que no trae rsyslog por defecto, usa `backend = systemd` también en los jails de correo.
- Fail2ban no arranca en Ubuntu 24.04 recién instalado con un error de Python (`ModuleNotFoundError: No module named 'asynchat'`): instala las actualizaciones pendientes (`sudo apt update && sudo apt full-upgrade`), que incluyen la versión corregida del paquete.
- Te bloqueas a ti mismo: entra por la consola web del proveedor y usa `unbanip`; añade tu IP a `ignoreip`.

12.8 Actualizaciones automáticas de seguridad

PASO 12.6 – Configurar unattended-upgrades

Objetivo: que los parches de seguridad se instalen solos cada día.

Comando:

TERMINAL

```
sudo apt install -y unattended-upgrades apt-listchanges
sudo dpkg-reconfigure -plow unattended-upgrades
```

Responde **Yes** a «Automatically download and install stable updates?». Esto crea `/etc/apt/apt.conf.d/20auto-upgrades`.

Para recibir un aviso por correo cuando se instalen actualizaciones, edita:

Archivo: `/etc/apt/apt.conf.d/50unattended-upgrades`. Busca estas líneas (están comentadas con `//`), quita las barras y deja estos valores:

ARCHIVO DE CONFIGURACIÓN

```
Unattended-Upgrade::Mail "root";
Unattended-Upgrade::MailReport "on-change";
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false";
```

Análisis: `Mail "root"` envía el informe a root (que es alias de `administracion@ejemplo1.com`); `on-change` solo cuando se instala algo; `Automatic-Reboot "false"` evita reinicios sorpresa: tú decides cuándo reiniciar (el archivo `/var/run/reboot-required` te avisa de que hace falta).

Comprobación:

TERMINAL

```
cat /etc/apt/apt.conf.d/20auto-upgrades
sudo unattended-upgrade --dry-run --debug 2>&1 | tail -5
```

SALIDA ESPERADA

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

> **CONSEJO:** Las actualizaciones de versión mayor (por ejemplo, de Ubuntu 24.04 a 26.04) **nunca** son automáticas. Requieren planificación, copia de seguridad completa y revisión de los cambios (en este caso, el paso a Dovecot 2.4).

12.9 Copias de seguridad automáticas

Regla **3-2-1**: al menos 3 copias de los datos, en 2 soportes distintos, y 1 fuera del servidor (en otro centro de datos). Una copia que solo está en el mismo disco no te protege de un fallo del disco, de un borrado del proveedor ni de un ataque de *ransomware*.

Qué hay que copiar:

Qué	Ruta	Por qué
Bases de datos	(volcado con `mariadb-dump`)	Los archivos de `/var/lib/mysql` no se pueden copiar en caliente de forma fiable.
Webs	`/var/www/`	Código, configuración y archivos subidos.
Buzones	`/var/mail/vhosts/`	Todo el correo.
Configuración	`/etc/`	Apache, PHP, Postfix, Dovecot, OpenDKIM, usuarios de correo...
Certificados y cuenta ACME	`/etc/letsencrypt/` (incluido en `/etc`)	Evita reemitir y conserva la cuenta.
Claves DKIM	`/etc/opendkim/` (incluido en `/etc`)	Si las pierdes, tendrás que publicar claves nuevas en el DNS.
Scripts propios	`/usr/local/sbin/`	Tus herramientas de administración.

PASO 12.7 – Crear el script de copia de seguridad

Archivo: `/usr/local/sbin/backup-servidor` (nuevo, contenido completo):

TERMINAL

```
#!/bin/bash
# ======================================================================
#  backup-servidor - Copia diaria de bases de datos, webs, correo y configuración
#  Se ejecuta desde cron. Conserva los últimos DIAS días.
# ======================================================================
set -euo pipefail

DESTINO=/var/backups/servidor
DIAS=7
FECHA=$(date +%F)
DIR="$DESTINO/$FECHA"
LOG=/var/log/backup-servidor.log

exec >>"$LOG" 2>&1
echo "=== $(date '+%F %T') Inicio de la copia ==="

umask 077
mkdir -p "$DIR"

# 1. Bases de datos: un archivo por base (excepto las internas)
```

```
for BD in $(mariadb -N -e "SHOW DATABASES" | grep -Ev
'^(information_schema|performance_schema|sys)$'); do
    mariadb-dump --single-transaction --routines --triggers --events --databases "$BD" | gzip
> "$DIR/bd_${BD}.sql.gz"
    echo "  Base de datos $BD copiada"
done

# 2. Archivos: webs, buzones, configuración y scripts
tar -czf "$DIR/www.tar.gz"     -C / var/www
tar -czf "$DIR/correo.tar.gz"  -C / var/mail/vhosts
tar -czf "$DIR/etc.tar.gz"     -C / etc
tar -czf "$DIR/scripts.tar.gz" -C / usr/local/sbin
echo "  Archivos copiados"

# 3. Lista de paquetes instalados (para reconstruir el servidor)
dpkg --get-selections > "$DIR/paquetes.txt"

# 4. Suma de verificación para detectar copias corruptas
( cd "$DIR" && sha256sum ./* > SHA256SUMS )

# 5. Borrar copias antiguas
find "$DESTINO" -mindepth 1 -maxdepth 1 -type d -mtime +"$DIAS" -exec rm -rf {} +

echo "  Tamaño de la copia: $(du -sh "$DIR" | cut -f1)"
echo "=== $(date '+%F %T') Copia terminada correctamente ==="
```

Análisis:

Línea	Qué hace
`exec >>"$LOG" 2>&1`	Todo lo que el script escriba (incluidos los errores) va al log.
`umask 077`	Los archivos creados solo los puede leer root.
`mariadb -N -e "SHOW DATABASES"`	Lista las bases sin cabecera (`-N`); así las nuevas se copian automáticamente.
`tar -czf archivo -C / var/www`	Comprime la carpeta guardando rutas relativas (`var/www/...`), lo que facilita restaurar.
`sha256sum`	Huellas para verificar la integridad de la copia antes de restaurarla.
`find ... -mtime +7 -exec rm -rf`	Elimina las carpetas de más de 7 días.

Comando:

TERMINAL

```
sudo chmod 700 /usr/local/sbin/backup-servidor
sudo /usr/local/sbin/backup-servidor
sudo tail -n 15 /var/log/backup-servidor.log
sudo ls -lh /var/backups/servidor/$(date +%F)/
```

Resultado esperado:

SALIDA ESPERADA

```
=== 2026-11-09 04:30:00 Inicio de la copia ===
  Base de datos mysql copiada
  Base de datos web1_db copiada
  Base de datos web2_db copiada
  Archivos copiados
  Tamaño de la copia: 14M
=== 2026-11-09 04:30:09 Copia terminada correctamente ===
```

PASO 12.8 – Programar la copia diaria con cron

Archivo: `/etc/cron.d/backup-servidor` (nuevo, contenido completo):

TAREA PROGRAMADA (CRON)

```
# Copia de seguridad diaria a las 03:30
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MAILTO=root
30 3 * * * root /usr/local/sbin/backup-servidor || echo "La copia de seguridad ha fallado.
Revisa /var/log/backup-servidor.log"
```

Formato de una línea de cron:

DIAGRAMA

```
 30   3    *    *    *    root  comando
 │    │    │    │    │     │
 │    │    │    │    │     └─ usuario que lo ejecuta (solo en /etc/cron.d)
 │    │    │    │    └─────── día de la semana (0-7, 0 y 7 = domingo)
 │    │    │    └──────────── mes (1-12)
 │    │    └───────────────── día del mes (1-31)
 │    └────────────────────── hora (0-23)
 └─────────────────────────── minuto (0-59)
```

Si el script falla, el `|| echo ...` produce una salida y cron la envía por correo a `MAILTO` (root → `administracion@ejemplo1.com`).

TERMINAL

```
sudo chmod 644 /etc/cron.d/backup-servidor
```

PASO 12.9 – Sacar las copias del servidor

Objetivo: cumplir la parte «1 copia fuera» de la regla 3-2-1.

Explicación: la forma más sencilla es que **otro** equipo (un segundo servidor o un NAS) descargue las copias por SSH cada día con `rsync`. Así, si alguien compromete tu servidor, no tiene credenciales para borrar las copias externas. Ejemplo, ejecutado **en el equipo de copias**, con un usuario que tenga una clave autorizada en el servidor:

TERMINAL

```
rsync -az --delete -e ssh sysadmin@mail.ejemplo1.com:/var/backups/servidor/
/srv/copias/mail.ejemplo1.com/
```

Para que `sysadmin` pueda leer las copias sin ser root, cambia el grupo de la carpeta de destino del script y su `umask` (por ejemplo, `umask 027` y `chgrp -R sysadmin "$DESTINO"`), o usa un usuario dedicado de solo lectura. Otra opción muy extendida es `rclone`, que envía las copias cifradas a un almacenamiento de objetos.

> **ATENCIÓN:** Prueba la restauración. Al menos una vez al mes, restaura en una máquina de pruebas una base de datos, una web y un buzón. Comprueba antes las sumas con `sha256sum -c SHA256SUMS`.

12.10 Monitorización de servicios

PASO 12.10 – Script de estado del servidor

Objetivo: tener un único comando que revise todo y avise por correo si algo va mal.

Archivo: `/usr/local/sbin/estado-servidor` (nuevo, contenido completo):

TERMINAL

```
#!/bin/bash
# ================================================================
#  estado-servidor - Revisión rápida de la salud del servidor
#  Uso: sudo estado-servidor           (muestra el informe)
#       sudo estado-servidor --avisar   (solo envía correo a root si hay problemas)
# ================================================================
SERVICIOS="apache2 php8.3-fpm mariadb postfix@- dovecot opendkim fail2ban ssh cron"
PUERTOS="22 25 80 443 465 587 993"
LIMITE_DISCO=85
DIAS_CERT=20
PROBLEMAS=()

informe() { [ "${1:-}" = "--avisar" ] || echo "$2"; }

for S in $SERVICIOS; do
    if systemctl is-active --quiet "$S"; then informe "$1" "[OK]    servicio $S activo"
    else PROBLEMAS+=("Servicio $S detenido"); informe "$1" "[FALLO] servicio $S detenido"; fi
done

for P in $PUERTOS; do
    if ss -tln | awk '{print $4}' | grep -qE "[:.]$P$"; then informe "$1" "[OK]    puerto $P
escuchando"
    else PROBLEMAS+=("Puerto $P sin escuchar"); informe "$1" "[FALLO] puerto $P sin escuchar";
fi
done

while read -r USO MONTAJE; do
    USO=${USO%\%}
    if [ "$USO" -ge "$LIMITE_DISCO" ]; then PROBLEMAS+=("Disco $MONTAJE al $USO%"); informe
"$1" "[FALLO] disco $MONTAJE al $USO%"
    else informe "$1" "[OK]    disco $MONTAJE al $USO%"; fi
done < <(df -P -x tmpfs -x devtmpfs -x squashfs -x overlay | awk 'NR>1 {print $5, $6}')

for C in /etc/letsencrypt/live/*/cert.pem; do
    [ -e "$C" ] || continue
    N=$(basename "$(dirname "$C")")
    if openssl x509 -checkend $((DIAS_CERT*86400)) -noout -in "$C" >/dev/null; then informe
"$1" "[OK]    certificado $N vigente más de $DIAS_CERT días"
    else PROBLEMAS+=("Certificado $N caduca en menos de $DIAS_CERT días"); informe "$1"
"[FALLO] certificado $N caduca pronto"; fi
done
```

```
COLA=$(postqueue -j 2>/dev/null | wc -l)
if [ "$COLA" -gt 50 ]; then PROBLEMAS+=("Cola de correo con $COLA mensajes"); informe "$1"
"[FALLO] cola de correo: $COLA mensajes"
else informe "$1" "[OK]    cola de correo: $COLA mensajes"; fi

[ -f /var/run/reboot-required ] && informe "$1" "[AVISO] el sistema necesita reiniciarse"

if [ "${#PROBLEMAS[@]}" -gt 0 ] && [ "${1:-}" = "--avisar" ]; then
    printf '%s\n' "${PROBLEMAS[@]}" | mail -s "[$(hostname)] ${#PROBLEMAS[@]} problema(s)
detectado(s)" root
fi
[ "${#PROBLEMAS[@]}" -eq 0 ]
```

Comando:

TERMINAL

```
sudo chmod 700 /usr/local/sbin/estado-servidor
sudo estado-servidor
```

Resultado esperado:

SALIDA ESPERADA

```
[OK]    servicio apache2 activo
[OK]    servicio php8.3-fpm activo
[OK]    servicio mariadb activo
[OK]    servicio postfix@- activo
[OK]    servicio dovecot activo
[OK]    servicio opendkim activo
[OK]    servicio fail2ban activo
[OK]    servicio ssh activo
[OK]    servicio cron activo
[OK]    puerto 22 escuchando
...
[OK]    disco / al 18%
[OK]    certificado ejemplo1.com vigente más de 20 días
[OK]    cola de correo: 0 mensajes
```

Prográmalo cada 15 minutos en modo aviso:

Archivo: `/etc/cron.d/estado-servidor` (nuevo, contenido completo):

TAREA PROGRAMADA (CRON)

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
*/15 * * * * root /usr/local/sbin/estado-servidor --avisar >/dev/null 2>&1
```

ATENCIÓN: Un servidor no puede avisar de que él mismo está caído o sin red. Complementa este script con un servicio de monitorización **externo** (hay opciones gratuitas de *uptime monitoring*) que compruebe cada pocos minutos `https://www.ejemplo1.com`, `https://www.ejemplo2.com` y el puerto 25 desde fuera.

Para que systemd reinicie automáticamente un servicio si se cae, puedes añadir un *drop-in* (ejemplo con Dovecot):

TERMINAL

```
sudo mkdir -p /etc/systemd/system/dovecot.service.d
printf '[Service]\nRestart=on-failure\nRestartSec=5s\n' | sudo tee
/etc/systemd/system/dovecot.service.d/reinicio.conf
sudo systemctl daemon-reload
```

12.11 Revisión de logs

Log	Contenido	Comando típico
`/var/log/auth.log`	SSH, sudo, inicios de sesión.	`sudo grep 'Failed\`
`/var/log/mail.log`	Postfix, Dovecot, OpenDKIM.	`sudo grep -E 'reject\`
`/var/log/apache2/*-error.log`	Errores de cada web.	`sudo tail -n 50 /var/log/apache2/ejemplo1.com-error.log`
`/var/log/apache2/*-access.log`	Visitas de cada web.	`sudo awk '{print $9}' /var/log/apache2/ejemplo1.com-access.log \`
`/var/www/*/logs/php-error.log`	Errores PHP de cada web.	`sudo tail /var/www/ejemplo1.com/logs/php-error.log`
`/var/log/mysql/error.log`	Errores de MariaDB.	`sudo tail /var/log/mysql/error.log`
`/var/log/fail2ban.log`	Bloqueos.	`sudo grep Ban /var/log/fail2ban.log \`
`/var/log/php8.3-fpm.log`	Gestor PHP-FPM.	`sudo tail /var/log/php8.3-fpm.log`
journal de systemd	Todo, por servicio.	`sudo journalctl -u dovecot --since today`

Comandos de journal especialmente útiles:

TERMINAL

```
# Errores de todo el sistema desde el último arranque
sudo journalctl -p err -b
# Seguir en directo varios servicios a la vez
sudo journalctl -f -u apache2 -u php8.3-fpm -u postfix@- -u dovecot
# Espacio que ocupa el journal y limpiarlo a 500 MB
journalctl --disk-usage
sudo journalctl --vacuum-size=500M
```

Rutina de revisión recomendada:

Frecuencia	Tarea
Diaria (automática)	Copia de seguridad, actualizaciones de seguridad, `estado-servidor --avisar`.
Semanal	Revisar `mail.log` (rechazos, colas), `fail2ban-client status`, errores de Apache y PHP, informes DMARC.
Mensual	Prueba de restauración, revisar usuarios y claves SSH, espacio en disco, `certbot certificates`, listas negras de la IP.
Trimestral	Revisar versiones de las aplicaciones web, rotar contraseñas sensibles, revisar reglas de UFW.

12.12 Recuperación ante fallos

Situación	Procedimiento
Un servicio no arranca	1. `systemctl status servicio` y `journalctl -u servicio -n 50`. 2. Prueba la configuración (`apachectl configtest`, `postfix check`, `doveconf -n`, `php-fpm8.3 -t`, `sshd -t`). 3. Si cambiaste algo, restaura la copia `.original`/`.bak` del archivo.
Disco lleno	`df -h` para ver la partición; `sudo du -xh / --max-depth=2 \`
Se ha borrado una web	`sudo tar -xzf /var/backups/servidor/FECHA/www.tar.gz -C /tmp/restaura var/www/ejemplo1.com` y copia lo necesario con `rsync -a` a su sitio; revisa propietarios (`web1:www-data`).
Se ha dañado una base de datos	PASO 4.13 con el volcado `bd_web1_db.sql.gz` del día elegido.
Un usuario ha borrado correos	Extrae su carpeta de `correo.tar.gz` en `/tmp`, copia los archivos de `cur/` a su Maildir, ajusta propietario `vmail:vmail` y ejecuta `sudo doveadm force-resync -u usuario '*'`.
Contraseña de correo robada (envía spam)	`sudo correo-clave usuario`; vacía la cola de spam (`mailq`, `postsuper -d`); revisa listas negras; avisa al usuario de que revise su equipo.
Servidor comprometido	No intentes «limpiarlo». Aísla el servidor, conserva evidencias, monta uno nuevo desde cero (Módulo 14) y restaura **solo datos** (bases de datos, contenidos, buzones) de copias anteriores al incidente, revisándolos. Cambia todas las contraseñas y claves (incluidas las DKIM).
Reconstrucción completa	Instala Ubuntu, sigue los módulos 1-12 o restaura `/etc` de forma selectiva, reinstala los paquetes (`sudo dpkg --set-selections < paquetes.txt && sudo apt-get dselect-upgrade`), restaura bases de datos, `/var/www` y `/var/mail/vhosts`, y comprueba todo con el Módulo 13.

12.13 Ejercicios prácticos del Módulo 12

1. Desde otra IP, falla tres veces la contraseña SSH (o la de IMAP) y comprueba con `fail2ban-client status` que la IP queda bloqueada. Desbloquéala.
2. Ejecuta `sudo estado-servidor` con Dovecot detenido. ¿Qué muestra? ¿Qué correo llega a root con `--avisar`?
3. Restaura en `/tmp/prueba` el buzón de `soporte@ejemplo2.com` de la copia de hoy y comprueba que los mensajes están completos.
4. Calcula cuánto espacio ocuparán 7 días de copias y decide si te conviene una partición o disco aparte.
5. Usa `curl -sI` para comprobar que ninguna de tus webs revela la versión de Apache ni de PHP (busca la cabecera `X-Powered-By`; si aparece, añade `php_admin_flag[expose_php] = off` en cada pool).

Módulo 13. Pruebas y diagnóstico

Objetivos del módulo

- Disponer de un procedimiento de verificación completo y ordenado de toda la infraestructura.
- Saber qué comando usar para comprobar cada servicio y cómo interpretar su resultado.
- Aplicar un método de diagnóstico sistemático ante cualquier fallo.
- Reconocer los errores reales más frecuentes y su solución.

13.1 Método de diagnóstico

Ante cualquier problema, sigue siempre el mismo orden, «de abajo arriba», como las capas de la red:

DIAGRAMA

```
1. ¿El servicio está funcionando?        systemctl status / is-active
2. ¿Escucha en el puerto correcto?       ss -tlnp
3. ¿El cortafuegos lo permite?           ufw status
4. ¿El DNS resuelve bien?                dig
5. ¿Se llega por red desde fuera?        nc -vz / curl / openssl s_client (desde otro equipo)
6. ¿La configuración es válida?          apachectl configtest / postfix check / doveconf -n
7. ¿Qué dice el log?                     tail / grep / journalctl
8. ¿Son correctos permisos y propietarios? ls -l / namei -l / sudo -u USUARIO
```

CONSEJO: Cambia **una sola cosa** cada vez y vuelve a probar. Si cambias cinco cosas a la vez y funciona, no sabrás cuál era el problema; si deja de funcionar, no sabrás cuál lo rompió. Y antes de cambiar un archivo, haz una copia: `sudo cp archivo archivo.$(date +%F)`.

13.2 Procedimiento de verificación completo

Ejecuta esta batería de pruebas al terminar la instalación, tras cada cambio importante y tras cada actualización mayor. Las pruebas marcadas como «externa» deben hacerse desde **otro** equipo.

PASO 13.1 – Apache funciona

TERMINAL

```
systemctl is-active apache2 && systemctl is-enabled apache2
sudo apachectl configtest
sudo apachectl -S 2>&1 | grep -E 'namevhost|default server'
sudo ss -tlnp | grep -E ':(80|443) '
```

Resultado esperado: `active`, `enabled`, `Syntax OK`, los VirtualHosts de ambos dominios y Apache escuchando en 80 y 443.

PASO 13.2 – Cada dominio carga su propia web

TERMINAL

```
for d in www.ejemplo1.com www.ejemplo2.com; do
  echo "== $d"
  curl -s -o /dev/null -w "HTTP %{http_code}  ->  %{redirect_url}\n" http://$d/
  curl -s https://$d/ | grep -o '<h1>[^<]*</h1>'
done
curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" https://ejemplo2.com/
```

Resultado esperado:

SALIDA ESPERADA

```
== www.ejemplo1.com
HTTP 301  ->  https://www.ejemplo1.com/
<h1>Agenda de contactos · ejemplo1.com</h1>
== www.ejemplo2.com
HTTP 301  ->  https://www.ejemplo2.com/
<h1>Agenda de contactos · ejemplo2.com</h1>
301 https://www.ejemplo2.com/
```

Prueba externa: abre las cuatro direcciones (`ejemplo1.com`, `www.ejemplo1.com`, `ejemplo2.com`, `www.ejemplo2.com`) en un navegador y en el móvil con datos móviles.

PASO 13.3 – PHP se ejecuta correctamente y en su pool

TERMINAL

```
systemctl is-active php8.3-fpm
sudo php-fpm8.3 -t
ls -l /run/php/*.sock
ps -eo user,cmd | grep '[p]hp-fpm: pool'
```

Resultado esperado: `active`, `test is successful`, los dos sockets y, si hay visitas recientes, procesos `php-fpm: pool web1` del usuario `web1` y `php-fpm: pool web2` del usuario `web2` (con `pm = ondemand` puede no haber procesos si nadie ha visitado las webs en los últimos 30 segundos; recarga una web y repite).

PASO 13.4 – MariaDB responde y cada aplicación accede a su base de datos

TERMINAL

```
systemctl is-active mariadb
sudo ss -tlnp | grep 3306
sudo mariadb -e "SELECT table_schema AS bd, COUNT(*) AS tablas FROM information_schema.tables
WHERE table_schema LIKE 'web%' GROUP BY table_schema;"
sudo mariadb -e "SHOW GRANTS FOR 'web1_user'@'localhost'; SHOW GRANTS FOR
'web2_user'@'localhost';"
sudo grep -h SQLSTATE /var/www/*/logs/php-error.log 2>/dev/null | tail -5
```

Resultado esperado: `active`, escucha solo en `127.0.0.1:3306`, dos tablas en cada base (`contactos` y `mensajes`), permisos limitados y ningún `SQLSTATE` reciente en los logs de PHP. La prueba definitiva es funcional: crear, editar y borrar un contacto en cada web (PASO 5.15).

PASO 13.5 – Postfix recibe mensajes

TERMINAL

```
systemctl is-active postfix@-
sudo postfix check
postconf -n | grep -E
'^(myhostname|mydestination|virtual_mailbox_domains|virtual_transport|smtpd_milters)'
postmap -q ejemplo2.com hash:/etc/postfix/virtual_domains
postmap -q soporte@ejemplo2.com hash:/etc/postfix/vmailbox
```

Prueba externa: envía desde Gmail a `info@ejemplo1.com` y a `soporte@ejemplo2.com`, y sigue la llegada:

TERMINAL

```
sudo tail -f /var/log/mail.log | grep -E 'postfix/smtpd|lmtp|Saved'
```

Debe aparecer la conexión desde un servidor de Google y `status=sent (250 2.0.0 ... Saved)`.

Prueba de relay abierto desde un equipo externo (debe ser **rechazado**):

TERMINAL

```
swaks --server mail.ejemplo1.com --port 25 --from prueba@example.org --to alguien@example.net
```

SALIDA ESPERADA

```
<** 454 4.7.1 <alguien@example.net>: Relay access denied
```

PASO 13.6 – Postfix envía mensajes (con autenticación)

TERMINAL

```
swaks --server mail.ejemplo1.com --port 587 --tls --auth LOGIN --auth-user
administracion@ejemplo1.com --from administracion@ejemplo1.com --to tu-direccion@gmail.com --
header "Subject: Verificacion 587"
swaks --server mail.ejemplo2.com --port 465 --tlsc --auth LOGIN --auth-user
soporte@ejemplo2.com --from soporte@ejemplo2.com --to tu-direccion@gmail.com --header
"Subject: Verificacion 465"
sudo grep -E 'status=(sent|deferred|bounced)' /var/log/mail.log | tail -4
```

Resultado esperado: `235 Authentication successful` en swaks y `status=sent (250 2.0.0 OK ...)` hacia Gmail en el log.

PASO 13.7 – Dovecot permite consultar los buzones

TERMINAL

```
systemctl is-active dovecot
sudo doveconf -n > /dev/null && echo "configuración OK"
sudo ss -tlnp | grep dovecot
sudo doveadm user '*'
sudo doveadm auth test info@ejemplo1.com
sudo doveadm mailbox status -u soporte@ejemplo2.com "messages unseen" INBOX
```

Prueba externa: abre las cuentas en Thunderbird o en el móvil con IMAP 993, comprueba que ves los mensajes y que al borrar uno en un dispositivo desaparece en el otro.

PASO 13.8 – Los certificados son válidos

TERMINAL

```
sudo certbot certificates | grep -E 'Certificate Name|Domains|Expiry'
for hp in www.ejemplo1.com:443 www.ejemplo2.com:443; do
  echo | openssl s_client -connect $hp -servername ${hp%:*} -verify_return_error 2>&1 | grep -
E '^subject|Verify return code'
done
for p in 465 993 995; do
  echo | openssl s_client -connect mail.ejemplo1.com:$p -verify_return_error 2>&1 | grep
'Verify return code'
done
echo | openssl s_client -connect mail.ejemplo2.com:587 -starttls smtp -verify_return_error
2>&1 | grep 'Verify return code'
sudo certbot renew --dry-run 2>&1 | tail -4
```

Resultado esperado: `Verify return code: 0 (ok)` en todos los puertos y `all simulated renewals succeeded`.

PASO 13.9 – SPF, DKIM y DMARC están correctamente configurados

TERMINAL

```
for d in ejemplo1.com ejemplo2.com; do
  echo "== $d"
  echo -n "MX:    "; dig +short MX $d
  echo -n "SPF:   "; dig +short TXT $d | grep spf1
  echo -n "DMARC: "; dig +short TXT _dmarc.$d
  sudo opendkim-testkey -d $d -s mail -vvv 2>&1 | tail -1
done
echo -n "PTR:   "; dig +short -x 203.0.113.10
```

Resultado esperado: MX, SPF y DMARC con los valores del Módulo 9, `key OK` para ambos dominios y PTR `mail.ejemplo1.com.`.

Prueba externa: envía un correo desde cada dominio a `check-auth@verifier.port25.com` o a la dirección que te dé `mail-tester.com`. Debes obtener `SPF check: pass`, `DKIM check: pass` y `DMARC check: pass` (y una puntuación de 9 o 10 sobre 10 en mail-tester).

PASO 13.10 – Los servicios arrancan automáticamente

TERMINAL

```
for s in apache2 php8.3-fpm mariadb postfix dovecot opendkim fail2ban ssh certbot.timer; do
  printf '%-14s %s\n' "$s" "$(systemctl is-enabled $s)"
done
```

Resultado esperado: `enabled` en todos. La prueba definitiva:

TERMINAL

```
sudo reboot
```

Espera un par de minutos, vuelve a conectarte y ejecuta:

TERMINAL

```
sudo estado-servidor
```

Todo debe aparecer `[OK]` sin que hayas arrancado nada a mano.

13.3 Hoja de verificación final

#	Comprobación	Comando o prueba	✓
1	Apache activo y sintaxis correcta	PASO 13.1	
2	Cada dominio carga su web y redirige a HTTPS www	PASO 13.2	
3	PHP se ejecuta en el pool de cada web	PASO 13.3	
4	MariaDB solo local; cada app usa su base	PASO 13.4	
5	Postfix recibe de Internet y no es relay abierto	PASO 13.5	
6	Envío autenticado por 587 y 465	PASO 13.6	
7	Lectura de buzones por IMAP/POP3 con TLS	PASO 13.7	
8	Certificados válidos y renovación simulada correcta	PASO 13.8	
9	SPF, DKIM, DMARC y PTR correctos	PASO 13.9	
10	Todo arranca tras reiniciar	PASO 13.10	
11	Fail2ban con 5 jails activos	`sudo fail2ban-client status`	
12	Copia de seguridad de hoy completa	`sudo ls -lh /var/backups/servidor/$(date +%F)`	

13.4 Catálogo de errores reales y soluciones

Web

Mensaje / síntoma	Dónde aparece	Causa	Solución
`AH00558: Could not reliably determine the server's fully qualified domain name`	`apachectl configtest`	Sin `ServerName` global.	PASO 2.5.
`AH00526: Syntax error on line 12 of /etc/apache2/sites-enabled/ejemplo1.com.conf: Invalid command 'RewriteEngine'`	configtest	Módulo no activado.	`sudo a2enmod rewrite`.
`AH00072: make_sock: could not bind to address [::]:80`	journal de apache2	Otro programa usa el puerto.	`sudo ss -tlnp \

Mensaje / síntoma	Dónde aparece	Causa	Solución
`AH01630: client denied by server configuration`	error.log	Falta `Require all granted` en `<Directory>` o la ruta no coincide.	Revisa el bloque `<Directory>`.
`AH01797`/`AH00035: access to / denied ... because search permissions are missing`	error.log	Falta permiso `x` en alguna carpeta de la ruta.	`namei -l ruta`; `chmod 750` + grupo `www-data`.
`AH01079: failed to make connection to backend: httpd-UDS` + 503	error.log	Socket de PHP-FPM inexistente.	`systemctl status php8.3-fpm`, revisa `listen` del pool.
`Primary script unknown`	error.log	PHP-FPM no encuentra el archivo (ruta o permisos).	El usuario del pool debe poder leer el script.
`PHP Warning: require(): open_basedir restriction in effect`	log PHP de la web	El código intenta salir de su carpeta.	Corrige la ruta en el código.
`PHP Fatal error: Uncaught PDOException: SQLSTATE[HY000] [1045] Access denied`	log PHP	Credenciales incorrectas.	Revisa `config.php` y PASO 4.11.
`PHP Fatal error: Uncaught PDOException: could not find driver`	log PHP	Falta `php-mysql`.	Instala y reinicia FPM.
`ERR_TOO_MANY_REDIRECTS`	navegador	Bucle de redirección (proxy/CDN en modo HTTP).	Ver PASO 3.14.
`NET::ERR_CERT_COMMON_NAME_INVALID`	navegador	El nombre no está en el certificado o SNI cae en el comodín.	Revisa `ServerName/ServerAlias` y el certificado del VirtualHost.

Base de datos

Mensaje	Causa	Solución
`ERROR 1698 (28000): Access denied for user 'root'@'localhost'`	Entrar como root sin sudo.	`sudo mariadb`. Es el comportamiento correcto.
`ERROR 1044 (42000): Access denied for user 'web1_user'@'localhost' to database 'web2_db'`	Intento de acceder a otra base.	Correcto: es el aislamiento.
`ERROR 1142 (42000): DROP command denied`	Permisos limitados.	Correcto, salvo que la aplicación necesite ese permiso.
`ERROR 2002 (HY000): Can't connect to local server through socket`	MariaDB detenido.	`systemctl start mariadb`; si no arranca, `/var/log/mysql/error.log` (suele ser disco lleno).
`ERROR 1062 (23000): Duplicate entry ... for key 'uk_email'`	Valor repetido en un campo único.	Comportamiento esperado; la aplicación lo traduce a un aviso.

Correo

Mensaje en /var/log/mail.log	Causa	Solución
NOQUEUE: reject: RCPT from ...: 454 4.7.1 <x@gmail.com>: Relay access denied	Alguien sin autenticar intenta enviar a fuera.	Correcto si es un desconocido. Si es tu usuario, está usando el puerto 25 en vez del 587.
NOQUEUE: reject: RCPT ...: 550 5.1.1 <x@ejemplo1.com>: Recipient address rejected: User unknown in virtual mailbox table	La dirección no existe.	Créala o añade un alias.
warning: SASL authentication failure: ... / SASL LOGIN authentication failed	Contraseña incorrecta (o ataque).	Si es masivo, Fail2ban lo bloqueará.
warning: SASL: Connect to private/auth failed: No such file or directory	Dovecot detenido o sin socket auth.	PASO 7.8.
connect to private/dovecot-lmtp: No such file or directory + status=deferred	Dovecot detenido.	Arranca Dovecot; Postfix reintentará solo.
connect to gmail-smtp-in.l.google.com[...]:25: Connection timed out	Puerto 25 saliente bloqueado.	PASO 6.1 / relayhost.
said: 550-5.7.26 Unauthenticated email from ejemplo1.com is not accepted due to domain's DMARC policy	SPF y DKIM fallan.	Módulo 9: revisa firma DKIM y SPF.
said: 550-5.7.25 ... The IP address sending this message does not have a PTR record setup	Falta PTR (a menudo en IPv6).	PASO 9.2, o inet_protocols = ipv4.
warning: hostname X does not resolve to address Y	El cliente remoto tiene DNS inverso incoherente.	Informativo; es problema del remitente.
warning: connect to Milter service inet:localhost:8891: Connection refused	OpenDKIM detenido o en otro socket.	PASO 9.8.
dovecot: imap-login: Disconnected (auth failed, 1 attempts in 2 secs): user=<info@ejemplo1.com>	Contraseña incorrecta en IMAP.	correo-clave si el usuario la olvidó.
dovecot: lmtp(...): Error: ... Permission denied	Propietario de los buzones incorrecto.	sudo chown -R vmail:vmail /var/mail/vhosts.
dovecot: auth: Error: passwd-file /etc/dovecot/users: User ... has invalid UID	Campo uid vacío o erróneo.	Revisa el formato de la línea (8 campos).

13.5 Caja de herramientas de diagnóstico

Herramienta	Uso	Ejemplo
`systemctl` / `journalctl`	Estado y logs de servicios.	`journalctl -u dovecot -n 50`
`ss`	Puertos en escucha y conexiones.	`sudo ss -tlnp`
`dig`	Consultas DNS.	`dig +short MX ejemplo1.com @1.1.1.1`
`curl`	Pruebas HTTP/HTTPS.	`curl -vI https://www.ejemplo1.com`
`openssl s_client`	Pruebas TLS en cualquier puerto.	`openssl s_client -connect mail.ejemplo1.com:993`
`nc`	Conectividad TCP y diálogos manuales.	`nc -vz mail.ejemplo1.com 25`
`swaks`	Pruebas SMTP completas.	Ver PASO 7.19
`doveadm`	Usuarios, autenticación y buzones de Dovecot.	`doveadm auth test usuario`
`postconf` / `postmap -q` / `mailq`	Configuración, tablas y cola de Postfix.	`postmap -q x@ejemplo1.com hash:/etc/postfix/vmailbox`
`namei -l`	Permisos de toda una ruta.	`namei -l /var/www/ejemplo1.com/public_html/index.php`
`sudo -u usuario`	Probar como otro usuario.	`sudo -u web1 cat /var/www/ejemplo1.com/config/config.php`
`fail2ban-client`	Estado de bloqueos.	`fail2ban-client status sshd`
`htop`, `free -h`, `df -h`	Recursos (CPU, memoria, disco).	`df -h`

13.6 Ejercicios prácticos del Módulo 13

1. Ejecuta toda la batería de pruebas del apartado 13.2 y rellena la hoja de verificación.
2. Pide a un compañero que provoque un fallo oculto (detener un servicio, cambiar un permiso, borrar una línea de configuración) y encuéntralo siguiendo el método del apartado 13.1.
3. Provoca deliberadamente tres errores del catálogo y verifica que aparecen exactamente los mensajes descritos.
4. Escribe tu propio script `verificar-infraestructura` que automatice los pasos 13.1 a 13.10 y devuelva un resumen.
5. Documenta en una tabla tus propios errores encontrados durante el curso, su causa y su solución: es la base del manual de operaciones de tu servidor.

Módulo 14. Proyecto final

Objetivos del proyecto

Construir **desde cero**, en un servidor nuevo y sin consultar soluciones salvo este manual, una infraestructura completa con dos dominios independientes. Al terminar, deberás poder:

- Acceder a `https://www.ejemplo1.com` y `https://www.ejemplo2.com`, cada una con su aplicación PHP y su base de datos.
- Usar desde un cliente de correo las cuentas de ambos dominios, enviando y recibiendo correo de Internet con SPF, DKIM y DMARC correctos.
- Demostrar que la infraestructura es segura, que se recupera sola tras un reinicio y que tiene copias de seguridad.

> **CONSEJO:** Para el proyecto, sustituye `ejemplo1.com` y `ejemplo2.com` por dominios reales tuyos (o por los nombres de tu laboratorio) y la IP `203.0.113.10` por la de tu servidor. Una forma cómoda de hacerlo es preparar una tabla con tus valores (apartado «Los datos del proyecto guía» de la introducción) antes de empezar.

14.1 Requisitos de partida

Requisito	Comprobado
Servidor con Ubuntu Server 24.04 LTS recién instalado, 2 GB de RAM o más.	
IP pública fija; puertos 25, 80, 443, 465, 587, 993 y 995 sin bloquear por el proveedor.	
Posibilidad de configurar el PTR de la IP.	
Dos dominios con acceso a su panel DNS.	
Acceso SSH o consola web del proveedor.	
Un ordenador con cliente SSH y un cliente de correo (Thunderbird).	

14.2 Plan de construcción

Sigue las fases en orden. Cada fase termina con una **verificación obligatoria**: no pases a la siguiente sin superarla.

Fase 1 – Sistema base (Módulo 1)

Tarea	Referencia
Primera conexión y actualización completa	PASOS 1.1 y 1.2
Hostname `mail.ejemplo1.com` y `/etc/hosts`	PASO 1.3
Zona horaria y NTP	PASO 1.4
Usuario `sysadmin` con sudo	PASO 1.5
Claves SSH y endurecimiento de SSH	PASOS 1.6 y 1.7

Tarea	Referencia
Herramientas básicas y UFW con SSH	PASOS 1.8 y 1.9

Verificación: `hostname -f` devuelve `mail.ejemplo1.com`; entras como `sysadmin` con clave; `ssh root@IP` es rechazado; `ufw status` activo.

Fase 2 – DNS inicial (Módulos 3 y 9)

Tarea	Referencia
Registros A (y AAAA si procede) para `@`, `www` y `mail` de ambos dominios	PASO 3.10
PTR de la IP → `mail.ejemplo1.com`	PASO 9.2
Registros MX de ambos dominios	PASO 9.1
SPF de dominio y de `mail`	PASO 9.3
DMARC en modo `p=none`	PASO 9.11

Hazlo pronto: así, mientras trabajas en las fases siguientes, el DNS tiene tiempo de propagarse. Verificación: `dig` devuelve los valores correctos desde `1.1.1.1` y `8.8.8.8`; `dig -x IP` devuelve `mail.ejemplo1.com.`

Fase 3 – Servidor web y VirtualHosts (Módulos 2 y 3)

Tarea	Referencia
Instalar Apache, abrir 80/443, ServerName global, módulos	PASOS 2.1 a 2.6
Usuarios `web1`/`web2`, carpetas y permisos	PASOS 3.1 a 3.3
Sitio comodín `000-default`	PASO 3.4
VirtualHosts HTTP y habilitación	PASOS 3.5 a 3.7
Certificados de las webs y del correo	PASOS 3.11 y 6.2
VirtualHosts HTTPS y comodín `default-ssl`	PASOS 3.12 a 3.14

Verificación: PASOS 13.1 y 13.2 (con las páginas HTML de prueba).

Fase 4 – Base de datos (Módulo 4)

Tarea	Referencia
Instalar MariaDB, comprobar `bind-address`, `mariadb-secure-installation`	PASOS 4.1 a 4.3
Contraseñas, bases `web1_db`/`web2_db`, usuarios y permisos	PASOS 4.5 a 4.8
Tablas `contactos` (y `mensajes`, PASO 11.1)	PASO 4.9
Prueba de aislamiento	PASO 4.10
Copia y restauración manual	PASOS 4.12 y 4.13

Verificación: `web1_user` lee `web1_db` y recibe `Access denied` en `web2_db`.

Fase 5 – PHP y aplicaciones (Módulos 5 y 11)

Tarea	Referencia
Instalar PHP-FPM y extensiones	PASO 5.1
Carpetas de logs, pools `web1` y `web2`, desactivar `www`	PASOS 5.2 a 5.5
`proxy_fcgi` y VirtualHosts definitivos	PASOS 5.6 a 5.8
Comprobar usuario de ejecución	PASO 5.9
`config.php`, `db.php`, aplicación CRUD y estilos	PASOS 5.10 a 5.14
Despliegue en ejemplo2.com	PASO 5.16

Verificación: PASOS 13.3, 13.4 y 5.15 (las cuatro operaciones CRUD en ambas webs).

Fase 6 – Correo (Módulos 6, 7 y 8)

Tarea	Referencia
Comprobar el puerto 25 saliente	PASO 6.1
Instalar Postfix, `main.cf`, alias y puerto 25	PASOS 6.3 a 6.6
Pruebas de envío y recepción locales	PASOS 6.7 a 6.9
Instalar Dovecot, usuario `vmail` y primera cuenta	PASOS 7.1 a 7.3
Autenticación, buzones, servicios, TLS y LMTP	PASOS 7.4 a 7.11
Integración con Postfix y puertos 587/465	PASOS 7.12 a 7.15
Tabla de remitentes y restricción en 587/465	PASOS 8.1 y 8.2
Scripts `correo-alta`, `correo-clave` y `correo-baja`	PASOS 8.3 a 8.5
Cuentas, alias y redirección de root	PASOS 8.6 a 8.9

Verificación: PASOS 13.5, 13.6 y 13.7, y configuración de Thunderbird (PASO 8.11) para `info@ejemplo1.com` y `soporte@ejemplo2.com`.

Fase 7 – Autenticación del correo (Módulo 9)

Tarea	Referencia
Instalar OpenDKIM y generar claves	PASOS 9.4 y 9.5
Tablas, `opendkim.conf` y socket	PASOS 9.6 a 9.8
Milter en Postfix	PASO 9.9
Publicar DKIM en el DNS	PASO 9.10

Verificación: PASO 13.9 y «Mostrar original» en Gmail con SPF, DKIM y DMARC en **PASS** para ambos dominios (PASO 9.12).

Fase 8 – TLS, integración y seguridad (Módulos 10, 11 y 12)

Tarea	Referencia
Temporizador, simulación y *deploy hook* de Certbot	PASOS 10.1 a 10.3
Formulario de contacto en ambas webs	PASOS 11.1 y 11.2
UFW definitivo, cabeceras de Apache, límites de Postfix	PASOS 12.1 a 12.4
Fail2ban	PASO 12.5
Actualizaciones automáticas	PASO 12.6
Copias de seguridad y cron	PASOS 12.7 a 12.9
Script de estado y monitorización	PASO 12.10

Verificación: PASOS 13.8, 13.10 y la hoja de verificación completa del apartado 13.3.

Fase 9 (opcional) – Servidor FTP cifrado (Módulo 15)

Tarea	Referencia
Registros DNS `ftp` y certificado `ftp.ejemplo1.com`	PASOS 15.1 y 15.2
Instalar vsftpd, contraseñas y jaula chroot con ACL	PASOS 15.3 a 15.5
PAM y lista de usuarios autorizados	PASOS 15.6 y 15.7
`vsftpd.conf` y puertos en UFW	PASOS 15.8 y 15.9
Renovación, Fail2ban y monitorización	PASOS 15.12 a 15.14

Verificación: PASO 15.10 (cifrado obligatorio, usuario no autorizado rechazado, jaula y subida con permisos correctos) y conexión desde FileZilla (PASO 15.11).

14.3 Pruebas de aceptación

El proyecto se considera superado cuando se cumplen **todas** estas pruebas, realizadas desde fuera del servidor salvo indicación:

#	Prueba	Resultado esperado
1	Abrir `http://ejemplo1.com`	Redirige a `https://www.ejemplo1.com` con candado válido y muestra la agenda de ejemplo1.
2	Abrir `http://ejemplo2.com`	Redirige a `https://www.ejemplo2.com` y muestra la agenda de ejemplo2.
3	Crear un contacto en la web 1	Aparece en la web 1 y **no** en la web 2.
4	Editar y borrar ese contacto	Los cambios se reflejan en `web1_db`.
5	Enviar el formulario de contacto de la web 2	El mensaje se guarda en `web2_db.mensajes` y llega al buzón de `soporte@ejemplo2.com`.
6	Desde Gmail, escribir a `info@ejemplo1.com` y `soporte@ejemplo2.com`	Ambos llegan a sus buzones en Thunderbird.

#	Prueba	Resultado esperado
7	Desde Thunderbird, responder a Gmail con cada cuenta	Llegan a la bandeja de entrada de Gmail con SPF, DKIM y DMARC en PASS.
8	Escribir a `postmaster@ejemplo2.com`	Llega al buzón de `soporte@ejemplo2.com`.
9	Escribir a `noexiste@ejemplo1.com`	Gmail informa de que la dirección no existe (rechazo 550).
10	Intentar relay sin autenticación por el puerto 25	`Relay access denied`.
11	Autenticarse como `info@` y enviar como `administracion@`	`Sender address rejected: not owned by user`.
12	`nc -vz IP 3306`, `nc -vz IP 143`, `nc -vz IP 110`	Las tres conexiones fallan.
13	4 intentos de contraseña erróneos en IMAP desde una IP de prueba	La IP queda bloqueada por Fail2ban.
14	Reiniciar el servidor	Todo vuelve a funcionar sin intervención; `estado-servidor` todo en `[OK]`.
15	Puntuación en mail-tester.com	9/10 o superior desde ambos dominios.
16	Borrar `web2_db` y restaurarla desde la copia automática de la noche	La web 2 vuelve a mostrar sus datos.
17 (Fase 9)	Subir por FTPS con FileZilla un archivo a `public_html` de ejemplo2.com como `web2`	Se ve en `https://www.ejemplo2.com`; `web2` no puede ver la carpeta de ejemplo1.com y el FTP sin cifrar es rechazado.

14.4 Entregables del proyecto

1. **Ficha de la infraestructura**: la tabla del apartado 11.1 con tus valores reales.
2. **Registro de DNS**: exportación o captura de los registros de ambos dominios.
3. **Configuraciones**: copia (sin contraseñas ni claves privadas) de los archivos que has creado o modificado. Puedes obtener las configuraciones efectivas con `postconf -n`, `sudo doveconf -n`, `sudo apachectl -S` y `sudo opendkim -n` (comprobación de la configuración de OpenDKIM).
4. **Evidencias**: salida de la batería de pruebas del Módulo 13, cabeceras `Authentication-Results` de Gmail y resultado de mail-tester.
5. **Manual de operaciones**: cómo crear una cuenta de correo, cómo añadir una tercera web y cómo restaurar una copia, escritos por ti con tus palabras.
6. **Registro de incidencias**: los errores que encontraste, su causa y su solución.

14.5 Retos de ampliación

Cuando domines el proyecto, amplíalo:

1. **Tercer dominio completo** (`ejemplo3.com`): usuario `web3`, carpeta, pool, VirtualHost, certificado, base de datos, dominio de correo en `virtual_domains`, clave DKIM y registros DNS. Escribe un script `sitio-alta` que automatice la parte web.
2. **Webmail**: instala Roundcube en un VirtualHost propio (`webmail.ejemplo1.com`) con su propia base de datos, conectado por IMAP a `127.0.0.1:993` y por SMTP a `127.0.0.1:587`.

3. **Usuarios de correo en MariaDB**: sustituye `/etc/dovecot/users` y las tablas `hash:` de Postfix por consultas a una base de datos `mailserver` (drivers `sql` en Dovecot y `mysql:` en Postfix, paquete `postfix-mysql`).
4. **Antispam**: añade Rspamd como milter para filtrar el correo entrante y mover el spam a la carpeta Junk con Sieve (`dovecot-sieve`).
5. **Cuotas de buzón** con el plugin `quota` de Dovecot y aviso al usuario al 90 %.
6. **MTA-STS y TLS-RPT**: publica la política que obliga a los servidores remitentes a usar TLS válido al entregarte correo.
7. **Copias externas cifradas** con `restic` o `borg` hacia otro servidor.
8. **Migración a Dovecot 2.4**: en una máquina de pruebas con Debian 13, traduce la configuración de los módulos 7 y 8 a la nueva sintaxis.

14.6 Archivo main.cf completo de referencia

Así queda `/etc/postfix/main.cf` al terminar todos los módulos. Úsalo para comparar con el tuyo:

ARCHIVO DE CONFIGURACIÓN

```
# ================================================================
#  /etc/postfix/main.cf  -  mail.ejemplo1.com  (versión final)
# ================================================================
compatibility_level = 3.6
smtpd_banner = $myhostname ESMTP
biff = no
append_dot_mydomain = no
readme_directory = no

# --- Identidad del servidor ---
myhostname = mail.ejemplo1.com
mydomain = ejemplo1.com
myorigin = $mydomain
mydestination = $myhostname, localhost.$mydomain, localhost
mynetworks = 127.0.0.0/8 [::ffff:127.0.0.0]/104 [::1]/128
inet_interfaces = all
inet_protocols = all
relayhost =

# --- Alias y límites ---
alias_maps = hash:/etc/aliases
alias_database = hash:/etc/aliases
recipient_delimiter = +
mailbox_size_limit = 0
message_size_limit = 52428800

# --- TLS entrante ---
smtpd_tls_cert_file = /etc/letsencrypt/live/mail.ejemplo1.com/fullchain.pem
smtpd_tls_key_file = /etc/letsencrypt/live/mail.ejemplo1.com/privkey.pem
smtpd_tls_security_level = may
smtpd_tls_auth_only = yes
smtpd_tls_protocols = >=TLSv1.2
smtpd_tls_mandatory_protocols = >=TLSv1.2
smtpd_tls_loglevel = 1
smtpd_tls_session_cache_database = btree:${data_directory}/smtpd_scache

# --- TLS saliente ---
smtp_tls_security_level = may
smtp_tls_CApath = /etc/ssl/certs
smtp_tls_loglevel = 1
smtp_tls_session_cache_database = btree:${data_directory}/smtp_scache
```

```
# --- Restricciones ---
smtpd_helo_required = yes
disable_vrfy_command = yes
smtpd_helo_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_invalid_helo_hostname,
    reject_non_fqdn_helo_hostname
smtpd_sender_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_non_fqdn_sender,
    reject_unknown_sender_domain
smtpd_relay_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    defer_unauth_destination
smtpd_recipient_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_unauth_destination,
    reject_non_fqdn_recipient,
    reject_unknown_recipient_domain

# --- Buzones virtuales (Dovecot) ---
virtual_mailbox_domains = hash:/etc/postfix/virtual_domains
virtual_mailbox_maps = hash:/etc/postfix/vmailbox
virtual_alias_maps = hash:/etc/postfix/virtual
virtual_transport = lmtp:unix:private/dovecot-lmtp

# --- SASL (Dovecot) ---
smtpd_sasl_type = dovecot
smtpd_sasl_path = private/auth
smtpd_sasl_auth_enable = no
smtpd_sasl_security_options = noanonymous, noplaintext
smtpd_sasl_tls_security_options = noanonymous

# --- Remitentes autorizados ---
smtpd_sender_login_maps = hash:/etc/postfix/sender_login

# --- DKIM (OpenDKIM) ---
milter_default_action = accept
milter_protocol = 6
smtpd_milters = inet:localhost:8891
non_smtpd_milters = $smtpd_milters

# --- Límites ---
smtpd_client_connection_rate_limit = 30
smtpd_client_message_rate_limit = 100
smtpd_client_recipient_rate_limit = 200
anvil_rate_time_unit = 60s
smtpd_error_sleep_time = 5s
smtpd_soft_error_limit = 5
smtpd_hard_error_limit = 10
```

14.7 Conclusión

Si has llegado hasta aquí y superado las pruebas de aceptación, has construido y entiendes una infraestructura que muchas empresas pequeñas y medianas pagan por tener: varias webs aisladas con sus bases de datos, un

servidor de correo con autenticación moderna, cifrado en todos los servicios, protección contra ataques, copias de seguridad y monitorización.

Lo más importante no son los comandos, que siempre podrás consultar, sino el método: **entender cada pieza, cambiar una cosa cada vez, verificar siempre y leer los logs**. Con ese método podrás administrar este servidor, ampliarlo y diagnosticar cualquier problema nuevo que aparezca.

Módulo 15. Servidor FTP con vsftpd (complementario)

Objetivos del módulo

- Entender cómo funciona FTP, sus modos activo y pasivo y sus riesgos de seguridad.
- Decidir con criterio entre FTP cifrado (FTPS) y SFTP.
- Instalar y configurar **vsftpd** para que cada web tenga su propio acceso FTP cifrado.
- Encerrar a cada usuario en la carpeta de su web (*chroot*) sin debilitar la seguridad.
- Obtener un certificado para `ftp.ejemplo1.com` y `ftp.ejemplo2.com` e integrarlo en la renovación automática.
- Proteger el servicio con UFW y Fail2ban, y probarlo desde la línea de comandos y desde FileZilla.

> **CONSEJO:** Este módulo amplía la infraestructura del proyecto final. Requiere haber completado los módulos 1 a 5 (usuarios `web1`/`web2` y carpetas de las webs), 10 (Certbot) y 12 (UFW y Fail2ban).

15.1 Qué es FTP y para qué lo queremos

FTP (*File Transfer Protocol*) es uno de los protocolos más antiguos de Internet (1971). Sirve para subir y descargar archivos entre un cliente y un servidor. En un alojamiento web se usa para que la persona responsable de cada web pueda **subir sus archivos** (HTML, imágenes, una aplicación PHP) sin tener acceso de administrador al servidor.

Nuestro objetivo:

Usuario FTP	Accede a	Lo que ve como «/»	Puede escribir en
`web1`	ejemplo1.com	`/var/www/ejemplo1.com`	`public_html`, `app`, `config`, `tmp`, `logs`
`web2`	ejemplo2.com	`/var/www/ejemplo2.com`	Las mismas carpetas de su web
`sysadmin`, `root` y cualquier otro	—	Acceso denegado	—

Así, los archivos subidos pertenecen automáticamente al usuario de la web (`web1`), que es el mismo con el que se ejecuta su PHP. No hay que corregir propietarios después de cada subida.

Cómo funciona FTP: dos conexiones

A diferencia de HTTP, FTP usa **dos** conexiones TCP:

- La **conexión de control** (puerto 21): por ella viajan los comandos (`USER`, `PASS`, `LIST`, `STOR`...) y las respuestas.
- La **conexión de datos**: se abre una nueva para cada listado de carpeta y cada archivo transferido.

La conexión de datos puede abrirse de dos formas:

DIAGRAMA

```
MODO ACTIVO (el servidor llama al cliente) – problemático con NAT y cortafuegos
  Cliente:5001 ──── control ────► Servidor:21     "PORT 192.168.1.5,19,137"
  Cliente:5001 ◄──── datos ─────── Servidor:20     el servidor se conecta al cliente

MODO PASIVO (el cliente llama al servidor) – el que usaremos
  Cliente:5001 ──── control ────► Servidor:21     "PASV"
                                                    "227 Entering Passive Mode (...,156,64)"
  Cliente:5002 ──── datos ──────► Servidor:40000  el servidor abre un puerto del rango pasivo
```

En **modo pasivo** es el cliente quien abre las dos conexiones, lo que funciona detrás de los routers domésticos. A cambio, el servidor necesita un **rango de puertos** abierto en el cortafuegos para las conexiones de datos (usaremos 40000–40100).

El problema de seguridad de FTP

El FTP clásico envía **todo en texto plano**, incluidos el usuario y la contraseña. Cualquiera que pueda observar el tráfico (una red Wi-Fi pública, un equipo comprometido en el camino) las lee directamente. Por eso **nunca** debe usarse FTP sin cifrar en un servidor público.

Hay dos alternativas seguras:

Característica	FTPS (FTP sobre TLS)	SFTP (SSH File Transfer Protocol)
Qué es	FTP de siempre con cifrado TLS (como HTTPS).	Un protocolo distinto que funciona dentro de SSH.
Servidor	vsftpd	OpenSSH (ya instalado)
Puertos	21 + rango pasivo (40000–40100)	Solo el 22
Certificado	Sí (Let's Encrypt)	No (usa las claves del servidor SSH)
Compatibilidad	Cualquier cliente FTP con «FTP explícito sobre TLS»	FileZilla, WinSCP, Cyberduck, `sftp`, `scp`...
Complejidad	Mayor (dos conexiones, rango pasivo, NAT)	Menor

> **CONSEJO: Recomendación profesional:** si puedes elegir, SFTP es más sencillo de proteger porque solo usa el puerto 22. FTPS sigue siendo muy útil cuando los usuarios o las herramientas (editores web, aplicaciones antiguas, algunos servicios de despliegue) solo hablan FTP. En este módulo montamos **FTPS con vsftpd** y en el apartado 15.10 verás cómo ofrecer SFTP con la misma separación por webs.

vsftpd

vsftpd (*Very Secure FTP Daemon*) es el servidor FTP de referencia en Linux: pequeño, rápido y diseñado con la seguridad como prioridad (separación de privilegios, *chroot*, procesos sin permisos). Ubuntu 24.04 incluye vsftpd 3.0.5.

15.2 Preparar el DNS y el certificado

PASO 15.1 – Crear los registros DNS de FTP

Objetivo: que los usuarios se conecten a un nombre propio de su dominio.

Registros (en el panel DNS de cada dominio):

Dominio	Tipo	Nombre	Valor
ejemplo1.com	A	ftp	203.0.113.10
ejemplo2.com	A	ftp	203.0.113.10

Comprobación:

TERMINAL
```
dig +short A ftp.ejemplo1.com
dig +short A ftp.ejemplo2.com
```

Ambos deben devolver `203.0.113.10`.

PASO 15.2 – Obtener el certificado de ftp.ejemplo1.com y ftp.ejemplo2.com

Objetivo: tener un certificado válido para los dos nombres FTP.

Explicación: igual que con el correo (PASO 6.2), validamos a través del sitio comodín de Apache. Los nombres `ftp.*` no están en ningún VirtualHost, así que Apache los atiende con `000-default.conf`, cuyo DocumentRoot es `/var/www/html`.

Comando:

TERMINAL
```
sudo certbot certonly --webroot -w /var/www/html -d ftp.ejemplo1.com -d ftp.ejemplo2.com --
email administracion@ejemplo1.com --agree-tos --no-eff-email
```

Resultado esperado:

SALIDA ESPERADA
```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/ftp.ejemplo1.com/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/ftp.ejemplo1.com/privkey.pem
```

Errores habituales: los mismos del PASO 3.11 (DNS aún no propagado, puerto 80 cerrado).

> CONSEJO: **Laboratorio sin dominio real:** usa en el PASO 15.6 el certificado autofirmado (`/etc/ssl/certs/ssl-cert-snakeoil.pem` y `/etc/ssl/private/ssl-cert-snakeoil.key`). Los clientes FTP te pedirán aceptar el certificado la primera vez.

15.3 Instalación de vsftpd

PASO 15.3 – Instalar vsftpd

Comando:

TERMINAL
```
sudo apt install -y vsftpd acl
vsftpd -v
systemctl status vsftpd --no-pager
```

Análisis del comando: `vsftpd` es el servidor; `acl` aporta los comandos `setfacl` y `getfacl`, que necesitaremos en el PASO 15.5.

Resultado esperado:

SALIDA ESPERADA

```
vsftpd: version 3.0.5
● vsftpd.service - vsftpd FTP server
     Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; enabled; preset: enabled)
     Active: active (running) since ...
```

ATENCIÓN: Tras la instalación, vsftpd arranca con una configuración por defecto que **todavía no cifra**. El puerto 21 aún está cerrado en UFW, así que no es accesible desde fuera; no lo abras hasta terminar el PASO 15.7.

Archivos de vsftpd en Ubuntu:

Archivo	Contenido
`/etc/vsftpd.conf`	Configuración principal.
`/etc/pam.d/vsftpd`	Cómo se autentican los usuarios (PAM).
`/etc/ftpusers`	Usuarios a los que PAM **prohíbe** el FTP (root, daemon...).
`/etc/vsftpd.userlist`	(Lo crearemos) usuarios a los que vsftpd **permite** el FTP.
`/var/log/vsftpd.log`	Registro de conexiones y transferencias.

15.4 Usuarios FTP y jaula chroot

Por qué usamos los usuarios de las webs

Podríamos crear usuarios FTP aparte (`ftp_web1`...), pero entonces los archivos subidos tendrían un propietario distinto del que ejecuta PHP y habría que ajustar permisos constantemente. Usar directamente `web1` y `web2` mantiene el diseño del Módulo 3: **un usuario por web, dueño de todos sus archivos**.

Recuerda que estos usuarios tienen como shell `/usr/sbin/nologin` y que SSH solo admite a `sysadmin` (`AllowUsers`, PASO 1.7). Darles contraseña les permitirá **solo** usar FTP: no podrán iniciar sesión en el servidor.

PASO 15.4 – Asignar contraseña a los usuarios FTP

Objetivo: que `web1` y `web2` puedan autenticarse en FTP.

Comando:

TERMINAL

```
sudo passwd web1
sudo passwd web2
```

Elige contraseñas largas y distintas (puedes generarlas con `openssl rand -base64 18`).

Comprobación: el shell sigue siendo `nologin` y SSH sigue cerrado para ellos:

```
TERMINAL
getent passwd web1
sudo sshd -T | grep -i allowusers
```

```
SALIDA ESPERADA
web1:x:998:998::/var/www/ejemplo1.com:/usr/sbin/nologin
allowusers sysadmin
```

PASO 15.5 – Preparar la carpeta de cada web como jaula chroot

Objetivo: que cada usuario quede encerrado en la carpeta de su web y que vsftpd acepte esa carpeta como raíz.

Explicación: con `chroot_local_user=YES`, vsftpd encierra a cada usuario en su carpeta personal (*home*), que para `web1` es `/var/www/ejemplo1.com`. Por seguridad, vsftpd **se niega** a usar como raíz de la jaula una carpeta en la que el propio usuario puede escribir, y da este error al conectar:

```
SALIDA ESPERADA
500 OOPS: vsftpd: refusing to run with writable root inside chroot()
```

El motivo es que un usuario que puede escribir en la raíz de la jaula podría crear allí carpetas como `etc` o `lib` y engañar a programas del sistema que se ejecuten dentro de ella. Existe la opción `allow_writeable_chroot=YES` para saltarse la comprobación, pero **no la usaremos**: hacemos que la raíz no sea escribible y mantenemos la protección.

Ahora `/var/www/ejemplo1.com` pertenece a `web1` (PASO 3.2). La cambiamos a `root`, conservando el acceso de Apache (grupo `www-data`) y dando a `web1` permiso de **lectura y paso** (sin escritura) mediante una **ACL** (*Access Control List*, lista de control de acceso): un permiso adicional para un usuario concreto, más allá de propietario, grupo y otros.

Comando:

```
TERMINAL
sudo chown root:www-data /var/www/ejemplo1.com /var/www/ejemplo2.com
sudo chmod 750 /var/www/ejemplo1.com /var/www/ejemplo2.com
sudo setfacl -m u:web1:rx /var/www/ejemplo1.com
sudo setfacl -m u:web2:rx /var/www/ejemplo2.com
getfacl /var/www/ejemplo1.com
```

Análisis del comando:

Orden	Efecto
`chown root:www-data`	El dueño pasa a ser root (nadie más puede escribir en la raíz); el grupo sigue siendo `www-data`.
`chmod 750`	root: todo; `www-data`: entrar y listar (Apache sigue llegando a `public_html`); otros (por ejemplo `web2`): nada.
`setfacl -m u:web1:rx`	Añade (`-m`, *modify*) una entrada para el usuario `web1` con lectura y paso, sin escritura.
`getfacl`	Muestra todos los permisos, incluidas las ACL.

Resultado esperado:

SALIDA ESPERADA

```
# file: var/www/ejemplo1.com
# owner: root
# group: www-data
user::rwx
user:web1:r-x
group::r-x
mask::r-x
other::---
```

Las subcarpetas **no cambian**: `public_html` sigue siendo de `web1` (con setgid y grupo `www-data`), y `config`, `app`, `tmp` y `logs` siguen siendo de `web1` con permisos `700`. Por tanto `web1` puede escribir en todas ellas, pero no crear archivos sueltos en la raíz de su web (no lo necesita).

Comprobación: la web y PHP deben seguir funcionando exactamente igual:

TERMINAL

```
sudo -u www-data ls /var/www/ejemplo1.com/public_html > /dev/null && echo "Apache llega a
public_html"
sudo -u web1 ls /var/www/ejemplo1.com && echo "web1 puede listar su carpeta"
sudo -u web1 touch /var/www/ejemplo1.com/prueba 2>&1 | head -1
sudo -u web2 ls /var/www/ejemplo1.com 2>&1 | head -1
curl -s -o /dev/null -w "%{http_code}\n" https://www.ejemplo1.com/
```

SALIDA ESPERADA

```
Apache llega a public_html
app  config  logs  public_html  tmp
web1 puede listar su carpeta
touch: cannot touch '/var/www/ejemplo1.com/prueba': Permission denied
ls: cannot open directory '/var/www/ejemplo1.com': Permission denied
200
```

Errores habituales: `setfacl: command not found` (falta el paquete `acl`); si la web da 403 o 503 tras el cambio, revisa con `namei -l /var/www/ejemplo1.com/public_html/index.php` y con `getfacl` que el grupo es `www-data` y la ACL de `web1` existe.

PASO 15.6 – Permitir el inicio de sesión a usuarios con shell nologin

Objetivo: que PAM no rechace a `web1` y `web2` por tener `/usr/sbin/nologin` como shell.

Explicación: el archivo PAM de vsftpd en Ubuntu termina con la línea `auth required pam_shells.so`, que solo deja entrar a usuarios cuyo shell esté en `/etc/shells`. `nologin` no está (y no debe añadirse, porque otros servicios usan esa lista). Desactivamos esa comprobación **solo para vsftpd**; el control de quién puede usar FTP lo haremos con una lista explícita (PASO 15.7).

Comando:

TERMINAL

```
sudo cp /etc/pam.d/vsftpd /etc/pam.d/vsftpd.original
sudo sed -i 's/^auth\s\+required\s\+pam_shells.so/# auth required pam_shells.so  (desactivado:
los usuarios FTP usan nologin)/' /etc/pam.d/vsftpd
cat /etc/pam.d/vsftpd
```

Resultado esperado:

```
SALIDA ESPERADA
# Standard behaviour for ftpd(8).
auth    required       pam_listfile.so item=user sense=deny file=/etc/ftpusers onerr=succeed

# Note: vsftpd handles anonymous logins on its own. Do not enable pam_ftp.so.

# Standard pam includes
@include common-account
@include common-session
@include common-auth
# auth required pam_shells.so  (desactivado: los usuarios FTP usan nologin)
```

La primera línea se mantiene: los usuarios de `/etc/ftpusers` (root y las cuentas del sistema) siguen sin poder usar FTP.

PASO 15.7 – Crear la lista de usuarios autorizados

Objetivo: que **solo** `web1` y `web2` puedan conectarse por FTP.

Archivo: `/etc/vsftpd.userlist` (nuevo, contenido completo):

```
ARCHIVO DE CONFIGURACIÓN
web1
web2
```

```
TERMINAL
sudo chmod 644 /etc/vsftpd.userlist
```

Con la configuración del paso siguiente (`userlist_deny=NO`), esta lista funciona como «lista blanca»: cualquier otro usuario (incluido `sysadmin`) es rechazado **antes** de pedir la contraseña, así que su contraseña nunca viaja por FTP.

15.5 Configuración de vsftpd

PASO 15.8 – Escribir la configuración vsftpd.conf

Objetivo: configurar vsftpd con usuarios locales enjaulados, TLS obligatorio y modo pasivo.

Comando:

```
TERMINAL
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.original
sudo nano /etc/vsftpd.conf
```

Archivo: `/etc/vsftpd.conf` (sustituye todo el contenido):

```
ARCHIVO DE CONFIGURACIÓN
# ================================================================
#  /etc/vsftpd.conf - FTPS para ejemplo1.com y ejemplo2.com
# ================================================================

# --- Escucha ----------------------------------------------------
```

```
# Un único proceso que escucha en IPv6 y también en IPv4
listen=NO
listen_ipv6=YES

# --- Usuarios -----------------------------------------------------------
# Sin acceso anónimo
anonymous_enable=NO
# Usuarios locales de Linux (web1, web2) con permiso de escritura
local_enable=YES
write_enable=YES
# Archivos nuevos 640 y carpetas nuevas 750
local_umask=027
# Solo pueden entrar los usuarios de /etc/vsftpd.userlist
userlist_enable=YES
userlist_deny=NO
userlist_file=/etc/vsftpd.userlist
pam_service_name=vsftpd

# --- Jaula chroot -------------------------------------------------------
# Cada usuario queda encerrado en su carpeta personal (/var/www/ejemploN.com)
chroot_local_user=YES
# La raíz de la jaula NO debe ser escribible (ver PASO 15.5)
allow_writeable_chroot=NO
secure_chroot_dir=/var/run/vsftpd/empty
# Los listados muestran "ftp" en lugar de los nombres reales de usuario y grupo
hide_ids=YES

# --- Cifrado TLS obligatorio (FTPS explícito) ---------------------------
ssl_enable=YES
rsa_cert_file=/etc/letsencrypt/live/ftp.ejemplo1.com/fullchain.pem
rsa_private_key_file=/etc/letsencrypt/live/ftp.ejemplo1.com/privkey.pem
allow_anon_ssl=NO
force_local_logins_ssl=YES
force_local_data_ssl=YES
ssl_sslv2=NO
ssl_sslv3=NO
ssl_ciphers=HIGH
# Algunos clientes (FileZilla) fallan si se exige reutilizar la sesión TLS
require_ssl_reuse=NO

# --- Modo pasivo --------------------------------------------------------
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=40100
# Si el servidor está detrás de NAT (IP privada en la nube), descomenta y
# pon la IP pública:
#pasv_address=203.0.113.10

# --- Modo activo --------------------------------------------------------
connect_from_port_20=YES

# --- Registro -----------------------------------------------------------
xferlog_enable=YES
xferlog_std_format=NO
vsftpd_log_file=/var/log/vsftpd.log
log_ftp_protocol=NO
use_localtime=YES

# --- Límites y cortesía -------------------------------------------------
ftpd_banner=Servidor FTP. Solo se admiten conexiones cifradas.
idle_session_timeout=600
data_connection_timeout=120
max_clients=50
```

```
max_per_ip=5
dirmessage_enable=NO
```

Análisis de las directivas principales:

Directiva	Significado
`listen=NO` + `listen_ipv6=YES`	Un solo socket IPv6 que acepta también IPv4. Si activaras las dos, vsftpd fallaría con `could not bind listening IPv4 socket`.
`anonymous_enable=NO`	Nadie puede entrar como `anonymous`.
`local_enable` / `write_enable`	Los usuarios locales pueden entrar y subir, modificar y borrar archivos.
`local_umask=027`	Permisos de lo que se sube: archivos `640` y carpetas `750`, iguales a los del Módulo 3. Gracias al bit setgid de `public_html`, el grupo será `www-data`.
`userlist_enable=YES` + `userlist_deny=NO`	La lista `/etc/vsftpd.userlist` es una lista de **permitidos**.
`chroot_local_user=YES`	Cada usuario ve su carpeta personal como `/` y no puede salir de ella.
`allow_writeable_chroot=NO`	Mantiene la comprobación de seguridad del PASO 15.5.
`hide_ids=YES`	Los listados no revelan nombres de usuarios del sistema.
`ssl_enable=YES`	Activa TLS.
`force_local_logins_ssl` / `force_local_data_ssl`	Exige cifrado para la contraseña **y** para los datos. Un cliente sin TLS recibe `530 Non-anonymous sessions must use encryption.`
`ssl_sslv2=NO`, `ssl_sslv3=NO`	Desactiva los protocolos obsoletos.
`ssl_ciphers=HIGH`	Solo cifrados robustos.
`require_ssl_reuse=NO`	Por defecto vsftpd exige que la conexión de datos reutilice la sesión TLS de la de control; es más seguro, pero muchos clientes fallan con `425 Unable to build data connection`.
`pasv_min_port` / `pasv_max_port`	Rango de puertos de datos en modo pasivo (101 puertos: suficiente para 50 transferencias simultáneas).
`pasv_address`	Solo si el servidor no tiene la IP pública en su interfaz (NAT de algunos proveedores de nube).
`xferlog_std_format=NO`	Registro en el formato propio de vsftpd en `/var/log/vsftpd.log`, que es el que entiende Fail2ban.
`max_clients` / `max_per_ip`	Máximo de conexiones totales y por IP.

Comprueba y aplica:

TERMINAL

```
sudo systemctl restart vsftpd
systemctl status vsftpd --no-pager
sudo ss -tlnp | grep vsftpd
```

Resultado esperado:

```
SALIDA ESPERADA
● vsftpd.service - vsftpd FTP server
     Active: active (running) since ...
LISTEN 0  32  *:21  *:*  users:(("vsftpd",pid=10240,fd=3))
```

`*:21` significa que escucha en IPv4 e IPv6.

Errores habituales:

Mensaje	Causa	Solución
`500 OOPS: could not bind listening IPv4 socket`	`listen=YES` y `listen_ipv6=YES` a la vez.	Deja solo `listen_ipv6=YES`.
`500 OOPS: SSL: cannot load RSA certificate`	Ruta del certificado incorrecta.	`sudo ls -l /etc/letsencrypt/live/ftp.ejemplo1.com/`.
`500 OOPS: unrecognised variable in config file: ...`	Errata en un nombre de directiva.	Corrige la línea indicada.
El servicio no arranca sin mensaje claro	Error de sintaxis (vsftpd no admite espacios alrededor del `=`).	`sudo journalctl -u vsftpd -n 20 --no-pager`; escribe `opcion=valor` sin espacios.

PASO 15.9 – Abrir los puertos en el cortafuegos

Comando:

```
TERMINAL
sudo ufw allow 21/tcp comment 'FTP control (FTPS explicito)'
sudo ufw allow 40000:40100/tcp comment 'FTP pasivo'
sudo ufw status numbered
```

Análisis: `40000:40100/tcp` abre el rango completo de puertos pasivos. Si tu proveedor tiene un cortafuegos externo, ábrelos también allí.

> **ATENCIÓN:** No abras el puerto 20: en modo pasivo no se usa, y en modo activo es el **servidor** quien sale desde él (UFW permite todo el tráfico saliente). Tampoco abras el 990 (FTPS implícito): no lo hemos configurado.

15.6 Pruebas de funcionamiento

PASO 15.10 – Probar el acceso con curl y lftp

Objetivo: verificar el cifrado, la autenticación, la jaula y la subida de archivos. Haz estas pruebas desde **otro equipo** (tu ordenador con Linux, macOS o WSL en Windows).

Comprobar el certificado del puerto 21 con STARTTLS:

TERMINAL

```
echo | openssl s_client -connect ftp.ejemplo1.com:21 -starttls ftp 2>/dev/null | openssl x509
-noout -subject -ext subjectAltName
```

SALIDA ESPERADA

```
subject=CN = ftp.ejemplo1.com
X509v3 Subject Alternative Name:
    DNS:ftp.ejemplo1.com, DNS:ftp.ejemplo2.com
```

Listar la carpeta con conexión cifrada (curl pedirá la contraseña):

TERMINAL

```
curl --ssl-reqd -u web1 ftp://ftp.ejemplo1.com/
```

Análisis: `--ssl-reqd` exige FTPS explícito (`AUTH TLS` sobre el puerto 21) y aborta si el servidor no lo admite. Como `-u web1` no lleva contraseña, curl la pide sin que quede en el historial.

SALIDA ESPERADA

```
drwx------    2 ftp      ftp          4096 Nov 09 10:12 app
drwx------    2 ftp      ftp          4096 Nov 09 10:12 config
drwx------    2 ftp      ftp          4096 Nov 09 10:12 logs
drwxr-s---    2 ftp      ftp          4096 Nov 09 10:12 public_html
drwx------    2 ftp      ftp          4096 Nov 09 10:12 tmp
```

Subir un archivo a la web:

TERMINAL

```
echo '<h1>Subido por FTPS</h1>' > subida.html
curl --ssl-reqd -u web1 -T subida.html ftp://ftp.ejemplo1.com/public_html/
curl -s https://www.ejemplo1.com/subida.html
```

SALIDA ESPERADA

```
<h1>Subido por FTPS</h1>
```

Comprobaciones de seguridad (todas deben **fallar**):

TERMINAL

```
# 1. Sin cifrado
curl -u web1 ftp://ftp.ejemplo1.com/
# 2. Usuario no autorizado
curl --ssl-reqd -u sysadmin ftp://ftp.ejemplo1.com/
# 3. Salir de la jaula
curl --ssl-reqd -u web1 ftp://ftp.ejemplo1.com/../../etc/
```

SALIDA ESPERADA

```
curl: (67) Access denied: 530          ← 530 Non-anonymous sessions must use encryption.
curl: (67) Access denied: 530          ← 530 Permission denied. (sin llegar a pedir la
contraseña)
                                         ← muestra de nuevo /var/www/ejemplo1.com: "/" es la
raíz de la jaula
```

Con **lftp**, un cliente FTP de terminal muy completo, puedes trabajar de forma interactiva:

TERMINAL

```
lftp -u web1 -e "set ftp:ssl-force true; set ssl:verify-certificate yes" ftp.ejemplo1.com
```

Dentro de lftp: `ls`, `cd public_html`, `put archivo`, `get archivo`, `mirror -R carpeta_local carpeta_remota` (sube una carpeta entera) y `bye` para salir.

Comprobación en el servidor: propietario y permisos de lo subido, y registro:

TERMINAL

```
sudo ls -l /var/www/ejemplo1.com/public_html/subida.html
sudo tail -n 5 /var/log/vsftpd.log
```

SALIDA ESPERADA

```
-rw-r----- 1 web1 www-data 25 nov  9 10:20 /var/www/ejemplo1.com/public_html/subida.html
Mon Nov  9 10:20:01 2026 [pid 10301] [web1] OK LOGIN: Client "::ffff:198.51.100.7"
Mon Nov  9 10:20:02 2026 [pid 10303] [web1] OK UPLOAD: Client "::ffff:198.51.100.7",
"/public_html/subida.html", 25 bytes, 0.41Kbyte/sec
```

El archivo pertenece a `web1`, su grupo es `www-data` y sus permisos `640`: exactamente lo que necesita Apache para servirlo. Bórralo al terminar:

TERMINAL

```
sudo rm /var/www/ejemplo1.com/public_html/subida.html
```

Errores habituales:

Síntoma	Causa	Solución
`530 Login incorrect.`	Contraseña errónea, o PAM rechaza el shell.	`sudo passwd web1`; revisa el PASO 15.6. En `/var/log/auth.log` verás el motivo.
`500 OOPS: vsftpd: refusing to run with writable root inside chroot()`	La raíz de la web es escribible por el usuario.	PASO 15.5.
El login funciona pero el listado se queda colgado o da `425`	Puertos pasivos cerrados, o `pasv_address` necesario por NAT.	UFW y cortafuegos del proveedor (40000–40100); configura `pasv_address`.
`425 Unable to build data connection: ... SSL`	El cliente no reutiliza la sesión TLS.	`require_ssl_reuse=NO`.
`553 Could not create file.`	El usuario no puede escribir en esa carpeta (por ejemplo, en la raíz de la jaula).	Sube a `public_html/`.
Error de certificado en el cliente	Conectaste con la IP o con `mail.*` en lugar de `ftp.*`.	Usa `ftp.ejemplo1.com` o `ftp.ejemplo2.com`.

PASO 15.11 – Configurar FileZilla

Objetivo: dar acceso a la persona responsable de cada web con un cliente gráfico.

En FileZilla: «Archivo» → «Gestor de sitios» → «Nuevo sitio» y rellena:

Campo	Valor para ejemplo1.com	Valor para ejemplo2.com
Protocolo	FTP – Protocolo de transferencia de archivos	Igual
Servidor	`ftp.ejemplo1.com`	`ftp.ejemplo2.com`
Puerto	21 (o vacío)	21 (o vacío)
Cifrado	**Requiere FTP explícito sobre TLS**	Igual
Modo de acceso	Normal	Normal
Usuario	`web1`	`web2`
Contraseña	La del PASO 15.4	La del PASO 15.4

En la pestaña «Opciones de transferencia», el modo debe ser **Pasivo** (es el predeterminado).

Comprobación: al conectar, FileZilla muestra el certificado; comprueba que está emitido para `ftp.ejemplo1.com`, acéptalo y verás las carpetas de la web. Arrastra un archivo a `public_html` y ábrelo en el navegador.

> **ATENCIÓN:** Explica a los usuarios que **nunca** elijan «Usar FTP plano (inseguro)». Con nuestra configuración el servidor lo rechazaría, pero el cliente habría enviado ya el nombre de usuario sin cifrar.

15.7 Integración con la infraestructura

PASO 15.12 – Recargar vsftpd cuando se renueve su certificado

Objetivo: incluir vsftpd en el script de renovación del Módulo 10.

Archivo: `/etc/letsencrypt/renewal-hooks/deploy/recargar-servicios.sh` (sustituye todo el contenido):

TERMINAL

```
#!/bin/bash
# Se ejecuta tras cada renovación correcta de un certificado.
# Certbot define RENEWED_LINEAGE, por ejemplo /etc/letsencrypt/live/mail.ejemplo1.com
set -u

logger -t certbot-deploy "Certificado renovado: ${RENEWED_LINEAGE}"

case "${RENEWED_LINEAGE}" in
    */mail.ejemplo1.com)
        # Certificado del correo: Postfix y Dovecot
        systemctl reload postfix
        systemctl reload dovecot
        logger -t certbot-deploy "Postfix y Dovecot recargados"
        ;;
    */ftp.ejemplo1.com)
        # Certificado del FTP: vsftpd lo carga al arrancar
        systemctl restart vsftpd
        logger -t certbot-deploy "vsftpd reiniciado"
        ;;
    *)
```

```
        # Certificados de las webs: Apache
        if apachectl configtest >/dev/null 2>&1; then
            systemctl reload apache2
            logger -t certbot-deploy "Apache recargado"
        else
            logger -t certbot-deploy "ERROR: configuración de Apache inválida, no se recarga"
        fi
        ;;
esac
```

Análisis: el nuevo caso reinicia vsftpd cuando se renueva el certificado `ftp.ejemplo1.com`. El reinicio corta las transferencias en curso, pero Certbot renueva de madrugada y solo una vez cada dos meses.

Prueba el script:

TERMINAL

```
sudo RENEWED_LINEAGE=/etc/letsencrypt/live/ftp.ejemplo1.com /etc/letsencrypt/renewal-hooks/deploy/recargar-servicios.sh
sudo journalctl -t certbot-deploy -n 3 --no-pager
```

PASO 15.13 – Proteger el FTP con Fail2ban

Objetivo: bloquear las IP que prueban contraseñas en el FTP.

Archivo: `/etc/fail2ban/jail.local`. Conserva todo lo del PASO 12.5 y **añade al final**:

ARCHIVO DE CONFIGURACIÓN

```
# --- vsftpd: contraseñas FTP incorrectas ---
[vsftpd]
enabled  = true
logpath  = /var/log/vsftpd.log
backend  = auto
maxretry = 4
```

Comando:

TERMINAL

```
sudo fail2ban-client -t && sudo systemctl restart fail2ban
sudo fail2ban-client status vsftpd
```

Análisis: el filtro `vsftpd` incluido en Fail2ban busca las líneas `FAIL LOGIN` de `/var/log/vsftpd.log`. Al bloquear una IP, se cierran a la vez el puerto 21 y el rango pasivo, porque la acción bloquea todo el tráfico de esa IP hacia los puertos indicados en el jail (que por defecto incluye `ftp`, `ftp-data` y `ftps`) y, sin el 21, el resto queda inservible.

Comprobación: falla la contraseña cuatro veces desde una IP de pruebas y comprueba que aparece en `Banned IP list`. Desbloquéala con `sudo fail2ban-client set vsftpd unbanip IP`.

PASO 15.14 – Añadir vsftpd a la monitorización

Objetivo: que el script de estado del Módulo 12 vigile también el FTP.

Archivo: `/usr/local/sbin/estado-servidor`. Modifica solo estas dos líneas del principio (el resto se conserva):

TERMINAL

```
SERVICIOS="apache2 php8.3-fpm mariadb postfix@- dovecot opendkim fail2ban ssh cron vsftpd"
PUERTOS="21 22 25 80 443 465 587 993"
```

Comprobación: `sudo estado-servidor | grep -E 'vsftpd|puerto 21'` debe mostrar dos líneas `[OK]`.

> **CONSEJO:** Las copias de seguridad no necesitan cambios: el script del PASO 12.7 ya incluye `/var/www` (los archivos subidos) y `/etc` (la configuración de vsftpd y PAM).

15.8 Gestión del día a día

Tarea	Cómo
Cambiar la contraseña FTP de una web	`sudo passwd web1`
Quitar temporalmente el FTP a una web	Borra su línea de `/etc/vsftpd.userlist` (efecto inmediato en nuevas conexiones).
Bloquear la cuenta sin tocar la lista	`sudo passwd -l web1` (y `sudo passwd -u web1` para desbloquearla).
Dar FTP a una nueva web (`web3`)	Crea la web (Anexo D), haz los PASOS 15.4 y 15.5 para `web3`, añádela a `vsftpd.userlist` y amplía el certificado con `ftp.ejemplo3.com`.
Ver quién está conectado	`ps -eo user,cmd \
Ver las subidas de hoy	`sudo grep "$(date '+%a %b %e')" /var/log/vsftpd.log \
Ver los fallos de acceso	`sudo grep 'FAIL LOGIN' /var/log/vsftpd.log \
Desactivar el FTP por completo	`sudo systemctl disable --now vsftpd` y cierra los puertos con `sudo ufw delete allow 21/tcp` y `sudo ufw delete allow 40000:40100/tcp`.

15.9 Diagrama de la integración

DIAGRAMA

```
FileZilla / lftp (persona responsable de ejemplo1.com)
     │ 1. TCP 21 · AUTH TLS · certificado ftp.ejemplo1.com
     │ 2. USER web1 → ¿está en /etc/vsftpd.userlist? → PASS (cifrada)
     ▼
┌──────────────────────────────── SERVIDOR ──────────────────────────────────┐
│  UFW: 21 y 40000-40100 permitidos · Fail2ban vigila /var/log/vsftpd.log    │
│                                                                            │
│  VSFTPD ── PAM (/etc/pam.d/vsftpd): /etc/ftpusers + contraseña de web1     │
│      │ 3. chroot en /var/www/ejemplo1.com (raíz root:www-data + ACL web1)  │
│      │ 4. PASV → conexión de datos cifrada por un puerto 40000-40100       │
│      ▼                                                                     │
│  /var/www/ejemplo1.com/public_html/nuevo.php  (web1:www-data, 640)         │
│      │                                                                     │
│      ├──→ APACHE lo sirve (grupo www-data)                                 │
│      └──→ PHP-FPM pool web1 lo ejecuta (usuario web1)                      │
└────────────────────────────────────────────────────────────────────────────┘
```

15.10 Alternativa: SFTP con OpenSSH

Si prefieres no abrir el puerto 21 ni el rango pasivo, OpenSSH puede ofrecer SFTP con la misma separación por webs. Esquema de la configuración (sustituye a vsftpd; no uses ambos sin necesidad):

Archivo: `/etc/ssh/sshd_config.d/10-sftp-webs.conf` (nuevo):

ARCHIVO DE CONFIGURACIÓN

```
# Los usuarios del grupo sftpwebs solo pueden usar SFTP, encerrados en su web
Match Group sftpwebs
    ChrootDirectory %h
    ForceCommand internal-sftp -u 0027
    PasswordAuthentication yes
    AllowTcpForwarding no
    X11Forwarding no
```

Comandos:

TERMINAL

```
sudo groupadd sftpwebs
sudo usermod -aG sftpwebs web1
sudo usermod -aG sftpwebs web2
sudo sed -i 's/^AllowUsers .*/AllowUsers sysadmin web1 web2/' /etc/ssh/sshd_config.d/00-
endurecimiento.conf
sudo sshd -t && sudo systemctl restart ssh
```

Análisis:

Elemento	Significado
`Match Group sftpwebs`	Lo que sigue solo se aplica a los miembros de ese grupo.
`ChrootDirectory %h`	Encierra al usuario en su carpeta personal. OpenSSH exige que esa carpeta **y todas las superiores** pertenezcan a root y no sean escribibles por nadie más: el PASO 15.5 ya lo cumple.
`ForceCommand internal-sftp -u 0027`	Solo SFTP (nunca un terminal), con archivos nuevos `640`.

Elemento	Significado
`PasswordAuthentication yes`	Permite contraseña solo a estos usuarios (lo ideal es usar también claves).
`AllowUsers`	Hay que añadir `web1` y `web2`, porque en el PASO 1.7 limitamos SSH a `sysadmin`.

Los clientes se conectan con protocolo **SFTP**, puerto 22 y servidor `ftp.ejemplo1.com` (o cualquier nombre del servidor). Fail2ban ya protege el puerto 22 con el jail `sshd`.

15.11 Ejercicios prácticos del Módulo 15

1. Conecta con FileZilla como `web2` y comprueba que no puedes ver ni entrar en `/var/www/ejemplo1.com`. ¿Qué ves como carpeta raíz?
2. Sube con `lftp mirror -R` una carpeta con varios archivos HTML a `public_html` de ejemplo2.com y verifica sus permisos en el servidor.
3. Cambia temporalmente `allow_writeable_chroot` y la propiedad de `/var/www/ejemplo1.com` para provocar el error `refusing to run with writable root inside chroot()`. Explica por qué existe esa protección y restaura la configuración.
4. Comenta el rango pasivo en UFW y observa qué hace FileZilla al listar una carpeta. Restaura la regla.
5. Configura la alternativa SFTP del apartado 15.10 en una máquina de pruebas y compara ambos métodos: puertos abiertos, configuración necesaria y experiencia de uso.

Glosario técnico

Término	Definición
A (registro)	Registro DNS que asocia un nombre a una dirección IPv4.
ACL (permisos)	Lista de control de acceso: permisos adicionales para usuarios o grupos concretos sobre un archivo, más allá de propietario, grupo y otros (`setfacl`, `getfacl`).
AAAA (registro)	Registro DNS que asocia un nombre a una dirección IPv6.
ACME	Protocolo con el que Let's Encrypt verifica el control de un dominio y emite certificados.
Alias (correo)	Dirección sin buzón propio que reenvía a una o varias direcciones.
APT	Gestor de paquetes de Debian y Ubuntu (`apt install`, `apt update`).
Backscatter	Rebotes enviados a remitentes falsificados cuando un servidor acepta correo para direcciones inexistentes y luego lo devuelve.
CA (Autoridad de Certificación)	Entidad de confianza que firma certificados digitales.
Certbot	Cliente oficial de Let's Encrypt para obtener y renovar certificados.
Chroot	Técnica que encierra un proceso en una carpeta, impidiéndole ver el resto del sistema. Postfix la usa en `/var/spool/postfix`.
CIDR	Notación de rangos de IP con barra (`192.168.1.0/24`).
CNAME	Registro DNS que convierte un nombre en alias de otro nombre.
Cron	Programador de tareas periódicas de Linux.
CRUD	*Create, Read, Update, Delete*: las cuatro operaciones básicas sobre datos.
CSRF	Ataque que engaña al navegador de un usuario para enviar una petición no deseada a otra web; se evita con tokens.
Daemon (demonio)	Programa que se ejecuta en segundo plano prestando un servicio.
DKIM	Firma criptográfica de los correos salientes, verificable con una clave pública publicada en el DNS.
DMARC	Política DNS que indica qué hacer con correos que no superan SPF/DKIM alineados con el remitente visible, y dónde enviar informes.
DNS	Sistema que traduce nombres de dominio a direcciones IP y otros datos.
DocumentRoot	Carpeta cuyo contenido publica un VirtualHost de Apache.
Dovecot	Servidor IMAP/POP3; en este manual también entrega el correo (LMTP) y autentica (SASL).
Envelope (sobre)	Remitente y destinatarios usados en la transmisión SMTP (`MAIL FROM`, `RCPT TO`), distintos de las cabeceras visibles.
Fail2ban	Herramienta que bloquea IP tras detectar fallos de autenticación repetidos en los logs.
FCrDNS	Coincidencia entre el DNS inverso de una IP y el DNS directo de ese nombre.
FQDN	Nombre de dominio completo (`mail.ejemplo1.com`).

Término	Definición
FTP	Protocolo de transferencia de archivos con una conexión de control (puerto 21) y conexiones de datos aparte. Sin cifrar es inseguro.
FTPS	FTP protegido con TLS. En modo explícito, el cliente pide el cifrado con `AUTH TLS` en el puerto 21.
Hash	Resultado de una función criptográfica de un solo sentido; se usa para guardar contraseñas sin almacenarlas en claro.
HSTS	Cabecera que obliga al navegador a usar siempre HTTPS con un sitio.
IMAP	Protocolo para consultar el correo manteniéndolo sincronizado en el servidor.
InnoDB	Motor de almacenamiento transaccional de MariaDB.
Inyección SQL	Ataque que introduce código SQL a través de datos no filtrados; se evita con consultas preparadas.
Let's Encrypt	Autoridad de certificación gratuita y automatizada.
LMTP	Protocolo de entrega local de correo, usado entre Postfix y Dovecot.
Log	Registro de eventos de un programa.
Maildir	Formato de buzón en el que cada mensaje es un archivo independiente.
MariaDB	Sistema gestor de bases de datos relacional, compatible con MySQL.
MDA / MTA / MUA / MSA	Agente de entrega, de transferencia, de usuario y de envío de correo (ver Módulo 6).
Milter	Protocolo para que Postfix pase los correos a filtros externos (OpenDKIM, antispam).
MPM	Módulo de Apache que define cómo atiende peticiones concurrentes (`event`, `prefork`, `worker`).
Modo pasivo (FTP)	Forma de abrir la conexión de datos FTP en la que el cliente se conecta a un puerto del rango que indica el servidor. Funciona detrás de NAT.
MX (registro)	Registro DNS que indica el servidor que recibe el correo de un dominio.
open_basedir	Directiva de PHP que restringe las carpetas a las que puede acceder el código.
OpenDKIM	Servicio que firma y verifica DKIM.
PDO	Interfaz de PHP para acceder a bases de datos de forma uniforme.
PHP-FPM	Gestor de procesos PHP que atiende peticiones FastCGI, con *pools* independientes.
Pool (PHP-FPM)	Grupo de procesos PHP con usuario, socket y configuración propios.
POP3	Protocolo para descargar el correo del servidor al dispositivo.
Postfix	Servidor SMTP (MTA) utilizado en este manual.
Puerto	Número que identifica un servicio dentro de un equipo (0–65535).
PTR	Registro de DNS inverso: de IP a nombre.
Relay abierto	Servidor de correo que reenvía mensajes de cualquiera a cualquiera; es una vulnerabilidad grave.

Término	Definición
SAN	Campo de un certificado con la lista de nombres que cubre.
SASL	Marco de autenticación usado por SMTP para comprobar usuario y contraseña.
Selector DKIM	Nombre que identifica una clave DKIM (`mail` en `mail._domainkey`).
SFTP	Transferencia de archivos dentro de una conexión SSH (puerto 22). No tiene relación con FTP salvo el propósito.
SMTP	Protocolo de envío y transporte de correo electrónico.
SNI	Extensión de TLS que indica el nombre del sitio al iniciar la conexión, permitiendo varios certificados en una IP.
Socket Unix	Archivo especial para comunicar procesos del mismo equipo sin pasar por la red.
SPF	Registro DNS que enumera las IP autorizadas a enviar correo de un dominio.
SSH	Protocolo de acceso remoto cifrado.
STARTTLS	Comando que convierte una conexión en texto plano en cifrada.
sudo	Comando para ejecutar órdenes con privilegios de administrador.
systemd / systemctl	Gestor de servicios de Linux y su herramienta de control.
TCP / UDP	Protocolos de transporte: con conexión y garantías (TCP) o sin ellas (UDP).
TLS	Protocolo criptográfico que cifra y autentica las comunicaciones (sucesor de SSL).
TTL	Tiempo durante el que una respuesta DNS puede guardarse en caché.
UFW	Interfaz sencilla del cortafuegos de Linux.
Usuario virtual	Cuenta de correo que no es una cuenta del sistema operativo.
VirtualHost	Bloque de configuración de Apache que define un sitio web.
vmail	Usuario técnico de Linux propietario de todos los buzones virtuales.
vsftpd	Servidor FTP de Linux orientado a la seguridad; en este manual sirve FTPS con usuarios enjaulados en su web.
XSS	Ataque que inyecta JavaScript en una página; se evita escapando la salida con `htmlspecialchars`.

Anexos

Anexo A. Tabla de puertos

Puerto	Protocolo	Servicio	Programa	Cifrado	Abierto a Internet
21	TCP	FTP control (FTPS explícito)	vsftpd	TLS obligatorio	Sí (Módulo 15)
22	TCP	SSH	sshd	Sí	Sí
25	TCP	SMTP (recepción entre servidores)	Postfix	STARTTLS opcional	Sí
80	TCP	HTTP	Apache	No (redirige a 443)	Sí
110	TCP	POP3	Dovecot	—	No (deshabilitado)
143	TCP	IMAP	Dovecot	—	No (deshabilitado)
443	TCP	HTTPS	Apache	TLS	Sí
465	TCP	SMTPS (envío autenticado)	Postfix	TLS implícito	Sí
587	TCP	Submission (envío autenticado)	Postfix	STARTTLS obligatorio	Sí
993	TCP	IMAPS	Dovecot	TLS implícito	Sí
995	TCP	POP3S	Dovecot	TLS implícito	Sí (opcional)
3306	TCP	MariaDB	mariadbd	—	**No** (127.0.0.1)
8891	TCP	Milter OpenDKIM	opendkim	—	**No** (127.0.0.1)
40000–40100	TCP	FTP pasivo (datos)	vsftpd	TLS obligatorio	Sí (Módulo 15)

Anexo B. Mapa de archivos de la infraestructura

Ruta	Contenido	Módulo
`/etc/hosts`, `/etc/hostname`	Nombre del servidor	1
`/etc/ssh/sshd_config.d/00-endurecimiento.conf`	Endurecimiento de SSH	1
`/etc/apache2/conf-available/servername.conf`	ServerName global	2
`/etc/apache2/sites-available/000-default.conf`	Sitio comodín HTTP	3
`/etc/apache2/sites-available/ejemplo1.com.conf` / `ejemplo2.com.conf`	VirtualHosts	3, 5
`/etc/apache2/conf-available/cabeceras-seguridad.conf`	Cabeceras de seguridad	12
`/var/www/ejemploN.com/{public_html,config,app,logs,tmp}`	Webs	3, 5
`/etc/mysql/mariadb.conf.d/50-server.cnf`	Configuración de MariaDB	4
`/etc/php/8.3/fpm/pool.d/web1.conf` / `web2.conf`	Pools de PHP-FPM	5
`/etc/letsencrypt/live/*/`	Certificados	3, 6, 10
`/etc/letsencrypt/renewal-hooks/deploy/recargar-servicios.sh`	Recarga tras renovar	10

Ruta	Contenido	Módulo
`/etc/postfix/main.cf`, `/etc/postfix/master.cf`	Postfix	6, 7, 8, 9, 12
`/etc/postfix/virtual_domains`, `vmailbox`, `virtual`, `sender_login`	Tablas de correo	7, 8
`/etc/aliases`	Alias del sistema	6, 8
`/etc/dovecot/conf.d/10-auth.conf`, `auth-passwdfile.conf.ext`	Autenticación de Dovecot	7
`/etc/dovecot/conf.d/10-mail.conf`, `15-mailboxes.conf`	Buzones	7
`/etc/dovecot/conf.d/10-master.conf`	Servicios y sockets	7
`/etc/dovecot/conf.d/10-ssl.conf`, `20-lmtp.conf`	TLS y LMTP	7
`/etc/dovecot/users`	Cuentas de correo	7, 8
`/var/mail/vhosts/dominio/usuario/Maildir`	Buzones	7
`/etc/opendkim.conf`, `/etc/opendkim/*`	DKIM	9
`/etc/systemd/system/opendkim.service.d/override.conf`	Socket de OpenDKIM	9
`/etc/fail2ban/jail.local`	Fail2ban	12
`/etc/apt/apt.conf.d/50unattended-upgrades`	Actualizaciones automáticas	12
`/usr/local/sbin/correo-alta`, `correo-clave`, `correo-baja`	Gestión de cuentas	8
`/usr/local/sbin/backup-servidor`, `estado-servidor`	Copias y monitorización	12
`/etc/cron.d/backup-servidor`, `/etc/cron.d/estado-servidor`	Tareas programadas	12
`/var/backups/servidor/`	Copias de seguridad	12
`/etc/vsftpd.conf`	Configuración de vsftpd	15
`/etc/vsftpd.userlist`	Usuarios con acceso FTP	15
`/etc/pam.d/vsftpd`	Autenticación PAM del FTP	15
`/var/log/vsftpd.log`	Registro del FTP	15

Anexo C. Comandos esenciales por servicio

Servicio	Comprobar configuración	Recargar	Estado / diagnóstico
Apache	`sudo apachectl configtest`	`sudo systemctl reload apache2`	`sudo apachectl -S`, `-M`
PHP-FPM	`sudo php-fpm8.3 -t`	`sudo systemctl reload php8.3-fpm`	`ls /run/php/`
MariaDB	—	`sudo systemctl restart mariadb`	`sudo mariadb -e "STATUS"`
Postfix	`sudo postfix check`	`sudo systemctl reload postfix`	`postconf -n`, `mailq`
Dovecot	`sudo doveconf -n`	`sudo doveadm reload`	`sudo doveadm who`, `doveadm auth test`

Servicio	Comprobar configuración	Recargar	Estado / diagnóstico
OpenDKIM	`sudo opendkim -n`	`sudo systemctl restart opendkim`	`opendkim-testkey -d dominio -s mail -vvv`
SSH	`sudo sshd -t`	`sudo systemctl restart ssh`	`sudo sshd -T`
Fail2ban	`sudo fail2ban-client -t`	`sudo fail2ban-client reload`	`sudo fail2ban-client status`
UFW	—	`sudo ufw reload`	`sudo ufw status verbose`
vsftpd	— (comprobar con `journalctl -u vsftpd` tras reiniciar)	`sudo systemctl restart vsftpd`	`sudo tail /var/log/vsftpd.log`
Certbot	—	—	`sudo certbot certificates`, `renew --dry-run`

Anexo D. Lista de tareas para añadir un nuevo dominio

1. DNS: A (`@`, `www`, `mail`), MX, SPF, DMARC.
2. Usuario Linux `webN`, carpetas y permisos (PASOS 3.1–3.3).
3. Pool PHP-FPM `webN.conf` (PASO 5.3) y recarga de PHP-FPM.
4. VirtualHost HTTP, certificado con `certbot certonly --webroot`, VirtualHost HTTPS con PHP (PASOS 3.5, 3.11, 5.7).
5. Base de datos, usuario y permisos (PASOS 4.6–4.8).
6. Aplicación y `config.php` (PASOS 5.10–5.16).
7. Correo: añadir el dominio a `/etc/postfix/virtual_domains` + `postmap`; crear cuentas con `correo-alta`; alias `postmaster@`, `abuse@`, `dmarc@`, `noreply@`.
8. Ampliar el certificado del correo con `mail.nuevodominio` (`certbot certonly --webroot -w /var/www/html --cert-name mail.ejemplo1.com -d mail.ejemplo1.com -d mail.ejemplo2.com -d mail.nuevodominio`).
9. DKIM: generar clave, añadirla a `key.table` y `signing.table`, reiniciar OpenDKIM, publicar el TXT.
10. Si usas FTP: PASOS 15.4 y 15.5 para el nuevo usuario, añadirlo a `/etc/vsftpd.userlist` y ampliar el certificado `ftp.ejemplo1.com` con `ftp.nuevodominio`.
11. Verificar con la batería del Módulo 13.